Algorithmic Optimization of Dense Sphere Packings in Non-Euclidean Spaces

Marian Ela J. Alibadbad

Region VIII

Mathematics and Computational Science – Individual Category (MCS-I)

December 3, 2026

Table of Contents

Chapter **Page**

How to Run the Code **0**

Software Environment 0

Commands 0

Source Code Listing **0**

build_all.py 0

geometry.py 0

algorithms.py 0

tests.py 0

experiments.py 0

analyze.py 0

make_figures.py 0

docbuilder.py 0

make_plan.py 0

make_paper.py 0

make_code_listing.py 0

make_package.py 0

supplementary.py 0

supp_section.py 0

omml.py 0

newalgs.py 0

confirm_design.py 0

record_env.py 0

confirm_tune.py 0

confirm_run.py 0

confirm_count.py 0

confirm_analyze.py 0

conf_figures.py 0

conf_section.py 0

confirm_configs.py 0

confirm_retune.py 0

confirm_run_retuned.py 0

confirm_extra.py 0

confirm_mini.py 0

confirm_followup.py 0

confirm_round3.py 0

r3_section.py 0

checks.py 0

fill.py 0

pilot.py 0

tune.py 0

tune2.py 0

tune3.py 0

Unit Test Output **0**

Algorithmic Optimization of Dense Sphere Packings in Non-Euclidean Spaces

How to Run the Code

This document lists all code used in the study. The sections below give the software environment and the exact commands that reproduce every result.

Software Environment

The reported runs used the versions below; an independent full rerun (python3 build_all.py --rerun) under Python 3.13.18 with numpy 2.5.3, scipy 1.18.1, pandas 3.0.5, and matplotlib 3.11.2 reproduced every score, test statistic, and p-value in the CSV files exactly; only the wall-clock times differ. No other libraries are needed, and the optimizers use only numpy and scipy-compatible routines from numpy.

Python 3.12.3
numpy 2.4.4
scipy 1.17.1
pandas 3.0.2
matplotlib 3.10.8
python-docx 1.2.0
Pygments 2.20.0
pdfplumber 0.11.9
Pillow 12.1.1

Commands

Place all files listed below in one folder and run the commands in that order. The first command runs the validation tests and stops if any test fails. The second command runs the unit tests, all experiments, the statistics, the figures, and the builds of the plan, paper, listing, and data package; add --rerun to recompute every run from scratch (about 17 minutes on one core for the first study). Seeds 1 to 30 are used for evaluation and are fixed, so a rerun reproduces every value. Run them from that folder.

```
python3 tests.py
python3 build_all.py
python3 build_all.py --rerun
```

Source Code Listing

Each file is listed in full with line numbers and syntax colors. A long line continues on the next row, marked with an arrow, and its indentation is kept.

build_all.py

Master script. Runs the unit tests, all experiments, the statistics, the figures, and builds every deliverable.

```
"""build_all.py -- master script.  Runs validation tests, every experiment, all statistics,
all figures, and then builds the plan, paper, code listing, and data package.

Usage:  python3 build_all.py            (reuses cached runs if present)
        python3 build_all.py --rerun    (recomputes every run; takes about 17 minutes)
All random seeds are fixed (seeds 1-30 for evaluation, 9001-9004 for pilot tuning, 9101-9105 for unit tests), so a
rerun reproduces every value.
"""
import os
import subprocess
import sys

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))   # folder that contains code/ and out/
STEPS = [
    ("unit tests", ["python3", "tests.py"], "unit_test_output.txt"),
    ("experiments", ["python3", "experiments.py"] + (["--rerun"] if "--rerun" in sys.argv else []), None),
    ("statistics", ["python3", "analyze.py"], None),
    ("supplementary analyses", ["python3", "supplementary.py"] + (["--rerun"] if "--rerun" in sys.argv else []), None),
    ("confirmatory round: environment", ["python3", "record_env.py"], None),
    ("confirmatory round: tuning", ["python3", "confirm_tune.py"], None),
    ("confirmatory round: runs", ["python3", "confirm_run.py"] + (["--rerun"] if "--rerun" in sys.argv else []), None),
    ("confirmatory round: count-based problem", ["python3", "confirm_count.py"], None),
    ("confirmatory round: analysis", ["python3", "confirm_analyze.py"], None),
    ("confirmatory round: saved configurations and wall metrics", ["python3", "confirm_configs.py"], None),
    ("confirmatory round: re-tuning (extended grids)", ["python3", "confirm_retune.py"], None),
    ("confirmatory round: re-tuned runs", ["python3", "confirm_run_retuned.py"], None),
```

```
    ("mini-round: runs", ["python3", "confirm_mini.py", "run"], None),
    ("mini-round: analysis", ["python3", "confirm_mini.py", "analyze"], None),
    ("follow-up: runs", ["python3", "confirm_followup.py", "run"], None),
    ("follow-up: L-BFGS-B on the mini-round conditions", ["python3", "confirm_followup.py", "run_mini_lbfgsb"],
↳ None),
    ("follow-up: analysis", ["python3", "confirm_followup.py", "analyze"], None),
    ("final round: part A runs", ["python3", "confirm_round3.py", "runA"], None),
    ("final round: part B runs", ["python3", "confirm_round3.py", "runB"], None),
    ("final round: analysis", ["python3", "confirm_round3.py", "analyze"], None),
    ("confirmatory round: additional analyses", ["python3", "confirm_extra.py"], None),
    ("figures", ["python3", "make_figures.py"], None),
    ("research paper", ["python3", "make_paper.py"], None),
    ("code listing", ["python3", "make_code_listing.py"], None),
    ("data package", ["python3", "make_package.py"], None),
    ("final checks", ["python3", "checks.py"], None),
]

if __name__ == "__main__":
    os.makedirs(os.path.join(ROOT, "out", "data"), exist_ok=True)
    for name, cmd, log in STEPS:
        print("== " + name, flush=True)
        if log:
            with open(os.path.join(ROOT, "out", log), "w") as fh:
                r = subprocess.run(cmd, stdout=fh, stderr=subprocess.STDOUT)
        else:
            r = subprocess.run(cmd)
        if r.returncode != 0:
            sys.exit("step failed: " + name)
    print("all steps finished")
```

geometry.py

Geometry of the sphere, the hyperbolic plane (hyperboloid model), and the Euclidean plane: distances, gradients, exponential maps, sampling.

```
"""geometry.py -- Geometry of the three spaces used in the study.

Algorithmic Optimization of Dense Sphere Packings in Non-Euclidean Spaces

Spaces
```

```
  S2 : unit 2-sphere, points are unit vectors in R^3 (curvature +1)
  H2 : hyperbolic plane, curvature -1, hyperboloid model; a point is
       X = (x0, x1, x2) with -x0^2 + x1^2 + x2^2 = -1 and x0 > 0.
       The Poincare disk coordinate is (x1, x2) / (1 + x0).
  R2 : Euclidean plane (control), points are vectors in R^2.

All three spaces share one interface so that the optimizers are written
once.  Only numpy is used.
"""
import numpy as np

TINY = 1e-12


class Space:
    """A constant-curvature plane (S2, H2 or R2) with a shared interface."""

    def __init__(self, kind):
        if kind not in ("S2", "H2", "R2"):
            raise ValueError(kind)
        self.kind = kind
        self.amb = 2 if kind == "R2" else 3  # ambient dimension

    # ------------------------------------------------------------------
    # basic algebra
    # ------------------------------------------------------------------
    def lorentz(self, A, B):
        """Row-wise Lorentz product <A,B> = -a0 b0 + a1 b1 + a2 b2."""
        return -A[:, 0] * B[:, 0] + A[:, 1] * B[:, 1] + A[:, 2] * B[:, 2]

    # ------------------------------------------------------------------
    # distances
    # ------------------------------------------------------------------
    def dist_from_c(self, c):
        if self.kind == "S2":
            return 2.0 * np.arcsin(np.sqrt(np.clip((1.0 - c) / 2.0, 0.0, 1.0)))
        # H2
        return 2.0 * np.arcsinh(np.sqrt(np.maximum((c - 1.0) / 2.0, 0.0)))

    def pair_c(self, X):
        """c_ij = cos d_ij (S2) or cosh d_ij (H2)."""
        if self.kind == "S2":
            return X @ X.T
        G = -(X[:, :1] @ X[:, :1].T) + X[:, 1:] @ X[:, 1:].T  # Lorentz product
        return -G
```

```
51
52      def pair_dist(self, X):
53          """Full n x n distance matrix with +inf on the diagonal."""
54          n = len(X)
55          if self.kind == "R2":
56              diff = X[:, None, :] - X[None, :, :]
57              D = np.sqrt((diff ** 2).sum(-1))
58          else:
59              D = self.dist_from_c(self.pair_c(X))
60          D[np.arange(n), np.arange(n)] = np.inf
61          return D
62
63      def dist_to_many(self, x, Y):
64          """Distance from one point x to each row of Y."""
65          if self.kind == "R2":
66              return np.sqrt(((Y - x) ** 2).sum(-1))
67          if self.kind == "S2":
68              c = Y @ x
69          else:
70              c = -self.lorentz(Y, np.broadcast_to(x, Y.shape))
71          return self.dist_from_c(c)
72
73      def dist(self, x, y):
74          """Distance between two single points."""
75          return float(self.dist_to_many(np.asarray(x, float), np.asarray(y, float)[None, :])[0])
76
77      # ------------------------------------------------------------------
78      # radial distance from the origin (ball problems on H2 and R2)
79      # ------------------------------------------------------------------
80      def origin(self):
81          if self.kind == "S2":
82              return np.array([0.0, 0.0, 1.0])
83          if self.kind == "H2":
84              return np.array([1.0, 0.0, 0.0])
85          if self.kind == "R2":
86              return np.zeros(2)
87          raise RuntimeError
88
89      def rho(self, X):
90          if self.kind == "R2":
91              return np.sqrt((X ** 2).sum(-1))
92          if self.kind == "S2":
93              return 2.0 * np.arcsin(np.sqrt(np.clip((1.0 - X[:, 2]) / 2.0, 0.0, 1.0)))
94          if self.kind == "H2":
95              return 2.0 * np.arcsinh(np.sqrt(np.maximum((X[:, 0] - 1.0) / 2.0, 0.0)))
```

```
        raise RuntimeError

    def grad_rho(self, X, w):
        """sum_i w_i * grad rho(X_i)  as an (n, amb) array of tangent vectors."""
        r = self.rho(X)
        if self.kind == "R2":
            nrm = np.maximum(r, TINY)[:, None]
            return w[:, None] * X / nrm
        O = self.origin()
        if self.kind == "S2":
            s = np.maximum(np.sin(r), TINY)
            c = X[:, 2]
        else:
            s = np.maximum(np.sinh(r), TINY)
            c = X[:, 0]
        return (w / s)[:, None] * (c[:, None] * X - O[None, :])

    # ------------------------------------------------------------------
    # Riemannian gradient of  sum_{i<j} f(d_ij)  given W_ij = f'(d_ij)
    # ------------------------------------------------------------------
    def grad_pairs(self, X, D, W):
        """Return G with G_i = sum_j W_ij grad_{X_i} d_ij  (tangent at X_i)."""
        n = len(X)
        if self.kind == "R2":
            a = W / np.where(np.isfinite(D), np.maximum(D, TINY), 1.0)
            np.fill_diagonal(a, 0.0)
            return X * a.sum(1)[:, None] - a @ X
        C = self.pair_c(X)
        if self.kind == "S2":
            s = np.sqrt(np.maximum(1.0 - C * C, 1e-18))
        else:
            s = np.sqrt(np.maximum(C * C - 1.0, 1e-18))
        a = W / s
        np.fill_diagonal(a, 0.0)
        return X * (a * C).sum(1)[:, None] - a @ X

    # ------------------------------------------------------------------
    # moving along geodesics
    # ------------------------------------------------------------------
    def tangent_norm(self, V, X=None):
        if self.kind == "H2":
            return np.sqrt(np.maximum(self.lorentz(V, V), 0.0))
        return np.sqrt((V ** 2).sum(-1))

    def project_tangent(self, X, U):
```

```
        """Project ambient vectors U onto the tangent space at X (row-wise)."""
        if self.kind == "R2":
            return U
        if self.kind == "S2":
            return U - (U * X).sum(-1, keepdims=True) * X
        return U + self.lorentz(X, U)[:, None] * X

    def retract(self, X, V):
        """Exponential map: move each X_i along the geodesic with initial velocity V_i."""
        if self.kind == "R2":
            return X + V
        nv = self.tangent_norm(V)
        nv_s = np.maximum(nv, TINY)
        if self.kind == "S2":
            Y = np.cos(nv)[:, None] * X + (np.sin(nv) / nv_s)[:, None] * V
            return Y / np.linalg.norm(Y, axis=1, keepdims=True)
        Y = np.cosh(nv)[:, None] * X + (np.sinh(nv) / nv_s)[:, None] * V
        Y[:, 0] = np.sqrt(1.0 + Y[:, 1] ** 2 + Y[:, 2] ** 2)
        return Y

    def transport_velocity(self, X, V):
        """Velocity of the geodesic started at X with velocity V after unit time
        (used by the compression method); returns (X_new, V_new)."""
        if self.kind == "R2":
            return X + V, V.copy()
        nv = self.tangent_norm(V)
        nv_s = np.maximum(nv, TINY)
        if self.kind == "S2":
            Xn = np.cos(nv)[:, None] * X + (np.sin(nv) / nv_s)[:, None] * V
            Vn = -(nv * np.sin(nv))[:, None] * X + np.cos(nv)[:, None] * V
            nrm = np.linalg.norm(Xn, axis=1, keepdims=True)
            Xn = Xn / nrm
            return Xn, Vn
        Xn = np.cosh(nv)[:, None] * X + (np.sinh(nv) / nv_s)[:, None] * V
        Vn = (nv * np.sinh(nv))[:, None] * X + np.cosh(nv)[:, None] * V
        Xn[:, 0] = np.sqrt(1.0 + Xn[:, 1] ** 2 + Xn[:, 2] ** 2)
        return Xn, Vn

    def renormalize(self, X):
        if self.kind == "S2":
            return X / np.linalg.norm(X, axis=1, keepdims=True)
        if self.kind == "H2":
            X = X.copy()
            X[:, 0] = np.sqrt(1.0 + X[:, 1] ** 2 + X[:, 2] ** 2)
        return X
```

```
186
187     def clip_to_ball(self, X, Rmax):
188         """Radially pull points with rho > Rmax back to rho = Rmax."""
189         r = self.rho(X)
190         out = r > Rmax
191         if not out.any():
192             return X
193         X = X.copy()
194         if self.kind == "R2":
195             X[out] *= (Rmax / np.maximum(r[out], TINY))[:, None]
196         elif self.kind == "S2":
197             u = X[out, :2]
198             un = np.maximum(np.linalg.norm(u, axis=1, keepdims=True), TINY)
199             X[out, :2] = np.sin(Rmax) * u / un
200             X[out, 2] = np.cos(Rmax)
201         else:
202             u = X[out, 1:]
203             un = np.maximum(np.linalg.norm(u, axis=1, keepdims=True), TINY)
204             X[out, 1:] = np.sinh(Rmax) * u / un
205             X[out, 0] = np.cosh(Rmax)
206         return X
207
208     # ------------------------------------------------------------------
209     # sampling and conversions
210     # ------------------------------------------------------------------
211     def random_points(self, n, rng, R=None):
212         """n random points: uniform on S2, or uniform in the ball of radius R."""
213         if self.kind == "S2" and R is None:
214             v = rng.normal(size=(n, 3))
215             return v / np.linalg.norm(v, axis=1, keepdims=True)
216         th = rng.uniform(0.0, 2.0 * np.pi, n)
217         u = rng.uniform(0.0, 1.0, n)
218         if self.kind == "R2":
219             r = R * np.sqrt(u)
220             return np.c_[r * np.cos(th), r * np.sin(th)]
221         if self.kind == "S2":
222             # cap: area element sin(rho) d rho d theta, cdf = (1 - cos r)/(1 - cos R)
223             r = np.arccos(1.0 - u * (1.0 - np.cos(R)))
224             return np.c_[np.sin(r) * np.cos(th), np.sin(r) * np.sin(th), np.cos(r)]
225         # H2: area element sinh(rho) d rho d theta, cdf = (cosh r - 1)/(cosh R - 1)
226         r = np.arccosh(1.0 + u * (np.cosh(R) - 1.0))
227         return np.c_[np.cosh(r), np.sinh(r) * np.cos(th), np.sinh(r) * np.sin(th)]
228
229     def from_polar(self, r, th):
230         """Point from (radial distance, angle); H2 and R2 only."""
```

```
        r = np.atleast_1d(r)
        th = np.atleast_1d(th)
        if self.kind == "R2":
            return np.c_[r * np.cos(th), r * np.sin(th)]
        if self.kind == "S2":
            return np.c_[np.sin(r) * np.cos(th), np.sin(r) * np.sin(th), np.cos(r)]
        return np.c_[np.cosh(r), np.sinh(r) * np.cos(th), np.sinh(r) * np.sin(th)]

    def to_plane(self, X):
        """Planar drawing coordinates: Poincare disk for H2, identity for R2."""
        if self.kind == "H2":
            return X[:, 1:] / (1.0 + X[:, :1])
        return X

    def from_disk(self, Z):
        """Hyperboloid point from Poincare disk coordinates."""
        z2 = (Z ** 2).sum(-1)
        x0 = (1.0 + z2) / (1.0 - z2)
        sp = 2.0 / (1.0 - z2)
        return np.c_[x0, sp * Z[:, 0], sp * Z[:, 1]]

    def on_manifold_error(self, X):
        if self.kind == "S2":
            return float(np.max(np.abs(np.linalg.norm(X, axis=1) - 1.0)))
        if self.kind == "H2":
            return float(np.max(np.abs(-X[:, 0] ** 2 + X[:, 1] ** 2 + X[:, 2] ** 2 + 1.0)))
        return 0.0


# ---------------------------------------------------------------------
# closed-form reference distances (independent of the Space class)
# ---------------------------------------------------------------------
def poincare_distance(z, w):
    """Poincare disk distance (curvature -1) between complex numbers z, w."""
    num = 2.0 * abs(z - w) ** 2
    den = (1.0 - abs(z) ** 2) * (1.0 - abs(w) ** 2)
    return float(np.arccosh(1.0 + num / den))


def sphere_distance_haversine(a, b):
    """Great-circle distance between unit vectors a, b using atan2 form."""
    return float(np.arctan2(np.linalg.norm(np.cross(a, b)), np.dot(a, b)))


def spherical_cap_area(R):
```

```
    """Area of a spherical cap of angular radius R on the unit sphere: 2 pi (1 - cos R)."""
    return 2.0 * np.pi * (1.0 - np.cos(R))


def hyperbolic_ball_area(R):
    """Area of a hyperbolic disk of radius R (curvature -1): 2 pi (cosh R - 1)."""
    return 2.0 * np.pi * (np.cosh(R) - 1.0)


def euclid_image_disk(s, r):
    """Euclidean center and radius (in the Poincare disk) of the hyperbolic disk
    of radius r whose center lies at hyperbolic distance s from the origin."""
    lo = np.tanh((s - r) / 2.0) if s >= r else -np.tanh((r - s) / 2.0)
    hi = np.tanh((s + r) / 2.0)
    return 0.5 * (lo + hi), 0.5 * (hi - lo)
```

algorithms.py

The packing problems and the three optimizers: gradient descent on a soft-minimum energy, simulated annealing, and compression.

```
"""algorithms.py -- Problems and the three optimizers (numpy only).

Problem A (S2): place n points to maximize the minimum pairwise geodesic distance.
Problem B (H2, R2): place n equal disks in a ball of radius R centered at the
origin and maximize the common disk radius r.  Disk centers must satisfy
rho_i <= R - r and d_ij >= 2 r, so with  l_min = min( min_ij d_ij , min_i 2(R - rho_i) )
the best common radius is r = l_min / 2.

All three optimizers receive the same budget measured in pair-distance
evaluations (PE), so comparisons do not depend on implementation speed.

  1. gd : gradient descent on a soft-minimum energy with an annealed exponent p
  2. sa : simulated annealing, adaptive step, hard-minimum objective
  3. ls : Lubachevsky-Stillinger-style compression (time-stepped, simplified)
"""
import time
import numpy as np
from geometry import Space

```

```
ALGS = {1: "gd", 2: "sa", 3: "ls"}
ALG_NAMES = {"gdp": "Gradient Descent + Polishing", "gd": "Gradient Descent", "sa": "Simulated Annealing", "ls":
 "Compression"}


class Problem:
    def __init__(self, kind, n, R=None):
        self.space = Space(kind)
        self.kind = kind
        self.n = n
        self.R = R
        self.ball = R is not None          # bounded problem (disks in a geodesic ball)
        self.iu = np.triu_indices(n, 1)
        self.sweep = n * (n - 1) / 2.0
        if kind == "S2" and R is None:
            area = 4.0 * np.pi
        elif kind == "S2":
            area = 2.0 * np.pi * (1.0 - np.cos(R))
        elif kind == "H2":
            area = 2.0 * np.pi * (np.cosh(R) - 1.0)
        else:
            area = np.pi * R * R
        self.scale = float(np.sqrt(area / n))

    def initial(self, rng):
        if not self.ball:
            return self.space.random_points(self.n, rng)
        return self.space.random_points(self.n, rng, R=self.R - 0.05)

    def lmin(self, X, D=None):
        """Minimum of pairwise distances and doubled wall clearances."""
        if D is None:
            D = self.space.pair_dist(X)
        m = float(D.min())
        if self.ball:
            m = min(m, float(2.0 * (self.R - self.space.rho(X)).min()))
        return m

    def score(self, X):
        """Min angular distance (S2) or common disk radius (ball problems)."""
        m = self.lmin(X)
        return m / 2.0 if self.ball else m

    def density(self, r):
        """Fraction of the ball covered by n disks of radius r."""
```

```
        if self.kind == "S2":
            return self.n * (1.0 - np.cos(r)) / (1.0 - np.cos(self.R))
        if self.kind == "H2":
            return self.n * (np.cosh(r) - 1.0) / (np.cosh(self.R) - 1.0)
        if self.kind == "R2":
            return self.n * r * r / (self.R * self.R)
        return float("nan")


def _to_score(P, l):
    return l / 2.0 if P.ball else l


# ---------------------------------------------------------------------
# 1. gradient descent with annealed exponent
# ---------------------------------------------------------------------
def _soft_energy(P, X, p, want_grad):
    S = P.space
    D = S.pair_dist(X)
    Lp = D[P.iu]
    if P.ball:
        Lw = np.maximum(2.0 * (P.R - S.rho(X)), 1e-9)
        L = np.concatenate([Lp, Lw])
    else:
        L = Lp
    L = np.maximum(L, 1e-9)
    z = -p * np.log(L)
    zm = z.max()
    w = np.exp(z - zm)
    Z = w.sum()
    E = (zm + np.log(Z)) / p
    lmin = float(L.min())
    if not want_grad:
        return E, None, lmin
    sft = w / Z
    npair = len(Lp)
    Wd = np.zeros((P.n, P.n))
    Wd[P.iu] = -sft[:npair] / L[:npair]
    Wd = Wd + Wd.T
    G = S.grad_pairs(X, D, Wd)
    if P.ball:
        wr = 2.0 * sft[npair:] / L[npair:]
        G = G + S.grad_rho(X, wr)
    return E, G, lmin

```

```
109
110 def run_gd(P, X0, budget, rng, p0=2.0, pmax=150.0):
111     S = P.space
112     X = X0.copy()
113     pe = 0.0
114     cost = P.sweep + (P.n if P.ball else 0)
115     trace = [(0.0, P.score(X))]
116     best, bestX = trace[0][1], X.copy()
117     it = 0
118     eta_mult = 0.5
119     Rclip = (P.R - 1e-6) if P.ball else None
120     while pe + 2 * cost <= budget:
121         frac = pe / budget
122         p = p0 * (pmax / p0) ** frac
123         E, G, lmin = _soft_energy(P, X, p, True)
124         pe += cost
125         sc = _to_score(P, lmin)
126         if sc > best:
127             best, bestX = sc, X.copy()
128         gnorm = S.tangent_norm(G)
129         gmax = max(float(gnorm.max()), 1e-12)
130         eta = eta_mult * lmin ** 2 / p
131         eta = min(eta, 0.25 * lmin / gmax)
132         accepted = False
133         for _ in range(5):
134             if pe + cost > budget:
135                 break
136             Xn = S.retract(X, -eta * G)
137             if P.ball:
138                 Xn = S.clip_to_ball(Xn, Rclip)
139             En, _, _ = _soft_energy(P, Xn, p, False)
140             pe += cost
141             if En < E:
142                 accepted = True
143                 break
144             eta *= 0.4
145         if accepted:
146             X = Xn
147             eta_mult = min(eta_mult * 1.3, 2.0)
148         else:
149             eta_mult = max(eta_mult * 0.5, 1e-3)
150         it += 1
151         trace.append((pe, best))
152     sc_final = P.score(X)
153     if sc_final > best:
```

```
        best, bestX = sc_final, X.copy()
    trace.append((pe, best))
    return dict(X=bestX, score=best, trace=trace, pe=pe, iters=it)


# ---------------------------------------------------------------------
# 2. simulated annealing
# ---------------------------------------------------------------------
def run_sa(P, X0, budget, rng, T0f=0.02, T1f=1e-5, focus=0.3):
    S = P.space
    n = P.n
    X = X0.copy()
    D = S.pair_dist(X)
    rho = S.rho(X) if P.ball else None
    pe = P.sweep
    nmoves = int(max((budget - pe) // (n - 1), 1))
    scale = P.scale
    T0, T1 = T0f * scale, T1f * scale
    delta = 0.3 * scale

    def current_l():
        m = float(D.min())
        if P.ball:
            m = min(m, float(2.0 * (P.R - rho).min()))
        return m

    lcur = current_l()
    best = _to_score(P, lcur)
    bestX = X.copy()
    trace = [(0.0, best)]
    noise = rng.normal(size=(8192, S.amb))
    unif = rng.random(size=(8192, 3))
    k = 0
    acc = 0
    tried = 0
    log_ratio = np.log(T1 / T0)
    rlimit = (P.R - 1e-6) if P.ball else None
    rec_every = max(nmoves // 400, 1)
    for m in range(nmoves):
        if k >= 8192:
            noise = rng.normal(size=(8192, S.amb))
            unif = rng.random(size=(8192, 3))
            k = 0
        T = T0 * np.exp(log_ratio * m / nmoves)
        # choose the point to move
```

```
        if unif[k, 0] < focus:
            flat = int(np.argmin(D))
            i = flat // n if unif[k, 1] < 0.5 else flat % n
            if P.ball:
                iw = int(np.argmax(rho))
                if 2.0 * (P.R - rho[iw]) < D.flat[flat]:
                    i = iw
        else:
            i = int(unif[k, 1] * n) % n
        xi = X[i:i + 1]
        v = S.project_tangent(xi, noise[k:k + 1])
        nv = float(S.tangent_norm(v)[0])
        if nv < 1e-12:
            k += 1
            continue
        step = delta * (0.1 + 0.9 * unif[k, 2])
        xn = S.retract(xi, v * (step / nv))
        if P.ball:
            rn = float(S.rho(xn)[0])
            if rn > rlimit:
                k += 1
                tried += 1
                continue
        row_old = D[i].copy()
        others = np.delete(X, i, axis=0)
        drow = S.dist_to_many(xn[0], others)
        newrow = np.insert(drow, i, np.inf)
        D[i, :] = newrow
        D[:, i] = newrow
        if P.ball:
            rho_old = rho[i]
            rho[i] = rn
        lnew = current_l()
        tried += 1
        if lnew >= lcur or rng.random() < np.exp((lnew - lcur) / T):
            X[i] = xn[0]
            lcur = lnew
            acc += 1
            sc = _to_score(P, lcur)
            if sc > best:
                best = sc
                bestX = X.copy()
        else:
            D[i, :] = row_old
            D[:, i] = row_old
```

```
            if P.ball:
                rho[i] = rho_old
        pe += n - 1
        k += 1
        if tried >= 100:
            rate = acc / tried
            delta = delta * (1.15 if rate > 0.4 else 0.87 if rate < 0.25 else 1.0)
            delta = min(max(delta, 1e-7 * scale), 1.0 * scale)
            acc = tried = 0
        if m % rec_every == 0:
            trace.append((pe, best))
    trace.append((pe, best))
    return dict(X=bestX, score=best, trace=trace, pe=pe, iters=nmoves)


# ---------------------------------------------------------------------
# 3. Lubachevsky-Stillinger-style compression (time-stepped, simplified)
# ---------------------------------------------------------------------
def _toward(S, Xi, Xj, d):
    """Unit tangent vectors at Xi pointing to Xj (row-wise)."""
    if S.kind == "R2":
        return (Xj - Xi) / np.maximum(d, 1e-12)[:, None]
    if S.kind == "S2":
        c = (Xi * Xj).sum(-1)
        s = np.maximum(np.sin(d), 1e-9)
    else:
        c = -S.lorentz(Xi, Xj)
        s = np.maximum(np.sinh(d), 1e-9)
    return (Xj - c[:, None] * Xi) / s[:, None]


def _inner(S, A, B):
    return S.lorentz(A, B) if S.kind == "H2" else (A * B).sum(-1)


def run_ls(P, X0, budget, rng, dt_f=0.02, g0=0.25, g1=0.01):
    S = P.space
    n = P.n
    X = X0.copy()
    V = S.project_tangent(X, rng.normal(size=X.shape))
    V = V / np.maximum(S.tangent_norm(V), 1e-12)[:, None]
    dt = dt_f * P.scale
    a = 0.0
    pe = 0.0
    cost = 2 * P.sweep
```

```
    Rclip = (P.R - 1e-6) if P.ball else None
    Dn = S.pair_dist(X)
    pe += P.sweep
    best = _to_score(P, P.lmin(X, Dn))
    bestX = X.copy()
    trace = [(0.0, best)]
    steps = 0
    nsteps_total = max(int((budget - pe) // cost), 1)
    rec_every = max(nsteps_total // 400, 1)
    while pe + cost <= budget:
        frac = pe / budget
        gamma = g0 + (g1 - g0) * frac
        # 1. free flight along geodesics
        X, Vdt = S.transport_velocity(X, V * dt)
        V = Vdt / dt
        # 2. wall reflection (ball problems)
        if P.ball:
            rho = S.rho(X)
            out = rho > (P.R - a)
            if out.any():
                u = S.grad_rho(X, np.ones(n))
                un = np.maximum(S.tangent_norm(u), 1e-12)
                u = u / un[:, None]
                vn = _inner(S, V, u)
                refl = out & (vn > 0)
                V[refl] -= 2.0 * vn[refl][:, None] * u[refl]
                over = np.where(out, rho - (P.R - a), 0.0)
                X = S.retract(X, -over[:, None] * u)
        # 3. pair collisions: exchange normal velocity components, separate overlaps
        D = S.pair_dist(X)
        ii, jj = np.where(np.triu(D < 2.0 * a, 1))
        if len(ii):
            d = D[ii, jj]
            uij = _toward(S, X[ii], X[jj], d)
            uji = _toward(S, X[jj], X[ii], d)
            ai = _inner(S, V[ii], uij)
            aj = _inner(S, V[jj], uji)
            appr = (ai + aj) > 0
            if appr.any():
                f = (ai + aj)[appr]
                np.add.at(V, ii[appr], -f[:, None] * uij[appr])
                np.add.at(V, jj[appr], -f[:, None] * uji[appr])
            ov = 0.5 * (2.0 * a - d)
            push = np.zeros_like(X)
            np.add.at(push, ii, -ov[:, None] * uij)
```

```
            np.add.at(push, jj, -ov[:, None] * uji)
            X = S.retract(X, push)
            V = S.project_tangent(X, V)
        if P.ball:
            X = S.clip_to_ball(X, Rclip)
        X = S.renormalize(X)
        # keep velocities tangent and the total kinetic energy fixed (guards against drift)
        V = S.project_tangent(X, V)
        ke = float((S.tangent_norm(V) ** 2).mean())
        if ke > 0:
            V = V / np.sqrt(ke)
        # 4. growth limited by the available room
        Dn = S.pair_dist(X)
        pe += cost
        lmin = P.lmin(X, Dn)
        feasible = lmin / 2.0
        a = min(a + gamma * dt, feasible)
        sc = _to_score(P, lmin)
        if sc > best:
            best, bestX = sc, X.copy()
        steps += 1
        if steps % rec_every == 0:
            trace.append((pe, best))
    trace.append((pe, best))
    return dict(X=bestX, score=best, trace=trace, pe=pe, iters=steps)


# ---------------------------------------------------------------------
# 4. gradient descent followed by a max-min polishing step (supplementary variant, not part of the main study)
# ---------------------------------------------------------------------
def _tangent_basis(S, X):
    """Two orthonormal tangent vectors at every point (Gram-Schmidt on the projected coordinate axes)."""
    n, amb = X.shape
    T = np.stack([S.project_tangent(X, np.tile(np.eye(amb)[k], (n, 1))) for k in range(amb)], axis=1)   # (n, amb, amb)
    nrm = np.array([[float(S.tangent_norm(T[:, k, :])[i]) for k in range(amb)] for i in range(n)])
    k1 = nrm.argmax(1)
    e1 = T[np.arange(n), k1, :]
    e1 = e1 / np.maximum(S.tangent_norm(e1), 1e-12)[:, None]
    best, e2 = np.full(n, -1.0), np.zeros_like(e1)
    for k in range(amb):
        r = T[:, k, :] - _inner(S, T[:, k, :], e1)[:, None] * e1
        rn = S.tangent_norm(r)
        better = rn > best
        e2[better] = (r / np.maximum(rn, 1e-12)[:, None])[better]
```

```
        best = np.where(better, rn, best)
    return e1, e2


def _polish(P, X, budget_left, eps0=0.25):
    """Sequential linear programming for the max-min problem: maximize t subject to the linearized
    constraints d_ij + (change) >= t and wall clearances >= t inside a trust region; exact re-evaluation decides
 acceptance."""
    from scipy.optimize import linprog
    S, n = P.space, P.n
    cost = P.sweep + (n if P.ball else 0)
    pe = cost
    D = S.pair_dist(X)
    lcur = P.lmin(X, D)
    eps = eps0 * lcur
    it = 0
    Rclip = (P.R - 1e-6) if P.ball else None
    while pe + cost <= budget_left and eps > 1e-11:
        e1, e2 = _tangent_basis(S, X)
        ii, jj = np.where(np.triu(D < lcur + 3.0 * eps, 1))
        d = D[ii, jj]
        uij, uji = _toward(S, X[ii], X[jj], d), _toward(S, X[jj], X[ii], d)
        rows, rhs = [], []
        for q in range(len(ii)):                                    # t - d - g.a <= 0   (g = -<e_k, u>)
            r = np.zeros(2 * n + 1)
            r[2 * ii[q]] = _inner(S, e1[ii[q]][None], uij[q][None])[0]
            r[2 * ii[q] + 1] = _inner(S, e2[ii[q]][None], uij[q][None])[0]
            r[2 * jj[q]] = _inner(S, e1[jj[q]][None], uji[q][None])[0]
            r[2 * jj[q] + 1] = _inner(S, e2[jj[q]][None], uji[q][None])[0]
            r[-1] = 1.0
            rows.append(r)
            rhs.append(d[q])
        if P.ball:
            rho = S.rho(X)
            w = S.grad_rho(X, np.ones(n))
            w = w / np.maximum(S.tangent_norm(w), 1e-12)[:, None]
            for i in np.where(2.0 * (P.R - rho) < lcur + 3.0 * eps)[0]:
                r = np.zeros(2 * n + 1)
                r[2 * i] = 2.0 * _inner(S, e1[i][None], w[i][None])[0]
                r[2 * i + 1] = 2.0 * _inner(S, e2[i][None], w[i][None])[0]
                r[-1] = 1.0
                rows.append(r)
                rhs.append(2.0 * (P.R - rho[i]))
        c = np.zeros(2 * n + 1)
        c[-1] = -1.0
```

```
        sol = linprog(c, A_ub=np.array(rows), b_ub=np.array(rhs), bounds=[(-eps, eps)] * (2 * n) + [(None, None)], method="highs")
        it += 1
        if sol.status != 0:
            eps *= 0.4
            continue
        a = sol.x[:-1].reshape(n, 2)
        V = a[:, :1] * e1 + a[:, 1:] * e2
        Xn = S.retract(X, V)
        if P.ball:
            Xn = S.clip_to_ball(Xn, Rclip)
        Dn = S.pair_dist(Xn)
        ln = P.lmin(Xn, Dn)
        pe += cost
        if ln > lcur:
            X, D, lcur = Xn, Dn, ln
            eps = min(eps * 1.5, 0.5 * lcur)
        else:
            eps *= 0.4
    return X, lcur, pe, it


def run_gdp(P, X0, budget, rng, polish_share=0.15):
    """Gradient descent on the first (1 - polish_share) of the budget, then max-min polishing on the rest."""
    res = run_gd(P, X0, budget * (1.0 - polish_share), rng)
    X, lcur, pe, it = _polish(P, res["X"], budget - res["pe"])
    sc = _to_score(P, lcur)
    if sc < res["score"]:
        X, sc = res["X"], res["score"]
    return dict(X=X, score=sc, trace=[(0.0, sc)], pe=res["pe"] + pe, iters=res["iters"] + it)


RUNNERS = {"gd": run_gd, "sa": run_sa, "ls": run_ls, "gdp": run_gdp}


def run_algorithm(name, P, X0, budget, seed_seq):
    """Run one optimizer; returns result dict including wall-clock time."""
    rng = np.random.default_rng(seed_seq)
    t0 = time.perf_counter()
    res = RUNNERS[name](P, X0, budget, rng)
    res["time"] = time.perf_counter() - t0
    res["final_check"] = P.score(res["X"])
    return res
```

tests.py

Validation tests: closed-form distances, metric axioms, finite-difference gradients, and the known Tammes optima for 4, 6, and 12 points.

```
"""tests.py -- Validation tests (all must pass before any experiment is run)."""
import sys
import numpy as np
from geometry import (Space, poincare_distance, sphere_distance_haversine,
                      hyperbolic_ball_area, spherical_cap_area, euclid_image_disk)
from algorithms import Problem, run_algorithm, _soft_energy

RESULTS = []


def check(name, ok, detail=""):
    RESULTS.append((name, bool(ok), detail))
    print(("PASS  " if ok else "FAIL  ") + name + (("  [" + detail + "]") if detail else ""))


def test_sphere_distance():
    S = Space("S2")
    rng = np.random.default_rng(101)
    X = S.random_points(200, rng)
    D = S.pair_dist(X)
    err = 0.0
    for _ in range(300):
        i, j = rng.integers(0, 200, 2)
        if i != j:
            err = max(err, abs(D[i, j] - sphere_distance_haversine(X[i], X[j])))
    check("S2 distance matches arctan2(|a x b|, a.b)", err < 1e-9, f"max err {err:.2e}")
    e = np.eye(3)
    ok = (abs(S.dist(e[0], e[1]) - np.pi / 2) < 1e-12 and abs(S.dist(e[0], -e[0]) - np.pi) < 1e-7
          and abs(S.dist(e[0], np.array([np.cos(0.3), np.sin(0.3), 0.0])) - 0.3) < 1e-12)
    check("S2 closed-form distances (quarter turn, antipode, 0.3 rad)", ok)


def test_hyperbolic_distance():
    S = Space("H2")
    rng = np.random.default_rng(102)
```

```
    Z = np.sqrt(rng.uniform(0, 0.97, 300)) * np.exp(1j * rng.uniform(0, 2 * np.pi, 300))
    Zr = np.c_[Z.real, Z.imag]
    X = S.from_disk(Zr)
    err = 0.0
    for i in range(0, 298, 2):
        d1 = S.dist(X[i], X[i + 1])
        d2 = poincare_distance(Z[i], Z[i + 1])
        err = max(err, abs(d1 - d2) / max(d2, 1e-12))
    check("H2 hyperboloid distance equals Poincare disk formula", err < 1e-7, f"max rel err {err:.2e}")
    ok = True
    for s in (0.1, 0.5, 1.0, 2.0, 4.0):
        p = S.from_disk(np.array([[np.tanh(s / 2.0), 0.0]]))[0]
        ok &= abs(S.dist(S.origin(), p) - s) < 1e-9 * max(1, s)
    check("H2 point at Euclidean radius tanh(s/2) is at hyperbolic distance s", ok)
    ok = True
    rng2 = np.random.default_rng(103)
    for _ in range(20):
        r1, r2 = rng2.uniform(0.2, 3, 2)
        t = rng2.uniform(0, 2 * np.pi)
        a = S.from_polar(r1, 0.0)[0]
        b = S.from_polar(r2, t)[0]
        d_law = np.arccosh(np.cosh(r1) * np.cosh(r2) - np.sinh(r1) * np.sinh(r2) * np.cos(t))
        ok &= abs(S.dist(a, b) - d_law) < 1e-8
    check("H2 distance equals hyperbolic law of cosines", ok)
    th = np.linspace(0, 2 * np.pi, 7)[:-1]
    A = hyperbolic_ball_area(2.0)
    rr = np.linspace(0, 2.0, 200001)
    num = 2 * np.pi * np.trapezoid(np.sinh(rr), rr)
    check("H2 disk area 2 pi (cosh R - 1) matches numeric integral", abs(A - num) < 1e-6, f"{A:.6f} vs {num:.6f}
↳ ")


def test_euclidean():
    S = Space("R2")
    check("R2 distance of (0,0) and (3,4) is 5", abs(S.dist([0, 0], [3, 4]) - 5.0) < 1e-12)
    R = 3.0
    P = Problem("R2", 7, R)
    ang = np.arange(6) * np.pi / 3
    X = np.vstack([[0, 0], np.c_[2 * (R / 3) * np.cos(ang), 2 * (R / 3) * np.sin(ang)]])
    check("R2 hexagonal 7-disk arrangement has radius R/3", abs(P.score(X) - R / 3) < 1e-12,
          f"score {P.score(X):.6f}")


def test_metric_axioms():
    ok_all = True
```

```
    for kind in ("S2", "H2", "R2"):
        S = Space(kind)
        rng = np.random.default_rng(104)
        X = S.random_points(60, rng) if kind == "S2" else S.random_points(60, rng, R=3.0)
        D = S.pair_dist(X)
        D0 = D.copy()
        np.fill_diagonal(D0, 0.0)
        sym = np.max(np.abs(D0 - D0.T))
        tri = 0.0
        for _ in range(4000):
            i, j, k = rng.integers(0, 60, 3)
            tri = max(tri, D0[i, k] - (D0[i, j] + D0[j, k]))
        ok = sym < 1e-12 and tri < 1e-9
        ok_all &= ok
        check(f"{kind} distance symmetry and triangle inequality (4000 triples)", ok,
              f"sym {sym:.1e}, worst violation {tri:.1e}")


def test_retraction():
    for kind in ("S2", "H2", "R2"):
        S = Space(kind)
        rng = np.random.default_rng(105)
        X = S.random_points(40, rng) if kind == "S2" else S.random_points(40, rng, R=2.0)
        V = S.project_tangent(X, rng.normal(size=X.shape)) * 0.3
        Y = S.retract(X, V)
        d = np.array([S.dist(X[i], Y[i]) for i in range(40)])
        nv = S.tangent_norm(V)
        err = np.max(np.abs(d - nv))
        check(f"{kind} exponential map moves each point by |v| and stays on the manifold",
              err < 1e-7 and S.on_manifold_error(Y) < 1e-9, f"dist err {err:.1e}, manifold err {S.on_manifold_er
ror(Y):.1e}")


def test_gradient():
    for kind, n, R in (("S2", 9, None), ("S2", 8, 2.0), ("H2", 8, 2.0), ("R2", 8, 2.0)):
        P = Problem(kind, n, R)
        rng = np.random.default_rng(106)
        X = P.initial(rng)
        p = 8.0
        E, G, _ = _soft_energy(P, X, p, True)
        S = P.space
        V = S.project_tangent(X, rng.normal(size=X.shape))
        V = V / np.sqrt(np.sum(S.tangent_norm(V) ** 2))
        h = 1e-6
        Ep, _, _ = _soft_energy(P, S.retract(X, h * V), p, False)
```

```
        Em, _, _ = _soft_energy(P, S.retract(X, -h * V), p, False)
        fd = (Ep - Em) / (2 * h)
        if kind == "H2":
            an = float(np.sum(-G[:, 0] * V[:, 0] + G[:, 1] * V[:, 1] + G[:, 2] * V[:, 2]))
        else:
            an = float(np.sum(G * V))
        check(f"{kind}{' cap' if R is not None and kind == 'S2' else ''} analytic Riemannian gradient equals finite difference", abs(fd - an) < 1e-5 * max(1, abs(an)),
              f"fd {fd:.8f}, analytic {an:.8f}")


def test_cap():
    S = Space("S2")
    pole, eq, south = np.array([[0.0, 0.0, 1.0]]), np.array([[1.0, 0.0, 0.0]]), np.array([[0.0, 0.0, -1.0]])
    ok = abs(S.rho(pole)[0]) < 1e-12 and abs(S.rho(eq)[0] - np.pi / 2) < 1e-12 and abs(S.rho(south)[0] - np.pi) < 1e-7
    check("S2 cap: distance from the pole is 0, pi/2 at the equator, pi at the antipode", ok)
    rr = np.linspace(0, 2.0, 200001)
    num = 2 * np.pi * np.trapezoid(np.sin(rr), rr)
    check("S2 cap: area 2 pi (1 - cos R) matches numeric integral", abs(spherical_cap_area(2.0) - num) < 1e-6, f"{spherical_cap_area(2.0):.6f} vs {num:.6f}")
    rng = np.random.default_rng(107)
    X = S.random_points(40000, rng, R=2.0)
    frac = float((S.rho(X) <= 1.0).mean())
    exp = (1 - np.cos(1.0)) / (1 - np.cos(2.0))
    se = np.sqrt(exp * (1 - exp) / 40000)
    check("S2 cap: random points are uniform by area (share within R/2 matches theory, 4 s.e.)", abs(frac - exp) < 4 * se and S.rho(X).max() <= 2.0 + 1e-9, f"{frac:.4f} vs {exp:.4f}")
    Rr = 0.05
    ang = np.arange(6) * np.pi / 3
    X7 = np.vstack([[0, 0, 1.0], S.from_polar(np.full(6, 2 * Rr / 3), ang)])
    P7 = Problem("S2", 7, Rr)
    check("S2 cap: Euclidean limit, 7 disks in a cap of radius 0.05 reach R/3 within 0.2%", abs(P7.score(X7) / (Rr / 3) - 1) < 2e-3, f"ratio {P7.score(X7) / (Rr / 3):.5f}")
    Y = S.clip_to_ball(S.random_points(10, rng) * 1.0, 1.0)
    check("S2 cap: clipping moves outside points exactly to the cap boundary", np.allclose(S.rho(Y), 1.0, atol=1e-9) or np.all(S.rho(Y) <= 1.0 + 1e-9))


def test_known_tammes():
    S = Space("S2")
    tet = np.array([[1, 1, 1], [1, -1, -1], [-1, 1, -1], [-1, -1, 1]], float) / np.sqrt(3)
    octa = np.vstack([np.eye(3), -np.eye(3)])
    ph = (1 + np.sqrt(5)) / 2
    ico = np.array([[0, s1, s2 * ph] for s1 in (1, -1) for s2 in (1, -1)]
```

```
                    + [[s1, s2 * ph, 0] for s1 in (1, -1) for s2 in (1, -1)]
                    + [[s2 * ph, 0, s1] for s1 in (1, -1) for s2 in (1, -1)], float)
    ico /= np.linalg.norm(ico, axis=1, keepdims=True)
    for name, X, ref in (("tetrahedron N=4", tet, np.arccos(-1 / 3)),
                         ("octahedron N=6", octa, np.pi / 2),
                         ("icosahedron N=12", ico, np.arctan(2.0))):
        d = Problem("S2", len(X)).score(X)
        check(f"Closed-form Tammes configuration, {name}: min distance {ref:.6f} rad", abs(d - ref) < 1e-12,
              f"computed {d:.12f}")


def test_recover_optima():
    """Pass criterion: the optimizer suite recovers each known optimum (best of the three
    optimizers over 5 test seeds).  Per-optimizer recovery is hypothesis H3 and is
    reported by the main experiment, not treated as a validation criterion."""
    refs = {4: np.arccos(-1 / 3), 6: np.pi / 2, 12: np.arctan(2.0)}
    for n, ref in refs.items():
        P = Problem("S2", n)
        per = {}
        for alg in ("gd", "sa", "ls"):
            gaps = []
            for seed in (9101, 9102, 9103, 9104, 9105):
                X0 = P.initial(np.random.default_rng(seed))
                r = run_algorithm(alg, P, X0, 1500 * P.sweep, [seed, 3])
                gaps.append(ref - r["score"])
            per[alg] = min(gaps)
        best = min(per.values())
        info = ", ".join(f"{k} {v:.1e}" for k, v in per.items())
        check(f"Optimizer suite recovers the N={n} Tammes optimum (best gap < 2e-3 rad over 5 test seeds)",
              best < 2e-3, f"best gap per optimizer: {info}")


def run_all():
    test_sphere_distance()
    test_hyperbolic_distance()
    test_euclidean()
    test_metric_axioms()
    test_retraction()
    test_gradient()
    test_cap()
    test_known_tammes()
    test_recover_optima()
    npass = sum(1 for _, ok, _ in RESULTS if ok)
    print(f"\n{npass} of {len(RESULTS)} tests passed")
    return npass == len(RESULTS)
```

```
208
209
210 if __name__ == "__main__":
211     sys.exit(0 if run_all() else 1)
```

experiments.py

Runs every condition with 30 paired seeds under an equal budget and writes the raw CSV files.

```
  1 """experiments.py -- runs every experiment and writes the raw CSV files.
  2
  3 Equal budget (pair-distance evaluations), shared seeds, fixed random streams.
  4 """
  5 import os
  6 import sys
  7 import time
  8 import pickle
  9 import numpy as np
 10 import pandas as pd
 11 from algorithms import Problem, run_algorithm, ALGS
 12
 13 ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))   # folder that contains code/ and out/
 14 OUT = os.path.join(ROOT, "out")
 15 DATA = OUT + "/data"
 16 CACHE = OUT + "/cache"
 17 SEEDS = list(range(1, 31))
 18 BUDGET_SWEEPS = 1500          # budget = 1500 x (n choose 2) pair-distance evaluations
 19 S2_N = [4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 24, 30, 40, 60]
 20 BALL_RN = [(2.0, 7), (2.0, 12), (2.0, 19), (3.0, 12), (3.0, 19), (3.0, 30)]
 21
 22 # Optimal minimum angular distances (degrees) for the Tammes problem, N = 4 to 14 and 24.
 23 # Optimality was proved by: L. Fejes Toth (1943) for N = 3, 4, 6, 12; K. Schutte and B. L. van der Waerden (1951
    ↳ ) for N = 5, 7, 8, 9;
 24 # R. M. Robinson (1961) for N = 24; L. Danzer (1963) for N = 10, 11; O. R. Musin and A. S. Tarasov (2012) for N
    ↳ = 13 and
 25 # (2015) for N = 14.  N = 4, 6, 12 are also derived in closed form and checked by the unit tests.  The numerical
    ↳  values
 26 # should be checked against the original papers or Sloane's tables of spherical codes before publication.
 27 BENCH_DEG = {4: float(np.degrees(np.arccos(-1.0 / 3.0))), 5: 90.0, 6: 90.0, 7: 77.8695, 8: 74.8585,
 28              9: 70.5288, 10: 66.1468, 11: 63.4349, 12: float(np.degrees(np.arctan(2.0))),
 29              13: 57.1367, 14: 55.6706, 24: 43.6908}
 30 BENCH_SRC = {4: "Fejes Toth (1943); closed form", 5: "Schutte & van der Waerden (1951)", 6: "Fejes Toth (1943);
```

```
    closed form",
31              7: "Schutte & van der Waerden (1951)", 8: "Schutte & van der Waerden (1951)", 9: "Schutte & van der
    Waerden (1951)",
32              10: "Danzer (1963)", 11: "Danzer (1963)", 12: "Fejes Toth (1943); closed form", 13: "Musin & Taraso
    v (2012)",
33              14: "Musin & Tarasov (2015)", 24: "Robinson (1961)"}
34
35  # Best-known (not proven) minimum angles from the published tables of spherical codes, used in place of the best
    value found
36  # by the runs.  N = 30: cos(theta) = 0.781551875094987327103586104096 50 (Clare & Kepert, 1986, as tabulated by H
    . Cohn;
37  # 38.597 degrees, the value in Sloane's table).  N = 40 and 60: values supplied by the author from Sloane's tabl
    es.
38  BEST_KNOWN_DEG = {30: 38.5971159, 40: 33.1583563, 60: 27.1928300}
39  BEST_KNOWN_SRC = {k: "Sloane's tables of spherical codes (as supplied by the author; N = 30 also matches Cohn's
    compilation)" for k in (30, 40, 60)}
40
41  COND_ID = {}
42  for i, N in enumerate(S2_N):
43      COND_ID[("S2", N, None)] = 100 + i
44  for sp, base in (("H2", 200), ("R2", 300), ("S2cap", 400)):
45      for i, (R, n) in enumerate(BALL_RN):
46          COND_ID[(sp, n, R)] = base + i
47
48
49  def conditions():
50      for N in S2_N:
51          yield ("S2", N, None)
52      for sp in ("H2", "R2", "S2cap"):
53          for R, n in BALL_RN:
54              yield (sp, n, R)
55
56
57  def run_condition(cond):
58      kind, n, R = cond
59      cid = COND_ID[cond]
60      P = Problem("S2" if kind == "S2cap" else kind, n, R)
61      budget = BUDGET_SWEEPS * P.sweep
62      rows, configs, traces = [], [], []
63      fr = np.logspace(-3, 0, 41)
64      for seed in SEEDS:
65          X0 = P.initial(np.random.default_rng([seed, cid, 0]))
66          for aid, alg in ALGS.items():
67              res = run_algorithm(alg, P, X0, budget, [seed, cid, aid])
68              tr = np.array(res["trace"])
```

```
            final = res["score"]
            idx = np.where(tr[:, 1] >= 0.99 * final)[0]
            sweeps99 = float(tr[idx[0], 0] / P.sweep) if len(idx) else float("nan")
            r_val = final if P.ball else float("nan")
            rows.append(dict(space=kind, condition_id=cid, n=n, R=(R if R is not None else float("nan")),
                             algorithm=alg, seed=seed, score=final,
                             density=(P.density(final) if P.ball else float("nan")),
                             pair_evals=res["pe"], sweeps_used=res["pe"] / P.sweep, iterations=res["iters"],
                             time_s=res["time"], sweeps_to_99pct=sweeps99,
                             recomputed_score=res["final_check"]))
            X = res["X"]
            for k in range(n):
                configs.append(dict(space=kind, condition_id=cid, n=n, R=(R if R is not None else float("nan")),
                                    algorithm=alg, seed=seed, index=k,
                                    x0=X[k, 0], x1=X[k, 1], x2=(X[k, 2] if X.shape[1] > 2 else float("nan"))))
            pes = tr[:, 0]
            for f in fr:
                j = np.searchsorted(pes, f * budget, side="right") - 1
                j = max(j, 0)
                traces.append(dict(space=kind, condition_id=cid, n=n, R=(R if R is not None else float("nan")),
                                   algorithm=alg, seed=seed, budget_fraction=f, best_score=tr[j, 1]))
    return rows, configs, traces


def main(rerun=False):
    os.makedirs(CACHE, exist_ok=True)
    os.makedirs(DATA, exist_ok=True)
    allrows, allcfg, alltr = [], [], []
    t_all = time.time()
    for cond in conditions():
        f = f"{CACHE}/{cond[0]}_{cond[1]}_{cond[2]}.pkl"
        if os.path.exists(f) and not rerun:
            rows, cfg, tr = pickle.load(open(f, "rb"))
        else:
            t0 = time.time()
            rows, cfg, tr = run_condition(cond)
            pickle.dump((rows, cfg, tr), open(f, "wb"))
            print(f"done {cond} in {time.time() - t0:.1f}s (total {time.time() - t_all:.0f}s)", flush=True)
        allrows += rows
        allcfg += cfg
        alltr += tr
    runs = pd.DataFrame(allrows)
    # reference values: proven optimum where one exists, published best-known value where tabulated, otherwise b
  ↳ est found over all runs
    ref_type, ref_val = [], []
```

```
    best_found = runs.groupby("condition_id")["score"].max()
    for _, r in runs.iterrows():
        if r["space"] == "S2" and int(r["n"]) in BENCH_DEG:
            ref_type.append("verified optimum")
            ref_val.append(float(np.radians(BENCH_DEG[int(r["n"])])))
        elif r["space"] == "S2" and int(r["n"]) in BEST_KNOWN_DEG:
            ref_type.append("published best known")
            ref_val.append(float(np.radians(BEST_KNOWN_DEG[int(r["n"])])))
        else:
            ref_type.append("best found")
            ref_val.append(float(best_found[r["condition_id"]]))
    runs["reference_type"] = ref_type
    runs["reference_value"] = ref_val
    runs["relative_score"] = runs["score"] / runs["reference_value"]
    runs["gap_to_reference"] = runs["reference_value"] - runs["score"]
    runs["success_0.1pct"] = (runs["relative_score"] >= 0.999).astype(int)
    runs["success_1pct"] = (runs["relative_score"] >= 0.99).astype(int)
    runs.to_csv(f"{DATA}/runs_raw.csv", index=False)
    pd.DataFrame(allcfg).to_csv(f"{DATA}/configurations.csv", index=False, float_format="%.15g")
    pd.DataFrame(alltr).to_csv(f"{DATA}/convergence_traces.csv", index=False, float_format="%.12g")
    pd.DataFrame([{"n": k, "theta_deg": v, "theta_rad": float(np.radians(v)), "status": "proven optimum", "sourc
↳ e": BENCH_SRC[k]}
                  for k, v in BENCH_DEG.items()] +
                 [{"n": k, "theta_deg": v, "theta_rad": float(np.radians(v)), "status": "published best known (n
↳ ot proven)", "source": BEST_KNOWN_SRC[k]}
                  for k, v in BEST_KNOWN_DEG.items()]).to_csv(f"{DATA}/benchmark_values.csv", index=False, float
↳ _format="%.10g")
    pd.DataFrame({"seed": SEEDS}).to_csv(f"{DATA}/seed_list.csv", index=False)
    print("experiments finished in %.0f s" % (time.time() - t_all))


if __name__ == "__main__":
    main(rerun="--rerun" in sys.argv)
```

analyze.py

Summary statistics, Friedman and Wilcoxon tests with Holm correction, effect sizes, and the hypothesis tests.

```
"""analyze.py -- summary statistics, paired nonparametric tests, hypothesis tests.

Reads data/runs_raw.csv, writes summary and test CSV files into data/.
Significance: Holm-adjusted p < .05 (family = all pairwise tests within one space).
```

```
"""
import os
import numpy as np
import pandas as pd
from scipy import stats

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))   # folder that contains code/ and out/
OUT = os.path.join(ROOT, "out")
DATA = OUT + "/data"
ALG = ["gd", "sa", "ls"]
PAIRS = [("gd", "sa"), ("gd", "ls"), ("sa", "ls")]
ALPHA = 0.05
PRACTICAL = 0.005   # practical-importance threshold: 0.5% of the reference value


def ci95(x):
    x = np.asarray(x, float)
    n = len(x)
    m = x.mean()
    if n < 2 or x.std(ddof=1) == 0:
        return m, m
    h = stats.t.ppf(0.975, n - 1) * x.std(ddof=1) / np.sqrt(n)
    return m - h, m + h


def holm(pvals):
    p = np.asarray(pvals, float)
    order = np.argsort(p)
    m = len(p)
    adj = np.empty(m)
    run = 0.0
    for rank, idx in enumerate(order):
        run = max(run, (m - rank) * p[idx])
        adj[idx] = min(run, 1.0)
    return adj


def wilcoxon_paired(a, b):
    d = np.asarray(a, float) - np.asarray(b, float)
    nz = d[d != 0]
    if len(nz) == 0:
        return dict(W=0.0, p=1.0, rbc=0.0, dz=float("nan"), mean_diff=0.0, ci_lo=0.0, ci_hi=0.0, n_nonzero=0)
    ranks = stats.rankdata(np.abs(nz))
    wpos = ranks[nz > 0].sum()
    wneg = ranks[nz < 0].sum()
```

```
    try:
        res = stats.wilcoxon(d, zero_method="wilcox", alternative="two-sided")
        p = float(res.pvalue)
    except ValueError:
        p = 1.0
    sd = d.std(ddof=1)
    lo, hi = ci95(d)
    return dict(W=float(min(wpos, wneg)), p=p, rbc=float((wpos - wneg) / (wpos + wneg)),
                dz=float(d.mean() / sd) if sd > 0 else float("nan"), mean_diff=float(d.mean()),
                ci_lo=float(lo), ci_hi=float(hi), n_nonzero=int(len(nz)))


def main():
    runs = pd.read_csv(f"{DATA}/runs_raw.csv")
    # ------------------------------------------------------------------ summary
    rows = []
    for (sp, cid, n, R, alg), g in runs.groupby(["space", "condition_id", "n", "R", "algorithm"], dropna=False):
        lo, hi = ci95(g["score"])
        rlo, rhi = ci95(g["relative_score"])
        rows.append(dict(space=sp, condition_id=cid, n=n, R=R, algorithm=alg, n_seeds=len(g),
                         mean_score=g["score"].mean(), sd_score=g["score"].std(ddof=1), ci95_lo=lo, ci95_hi=hi,
                         median_score=g["score"].median(), min_score=g["score"].min(), max_score=g["score"].max(
),
                         mean_relative=g["relative_score"].mean(), sd_relative=g["relative_score"].std(ddof=1),
                         rel_ci95_lo=rlo, rel_ci95_hi=rhi,
                         reference_type=g["reference_type"].iloc[0], reference_value=g["reference_value"].iloc[0
], 
                         mean_gap=g["gap_to_reference"].mean(),
                         success_rate_0p1pct=g["success_0.1pct"].mean(), success_rate_1pct=g["success_1pct"].mea
n(),
                         mean_density=g["density"].mean(), sd_density=g["density"].std(ddof=1),
                         mean_time_s=g["time_s"].mean(), sd_time_s=g["time_s"].std(ddof=1),
                         mean_sweeps_to_99=g["sweeps_to_99pct"].mean(), mean_pair_evals=g["pair_evals"].mean()))
    summ = pd.DataFrame(rows).sort_values(["space", "condition_id", "algorithm"])
    summ.to_csv(f"{DATA}/summary_by_condition.csv", index=False, float_format="%.10g")

    # ------------------------------------------------------------ Friedman + pairwise
    fr_rows, pw_rows, rank_rows = [], [], []
    for (sp, cid, n, R), g in runs.groupby(["space", "condition_id", "n", "R"], dropna=False):
        piv = g.pivot(index="seed", columns="algorithm", values="score")[ALG]
        arr = piv.values
        if np.all(arr == arr[:, [0]]):
            chi, p = 0.0, 1.0
        else:
            chi, p = stats.friedmanchisquare(arr[:, 0], arr[:, 1], arr[:, 2])
```

```
        W = chi / (len(arr) * (3 - 1))
        fr_rows.append(dict(space=sp, condition_id=cid, n=n, R=R, n_seeds=len(arr), chi2=float(chi), df=2,
                            p=float(p), kendall_W=float(W)))
        ranks = np.apply_along_axis(lambda r: stats.rankdata(-r), 1, arr)   # rank 1 = highest score
        for k, a in enumerate(ALG):
            rank_rows.append(dict(space=sp, condition_id=cid, n=n, R=R, algorithm=a, mean_rank=float(ranks[:, k].mean())))
        refv = float(g["reference_value"].iloc[0])
        for a, b in PAIRS:
            r = wilcoxon_paired(piv[a].values, piv[b].values)
            dd = piv[a].values - piv[b].values
            r.update(space=sp, condition_id=cid, n=n, R=R, alg_a=a, alg_b=b, mean_rel_diff=float(dd.mean() / refv),
                     share_zero_diff=float(np.mean(dd == 0)), skewness=float(stats.skew(dd)) if dd.std() > 0 else 0.0)
            pw_rows.append(r)
    fr = pd.DataFrame(fr_rows)
    pw = pd.DataFrame(pw_rows)
    pw["p_holm"] = np.nan
    for sp in pw["space"].unique():
        m = pw["space"] == sp
        pw.loc[m, "p_holm"] = holm(pw.loc[m, "p"].values)
    fr["p_holm"] = np.nan
    for sp in fr["space"].unique():
        m = fr["space"] == sp
        fr.loc[m, "p_holm"] = holm(fr.loc[m, "p"].values)
    pw["significant"] = (pw["p_holm"] < ALPHA).astype(int)
    pw["practical"] = (pw["mean_rel_diff"].abs() >= PRACTICAL).astype(int)
    pw["significant_and_practical"] = pw["significant"] * pw["practical"]
    pw["winner"] = np.where(pw["significant"] == 1, np.where(pw["mean_diff"] > 0, pw["alg_a"], pw["alg_b"]), "none")
    fr["significant"] = (fr["p_holm"] < ALPHA).astype(int)
    fr.to_csv(f"{DATA}/test_friedman.csv", index=False, float_format="%.10g")
    pw.to_csv(f"{DATA}/test_pairwise_wilcoxon.csv", index=False, float_format="%.10g")
    ranks_df = pd.DataFrame(rank_rows)
    ranks_df.to_csv(f"{DATA}/mean_ranks.csv", index=False, float_format="%.10g")

    # ------------------------------------------------------------------ hypotheses
    hyp = []
    # H1: does curvature change the algorithm ordering?  per-condition mean paired difference
    # in relative score, S2 conditions versus H2 conditions (Mann-Whitney U), Holm over 3 pairs.
    cm = runs.groupby(["space", "condition_id", "algorithm"])["relative_score"].mean().unstack("algorithm")
    h1 = []
    for a, b in PAIRS:
        d = cm[a] - cm[b]
```

```
        s2 = d[d.index.get_level_values("space") == "S2"].values
        h2 = d[d.index.get_level_values("space") == "H2"].values
        u = stats.mannwhitneyu(s2, h2, alternative="two-sided")
        h1.append(dict(hypothesis="H1", test=f"{a}-{b}", n_S2=len(s2), n_H2=len(h2), median_diff_S2=float(np.median(s2)),
                       median_diff_H2=float(np.median(h2)), U=float(u.statistic), p=float(u.pvalue)))
    ph = holm([x["p"] for x in h1])
    for x, pv in zip(h1, ph):
        x["p_holm"] = float(pv)
        x["significant"] = int(pv < ALPHA)
    hyp += h1
    # overall best algorithm per space (mean rank and mean relative score)
    best_rows = []
    for sp in ("S2", "H2", "R2", "S2cap"):
        mr = ranks_df[ranks_df["space"] == sp].groupby("algorithm")["mean_rank"].mean()
        mrel = runs[runs["space"] == sp].groupby("algorithm")["relative_score"].mean()
        wins = pw[(pw["space"] == sp)]
        for a in ALG:
            best_rows.append(dict(space=sp, algorithm=a, mean_rank=float(mr[a]), mean_relative_score=float(mrel[a])))
    pd.DataFrame(best_rows).to_csv(f"{DATA}/overall_by_space.csv", index=False, float_format="%.10g")
    # winners per condition by mean score
    wrows = []
    for (sp, cid, n, R), g in summ.groupby(["space", "condition_id", "n", "R"], dropna=False):
        best = g.sort_values("mean_score", ascending=False).iloc[0]["algorithm"]
        wrows.append(dict(space=sp, condition_id=cid, n=n, R=R, best_mean_algorithm=best))
    pd.DataFrame(wrows).to_csv(f"{DATA}/winner_by_condition.csv", index=False)

    # H1c: common formulation (disks in a geodesic ball) in three curvatures; Kruskal-Wallis on per-setting
    # mean paired differences in relative score, Holm over the 3 pairs; plus best mean-rank algorithm per space.
    if "S2cap" in set(runs.space):
        h1c = []
        for a, b in PAIRS:
            d = cm[a] - cm[b]
            grp = {sp: d[d.index.get_level_values("space") == sp].values for sp in ("S2cap", "R2", "H2")}
            kw = stats.kruskal(grp["S2cap"], grp["R2"], grp["H2"])
            h1c.append(dict(hypothesis="H1c", test=f"{a}-{b} across K=+1,0,-1", median_diff_S2cap=float(np.median(grp["S2cap"])),
                            median_diff_R2=float(np.median(grp["R2"])), median_diff_H2=float(np.median(grp["H2"])),
                            chi2=float(kw.statistic), p=float(kw.pvalue)))
        for x, pv in zip(h1c, holm([x["p"] for x in h1c])):
            x["p_holm"] = float(pv)
            x["significant"] = int(pv < ALPHA)
        hyp += h1c
```

```
        # paired setting-level contrasts for GD - SA relative to the Euclidean control (exploratory)
        d = cm["gd"] - cm["sa"]
        e = {sp: d[d.index.get_level_values("space") == sp].values for sp in ("S2cap", "R2", "H2")}
        cons = []
        for sp in ("S2cap", "H2"):
            diff = e[sp] - e["R2"]
            try:
                w = stats.wilcoxon(diff)
                pval = float(w.pvalue)
            except ValueError:
                pval = 1.0
            cons.append(dict(hypothesis="H1c", test=f"GD-SA difference, {sp} minus R2, paired over 6 settings", mean_diff=float(diff.mean()), p=pval))
        for x, pv in zip(cons, holm([x["p"] for x in cons])):
            x["p_holm"] = float(pv)
            x["significant"] = int(pv < ALPHA)
        hyp += cons
        bm = {}
        for sp in ("S2cap", "R2", "H2"):
            mr = ranks_df[ranks_df["space"] == sp].groupby("algorithm")["mean_rank"].mean()
            bm[sp] = str(mr.idxmin())
        hyp.append(dict(hypothesis="H1c", test="best mean-rank algorithm per space (S2cap, R2, H2): " + ", ".join(f"{k}={v}" for k, v in bm.items()),
                        supported=int(len(set(bm.values())) > 1)))
    # H2: scaling with N on S2 (N >= 13): per-seed slope of relative score on log10 N
    big = runs[(runs["space"] == "S2") & (runs["n"] >= 13)]
    slopes = {}
    for a in ALG:
        sl = []
        for seed in sorted(big["seed"].unique()):
            g = big[(big["algorithm"] == a) & (big["seed"] == seed)].sort_values("n")
            sl.append(np.polyfit(np.log10(g["n"].values), g["relative_score"].values, 1)[0])
        slopes[a] = np.array(sl)
    pd.DataFrame(slopes).assign(seed=sorted(big["seed"].unique())).to_csv(f"{DATA}/scaling_slopes_per_seed.csv", index=False, float_format="%.10g")
    h2rows = []
    for a, b in (("ls", "gd"), ("ls", "sa")):
        r = wilcoxon_paired(slopes[a], slopes[b])
        h2rows.append(dict(hypothesis="H2", test=f"slope {a} - slope {b}", mean_slope_a=float(slopes[a].mean()),
                           mean_slope_b=float(slopes[b].mean()), mean_diff=r["mean_diff"], p=r["p"], rbc=r["rbc"], dz=r["dz"]))
    ph = holm([x["p"] for x in h2rows])
    for x, pv in zip(h2rows, ph):
        x["p_holm"] = float(pv)
        x["significant"] = int(pv < ALPHA)
```

```
    hyp += h2rows
    ch, cp = stats.friedmanchisquare(slopes["gd"], slopes["sa"], slopes["ls"])
    hyp.append(dict(hypothesis="H2", test="Friedman on slopes", chi2=float(ch), p=float(cp)))
    # runtime scaling exponents
    srows = []
    for a in ALG:
        g = summ[(summ["space"] == "S2") & (summ["algorithm"] == a) & (summ["n"] >= 13)]
        k, c = np.polyfit(np.log10(g["n"].values), np.log10(g["mean_time_s"].values), 1)
        srows.append(dict(algorithm=a, time_exponent=float(k), time_intercept=float(c)))
    pd.DataFrame(srows).to_csv(f"{DATA}/runtime_scaling_exponents.csv", index=False, float_format="%.10g")

    # H3: success on verified optima, N = 4..14
    h3 = []
    small = runs[(runs["space"] == "S2") & (runs["n"] <= 14)]
    for a in ALG:
        sr = small[small["algorithm"] == a].groupby("n")["success_0.1pct"].mean()
        h3.append(dict(hypothesis="H3", test=f"{a}: success rate >= 0.90 at every N in 4..14",
                       min_success_rate=float(sr.min()), n_N_below_0p9=int((sr < 0.9).sum()),
                       supported=int((sr >= 0.9).all())))
    hyp += h3
    pd.DataFrame(hyp).to_csv(f"{DATA}/hypothesis_tests.csv", index=False, float_format="%.10g")

    asm = pw[["space", "n", "R", "alg_a", "alg_b", "n_nonzero", "share_zero_diff", "skewness"]].copy()
    asm["wilcoxon_symmetry_flag"] = (asm["skewness"].abs() > 1.0).astype(int)
    asm.to_csv(f"{DATA}/assumption_checks.csv", index=False, float_format="%.10g")
    # ------------------------------------------------------------ derived ball quantities
    from geometry import hyperbolic_ball_area
    arows = []
    for R in (2.0, 3.0, 4.0, 6.0):
        s_total = 2 * np.pi * (1 - np.cos(R)) if R <= np.pi else float("nan")
        s_in = 2 * np.pi * (1 - np.cos(R - 1)) if R <= np.pi else float("nan")
        h_total = 2 * np.pi * (np.cosh(R) - 1)
        h_in = 2 * np.pi * (np.cosh(R - 1) - 1)
        e_total = np.pi * R * R
        e_in = np.pi * (R - 1) ** 2
        arows.append(dict(R=R, spherical_area=s_total, spherical_fraction_within_1_of_boundary=1 - s_in / s_tota
↳ l,
                          hyperbolic_area=h_total, hyperbolic_fraction_within_1_of_boundary=1 - h_in / h_total,
                          euclidean_area=e_total, euclidean_fraction_within_1_of_boundary=1 - e_in / e_total))
    pd.DataFrame(arows).to_csv(f"{DATA}/area_fractions.csv", index=False, float_format="%.10g")
    caps = []
    ball = runs[runs["space"] != "S2"]
    for sp in ("S2cap", "H2", "R2"):
        for R in (2.0, 3.0):
            g = ball[(ball["space"] == sp) & (ball["R"] == R)]
```

```
            bestn = g.groupby("n")["score"].max()
            for r0 in (0.4, 0.5):
                ok = bestn[bestn >= r0]
                caps.append(dict(space=sp, R=R, target_radius=r0, largest_tested_n_that_fits=int(ok.index.max())
  ↳ if len(ok) else 0,
                                 best_radius_at_that_n=float(ok.iloc[-1]) if len(ok) else float("nan")))
    pd.DataFrame(caps).to_csv(f"{DATA}/capacity_fixed_radius.csv", index=False, float_format="%.10g")
    print("analysis finished")


if __name__ == "__main__":
    main()
```

make_figures.py

All figures, drawn only from the CSV files, with the mathematical accuracy assertions.

```
"""make_figures.py -- all figures, drawn only from the CSV files, with accuracy assertions.

Colors: gradient descent = blue, simulated annealing = red, compression = green.
Fonts: Times New Roman (Liberation Serif if Times New Roman is not installed).
"""
import os
import subprocess
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import Circle, Patch
from matplotlib.lines import Line2D
from geometry import Space, euclid_image_disk

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))   # folder that contains code/ and out/
OUT = os.path.join(ROOT, "out")
DATA = OUT + "/data"
FIG = OUT + "/figures"
COL = {"gd": "#1F4E9E", "sa": "#C62828", "ls": "#2E7D32"}
MARK = {"gd": "o", "sa": "s", "ls": "^"}
LSTY = {"gd": "-", "sa": "--", "ls": "-."}
NAME = {"gd": "Gradient descent", "sa": "Simulated annealing", "ls": "Compression"}
ALG = ["gd", "sa", "ls"]
```

```
TOL = 1e-9
ASSERTIONS = []

fams = {f.name for f in matplotlib.font_manager.fontManager.ttflist}
SERIF = "Times New Roman" if "Times New Roman" in fams else "Liberation Serif"
plt.rcParams.update({"font.family": "serif", "font.serif": [SERIF], "mathtext.fontset": "stix",
                     "font.size": 12, "axes.labelsize": 12, "xtick.labelsize": 12, "ytick.labelsize": 12,
                     "legend.fontsize": 12, "axes.titlesize": 12, "figure.dpi": 100, "savefig.dpi": 300,
                     "axes.linewidth": 0.8, "grid.color": "#CCCCCC", "grid.linewidth": 0.5})


def legend_box(ax_or_fig, handles, **kw):
    lg = ax_or_fig.legend(handles=handles, frameon=True, fancybox=False, edgecolor="black", facecolor="white",
                          framealpha=1.0, prop={"family": SERIF, "size": 12}, **kw)
    lg.get_frame().set_linewidth(1.0)
    return lg


def alg_handles():
    return [Line2D([0], [0], color=COL[a], marker=MARK[a], linestyle=LSTY[a], lw=1.6, ms=6, label=NAME[a]) for a
 in ALG]


def save(fig, name):
    os.makedirs(FIG, exist_ok=True)
    path = f"{FIG}/{name}.png"
    fig.savefig(path, dpi=300, bbox_inches=None)
    plt.close(fig)
    return path


def check(name, ok, detail=""):
    ASSERTIONS.append((name, bool(ok), detail))
    if not ok:
        raise AssertionError(name + " " + detail)


def load():
    runs = pd.read_csv(f"{DATA}/runs_raw.csv")
    cfg = pd.read_csv(f"{DATA}/configurations.csv")
    tr = pd.read_csv(f"{DATA}/convergence_traces.csv")
    summ = pd.read_csv(f"{DATA}/summary_by_condition.csv")
    return runs, cfg, tr, summ


```

```
def get_config(cfg, runs, cid, alg, seed):
    g = cfg[(cfg.condition_id == cid) & (cfg.algorithm == alg) & (cfg.seed == seed)].sort_values("index")
    return g[["x0", "x1", "x2"]].values


def best_run(runs, cid, alg=None):
    g = runs[runs.condition_id == cid]
    if alg:
        g = g[g.algorithm == alg]
    return g.sort_values("score", ascending=False).iloc[0]


# ------------------------------------------------------------------ Figure 1
def fig_framework():
    from matplotlib.patches import FancyBboxPatch, Rectangle, Polygon
    W, H = 6.5, 7.6
    fig = plt.figure(figsize=(W, H))
    ax = fig.add_axes([0, 0, 1, 1])
    ax.set_xlim(0, W); ax.set_ylim(0, H); ax.axis("off")
    xa, xb, bw = 1.95, 5.05, 2.7
    ys = [H - 0.62 - 1.07 * k for k in range(7)]
    def box(xc, yc, text, kind="proc", h=0.7):
        if kind == "io":
            ax.add_patch(FancyBboxPatch((xc - bw / 2, yc - h / 2), bw, h, boxstyle="round,pad=0,rounding_size=0.
↳ 14", fc="white", ec="black", lw=1.0))
        else:
            ax.add_patch(Rectangle((xc - bw / 2, yc - h / 2), bw, h, fc="white", ec="black", lw=1.0))
        ax.text(xc, yc, text, ha="center", va="center", fontsize=12, color="black", family=SERIF, linespacing=1.
↳ 15)
        return h
    def arrow(p, q):
        ax.annotate("", xy=q, xytext=p, arrowprops=dict(arrowstyle="-|>", color="black", lw=1.0, shrinkA=0, shri
↳ nkB=0, mutation_scale=11))
    def line(p, q):
        ax.plot([p[0], q[0]], [p[1], q[1]], color="black", lw=1.0, solid_capstyle="butt")
    hs = [0.7, 0.95, 0.7, 0.95, 0.5, 0.5]
    texts = ["Research Problem\n(dense packing under curvature)", "Geometric Spaces\nSphere S\u00b2, Hyperbolic
↳ Plane H\u00b2,\nEuclidean Control R\u00b2",
             "Distance Function and\nEnergy or Density Definition", "Three Optimization Algorithms\n(gradient de
↳ scent, simulated\nannealing, compression)",
             "Implementation", "Unit Tests and Known Optima"]
    kinds = ["io", "proc", "proc", "proc", "proc", "proc"]
    for k in range(6):
        box(xa, ys[k], texts[k], kinds[k], hs[k])
    for k in range(5):
```

```
        arrow((xa, ys[k] - hs[k] / 2), (xa, ys[k + 1] + hs[k + 1] / 2))
    # decision diamond
    dy, dw, dh = ys[6], 1.5, 0.8
    ax.add_patch(Polygon([(xa, dy + dh / 2), (xa + dw / 2, dy), (xa, dy - dh / 2), (xa - dw / 2, dy)], closed=Tr
↳ ue, fc="white", ec="black", lw=1.0))
    ax.text(xa, dy, "Pass?", ha="center", va="center", fontsize=12, family=SERIF)
    arrow((xa, ys[5] - hs[5] / 2), (xa, dy + dh / 2))
    # "No": left, up, into Implementation
    xl = xa - bw / 2 - 0.55
    line((xa - dw / 2, dy), (xl, dy)); line((xl, dy), (xl, ys[4])); arrow((xl, ys[4]), (xa - bw / 2, ys[4]))
    ax.text(xl + 0.08, dy + 0.12, "No", fontsize=12, family=SERIF, ha="left", va="bottom")
    # "Yes": right into Trials, then upward
    bh = [0.7, 0.85, 0.85, 0.7]
    rtexts = ["Repeated Trials With\nFixed Seeds", "Performance Measures\n(distance or density, convergence,\nru
↳ ntime, success rate)",
              "Statistical Analysis\n(confidence intervals, paired\nnonparametric tests)", "Findings and\nHypoth
↳ esis Evaluation"]
    rk = ["proc", "io", "proc", "io"]
    ry = [ys[6], ys[5], ys[4], ys[3]]
    for t, k, y, h in zip(rtexts, rk, ry, bh):
        box(xb, y, t, k, h)
    arrow((xa + dw / 2, dy), (xb - bw / 2, dy))
    ax.text((xa + dw / 2 + xb - bw / 2) / 2, dy + 0.12, "Yes", fontsize=12, family=SERIF, ha="center", va="botto
↳ m")
    for i in range(3):
        arrow((xb, ry[i] + bh[i] / 2), (xb, ry[i + 1] - bh[i + 1] / 2))
    os.makedirs(FIG, exist_ok=True)
    fig.savefig(f"{FIG}/framework.png", dpi=300)
    plt.close(fig)
    from PIL import Image
    im = Image.open(f"{FIG}/framework.png").convert("RGB")
    arr = np.array(im)
    check("Framework diagram contains only gray levels (no color)", np.all(arr[..., 0] == arr[..., 1]) and np.al
↳ l(arr[..., 1] == arr[..., 2]))
    return f"{FIG}/framework.png", im.size


# ---------------------------------------------------------------------- Figure 2
def ortho_view(az, el):
    ca, sa_ = np.cos(az), np.sin(az)
    ce, se = np.cos(el), np.sin(el)
    Rz = np.array([[ca, -sa_, 0], [sa_, ca, 0], [0, 0, 1]])
    Rx = np.array([[1, 0, 0], [0, ce, -se], [0, se, ce]])
    return Rx @ Rz

```

```
150
151 def cap_circle(p, ang, m=200):
152     p = p / np.linalg.norm(p)
153     a = np.cross(p, [0.3, 0.5, 0.8])
154     a /= np.linalg.norm(a)
155     b = np.cross(p, a)
156     t = np.linspace(0, 2 * np.pi, m)
157     return np.cos(ang) * p[None, :] + np.sin(ang) * (np.cos(t)[:, None] * a[None, :] + np.sin(t)[:, None] * b[No
    ↳ ne, :])
158
159
160 def draw_sphere_panel(ax, X, dmin, color, title):
161     check("Sphere points have unit norm (1e-9)", np.max(np.abs(np.linalg.norm(X, axis=1) - 1)) < TOL)
162     S = Space("S2")
163     D = S.pair_dist(X)
164     check("Plotted configuration reproduces min distance", abs(D.min() - dmin) < TOL, f"{D.min()} vs {dmin}")
165     V = ortho_view(np.radians(25), np.radians(-70))
166     Y = X @ V.T
167     th = np.linspace(0, 2 * np.pi, 400)
168     ax.plot(np.cos(th), np.sin(th), color="black", lw=0.8)
169     half = dmin / 2.0
170     for p in X:
171         c = cap_circle(p, half) @ V.T
172         vis = c[:, 2] > 0
173         seg = np.where(vis)[0]
174         if len(seg):
175             cc = np.where(vis[:, None], c[:, :2], np.nan)
176             ax.plot(cc[:, 0], cc[:, 1], color=color, lw=0.5, alpha=0.7)
177     iu = np.triu_indices(len(X), 1)
178     for i, j in zip(*iu):
179         if D[i, j] < dmin + 1e-4:
180             ts = np.linspace(0, 1, 40)
181             om = D[i, j]
182             arc = (np.sin((1 - ts) * om)[:, None] * X[i] + np.sin(ts * om)[:, None] * X[j]) / np.sin(om)
183             arc = arc @ V.T
184             if np.all(arc[:, 2] > -0.02):
185                 ax.plot(arc[:, 0], arc[:, 1], color="black", lw=0.8)
186     front = Y[:, 2] > 0
187     ax.scatter(Y[front, 0], Y[front, 1], s=14, color=color, zorder=5)
188     ax.scatter(Y[~front, 0], Y[~front, 1], s=14, facecolors="white", edgecolors=color, linewidths=0.7, zorder=4)
189     ax.set_aspect("equal")
190     ax.set_xlim(-1.05, 1.05)
191     ax.set_ylim(-1.05, 1.05)
192     ax.axis("off")
193     ax.set_title(title, fontsize=12)
```

```
194
195
196 def fig_sphere(runs, cfg):
197     sel = [12, 13, 14, 24, 30, 60]
198     fig, axs = plt.subplots(2, 3, figsize=(6.5, 4.6))
199     for ax, N in zip(axs.ravel(), sel):
200         cid = 100 + [4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 24, 30, 40, 60].index(N)
201         b = best_run(runs, cid)
202         X = get_config(cfg, runs, cid, b.algorithm, int(b.seed))
203         draw_sphere_panel(ax, X, float(b.score), COL[b.algorithm], f"$N$ = {N}")
204     fig.subplots_adjust(left=0.01, right=0.99, top=0.93, bottom=0.01, wspace=0.05, hspace=0.12)
205     return save(fig, "fig_sphere_configs")
206
207
208 # ------------------------------------------------------------------ Figure 3
209 CAP_IDS = {}
210 BALL_SET = [(2.0, 7), (2.0, 12), (2.0, 19), (3.0, 12), (3.0, 19), (3.0, 30)]
211
212
213 def draw_hyperbolic_panel(ax, X, r, R, color, title):
214     S = Space("H2")
215     check("Hyperboloid points satisfy the model equation", S.on_manifold_error(X) < 1e-9)
216     D = S.pair_dist(X)
217     rho = S.rho(X)
218     r_re = min(D.min() / 2.0, float((R - rho).min()))
219     check("H2: recomputed disk radius matches CSV value (1e-9)", abs(r_re - r) < TOL, f"{r_re} vs {r}")
220     check("H2: disks do not overlap (tolerance 1e-9)", D.min() / 2.0 - r > -TOL)
221     check("H2: disks lie inside the container (tolerance 1e-9)", float((R - rho).min()) - r > -TOL)
222     Z = S.to_plane(X)
223     ax.add_patch(Circle((0, 0), np.tanh(R / 2.0), fill=False, ec="black", lw=1.0))
224     ax.add_patch(Circle((0, 0), 1.0, fill=False, ec="#888888", lw=0.6, ls=":"))
225     for k in range(len(X)):
226         zn = np.hypot(*Z[k])
227         cen, rad = euclid_image_disk(rho[k], r)
228         u = Z[k] / zn if zn > 1e-12 else np.array([1.0, 0.0])
229         c_xy = cen * u
230         ang = np.linspace(0, 2 * np.pi, 13)
231         pts = c_xy[None, :] + rad * np.c_[np.cos(ang), np.sin(ang)]
232         P3 = S.from_disk(pts)
233         dd = np.array([S.dist(X[k], q) for q in P3])
234         check("H2: drawn Euclidean image circle lies at hyperbolic distance r from the center", np.max(np.abs(dd
    ↳  - r)) < 1e-7)
235         ax.add_patch(Circle(c_xy, rad, fc=color, alpha=0.25, ec=color, lw=0.8))
236         ax.plot([Z[k, 0]], [Z[k, 1]], "o", color=color, ms=2)
237     lim = np.tanh(R / 2.0) * 1.04
```

```
    ax.set_xlim(-lim, lim); ax.set_ylim(-lim, lim); ax.set_aspect("equal"); ax.axis("off")
    ax.set_title(title, fontsize=12)


def draw_euclid_panel(ax, X, r, R, color, title):
    S = Space("R2")
    D = S.pair_dist(X)
    rho = S.rho(X)
    r_re = min(D.min() / 2.0, float((R - rho).min()))
    check("R2: recomputed disk radius matches CSV value (1e-9)", abs(r_re - r) < TOL, f"{r_re} vs {r}")
    check("R2: disks do not overlap and lie inside the container (1e-9)", D.min() / 2.0 - r > -TOL and float((R - rho).min()) - r > -TOL)
    ax.add_patch(Circle((0, 0), R, fill=False, ec="black", lw=1.0))
    for k in range(len(X)):
        ax.add_patch(Circle(X[k], r, fc=color, alpha=0.25, ec=color, lw=0.8))
        ax.plot([X[k, 0]], [X[k, 1]], "o", color=color, ms=2)
    lim = R * 1.04
    ax.set_xlim(-lim, lim); ax.set_ylim(-lim, lim); ax.set_aspect("equal"); ax.axis("off")
    ax.set_title(title, fontsize=12)


def draw_cap_panel(ax, X, r, R, color, title):
    """Spherical cap in azimuthal equidistant projection: plane radius = geodesic distance from the center."""
    S = Space("S2")
    check("Cap points have unit norm (1e-9)", np.max(np.abs(np.linalg.norm(X, axis=1) - 1)) < TOL)
    D = S.pair_dist(X)
    rho = S.rho(X)
    r_re = min(D.min() / 2.0, float((R - rho).min()))
    check("S2 cap: recomputed disk radius matches CSV value (1e-9)", abs(r_re - r) < TOL, f"{r_re} vs {r}")
    check("S2 cap: disks do not overlap and lie inside the cap (1e-9)", D.min() / 2.0 - r > -TOL and float((R - rho).min()) - r > -TOL)
    ax.add_patch(Circle((0, 0), R, fill=False, ec="black", lw=1.0))
    for k in range(len(X)):
        c = cap_circle(X[k], r, 240)
        dd = np.array([S.dist(X[k], q) for q in c[::20]])
        check("S2 cap: every drawn boundary point is at geodesic distance r from its center", np.max(np.abs(dd - r)) < 1e-7)
        rr = np.arccos(np.clip(c[:, 2], -1, 1))
        th = np.arctan2(c[:, 1], c[:, 0])
        ax.fill(rr * np.cos(th), rr * np.sin(th), fc=color, alpha=0.25, ec=color, lw=0.8)
        pr, pt = rho[k], np.arctan2(X[k, 1], X[k, 0])
        ax.plot([pr * np.cos(pt)], [pr * np.sin(pt)], "o", color=color, ms=2)
    lim = R * 1.04
    ax.set_xlim(-lim, lim); ax.set_ylim(-lim, lim); ax.set_aspect("equal"); ax.axis("off")
    ax.set_title(title, fontsize=12)
```

```
280
281
282 def fig_curvature(runs, cfg):
283     R, n = 3.0, 19
284     k = BALL_SET.index((R, n))
285     fig, axs = plt.subplots(1, 3, figsize=(6.5, 2.5))
286     spec = [("S2cap", 400 + k, draw_cap_panel, "Spherical cap, $K$ = +1"), ("R2", 300 + k, draw_euclid_panel, "E
    ↳ uclidean disk, $K$ = 0"),
287             ("H2", 200 + k, draw_hyperbolic_panel, "Hyperbolic disk, $K$ = \u22121")]
288     info = []
289     for ax, (sp, cid, fn, ttl) in zip(axs, spec):
290         b = best_run(runs, cid)
291         X = get_config(cfg, runs, cid, b.algorithm, int(b.seed))
292         if sp == "R2":
293             X = X[:, :2]
294         fn(ax, X, float(b.score), R, COL[b.algorithm], f"{ttl}\n$r$ = {b.score:.4f}")
295         info.append((sp, b.algorithm, float(b.score)))
296     fig.subplots_adjust(left=0.01, right=0.99, top=0.80, bottom=0.01, wspace=0.05)
297     return save(fig, "fig_curvature_packing"), info
298
299
300 # ---------------------------------------------------------------------- Figure 4
301 def ci_band(g, col):
302     m = g.groupby("budget_fraction")[col].mean()
303     s = g.groupby("budget_fraction")[col].std(ddof=1)
304     n = g.groupby("budget_fraction")[col].count()
305     h = 2.045 * s / np.sqrt(n)
306     return m.index.values, m.values, (m - h).values, (m + h).values
307
308
309 def fig_convergence(runs, tr):
310     conds = [(100 + 9, "$S^2$ Tammes, $N$ = 13"), (100 + 14, "$S^2$ Tammes, $N$ = 60"), (400 + 4, "Spherical cap
    ↳ , $R$ = 3, $n$ = 19"),
311              (300 + 4, "Euclidean disk, $R$ = 3, $n$ = 19"), (200 + 4, "Hyperbolic disk, $R$ = 3, $n$ = 19"), (1
    ↳ 00 + 12, "$S^2$ Tammes, $N$ = 30")]
312     ref = runs.groupby("condition_id")["reference_value"].first()
313     fig, axs = plt.subplots(3, 2, figsize=(6.5, 7.4), sharex=True)
314     for ax, (cid, title) in zip(axs.ravel(), conds):
315         lo_all = []
316         for a in ALG:
317             g = tr[(tr.condition_id == cid) & (tr.algorithm == a)].copy()
318             g["rel"] = g["best_score"] / ref[cid]
319             x, m, lo, hi = ci_band(g, "rel")
320             ax.plot(x, m, color=COL[a], ls=LSTY[a], marker=MARK[a], ms=3.5, lw=1.4, markevery=5)
321             ax.fill_between(x, lo, hi, color=COL[a], alpha=0.18, lw=0)
```

```
            lo_all.append(np.nanmin(lo))
        ax.axhline(1.0, color="#333333", ls=":", lw=1.0)
        ax.set_xscale("log")
        ax.set_ylim(max(0.0, min(lo_all) - 0.03), 1.04)
        ax.set_title(title, fontsize=11)
        ax.grid(True)
    for ax in axs[2]:
        ax.set_xlabel("Fraction of budget used")
    for ax in axs[:, 0]:
        ax.set_ylabel("Best score / reference")
    fig.subplots_adjust(left=0.11, right=0.98, top=0.96, bottom=0.17, hspace=0.35, wspace=0.28)
    h = alg_handles() + [Line2D([0], [0], color="#333333", ls=":", lw=1.0, label="Reference value")]
    legend_box(fig, h, loc="lower center", ncol=2, bbox_to_anchor=(0.54, 0.0))
    return save(fig, "fig_convergence")


# ---------------------------------------------------------------------- Figure 5
def fig_performance(summ):
    fig, axs = plt.subplots(4, 1, figsize=(6.5, 8.6))
    s2 = summ[summ.space == "S2"]
    for a in ALG:
        g = s2[s2.algorithm == a].sort_values("n")
        axs[0].errorbar(np.arange(len(g)), g.mean_relative, yerr=[g.mean_relative - g.rel_ci95_lo, g.rel_ci95_hi
 - g.mean_relative],
                        color=COL[a], marker=MARK[a], ls=LSTY[a], ms=5, lw=1.3, capsize=2.5, elinewidth=0.9)
    labs = [str(int(v)) for v in sorted(s2.n.unique())]
    axs[0].set_xticks(np.arange(len(labs)))
    axs[0].set_xticklabels(labs)
    axs[0].set_xlabel("Number of points $N$ (Tammes problem on $S^2$)")
    axs[0].set_ylabel("Distance / reference")
    axs[0].axhline(1.0, color="#333333", ls=":", lw=1.0)
    for ax, sp, t in ((axs[1], "S2cap", "spherical cap, $K$ = +1"), (axs[2], "R2", "Euclidean disk, $K$ = 0"), (
axs[3], "H2", "hyperbolic disk, $K$ = \u22121")):
        g0 = summ[summ.space == sp]
        conds = g0.drop_duplicates("condition_id").sort_values("condition_id")
        for a in ALG:
            g = g0[g0.algorithm == a].sort_values("condition_id")
            ax.errorbar(np.arange(len(g)), g.mean_relative, yerr=[g.mean_relative - g.rel_ci95_lo, g.rel_ci95_hi
 - g.mean_relative],
                        color=COL[a], marker=MARK[a], ls=LSTY[a], ms=5, lw=1.3, capsize=2.5, elinewidth=0.9)
        ax.set_xticks(np.arange(len(conds)))
        ax.set_xticklabels([f"{r:g}, {int(n)}" for r, n in zip(conds.R, conds.n)])
        ax.set_xlabel(f"Ball radius $R$, number of disks $n$ ({t})")
        ax.set_ylabel("Radius / best found")
        ax.axhline(1.0, color="#333333", ls=":", lw=1.0)
```

```
    for ax in axs:
        ax.grid(True)
        lo, hi = ax.get_ylim()
        ax.set_ylim(lo, max(hi, 1.01))
    fig.subplots_adjust(left=0.12, right=0.98, top=0.985, bottom=0.17, hspace=0.62)
    h = alg_handles() + [Line2D([0], [0], color="#333333", ls=":", lw=1.0, label="Reference value")]
    legend_box(fig, h, loc="lower center", ncol=2, bbox_to_anchor=(0.54, 0.0))
    return save(fig, "fig_performance_vs_size")


# ---------------------------------------------------------------------- Figure 6
def fig_runtime(summ):
    fig, ax = plt.subplots(figsize=(6.0, 3.9))
    s2 = summ[summ.space == "S2"]
    for a in ALG:
        g = s2[s2.algorithm == a].sort_values("n")
        h = 2.045 * g.sd_time_s / np.sqrt(g.n_seeds)
        ax.errorbar(g.n, g.mean_time_s, yerr=h, color=COL[a], marker=MARK[a], ls=LSTY[a], ms=5, lw=1.3, capsize=
↳ 2.5, elinewidth=0.9)
    ax.set_xscale("log")
    ax.set_yscale("log")
    ax.set_xlabel("Number of points $N$ (log scale)")
    ax.set_ylabel("Mean run time (s, log scale)")
    ax.grid(True, which="both")
    fig.subplots_adjust(left=0.13, right=0.97, top=0.97, bottom=0.34)
    legend_box(fig, alg_handles(), loc="lower center", ncol=2, bbox_to_anchor=(0.55, 0.0))
    return save(fig, "fig_runtime_scaling")


# ---------------------------------------------------------------------- Figure 7
def fig_distribution(runs):
    conds = [(100 + 9, "$S^2$\n$N$ = 13"), (100 + 12, "$S^2$\n$N$ = 30"), (100 + 14, "$S^2$\n$N$ = 60"),
             (400 + 4, "Cap\n(3, 19)"), (300 + 4, "$R^2$\n(3, 19)"), (200 + 4, "$H^2$\n(3, 19)")]
    fig, ax = plt.subplots(figsize=(6.5, 3.7))
    w = 0.22
    for k, a in enumerate(ALG):
        data = [runs[(runs.condition_id == cid) & (runs.algorithm == a)]["relative_score"].values for cid, _ in
↳ conds]
        pos = np.arange(len(conds)) + (k - 1) * (w + 0.03)
        bp = ax.boxplot(data, positions=pos, widths=w, patch_artist=True, showfliers=True,
                        medianprops=dict(color="black", lw=1.2), whiskerprops=dict(color=COL[a]), capprops=dict(
↳ color=COL[a]),
                        flierprops=dict(marker=MARK[a], mfc="none", mec=COL[a], ms=3))
        for b_ in bp["boxes"]:
            b_.set_facecolor(COL[a]); b_.set_alpha(0.35); b_.set_edgecolor(COL[a])
```

```
    ax.set_xticks(np.arange(len(conds)))
    ax.set_xticklabels([t for _, t in conds])
    ax.set_ylabel("Score / reference")
    ax.axhline(1.0, color="#333333", ls=":", lw=1.0)
    ax.grid(True, axis="y")
    handles = [Patch(facecolor=COL[a], alpha=0.5, edgecolor=COL[a], label=NAME[a]) for a in ALG]
    fig.subplots_adjust(left=0.11, right=0.98, top=0.97, bottom=0.30)
    legend_box(fig, handles, loc="lower center", ncol=3, bbox_to_anchor=(0.54, 0.0))
    return save(fig, "fig_distribution")


def main():
    runs, cfg, tr, summ = load()
    check("All plotted scores are recomputed from coordinates in the CSV files (runs match)",
          np.max(np.abs(runs.score - runs.recomputed_score)) < 1e-9)
    out = {}
    out["framework"] = fig_framework()
    out["sphere"] = fig_sphere(runs, cfg)
    out["curv"] = fig_curvature(runs, cfg)
    out["conv"] = fig_convergence(runs, tr)
    out["perf"] = fig_performance(summ)
    out["runtime"] = fig_runtime(summ)
    out["dist"] = fig_distribution(runs)
    pd.DataFrame(ASSERTIONS, columns=["assertion", "passed", "detail"]).to_csv(f"{DATA}/figure_assertions.csv",
↳ index=False)
    print(f"{len(ASSERTIONS)} figure assertions passed; font {SERIF}")
    return out


if __name__ == "__main__":
    main()
```

docbuilder.py

Shared APA 7 document builder used for the plan, paper, and this listing.

```
"""docbuilder.py -- APA 7 student-paper DOCX builder (python-docx + raw XML).

Used for Research_Paper, Research_Plan and Code_Listing.  Features: exact title
page, header page numbers, APA heading levels, APA tables (three rules), APA
figures, hanging-indent references, a Table of Contents wrapped in a real Word
TOC field, and list pages with spaced dot leaders computed from font metrics.
```

```
"""
import re
import subprocess
import os
import copy
from docx import Document
from docx.shared import Pt, Inches, RGBColor, Emu
from docx.enum.text import WD_ALIGN_PARAGRAPH, WD_TAB_ALIGNMENT, WD_TAB_LEADER, WD_BREAK
from docx.enum.section import WD_SECTION, WD_ORIENT
from docx.oxml.ns import qn
from docx.oxml import OxmlElement
from PIL import ImageFont

FONT = "Times New Roman"
TITLE = "Algorithmic Optimization of Dense Sphere Packings in Non-Euclidean Spaces"
AUTHOR = "Marian Ela J. Alibadbad"
CATEGORY = "Mathematics and Computational Science \u2013 Individual Category (MCS-I)"
REGION = "Region VIII"
_FDIR = "/usr/share/fonts/truetype/liberation/"
_FONTS = {}


def _font(bold=False, size=12):
    key = (bold, size)
    if key not in _FONTS:
        f = "LiberationSerif-Bold.ttf" if bold else "LiberationSerif-Regular.ttf"
        _FONTS[key] = ImageFont.truetype(_FDIR + f, size * 10)  # 10x for precision
    return _FONTS[key]


def text_width_pt(text, bold=False, size=12):
    return _font(bold, size).getlength(text) / 10.0


def set_run_font(run, name=FONT, size=12, bold=None, italic=None, color=None):
    run.font.name = name
    run.font.size = Pt(size)
    rpr = run._element.get_or_add_rPr()
    rf = rpr.find(qn("w:rFonts"))
    if rf is None:
        rf = OxmlElement("w:rFonts")
        rpr.insert(0, rf)
    for a in ("w:ascii", "w:hAnsi", "w:eastAsia", "w:cs"):
        rf.set(qn(a), name)
    if bold is not None:
```

```
        run.bold = bold
    if italic is not None:
        run.italic = italic
    if color is not None:
        run.font.color.rgb = RGBColor.from_string(color)


def _style_font(style, name=FONT, size=12, bold=False, italic=False):
    style.font.name = name
    style.font.size = Pt(size)
    style.font.bold = bold
    style.font.italic = italic
    style.font.color.rgb = RGBColor(0, 0, 0)
    rpr = style.element.get_or_add_rPr()
    rf = rpr.find(qn("w:rFonts"))
    if rf is None:
        rf = OxmlElement("w:rFonts")
        rpr.insert(0, rf)
    for a in list(rf.attrib):
        del rf.attrib[a]
    for a in ("w:ascii", "w:hAnsi", "w:eastAsia", "w:cs"):
        rf.set(qn(a), name)


def add_field(paragraph, instr, result_text="", bold=None):
    """Insert a complex field (begin/instr/separate/result/end) into a paragraph."""
    def fc(t):
        r = paragraph.add_run()
        e = OxmlElement("w:fldChar")
        e.set(qn("w:fldCharType"), t)
        r._element.append(e)
        return r
    fc("begin")
    r = paragraph.add_run()
    it = OxmlElement("w:instrText")
    it.set(qn("xml:space"), "preserve")
    it.text = " " + instr + " "
    r._element.append(it)
    fc("separate")
    rr = paragraph.add_run(result_text)
    set_run_font(rr, bold=bold)
    fc("end")


MARK = re.compile(r"(<i>|</i>|<b>|</b>|<sup>|</sup>|<sub>|</sub>)")
```

```
 97
 98
 99  class APADoc:
100      def __init__(self, title=TITLE, landscape_code=False):
101          self.title = title
102          d = self.doc = Document()
103          sec = d.sections[0]
104          sec.page_width, sec.page_height = Inches(8.5), Inches(11)
105          for m in ("left_margin", "right_margin", "top_margin", "bottom_margin"):
106              setattr(sec, m, Inches(1))
107          sec.header_distance = Inches(0.5)
108          sec.footer_distance = Inches(0.5)
109          self._styles()
110          self._header(sec)
111          d.core_properties.title = title
112          d.core_properties.author = AUTHOR
113          d.core_properties.subject = "NSTF SY 2026-2027, MCS-I"
114          d.core_properties.keywords = ""
115          d.core_properties.comments = ""
116          self.headings = []   # (level, text)
117          self.captions = []   # (kind, num, title)
118          self.nfig = 0
119          self.ntab = 0
120
121      # ------------------------------------------------------------------ styles
122      def _styles(self):
123          st = self.doc.styles
124          n = st["Normal"]
125          _style_font(n)
126          pf = n.paragraph_format
127          pf.line_spacing = 2.0
128          pf.space_before = Pt(0)
129          pf.space_after = Pt(0)
130          pf.widow_control = True
131          for lvl in (1, 2, 3, 4, 5):
132              h = st["Heading %d" % lvl]
133              _style_font(h, bold=True, italic=lvl in (3, 5))
134              hp = h.paragraph_format
135              hp.line_spacing = 2.0
136              hp.space_before = Pt(0)
137              hp.space_after = Pt(0)
138              hp.keep_with_next = True
139              hp.first_line_indent = Inches(0.5) if lvl >= 4 else Inches(0)
140              hp.left_indent = Inches(0)
141              hp.alignment = WD_ALIGN_PARAGRAPH.CENTER if lvl == 1 else WD_ALIGN_PARAGRAPH.LEFT
```

```
        for name in ("TOC 1", "TOC 2", "TOC 3"):
            try:
                s = st[name]
            except KeyError:
                s = st.add_style(name, 1)
            _style_font(s, bold=(name == "TOC 1"))
            s.paragraph_format.line_spacing = 2.0
            s.paragraph_format.space_after = Pt(0)
            s.paragraph_format.keep_together = True
        st["TOC 1"].paragraph_format.left_indent = Inches(0)
        st["TOC 2"].paragraph_format.left_indent = Inches(0.5)
        st["TOC 3"].paragraph_format.left_indent = Inches(1.0)
        for nm in ("TOC 1", "TOC 2", "TOC 3"):
            ts = st[nm].paragraph_format.tab_stops
            ts.add_tab_stop(Inches(6.5), WD_TAB_ALIGNMENT.RIGHT, WD_TAB_LEADER.DOTS)

    def _header(self, sec):
        hdr = sec.header
        hdr.is_linked_to_previous = False
        p = hdr.paragraphs[0]
        p.alignment = WD_ALIGN_PARAGRAPH.RIGHT
        p.paragraph_format.line_spacing = 1.0
        add_field(p, "PAGE", "1")
        for r in p.runs:
            set_run_font(r)

    # ---------------------------------------------------------- basic blocks
    def _para(self, style=None, align=None, indent_first=None, spacing=None, left=None,
              hanging=None, keep_next=False, space_before=None, space_after=None):
        p = self.doc.add_paragraph(style=style)
        pf = p.paragraph_format
        if align is not None:
            pf.alignment = align
        if indent_first is not None:
            pf.first_line_indent = Inches(indent_first)
        if left is not None:
            pf.left_indent = Inches(left)
        if hanging is not None:
            pf.left_indent = Inches(hanging if left is None else left)
            pf.first_line_indent = Inches(-hanging)
        if spacing is not None:
            pf.line_spacing = spacing
        if keep_next:
            pf.keep_with_next = True
        if space_before is not None:
```

```
            pf.space_before = Pt(space_before)
        if space_after is not None:
            pf.space_after = Pt(space_after)
        return p

    def add_marked(self, p, text, bold=False, italic=False, size=12):
        """Add text with <i>, <b>, <sup>, <sub> inline markup."""
        b, i, sup, sub = bold, italic, False, False
        for tok in MARK.split(text):
            if tok == "<i>": i = True
            elif tok == "</i>": i = italic
            elif tok == "<b>": b = True
            elif tok == "</b>": b = bold
            elif tok == "<sup>": sup = True
            elif tok == "</sup>": sup = False
            elif tok == "<sub>": sub = True
            elif tok == "</sub>": sub = False
            elif tok:
                r = p.add_run(tok)
                set_run_font(r, size=size, bold=b, italic=i)
                if sup: r.font.superscript = True
                if sub: r.font.subscript = True
        return p

    def body(self, text, indent=True, keep_next=False):
        p = self._para(indent_first=0.5 if indent else 0.0, align=WD_ALIGN_PARAGRAPH.LEFT, keep_next=keep_next)
        self.add_marked(p, text)
        return p

    def front_heading(self, text):
        """Centered bold heading that is NOT collected by the TOC field (no outline level)."""
        st = self.doc.styles
        try:
            fh = st["Front Heading"]
        except KeyError:
            fh = st.add_style("Front Heading", 1)
            fh.base_style = st["Normal"]
            _style_font(fh, bold=True)
            fh.paragraph_format.alignment = WD_ALIGN_PARAGRAPH.CENTER
            fh.paragraph_format.keep_with_next = True
            fh.paragraph_format.line_spacing = 2.0
        p = self.doc.add_paragraph(style="Front Heading")
        set_run_font(p.add_run(text), bold=True)
        return p

```

```
    def heading(self, text, level=1):
        p = self.doc.add_paragraph(style="Heading %d" % level)
        if level >= 4:
            text = text if text.endswith(".") else text + "."
        r = p.add_run(text)
        set_run_font(r, bold=True, italic=level in (3, 5))
        self.headings.append((level, text))
        return p

    def inline_heading_body(self, text, level, body_text):
        """APA level 4/5: heading ends with a period and body continues on the same line."""
        p = self.doc.add_paragraph(style="Heading %d" % level)
        r = p.add_run(text.rstrip(".") + ".")
        set_run_font(r, bold=True, italic=level == 5)
        p.paragraph_format.keep_with_next = False
        r2 = p.add_run(" ")
        set_run_font(r2, bold=False, italic=False)
        self.add_marked(p, body_text)
        self.headings.append((level, text.rstrip(".") + "."))
        return p

    def new_page(self):
        """Next-page section break (only break type used)."""
        self.doc.add_section(WD_SECTION.NEW_PAGE)
        sec = self.doc.sections[-1]
        sec.header.is_linked_to_previous = True

    # ------------------------------------------------------------- title page
    def title_page(self, date_text):
        p = self._para(align=WD_ALIGN_PARAGRAPH.CENTER, space_before=72)
        r = p.add_run(self.title)
        set_run_font(r, bold=True)
        first = True
        for line in (AUTHOR, REGION, CATEGORY, date_text):
            p = self._para(align=WD_ALIGN_PARAGRAPH.CENTER, space_before=24 if first else 0)
            first = False
            r = p.add_run(line)
            set_run_font(r)

    # ------------------------------------------------------------------ tables
    def _cell_border(self, cell, **kw):
        tcPr = cell._element.get_or_add_tcPr()
        b = tcPr.find(qn("w:tcBorders"))
        if b is None:
            b = OxmlElement("w:tcBorders")
```

```
            tcPr.append(b)
        for edge, val in kw.items():
            e = OxmlElement("w:" + edge)
            e.set(qn("w:val"), val)
            e.set(qn("w:sz"), "8")
            e.set(qn("w:space"), "0")
            e.set(qn("w:color"), "000000")
            b.append(e)

    def table(self, title, header, rows, note, aligns=None, widths=None, bold_cells=None, label=None, split=Fals
↳ e):
        """APA table: number line, italic title, three horizontal rules, Note."""
        if label is None:
            self.ntab += 1
            n = self.ntab
        else:
            n = label
        self.captions.append(("Table", n, title))
        p = self._para(keep_next=True, align=WD_ALIGN_PARAGRAPH.LEFT)
        set_run_font(p.add_run("Table "), bold=True)
        if label is None:
            add_field(p, "SEQ Table \\* ARABIC", str(n), bold=True)
        else:
            set_run_font(p.add_run(str(n)), bold=True)
        p = self._para(keep_next=True, align=WD_ALIGN_PARAGRAPH.LEFT)
        set_run_font(p.add_run(title), italic=True)
        ncol = len(header)
        t = self.doc.add_table(rows=1 + len(rows), cols=ncol)
        t.autofit = False
        t.alignment = 0
        tblPr = t._element.tblPr
        lay = OxmlElement("w:tblLayout")
        lay.set(qn("w:type"), "fixed")
        tblPr.append(lay)
        total = 6.5
        widths = widths or [total / ncol] * ncol
        aligns = aligns or ["l"] + ["c"] * (ncol - 1)
        amap = {"l": WD_ALIGN_PARAGRAPH.LEFT, "c": WD_ALIGN_PARAGRAPH.CENTER, "r": WD_ALIGN_PARAGRAPH.RIGHT}
        bold_cells = bold_cells or set()
        for ci, w_ in enumerate(widths):
            t.columns[ci].width = Inches(w_)
        for ri in range(1 + len(rows)):
            row = t.rows[ri]
            trPr = row._tr.get_or_add_trPr()
            cs = OxmlElement("w:cantSplit")
```

```
            trPr.append(cs)
            if split and ri == 0:
                trPr.append(OxmlElement("w:tblHeader"))        # header row repeats on the next page
            for ci in range(ncol):
                c = row.cells[ci]
                c.width = Inches(widths[ci])
                txt = header[ci] if ri == 0 else rows[ri - 1][ci]
                para = c.paragraphs[0]
                para.paragraph_format.line_spacing = 1.0
                para.paragraph_format.keep_with_next = (ri < 2 or ri >= len(rows) - 1) if split else (bool(note)
 or ri < len(rows))
                para.paragraph_format.alignment = amap["c"] if ri == 0 and aligns[ci] != "l" else amap[aligns[ci
]]
                self.add_marked(para, str(txt), bold=(ri > 0 and (ri - 1, ci) in bold_cells))
                if ri == 0:
                    self._cell_border(c, top="single", bottom="single")
                if ri == len(rows):
                    self._cell_border(c, bottom="single")
        if note:
            p = self._para(align=WD_ALIGN_PARAGRAPH.LEFT)
            set_run_font(p.add_run("Note"), italic=True)
            set_run_font(p.add_run(". "))
            self.add_marked(p, note)
        return n

    # ------------------------------------------------------------------ figures
    def figure(self, title, image_path, width_in, caption, legend, alt):
        """Image, then the caption (identifies and describes the figure), then the legend
        (explains symbols, colors, lines, markers, abbreviations, units).  All three stay together."""
        self.nfig += 1
        n = self.nfig
        self.captions.append(("Figure", n, title))
        p = self._para(keep_next=True, align=WD_ALIGN_PARAGRAPH.CENTER, spacing=1.0)
        run = p.add_run()
        run.add_picture(image_path, width=Inches(width_in))
        dp = run._element.find(".//" + qn("wp:docPr"))
        if dp is not None:
            dp.set("descr", alt)
            dp.set("title", "Figure %d" % n)
        p = self._para(keep_next=bool(legend), align=WD_ALIGN_PARAGRAPH.LEFT)
        set_run_font(p.add_run("Figure "), bold=True)
        add_field(p, "SEQ Figure \\* ARABIC", str(n), bold=True)
        set_run_font(p.add_run("."), bold=True)
        set_run_font(p.add_run(" "))
        set_run_font(p.add_run(title + "."), italic=True)
```

```
364          set_run_font(p.add_run(" "))
365          self.add_marked(p, caption)
366          if legend:
367              p = self._para(align=WD_ALIGN_PARAGRAPH.LEFT)
368              set_run_font(p.add_run("Legend."), italic=True)
369              set_run_font(p.add_run(" "))
370              self.add_marked(p, legend)
371          return n
372
373      def equation(self, text, number):
374          p = self._para(align=WD_ALIGN_PARAGRAPH.LEFT, indent_first=0.0)
375          ts = p.paragraph_format.tab_stops
376          ts.add_tab_stop(Inches(3.25), WD_TAB_ALIGNMENT.CENTER)
377          ts.add_tab_stop(Inches(6.5), WD_TAB_ALIGNMENT.RIGHT)
378          p.add_run("\t")
379          import omml
380          from docx.oxml import parse_xml
381          if omml.MODE == "image" and text in omml.LATEX:
382              png, w_in, h_in, desc_in = omml.render_png(text, os.path.join(os.path.dirname(os.path.dirname(os.pat
    ↳ h.abspath(__file__))), "out", "eq"))
383              run = p.add_run()
384              run.add_picture(png, height=Inches(h_in))
385              raise_hp = int(round(desc_in * 72 * 2))               # equation number is raised to the equation b
    ↳ aseline
386          elif text in omml.EQUATIONS:
387              p._p.append(parse_xml(omml.EQUATIONS[text]))     # native Word equation (OMML)
388          else:
389              self.add_marked(p, text)
390          p.add_run("\t")
391          numrun = p.add_run("(%d)" % number)
392          set_run_font(numrun)
393          if omml.MODE == "image" and text in omml.LATEX:
394              pos = OxmlElement("w:position")
395              pos.set(qn("w:val"), str(raise_hp))
396              numrun._r.get_or_add_rPr().append(pos)
397          for r in p.runs:
398              set_run_font(r, italic=r.italic)
399          return p
400
401      def reference(self, text):
402          p = self._para(hanging=0.5, align=WD_ALIGN_PARAGRAPH.LEFT)
403          self.add_marked(p, text)
404          return p
405
406      # ------------------------------------------------ contents-type pages
```

```
    def _dots_line(self, p, text, page, bold, left_in, first_hang_in, num_col=None, stop_in=6.0,
                   page_col_in=6.5, title_x=None):
        """Entry with spaced dot leader.  The dots end at stop_in; the page number is
        right-aligned at page_col_in.  Handles wrapped titles by simulating word wrap."""
        unit = ". "
        uw = text_width_pt(unit)
        avail_first = (stop_in - left_in) * 72.0
        # wrap simulation for the title text
        words = text.split(" ")
        lines, cur = [], ""
        limit = (stop_in - left_in) * 72.0 - 36.0
        for w in words:
            trial = (cur + " " + w).strip()
            if text_width_pt(trial, bold) <= limit:
                cur = trial
            else:
                lines.append(cur)
                cur = w
        lines.append(cur)
        last = lines[-1]
        lastw = text_width_pt(last, bold)
        offset = 0.0 if len(lines) > 1 else 0.0
        room = (stop_in - left_in) * 72.0 - lastw - text_width_pt(" ")
        if len(lines) > 1:
            room += 0  # continuation lines start at the same hanging indent
        count = max(int(room // uw), 1)
        return lines, count

    def entry(self, p, text, page, bold, left_in, hang_in, num=None, num_col_w=0.75):
        """Write one contents entry into paragraph p (already created)."""
        pf = p.paragraph_format
        pf.line_spacing = 2.0
        pf.space_after = Pt(0)
        pf.keep_together = True
        ts = pf.tab_stops
        if num is not None:
            # list-of-tables/figures entry: number at margin, title at 0.75 in hanging indent
            pf.left_indent = Inches(num_col_w)
            pf.first_line_indent = Inches(-num_col_w)
            ts.add_tab_stop(Inches(num_col_w), WD_TAB_ALIGNMENT.LEFT)
            r = p.add_run(str(num) + "\t")
            set_run_font(r, bold=False)
            left_eff = num_col_w
        else:
            pf.left_indent = Inches(left_in + hang_in)
```

```
            pf.first_line_indent = Inches(-hang_in)
            left_eff = left_in + hang_in
        ts.add_tab_stop(Inches(6.5), WD_TAB_ALIGNMENT.RIGHT)
        lines, count = self._dots_line(p, text, page, bold, left_eff, hang_in)
        # emit wrapped title as words (Word wraps on its own; widths identical in
        # Times New Roman / Liberation Serif)
        r = p.add_run(text + " ")
        set_run_font(r, bold=bold)
        r = p.add_run((". " * count).rstrip(" ") + " ")
        set_run_font(r, bold=False)
        r = p.add_run("\t" + str(page))
        set_run_font(r, bold=bold)

    def contents_page(self, heading, label, entries, kind, field_instr=None, in_toc=False):
        """kind 'toc' : entries = [(level, text, page)]
           kind 'list': entries = [(num, title, page)]"""
        if in_toc:
            self.heading(heading, 1)
        else:
            self.front_heading(heading)
        p = self._para(spacing=2.0, space_after=0, align=WD_ALIGN_PARAGRAPH.LEFT)
        p.paragraph_format.tab_stops.add_tab_stop(Inches(6.5), WD_TAB_ALIGNMENT.RIGHT)
        set_run_font(p.add_run(label), bold=True)
        set_run_font(p.add_run("\t"), bold=True)
        set_run_font(p.add_run("Page"), bold=True)
        paras = []
        for e in entries:
            if kind == "toc":
                lvl, text, page = e
                p = self.doc.add_paragraph(style="TOC %d" % lvl)
                self.entry(p, text, page, bold=(lvl == 1), left_in={1: 0.0, 2: 0.5, 3: 1.0}[lvl], hang_in=0.0)
            else:
                num, title, page = e
                p = self.doc.add_paragraph()
                self.entry(p, title, page, bold=False, left_in=0.0, hang_in=0.0, num=num)
            paras.append(p)
        if kind == "toc" and field_instr and paras:
            # wrap the cached entries in a real TOC field
            first, last = paras[0], paras[-1]
            def fc(par, t, at_start):
                r = OxmlElement("w:r")
                e = OxmlElement("w:fldChar")
                e.set(qn("w:fldCharType"), t)
                r.append(e)
                if at_start:
```

```
                    par._p.insert(par._p.index(par._p.pPr) + 1 if par._p.pPr is not None else 0, r)
                else:
                    par._p.append(r)
            def instr(par):
                r = OxmlElement("w:r")
                it = OxmlElement("w:instrText")
                it.set(qn("xml:space"), "preserve")
                it.text = " " + field_instr + " "
                r.append(it)
                par._p.insert(par._p.index(par._p.pPr) + 1, r)
            # begin, instr, separate must precede the first entry text (insert in reverse)
            for t in ("separate", None, "begin"):
                if t is None:
                    instr(first)
                else:
                    fc(first, t, True)
            fc(last, "end", False)
        return paras

    def save(self, path):
        self.doc.save(path)


# ---------------------------------------------------------------- conversion
def to_pdf(docx_path, outdir):
    subprocess.run(["soffice", "--headless", "--convert-to", "pdf", "--outdir", outdir, docx_path],
                   check=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL, timeout=300)
    return os.path.join(outdir, os.path.splitext(os.path.basename(docx_path))[0] + ".pdf")


def pdf_pages(pdf_path):
    import pdfplumber
    out = []
    with pdfplumber.open(pdf_path) as pdf:
        for pg in pdf.pages:
            out.append(pg.extract_text() or "")
    return out


def find_pages(pages, items, start_page=0):
    """items: list of strings.  Returns first page number (1-based, >= start_page+1) whose
    text contains each string at the start of a line; 0 if missing."""
    res = []
    for s in items:
        found = 0
```

```
        norm = " ".join(s.split())
        for i in range(start_page, len(pages)):
            for line in pages[i].split("\n"):
                if " ".join(line.split()).startswith(norm[:60]):
                    found = i + 1
                    break
            if found:
                break
        res.append(found)
    return res


def find_exact(pages, items, start_page=0):
    """First page (1-based) with a line exactly equal to each string; 0 if missing."""
    res = []
    for s in items:
        found = 0
        for i in range(start_page, len(pages)):
            if any(" ".join(l.split()) == s for l in pages[i].split("\n")):
                found = i + 1
                break
        res.append(found)
    return res
```

make_plan.py

Builds the Research Plan (written before the experiments).

```
"""make_plan.py -- builds Research_Plan.docx / .pdf .

This plan states every design value
exactly as executed, adds the common-formulation spherical-cap condition family, and contains a Version History
↳ and Timing.
It contains no results.
"""
import os
import json
import inspect
from docbuilder import *
import experiments as E
import algorithms as A

```

```
PLAN_DATE = "December 3, 2026"
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))   # folder that contains code/ and out/
OUT = os.path.join(ROOT, "out")

REFS = [
    "Absil, P.-A., Mahony, R., & Sepulchre, R. (2008). <i>Optimization algorithms on matrix manifolds</i>. Princeton University Press.",
    "Byrd, R. H., Lu, P., Nocedal, J., & Zhu, C. (1995). A limited memory algorithm for bound constrained optimization. <i>SIAM Journal on Scientific Computing, 16</i>(5), 1190–1208.",
    "Cohn, H. (n.d.). <i>Spherical codes</i> [Table of best known spherical codes]. Massachusetts Institute of Technology. https://cohn.mit.edu/spherical-codes/",
    "Danzer, L. (1963). <i>Endliche Punktmengen auf der 2-Sphäre mit möglichst grossem Minimalabstand</i> [Habilitation thesis]. Universität Göttingen.",
    "Fejes Tóth, L. (1943). Über eine Abschätzung des kürzesten Abstandes zweier Punkte eines auf einer Kugelfläche liegenden Punktsystems. <i>Jahresbericht der Deutschen Mathematiker-Vereinigung, 53</i>, 66–68.",
    "Hajek, B. (1988). Cooling schedules for optimal annealing. <i>Mathematics of Operations Research, 13</i>(2), 311–329. https://doi.org/10.1287/moor.13.2.311",
    "Liu, D. C., & Nocedal, J. (1989). On the limited memory BFGS method for large scale optimization. <i>Mathematical Programming, 45</i>(1–3), 503–528.",
    "Musin, O. R., & Tarasov, A. S. (2015). The Tammes problem for N = 14. <i>Experimental Mathematics, 24</i>(4), 460–468. https://doi.org/10.1080/10586458.2015.1022842",
    "Robinson, R. M. (1961). Arrangement of 24 points on a sphere. <i>Mathematische Annalen, 144</i>, 17–48.",
    "Saff, E. B., & Kuijlaars, A. B. J. (1997). Distributing many points on a sphere. <i>The Mathematical Intelligencer, 19</i>(1), 5–11.",
    "Schütte, K., & van der Waerden, B. L. (1951). Auf welcher Kugel haben 5, 6, 7, 8 oder 9 Punkte mit Mindestabstand Eins Platz? <i>Mathematische Annalen, 123</i>, 96–124.",
    "Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., ... SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. <i>Nature Methods, 17</i>(3), 261\u2013272. https://doi.org/10.1038/s41592-019-0686-2",
    "Wales, D. J., & Doye, J. P. K. (1997). Global optimization by basin-hopping and the lowest energy structures of Lennard-Jones clusters containing up to 110 atoms. <i>The Journal of Physical Chemistry A, 101</i>(28), 5111\u20135116. https://doi.org/10.1021/jp970984n",
    "Specht, E. (n.d.). <i>Packomania: Circles in a circle</i> [Data set]. http://www.packomania.com/",
    "Sloane, N. J. A., Hardin, R. H., Smith, W. D., & others. (n.d.). <i>Tables of spherical codes</i> [Data set]. http://neilsloane.com/packings/",
    "Tammes, P. M. L. (1930). On the origin of number and arrangement of the places of exit on the surface of pollen-grains. <i>Recueil des Travaux Botaniques Néerlandais, 27</i>, 1–84.",
    "Böröczky, K. (1978). Packing of spheres in spaces of constant curvature. <i>Acta Mathematica Academiae Scientiarum Hungaricae, 32</i>, 243\u2013261. https://doi.org/10.1007/BF01902361",
    "Cannon, J. W., Floyd, W. J., Kenyon, R., & Parry, W. R. (1997). Hyperbolic geometry. In S. Levy (Ed.), <i>Flavors of geometry</i> (pp. 59\u2013115). Cambridge University Press.",
    "Holm, S. (1979). A simple sequentially rejective multiple test procedure. <i>Scandinavian Journal of Statistics, 6</i>(2), 65\u201370.",
```

```
    "Kirkpatrick, S., Gelatt, C. D., Jr., & Vecchi, M. P. (1983). Optimization by simulated annealing. <i>Science, 220</i>(4598), 671\u2013680. https://doi.org/10.1126/science.220.4598.671",
    "Lubachevsky, B. D., & Stillinger, F. H. (1990). Geometric properties of random disk packings. <i>Journal of Statistical Physics, 60</i>, 561\u2013583. https://doi.org/10.1007/BF01025983",
    "Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H., & Teller, E. (1953). Equation of state calculations by fast computing machines. <i>The Journal of Chemical Physics, 21</i>(6), 1087\u20131092. https://doi.org/10.1063/1.1699114",
    "Musin, O. R., & Tarasov, A. S. (2012). The strong thirteen spheres problem. <i>Discrete & Computational Geometry, 48</i>(1), 128\u2013141. https://doi.org/10.1007/s00454-011-9392-2",
    "Wilcoxon, F. (1945). Individual comparisons by ranking methods. <i>Biometrics Bulletin, 1</i>(6), 80\u20138
3. https://doi.org/10.2307/3001968",
]

_hs = open(OUT + '/conf/PREREGISTRATION.sha256').read().split()
PRE_HASH, PRE_TIME = _hs[0], _hs[-1].replace('T', ' at ').replace('Z', '')
NS = E.S2_N
BALL = E.BALL_RN
NSEED = len(E.SEEDS)
dg = {n: {k: v.default for k, v in inspect.signature(fn).parameters.items() if v.default is not inspect._empty}
      for n, fn in (("gd", A.run_gd), ("sa", A.run_sa), ("ls", A.run_ls))}
N_TAM_RUNS = len(NS) * NSEED * 3
N_BALL_RUNS = 3 * len(BALL) * NSEED * 3
SET = ", ".join(f"({r:g}, {n})" for r, n in BALL)



def p10(x):
    """Format a power of ten such as 1e-05 as 10<sup>-5</sup>."""
    import math
    e = int(round(math.log10(x)))
    return f"10<sup>\u2212{abs(e)}</sup>" if e < 0 else f"10<sup>{e}</sup>"

def build(pg):
    d = APADoc(title=TITLE)
    d.title_page(PLAN_DATE)
    d.new_page()
    d.contents_page("Table of Contents", "Chapter", pg["toc"], "toc", field_instr='TOC \\o "1-3" \\z \\u')
    d.new_page()
    d.contents_page("List of Tables", "Table", pg["tables"], "list", in_toc=True)
    d.new_page()
    floats = []

    def F(fn, *a, **k):
        floats.append((fn, a, k))

```

```
    def FL():
        while floats:
            fn, a, k = floats.pop(0)
            fn(*a, **k)

    def HD(text, level=1):
        FL()
        return d.heading(text, level)

    p = d._para(align=WD_ALIGN_PARAGRAPH.CENTER)
    set_run_font(p.add_run(TITLE), bold=True)

    # ============================================================ Rationale
    HD("Rationale", 1)
    d.body("This section explains the problem, the existing knowledge on which the study builds, the gap it addr
↳ esses, and its significance.")
    HD("Background", 2)
    d.body("Packing equal disks or points as densely as possible is a classical problem of geometry. On the unit
↳  sphere, the Tammes problem asks for <i>N</i> points whose smallest pairwise geodesic distance is as large as po
↳ ssible. The problem is named for Tammes (1930). Optimal configurations have been proved for <i>N</i> = 3, 4, 6,
↳ and 12 (Fejes Tóth, 1943), <i>N</i> = 5, 7, 8, and 9 (Schütte & van der Waerden, 1951), <i>N</i> = 24 (Robinson,
↳  1961), <i>N</i> = 10 and 11 (Danzer, 1963), <i>N</i> = 13 (Musin & Tarasov, 2012), and <i>N</i> = 14 (Musin & T
↳ arasov, 2015); for most other <i>N</i>, the best known configurations come from computer search (Sloane et al.,
↳ n.d.; Cohn, n.d.) and are not proven optimal. Methods for spreading many points over a sphere are surveyed by Sa
↳ ff and Kuijlaars (1997).")
    d.body("Curvature changes the geometry of packing. In the Euclidean plane the area of a disk grows as the sq
↳ uare of its radius; in the hyperbolic plane it grows exponentially (Cannon et al., 1997), so most of the area of
↳  a large hyperbolic disk lies near its boundary. Böröczky (1978) proved density bounds for packings in spaces of
↳  constant curvature, where the definition of density is itself delicate. Computational methods for packing inclu
↳ de energy minimization, Monte Carlo and annealing methods (Kirkpatrick et al., 1983; Metropolis et al., 1953), a
↳ nd compression of growing disks (Lubachevsky & Stillinger, 1990).")
    HD("Research Gap, Importance, and Societal Impact", 2)
    d.body("Established knowledge in this area includes the optimal Tammes configurations listed above, the area
↳  formulas of the three plane geometries, and the existence of the three families of methods. The proposed contri
↳ bution of this project is narrower: a transparent, reproducible comparison of three simple optimizers under one
↳ shared protocol, with an equal computational budget and paired random seeds, on one bounded disk-packing problem
↳  posed identically in geodesic balls of the sphere (curvature +1), the Euclidean plane (0), and the hyperbolic p
↳ lane (\u22121), together with the Tammes problem on the whole sphere. I did not find a study that compares these
↳  methods this way. The project proves no theorem and does not aim to set packing records.")
    d.body("Well-spread point sets are used to sample spheres and to place sensors or antennas over curved regio
↳ ns, and hyperbolic packings relate to embeddings of tree-like networks. A reproducible comparison can help stude
↳ nts and researchers choose a method and see where each method fails.")

    # ======================================================== Research questions
    HD("Research Questions", 1)
```

```
    d.body("The study addresses four questions. RQ1: under an equal budget, which of gradient descent, simulated annealing, and compression reaches the best packings in each problem? RQ2: how does performance change as the number of points or disks grows? RQ3: for numbers of points with a proven optimum, how often does each method recover that optimum within a stated tolerance? RQ4 (exploratory): when the same bounded-disk problem is solved in spaces of curvature +1, 0, and −1, does the ranking of the methods change? This question is exploratory because curvature and area are confounded (a ball of the same radius has a different area in each space), only six settings per space are run, and the ball of radius 3 on the sphere covers nearly the whole sphere.")

    # ============================================================ Goals
    HD("Goals, Expected Outcomes, and Hypotheses", 1)
    d.body("This section lists the objectives, states the hypotheses, and fixes the rule that decides whether each hypothesis is supported.")
    HD("Goals and Expected Outcomes", 2)
    d.body(f"The objectives are (O1) to implement and validate the geometry and the three optimizers from scratch, (O2) to run {NSEED} paired seeds in each of {len(NS) + 3 * len(BALL)} conditions under an equal budget, (O3) to report all results, including failures, and (O4) to release code and data so that every table and figure can be regenerated. The expected outcome is a reproducible package that states which method performs best in which setting, with the size and the limits of each difference. No winner is assumed, and negative results will be reported.")
    HD("Hypotheses and Decision Rules", 2)
    d.body("Three hypotheses are tested, each with a decision rule stated in Table 1. This plan was finalized after some results had been seen (see Appendix A, History of Work Done, which states which rules were fixed before, and which after, results were seen), so every hypothesis result is to be reported as exploratory, and H1 most of all. The wording of H1 refers to the space, not to curvature alone, because only the bounded-disk problem is posed identically in all three spaces. Table 1 gives the rules.")
    F(d.table, "Hypotheses and Decision Rules",
      ["Hypothesis", "Test and measurable criterion", "Decision"],
      [["H1 (exploratory): the ranking of the algorithms depends on the space",
        "Common formulation (disks in a geodesic ball, six settings per space): Kruskal\u2013Wallis test across the three spaces of the per-setting mean paired difference in relative score, for each of the three algorithm pairs, Holm-adjusted over the three pairs; and the algorithm with the lowest mean rank in each space.",
        "Supported: some pair has adjusted <i>p</i> < .05 and the lowest-mean-rank algorithm is not the same in all three spaces. Partly supported: exactly one of the two. Not supported: neither."],
       ["H2: compression scales better with <i>N</i>",
        "Tammes problem, <i>N</i> \u2265 13: per-seed slope of relative score on log<sub>10</sub> <i>N</i>; Wilcoxon signed-rank tests of the compression slope against each other slope, Holm-adjusted over two tests.",
        "Supported: compression slope higher than both, adjusted <i>p</i> < .05. Partly supported: higher than one. Not supported: otherwise, including an opposite direction."],
       ["H3: every algorithm recovers the proven optimum for small <i>N</i>",
        "Tammes problem, <i>N</i> = 4 to 14: share of seeds with relative score \u2265 .999 (0.1% tolerance) at each <i>N</i>.",
        "Supported: share \u2265 .90 at every <i>N</i> for all three algorithms. Partly supported: for one or two algorithms. Not supported: for none."]],
      "Relative score = final score divided by the reference value (see Experimental Conditions). A 1% tolerance (relative score \u2265 .99) is reported as a sensitivity check only.",
```

```
122         aligns=["l", "l", "l"], widths=[1.5, 2.9, 2.1], split=True)
123
124     # ============================================================ Procedures
125     HD("Procedures", 1)
126     d.body("This section defines the spaces, problems, experimental conditions, algorithms, metrics, and validat
    ↳ ion tests. The parameter values given are tunable defaults, not requirements; the ranges examined in the sensiti
    ↳ vity analysis are stated in the Data Analysis section.")
127     HD("Spaces and Problems", 2)
128     d.body("Three constant-curvature planes are used. The unit sphere <i>S</i><sup>2</sup> has curvature +1, wit
    ↳ h points as unit vectors <b><i>x</i></b> in <i>R</i><sup>3</sup> and geodesic distance given by Equation 1. The
    ↳ hyperbolic plane <i>H</i><sup>2</sup> has curvature \u22121; computations use the hyperboloid model, where a poi
    ↳ nt satisfies [<b><i>x</i></b>, <b><i>x</i></b>] = \u22121 and <i>x</i><sub>0</sub> > 0 for the Lorentz product [
    ↳ <b><i>x</i></b>, <b><i>y</i></b>] = \u2212<i>x</i><sub>0</sub><i>y</i><sub>0</sub> + <i>x</i><sub>1</sub><i>y</i
    ↳ ><sub>1</sub> + <i>x</i><sub>2</sub><i>y</i><sub>2</sub>, and distance is given by Equation 2; figures use the P
    ↳ oincaré disk. The Euclidean plane <i>R</i><sup>2</sup> has the usual distance.")
129     d.equation("cos <i>d</i> = <b><i>x</i></b> \u00b7 <b><i>y</i></b>", 1)
130     d.equation("cosh <i>d</i> = \u2212[<b><i>x</i></b>, <b><i>y</i></b>]", 2)
131     d.body("Problem A (Tammes) places <i>N</i> points on <i>S</i><sup>2</sup> to maximize the smallest pairwise
    ↳ distance. Problem B places <i>n</i> equal disks in a closed geodesic ball of radius <i>R</i> about a fixed cente
    ↳ r, in each of the three spaces (a spherical cap, a Euclidean disk, and a hyperbolic disk), and maximizes the com
    ↳ mon radius <i>r</i>. A disk of radius <i>r</i> whose center is at distance <i>\u03c1</i><sub><i>i</i></sub> from
    ↳  the ball center lies inside the ball when <i>\u03c1</i><sub><i>i</i></sub> \u2264 <i>R</i> \u2212 <i>r</i>, and
    ↳  two disks do not overlap when <i>d</i><sub><i>ij</i></sub> \u2265 2<i>r</i>. Both problems maximize <i>\u2113</
    ↳ i><sub>min</sub> in Equation 3, where the second term is absent for Problem A; the score is <i>\u2113</i><sub>mi
    ↳ n</sub> for Problem A and <i>r</i> = <i>\u2113</i><sub>min</sub>/2 for Problem B. Problem B is a maximum-radius
    ↳ formulation for fixed <i>n</i> and <i>R</i>; the number of disks of a fixed radius that fit is derived from it."
    ↳ )
132     d.equation("<i>\u2113</i><sub>min</sub> = min{ min<sub><i>i</i> < <i>j</i></sub> <i>d</i><sub><i>ij</i></sub
    ↳ >, min<sub><i>i</i></sub> 2(<i>R</i> \u2212 <i>\u03c1</i><sub><i>i</i></sub>) }", 3)
133     d.body("The area of a disk of radius <i>r</i> is 2\u03c0(1 \u2212 cos <i>r</i>) on <i>S</i><sup>2</sup> (<i>
    ↳ r</i> \u2264 \u03c0), \u03c0<i>r</i><sup>2</sup> on <i>R</i><sup>2</sup>, and 2\u03c0(cosh <i>r</i> \u2212 1) on
    ↳  <i>H</i><sup>2</sup>. Packing density is <i>n</i> times the area of a disk of radius <i>r</i>, divided by the a
    ↳ rea of the ball of radius <i>R</i>.")
134     HD("Comparability of the Problems", 2)
135     d.body("Problem A and Problem B differ in their objective (a minimum distance against a common radius), in t
    ↳ heir constraints (no boundary against a boundary), and in scale, so their results are not compared with one anot
    ↳ her. Statements about curvature rest only on Problem B, which has the same formulation, the same <i>R</i> and <i
    ↳ >n</i>, and the same code in all three spaces. Even then, a ball of radius <i>R</i> has a different area in each
    ↳  space, so a difference between spaces can come from curvature, from area, or from both; the study does not sepa
    ↳ rate them. Problem A is used for the comparison of the algorithms on the sphere, the recovery of proven optima,
    ↳ and scaling with <i>N</i>.")
136     HD("Experimental Conditions", 2)
137     d.body(f"Problem A uses <i>N</i> = {', '.join(str(n) for n in NS[:-1])}, and {NS[-1]}. Problem B uses the si
    ↳ x settings (<i>R</i>, <i>n</i>) = {SET} in each of the three spaces. The evaluation seeds are 1 to {NSEED}; the
    ↳ initial configuration depends only on the seed and the condition, so the three algorithms start from the same co
```

```
    nfiguration. Pilot runs for parameter tuning use seeds 9001 to 9004, which are never used in the evaluation. The
     total is {N_TAM_RUNS + N_BALL_RUNS:,} runs.")
138     d.body(f"The Tammes problem accounts for {N_TAM_RUNS:,} runs ({len(NS)} values of <i>N</i> \u00d7 {NSEED} se
    eds \u00d7 3 algorithms), and each space accounts for {len(BALL) * NSEED * 3:,} runs in Problem B ({len(BALL)} s
    ettings \u00d7 {NSEED} seeds \u00d7 3 algorithms). The reference value of a condition is the proven optimum for
    Problem A with <i>N</i> = 4 to 14 and 24 (credited to Fejes Tóth, 1943; Schütte & van der Waerden, 1951; Robinso
    n, 1961; Danzer, 1963; and Musin & Tarasov, 2012, 2015), the published best-known value where one is entered (Sl
    oane et al., n.d.; Cohn, n.d.), and otherwise the highest score found by any run in that condition; the last two
     are not claimed to be optimal.")
139     HD("Optimization Algorithms", 2)
140     d.body("Three algorithms are implemented in Python with numpy. Table 2 gives the default value of every para
    meter; each is tunable. All three move points along geodesics and keep points on the manifold, and all report th
    e best configuration they visited, scored exactly from its coordinates.")
141     F(d.table, "Algorithm Parameters", ["Algorithm and parameter", "Value"],
142       [["GD: energy, exponent", f"<i>E<sub>p</sub></i> = (1/<i>p</i>) ln \u03a3 <i>\u2113<sub>k</sub></i><sup>\u
    2212<i>p</i></sup> over pair distances and, in Problem B, doubled wall clearances; <i>p</i> from {dg['gd']['p0']
    :g} to {dg['gd']['pmax']:g}, geometric in the share of budget used"],
143        ["GD: step, line search", "<i>\u03b7</i> = min(<i>m</i> \u2113<sub>min</sub><sup>2</sup>/<i>p</i>, 0.25 \
    u2113<sub>min</sub>/max|<i>g</i>|); <i>m</i> starts at 0.5, \u00d7 1.3 after success (maximum 2), \u00d7 0.5 aft
    er failure (minimum 0.001); on failure multiply <i>\u03b7</i> by 0.4, at most 5 trials"],
144        ["SA: temperature", f"Geometric from {dg['sa']['T0f']:g} to {p10(dg['sa']['T1f'])} times the length scale
     \u221a(area/<i>n</i>)"],
145        ["SA: move, adaptation", "Random geodesic step of length \u03b4(0.1 + 0.9<i>u</i>), <i>u</i> uniform on (
    0, 1); \u03b4 starts at 0.3 times the length scale; every 100 proposals \u03b4 \u00d7 1.15 if acceptance > .40,
    \u00d7 0.87 if < .25"],
146        ["SA: point choice", f"Probability {dg['sa']['focus']:g}: an endpoint of the limiting pair or the point n
    earest the wall; otherwise uniform"],
147        ["CP: time step, growth", f"Time step {dg['ls']['dt_f']:g} times the length scale at unit root-mean-squar
    e speed; common radius grows by \u03b3 d<i>t</i> per step, \u03b3 falling linearly from {dg['ls']['g0']:g} to {d
    g['ls']['g1']:g}, capped by half the smallest distance"],
148        ["CP: collisions", "Velocity components along the line of centers exchanged if approaching; overlaps remo
    ved; wall reflection in Problem B; kinetic energy renormalized each step"]],
149       "GD = gradient descent; SA = simulated annealing; CP = compression. The length scale is \u221a(area/<i>n</
    i>). The compression method is a simplified, time-stepped version of the method of Lubachevsky and Stillinger (1
    990). The values are defaults used in the reported runs, chosen on pilot seeds; they are not fixed by the design
    , and the ranges examined later are listed under Supplementary Exploratory Analyses.",
150       aligns=["l", "l"], widths=[1.6, 4.9])
151     HD("Budget, Initialization, and Stopping", 2)
152     d.body(f"Every run receives a budget of {E.BUDGET_SWEEPS:,} sweeps, where one sweep is the <i>n</i>(<i>n</i>
     \u2212 1)/2 pair-distance evaluations needed to compute all distances once; a gradient step costs one sweep plu
    s one sweep per line-search trial (in Problem B each evaluation also includes the <i>n</i> wall clearances), a s
    imulated-annealing move costs <i>n</i> \u2212 1 evaluations, and a compression step costs two sweeps. Gradient d
    escent and compression stop when their next step would exceed the budget; simulated annealing fixes its number o
    f moves from the budget in advance and does not charge proposals rejected for lying outside the ball, so it may
    use slightly less than the budget. Initial points are uniform on <i>S</i><sup>2</sup> for Problem A and uniform
```

```
   ↳ by area in the ball of radius <i>R</i> \u2212 0.05 for Problem B.")
153      HD("Evaluation Metrics", 2)
154      d.body("The primary metric is the relative score, the final score divided by the reference value. A run succ
   ↳ eeds if its relative score is at least .999; success rates are reported with a 1% sensitivity check (relative sc
   ↳ ore at least .99). Convergence is the number of sweeps used when the best score first reached 99% of the run\u20
   ↳ 19s own final score. Scaling is the per-seed slope of relative score on log<sub>10</sub> <i>N</i> for <i>N</i> \
   ↳ u2265 13, and the log\u2013log slope of mean run time on <i>N</i>. Computational cost is the wall-clock time, re
   ↳ ported as descriptive information because the budget is fixed in distance evaluations. Mathematical optimality i
   ↳ s claimed only where a published proof exists; elsewhere a result is called the best score found.")
155      HD("Final Round: Design, Rules, and Timestamp", 2)
156      d.body("A last round is planned to answer the one question that earlier rounds left open, whether curvature
   ↳ changes the density deficit once the boundary share is in the model, and to test one original rule. Its hypothes
   ↳ es, decision rules, seeds, and analysis code are fixed in PREREGISTRATION_ROUND3.md before any run; Table 3 give
   ↳ s its decision rules and Table 4 its design. Everything that happened in earlier rounds is in Appendix A.")
157      _r3 = open(OUT + "/conf/ROUND3.sha256").read().split("\n")
158      F(d.table, "Decision Rules of the Final Round",
159        ["Question", "Decision rule"],
160        [["A1: curvature adds a deficit at a given boundary share", "Fit <i>\u03b4</i> = <i>a</i> + <i>\u03b2 b</i
   ↳ > + <i>g</i><sub>H</sub> [hyperbolic] + <i>g</i><sub>S</sub> [cap]; the 95% interval of <i>g</i><sub>H</sub> is
   ↳ from 10,000 bootstrap resamples of the 48 cells. Detectable if the interval excludes 0 and <i>g</i><sub>H</sub>
   ↳ \u2265 .02; no detectable effect if the interval lies within \u00b1.02; otherwise inconclusive. The same rule is
   ↳  applied to the cap term."],
161         ["A2: paired differences (descriptive)", "Mean difference in <i>\u03b4</i> between the hyperbolic disk (a
   ↳ nd the cap) and the Euclidean disk over the 48 cells, with a bootstrap interval."],
162         ["B1: the number of disks can be predicted", "<i>K</i><sub>pred</sub> = floor(<i>n</i><sub>ref</sub>(1 \u
   ↳ 2212 0.46 <i>b</i><sub>0</sub>) + 0.5), with <i>n</i><sub>ref</sub> the number of disks that would fit at the up
   ↳ per bound on the density and <i>b</i><sub>0</sub> the design boundary share. Supported if at least 80% of condit
   ↳ ions have |<i>K</i> \u2212 <i>K</i><sub>pred</sub>| \u2264 1, the mean absolute error is at most 1.0 disk, the m
   ↳ ean relative error is at most .10, and each space has a mean signed error within \u00b11 disk; not supported if
   ↳ fewer than 60% are within 1 or the mean absolute error exceeds 1.5; otherwise inconclusive."]],
163        "The coefficient 0.46 was fitted in an earlier round and is not refitted. With the residual spread of the
   ↳ earlier round, the expected half-width of the interval of <i>g</i><sub>H</sub> is about .03, so the round can sh
   ↳ ow an effect of .03 or more but may remain inconclusive; the plan states this before the data. Data: PREREGISTRA
   ↳ TION_ROUND3.md.",
164        aligns=["l", "l"], widths=[2.0, 4.5], split=True)
165      F(d.table, "Design of the Final Round",
166        ["Part", "Conditions", "Methods, budget, and seeds"],
167        [["A: curvature at equal area", "Three spaces (Euclidean disk, hyperbolic disk, spherical cap) \u00d7 eigh
   ↳ t areas (2, 3, 4, 5, 6, 8, 10, 12) \u00d7 six numbers of disks (8, 12, 16, 20, 24, 32) = 144 conditions in 48 ce
   ↳ lls of equal area and equal <i>n</i>.", "Polished gradient descent and L-BFGS-B; 1,500 sweeps; fresh seeds 501 t
   ↳ o 520; 5,760 runs."],
168         ["B: a rule for the number of disks", "Disk radii 0.25 and 0.35 \u00d7 four design boundary shares (0.375
   ↳  to 0.525) in the hyperbolic disk and the cap, and the four boundary shares in the Euclidean disk (no length sca
   ↳ le) = 20 conditions.", "Polished gradient descent; 1,500 sweeps; fresh seeds 601 to 620; the number of disks <i>
   ↳ K</i> is the largest <i>n</i> reached in at least 10 of 20 seeds."]],
```

```
      "The ball radius of each condition follows from its area (Part A) or from the design boundary share and the disk radius (Part B). Data files named in PREREGISTRATION_ROUND3.md.",
      aligns=["l", "l", "l"], widths=[1.4, 3.3, 1.8], split=True)
    d.body(f"<b>Timestamp.</b> The preregistration file has the SHA-256 hash {_r3[0].split()[0]}, recorded by the author on {_r3[1].split(': ')[1].replace('T', ' at ').replace('Z', '')} UTC. A hash recorded by the author is not an independent time stamp. Before any run of the final round, the author will email the preregistration file, this plan, and the earlier preregistration files to the research adviser, and the date of that email is the time stamp; no run is started before it is sent. Any deviation will be listed in the Research Paper. This plan was finished before the Research Paper, and its body will not be changed after the final round has run; later events are recorded in the paper.")
    HD("Validation Tests", 2)
    d.body("The tests in Table 5 are executed before the experiments that depend on the code they check, and those experiments proceed only if every test passes. No test result is reported unless the test was executed.")
    F(d.table, "Validation Tests and Pass Criteria",
      ["Test group", "How it is performed", "Pass criterion"],
      [["Sphere distance", "300 pairs vs. arctangent form; closed forms", "Error < 10<sup>\u22129</sup>; closed forms < 10<sup>\u221212</sup>"],
       ["Hyperbolic distance", "Poincaré formula; law of cosines; area integral", "Relative error < 10<sup>\u2127</sup>; others < 10<sup>\u22126</sup>"],
       ["Euclidean control", "Distance of (0,0), (3,4); 7-disk hexagon", "Error < 10<sup>\u221212</sup>"],
       ["Spherical cap", "Poles, area, sampling, Euclidean limit", "Area < 10<sup>\u22126</sup>; limit within 0.2%"],
       ["Metric axioms", "Symmetry; 4,000 triples per space", "Asymmetry < 10<sup>\u221212</sup>; triangle < 10<sup>\u22129</sup>"],
       ["Exponential map", "Step length; point stays on manifold", "Error < 10<sup>\u22127</sup>"],
       ["Gradients", "Analytic gradient vs. finite differences", "Difference < 10<sup>\u2125</sup>"],
       ["Known Tammes optima", "Three polyhedra; optimizers, seeds 9101\u20139105", "Gap < 0.002 rad"]],
      "A failing test blocks the experiments. Recovery by each optimizer is hypothesis H3, not a validation criterion.",
      aligns=["l", "l", "l"], widths=[1.5, 2.9, 2.1], split=True)
    # ============================================================ Risk and safety
    HD("Risk and Safety", 1)
    d.body("The project is computational, so its risks are procedural. This section lists approvals and the measures that protect data integrity.")
    HD("Hazards and Approvals", 2)
    d.body("The project uses no hazardous materials, human participants, or vertebrate animals. It requires a risk assessment: Form 3 (Risk Assessment) is required for all projects, together with Form 1, Form 1A, and Form 1B and this Research Plan. Form 7 is required only if the project continues earlier research.")
    HD("Disclosure of AI Assistance and Researcher\u2019s Contribution", 2)
    d.body("Claude, an AI assistant developed by Anthropic, was used to assist with the writing of this document, including drafting, grammar checking, and language refinement. Other uses of AI assistance in this project, including its use in programming and analysis, are described in the separate AI Disclosure Agreement submitted to the research adviser and the Science Research Committee. GitHub was used to support code management and access to programming resources. The author\u2019s original contributions include the selection of the research problem, the formulation of the research questions, and the decisions regarding the research design. The author personal
```

```
   ↳ ly ran and checked the code, verified the results, and can explain and defend the programming procedures, method
   ↳ s, and findings. The author accepts full responsibility for the accuracy and integrity of every statement, numer
   ↳ ical result, citation, and conclusion presented in this research. The code and data are provided to support repr
   ↳ oducibility and independent verification of the results.")
193        HD("Data Integrity, Backups, and Reproducibility", 2)
194        d.body("One master script, build_all.py, runs the tests, experiments, statistics, figures, and documents wit
   ↳ h fixed seeds. Every result is saved to a CSV file; SHA-256 hashes of the delivered files are listed in MANIFEST
   ↳ .sha256 and can be checked with sha256sum. Copies of the code and data are to be kept in at least two separate p
   ↳ laces. Software versions are listed in the code listing, README.md describes every column of every file, and all
   ↳  results, including failures and negative findings, are retained.")
195
196        # ============================================================ Data analysis
197        HD("Data Analysis", 1)
198        d.body("This section states exactly which observations are compared, which tests are used and why, how their
   ↳  assumptions are checked, and how significance is decided.")
199        HD("Comparisons and Tests", 2)
200        d.body(f"For each condition the observations are the {NSEED} final scores of each algorithm; scores of the s
   ↳ ame seed come from the same starting configuration and form a block. The Friedman test compares the three algori
   ↳ thms across the {NSEED} blocks, and the Wilcoxon signed-rank test (Wilcoxon, 1945) compares each pair on the {NS
   ↳ EED} within-seed differences. Nonparametric paired tests are chosen because scores are bounded, are tied or clus
   ↳ tered at local optima, and are not normally distributed. Effect sizes are Kendall\u2019s <i>W</i> for the Friedm
   ↳ an test and the matched-pairs rank-biserial correlation with the standardized mean difference <i>d<sub>z</sub></
   ↳ i> for pairs. Means carry 95% confidence intervals based on the <i>t</i> distribution; with 30 seeds these inter
   ↳ vals are approximate for non-normal scores and are used descriptively.")
201        HD("Assumptions, Multiplicity, and Significance", 2)
202        d.body("The tests assume that blocks are independent, which holds because each seed gives an independent ran
   ↳ dom start. The Wilcoxon test also assumes that paired differences are symmetric about their median; this is chec
   ↳ ked by the skewness of each set of differences (an absolute value above 1 is flagged), and the shares of zero di
   ↳ fferences are reported. Tests are corrected with the Holm procedure (Holm, 1979), where one family is all tests
   ↳ of the same kind within one space, and the hypothesis tests are corrected across their own pairs. A result is st
   ↳ atistically significant when the Holm-adjusted <i>p</i> is below .05. Practical importance is judged separately:
   ↳  a difference counts as practically important when its mean is at least 0.5% of the reference value, so a signif
   ↳ icant but smaller difference is reported as such.")
203
204        HD("Properties of the Algorithms and Supplementary Analyses", 2)
205        d.body("Only some properties of the algorithms are proven. Proven: the soft-minimum energy lies between \u22
   ↳ 12ln \u2113<sub>min</sub> and \u2212ln \u2113<sub>min</sub> + (ln <i>m</i>)/<i>p</i>, so it approaches the true
   ↳ objective as <i>p</i> grows; the gradient is checked against finite differences. Not proven: convergence of any
   ↳ of the three optimizers to a global optimum, and any statement that one is better in general; such statements ar
   ↳ e empirical and are limited to the conditions run. Supplementary analyses, all exploratory and reported separate
   ↳ ly in the paper, are planned or have been run for: (a) gradient descent followed by a max-min polishing step (a
   ↳ sequential linear program whose cost is not charged in the sweep count), with wall-clock time reported beside eq
   ↳ ual-sweep results, because simulated annealing costs much more time per sweep; (b) sensitivity to budget, to the
   ↳  soft-minimum exponent and annealing schedule, and to the starting configuration; (c) an equal-area version of t
   ↳ he ball problem and larger balls; and (d) a check of scores against an independent computation. Parameter values
```

```
    named in this plan are defaults; the ranges tried are listed in the paper.")
    FL()
    d.new_page()
    HD("References", 1)
    for r in sorted(REFS, key=lambda s: s.replace("ö", "o").lower()):
        d.reference(r)
    d.new_page()
    HD("Appendix A: History of Work Done", 1)
    d.body("This appendix records what was done before the final round, in the order it happened, with the dates and hashes that the author recorded. It is a record of past work, not a plan; the plan is the body of this document. The Research Paper reports the results.")
    HD("Version History and Timing", 2)
    d.body("The plan was finalized after results existed, so the decision rules in Table 1 are not pre-registered in the strict sense; the dated history below states what was changed, and when, after results had been seen. Only the confirmatory round (version 3), the mini-round (version 4), and the follow-up (version 5) are preregistered, and one decision of the mini-round has been withdrawn (version 5.1), and each preregistration is dated by the hash given below, which the author recorded. The results of earlier versions are never changed by later ones.")
    d.body("<b>Version 1, October 25, 2026.</b> An earlier draft exists. This plan makes no claim that it preceded any experiment.")
    d.body("<b>Version 2, October 26, 2026.</b> States exact design values as executed. By then the 27 conditions of Problem A and of Problem B in <i>H</i><sup>2</sup> and <i>R</i><sup>2</sup> had been run, and the spherical-cap results had been seen. To the author\u2019s knowledge their analysis code was written before their results were inspected; the data package contains no dated record of the order. After those results were seen, the spherical-cap family (six settings) was added to study curvature in one common formulation; its analysis (Table 1, H1) was written into the analysis code before the cap runs were executed, but H1 as now worded is exploratory with respect to the earlier results. The 0.5% practical-importance threshold and the assumption checks were formalized after the first 27 conditions had been run; the compression method was simplified to a time-stepped scheme; the validation criterion for optimizer recovery was changed to the best of the three optimizers after compression failed an earlier per-optimizer version; and the wording of the second H1 criterion was clarified after the cap results were seen, from \u201cdiffers in at least two of the three spaces\u201d to \u201cis not the same in all three spaces\u201d (the two readings give the same decision).")
    d.body("<b>Version 2.1, October 28, 2026.</b> Statements about timing were corrected to match the records; the budget description in Procedures was clarified to match the code; and a sentence about work breaks was removed from Risk and Safety. After results were seen: the reference values for Tammes problems at <i>N</i> \u2265 30 were taken from published best-known values (Sloane\u2019s tables, via Cohn\u2019s compilation) rather than the best score found by the runs, and the relative scores were recomputed; the citations for the proven optima were corrected to Fejes T\u00f3th, Sch\u00fctte and van der Waerden, Danzer, Robinson, and Musin and Tarasov; all hypothesis results and RQ4 were labeled exploratory; supplementary exploratory analyses were added (gradient descent with a final polishing step, wall-clock time, sensitivity to budget, parameters and starting configuration, an equal-area comparison and larger balls); and an AI-assistance statement was added. The 27 conditions, the six cap settings, the three optimizers, the budget, and the decision rules in Table 1 were not changed.")
    d.body(f"<b>Version 3, October 29, 2026.</b> Adds a preregistered confirmatory round (see the next section), which uses fresh seeds, three budget levels, equal-area balls, added baselines, and a hardware record. Its preregistration file has the SHA-256 hash {PRE_HASH}, computed on {PRE_TIME} UTC before the pilot tuning and before
```

```
    ↳ any evaluation seed was run; the tuned parameters were hashed before the evaluation run. A hash recorded by the
    ↳ author is not an independent time stamp. No hypothesis, decision rule, or result of versions 1 to 2.1 was change
    ↳ d. Any further change will be recorded here and in the paper.")
220     _ad = open(OUT + "/conf/PREREG_ADDENDA.sha256").read().split("\n")
221     _h = lambda n: [l.split()[0] for l in _ad if l.endswith(n)][0]
222     _rt = open(OUT + "/conf/RETUNED.sha256").read().split("\n")
223     d.body(f"<b>Version 4, October 30, 2026.</b> After the results of the confirmatory round had been seen, the
    ↳ author recorded the following in a second file (PREREGISTRATION_ADDENDUM_1.md, SHA-256 {_h('ADDENDUM_1.md')}) an
    ↳ d a mini-round file (PREREGISTRATION_MINIROUND.md, SHA-256 {_h('MINIROUND.md')}), both hashed on {[l for l in _a
    ↳ d if l.startswith('hashed')][0].split(': ')[1].replace('T', ' at ').replace('Z', '')} UTC before any run they de
    ↳ scribe: (1) the Packomania values for the Euclidean conditions were entered after the first analysis, which had
    ↳ used the best score found (the decisions were the same with both references); (2) the candidate ranges of compre
    ↳ ssion and L-BFGS-B were extended, because their selected values lay at the edge of the old ranges, and both meth
    ↳ ods were run again (new tuned values hashed {[l for l in _rt if l.startswith('hashed')][0].split(': ')[1].replac
    ↳ e('T', ' at ').replace('Z', '')} UTC, SHA-256 {[l.split()[0] for l in _rt if l.endswith('tuned_params_retuned.js
    ↳ on')][0]}, before the evaluation); (3) the confirmatory round was run again with the same seeds and saved the fi
    ↳ nal configuration of every run, and the wall contact is reported for all runs; (4) a mini-round with a fixed dis
    ↳ k radius and a matched boundary share was added (seeds 301 to 320); and (5) the reference density is described a
    ↳ s an upper bound on the density, not as the density of a densest packing. The first preregistration file was not
    ↳  edited, and no result of versions 1 to 3 was changed. Because the author recorded the hashes, they are not inde
    ↳ pendent time stamps; the author reports having emailed the four preregistration files to the research adviser on
    ↳  October 30, 2026, after the runs of the confirmatory round and the mini-round and before the runs of the follow
    ↳ -up, and the adviser\u2019s email, not the author\u2019s hash, is the independent record (it cannot show that th
    ↳ e first three files preceded their runs).")
224     _fu = open(OUT + "/conf/FOLLOWUP.sha256").read().split("\n")
225     _r3h = open(OUT + "/conf/ROUND3.sha256").read().split()[0]
226     d.body(f"<b>Version 5, October 30, 2026.</b> Adds a preregistered follow-up to the mini-round (PREREGISTRATI
    ↳ ON_FOLLOWUP.md, SHA-256 {_fu[0].split()[0]}, hashed on {_fu[1].split(': ')[1].replace('T', ' at ').replace('Z',
    ↳ '')} UTC before its runs). It corrects a flaw of the mini-round: the Euclidean plane has no length scale, so the
    ↳  Euclidean conditions at disk radii 0.30 and 0.40 were the same problem and the six Euclidean conditions were th
    ↳ ree, which also overstated the degrees of freedom of rule M2. The follow-up uses five design boundary shares, th
    ↳ ree disk radii for the hyperbolic disk and the cap, one Euclidean condition per boundary share, seeds 401 to 420
    ↳ , and two methods (polished gradient descent and L-BFGS-B) for the number of disks <i>K</i>. No rule, result, or
    ↳  hash of versions 1 to 4 was changed; the mini-round decisions are reported as they were, with a sensitivity ana
    ↳ lysis without the duplicates.")
227     _cr = open(OUT + "/conf/CORRECTION.sha256").read().split("\n")
228     d.body(f"<b>Version 5.1, October 30, 2026 (a withdrawal).</b> The decision of rule M2 of the mini-round, \u2
    ↳ 01cno detectable curvature effect\u201d (hyperbolic term 0.007, interval \u22120.032 to 0.046, 13 degrees of fre
    ↳ edom), is withdrawn. The Euclidean conditions at disk radii 0.30 and 0.40 are the same problem, so the 18 condit
    ↳ ions were 15 and the degrees of freedom 10; without the duplicates (a post hoc analysis, decided after the resul
    ↳ t was seen) the interval is \u22120.058 to 0.060 and the decision is inconclusive. Earlier text, including the p
    ↳ aper written before this correction, said that no curvature effect larger than .05 was found in the mini-round;
    ↳ that statement is withdrawn. The correction note (CORRECTION_NOTE_1.md, SHA-256 {_cr[0].split()[0]}, hashed {_cr
    ↳ [1].split(': ')[1].replace('T', ' at ').replace('Z', '')} UTC, recorded by the author) states this, and the mini
    ↳ -round decisions as originally computed are kept in mini_decisions.json and reported next to the corrected analy
```

```
   ↳ sis.")
229      d.body("The runs of the follow-up began about one minute after the hash of its file was recorded (23:34 UTC
   ↳ on October 29, 2026; run times in FOLLOWUP_RUNTIMES.txt), and the replacement boundary shares and the second met
   ↳ hod were in that file; the author reports that the four files had been emailed to the research adviser before th
   ↳ e runs began. F1 to F3 are therefore preregistered by the author\u2019s hash, and the adviser\u2019s email is th
   ↳ e independent record of the date of the files.")
230      d.body(f"<b>Version 6, October 30, 2026.</b> The plan was restructured so that its body looks forward: it st
   ↳ ates the hypotheses, decision rules, seeds, and timestamp of one final round (PREREGISTRATION_ROUND3.md, SHA-256
   ↳  {_r3h}), and the history of earlier work, including every version above, is kept in this appendix. The final ro
   ↳ und was preregistered to settle the curvature question that three earlier rounds left inconclusive (more equal-a
   ↳ rea conditions, with the bootstrap taken over cells of equal area and equal number of disks) and to test a rule
   ↳ for the number of disks. No rule, result, or hash of versions 1 to 5.1 was changed.")
231      # ---- confirmatory round
232      HD("Confirmatory Round", 2)
233      d.body("This section describes a second round of experiments whose hypotheses, thresholds, and decision rule
   ↳ s were written and hashed before any of its evaluation seeds was run. The full text is PREREGISTRATION.md in the
   ↳  data package.")
234      HD("Prediction and Design", 3)
235      d.body("Curvature and area are confounded in the first study, because a ball of a given radius has a differe
   ↳ nt area in each space. The confirmatory round therefore compares balls of equal area. For <i>n</i> disks of radi
   ↳ us <i>r</i> in a ball of radius <i>R</i>, the density deficit is <i>\u03b4</i> = 1 \u2212 [<i>n</i> Area(<i>r</i
   ↳ >) / Area(<i>R</i>)] / <i>\u03c1</i>*, where <i>\u03c1</i>* is an upper bound on the density of any packing of d
   ↳ isks of radius <i>r</i> in the whole space (Fejes T\u00f3th, 1943; B\u00f6r\u00f6czky, 1978), and the boundary s
   ↳ hare is <i>b</i> = 1 \u2212 Area(<i>R</i> \u2212 <i>r</i>) / Area(<i>R</i>). The prediction is that <i>\u03b4</i
   ↳ > increases with <i>b</i> in every space, so that most of the deficit is a boundary effect and not a curvature e
   ↳ ffect.")
236      d.body("The design uses fresh evaluation seeds 101 to 120, three budget levels of 500, 1,500, and 3,000 swee
   ↳ ps as part of the main design, and six methods: gradient descent, gradient descent with a polishing step, simula
   ↳ ted annealing, compression, SciPy L-BFGS-B (Virtanen et al., 2020; Byrd et al., 1995), and basin hopping (Wales
   ↳ & Doye, 1997). Each method has one tuned parameter with six candidate settings, tuned on the same pilot conditio
   ↳ ns and pilot seeds 9101 to 9104. The 33 conditions are the Tammes problem for <i>N</i> = 9, 13, 14, 24, 30, 40,
   ↳ and 60; balls of equal area 3, 6, and 12 with 12 and 24 disks in the spherical cap, the Euclidean disk, and the
   ↳ hyperbolic disk; and hyperbolic balls of radius 2, 3, 6, and 8 with 24, 36, 50, and 70 disks, each paired with a
   ↳  Euclidean disk of the same area. Equal area in all three spaces is possible only up to 4\u03c0, so the large ba
   ↳ lls compare two spaces. The parameter values are tunable defaults, not requirements. The hardware and software a
   ↳ re recorded before the run, and no run is excluded.")
237      d.body("Reference values are the proven optimum or the published best-known value (Sloane\u2019s tables) for
   ↳  the Tammes problem, the Packomania value (Specht, n.d.) for the ten Euclidean conditions with 12, 24, 36, 50, o
   ↳ r 70 disks (entered after the first analysis), and otherwise the best score found in the round, which is stated
   ↳ wherever it is used. As an original element, a count-based version of the disk problem asks for the largest numb
   ↳ er of disks of a fixed radius that fit in a ball of a given area; it is exploratory and has no decision rule.")
238      HD("Decision Rules", 3)
239      F(d.table, "Decision Rules of the Confirmatory Round",
240        ["Question", "Decision rule (1,500 sweeps unless stated)"],
241        [["C1: the density deficit follows the boundary share (26 ball conditions)", "Supported if Spearman <i>r</
```

```
   ↳ i><sub>s</sub> between <i>b</i> and <i>\u03b4</i> is at least .70 and the one-sided permutation <i>p</i> is belo
   ↳ w .05; not supported if <i>r</i><sub>s</sub> is below .30; otherwise inconclusive."],
242        ["C2: curvature adds a deficit beyond the boundary share", "Least-squares fit of <i>\u03b4</i> on <i>b</i
   ↳ > and indicators of space; a detectable effect if the 95% bootstrap interval of the hyperbolic term excludes 0 a
   ↳ nd the term is at least .02; no effect if the whole interval lies within \u00b1.02."],
243        ["C3: the ranking of gradient descent and annealing depends on the space (18 equal-area conditions)", "De
   ↳ pendent if the Kruskal\u2013Wallis <i>p</i> is below .05; independent if each 90% bootstrap interval of a differ
   ↳ ence of medians lies within \u00b1.01."],
244        ["C4: stronger baselines matter; polished gradient descent is competitive", "L-BFGS-B or basin hopping be
   ↳ ats both gradient descent and annealing in at least half of the 33 conditions; polished gradient descent ranks f
   ↳ irst or second in at least half."],
245        ["C5: the ranking of the methods is stable across budgets", "Median Kendall\u2019s <i>W</i> over the thre
   ↳ e budget levels is at least .80."],
246        ["C6: some method recovers the proven optimum for <i>N</i> = 9, 13, 14, 24", "The best share of seeds wit
   ↳ h relative score at least .999 is at least .90 for all four."]],
247       "All six questions are separate, are reported whatever the outcome, and are not corrected for one another.
   ↳  Analyses not listed here are exploratory. Data: PREREGISTRATION.md.",
248       aligns=["l", "l"], widths=[2.4, 4.1], split=True)
249     HD("Mini-Round With a Matched Boundary Share", 3)
250     d.body("The main round could not separate curvature from the boundary effect (C2 was inconclusive), and its
   ↳ boundary share depends on the radius each method finds. A mini-round fixes the disk radius <i>r</i><sub>0</sub>
   ↳ (0.30 or 0.40) and a design boundary share <i>b</i><sub>0</sub> (0.40, 0.45, or 0.50) and chooses the radius of
   ↳ each ball so that <i>b</i><sub>0</sub> is the same in the Euclidean disk, the hyperbolic disk, and the spherical
   ↳  cap (18 conditions). For each ball, <i>K</i> is the largest number of disks for which polished gradient descent
   ↳  at 1,500 sweeps reaches radius <i>r</i><sub>0</sub> in at least 10 of 20 fresh seeds (301 to 320), and the fixe
   ↳ d-radius deficit is <i>\u03b4</i><sub>0</sub> = 1 \u2212 <i>K</i> Area(<i>r</i><sub>0</sub>) / (Area(<i>R</i>) <
   ↳ i>\u03c1</i>*). The full text is PREREGISTRATION_MINIROUND.md.")
251     F(d.table, "Decision Rules of the Mini-Round",
252       ["Question", "Decision rule"],
253       [["M1: the fixed-radius deficit follows the design boundary share", "Supported if Spearman <i>r</i><sub>s<
   ↳ /sub> between <i>b</i><sub>0</sub> and <i>\u03b4</i><sub>0</sub> is at least .50 and the one-sided permutation <
   ↳ i>p</i> (20,000 permutations) is below .05; not supported if <i>r</i><sub>s</sub> is below .20; otherwise inconc
   ↳ lusive."],
254        ["M2: curvature adds a deficit at a matched boundary share", "Least-squares fit of <i>\u03b4</i><sub>0</s
   ↳ ub> on <i>b</i><sub>0</sub>, indicators of the hyperbolic disk and the cap, and <i>r</i><sub>0</sub> (13 degrees
   ↳  of freedom); a detectable effect if the 95% <i>t</i> interval of the hyperbolic term excludes 0 and the term is
   ↳  at least .05 in size; no detectable effect if the whole interval lies within \u00b1.05; otherwise inconclusive.
   ↳  The preregistered fit counted 13 degrees of freedom, which included three duplicated Euclidean conditions; the
   ↳ decision it produced is withdrawn (Version 5.1)."],
255        ["M3: a boundary-loss model describes the other spaces", "<i>\u03b4</i><sub>0</sub> = <i>\u03bb b</i><sub
   ↳ >0</sub>, with <i>\u03bb</i> fitted on the six Euclidean conditions and not refitted; the model describes the ot
   ↳ her twelve conditions if their mean absolute error is at most .05, and does not if it exceeds .10; otherwise inc
   ↳ onclusive."]],
256       "The three rules are separate and are not corrected for the six rules of the main round. Analyses not list
   ↳ ed here are exploratory. The decision of M2 is withdrawn (Version 5.1). Data: PREREGISTRATION_MINIROUND.md.",
```

```
      aligns=["l", "l"], widths=[2.4, 4.1], split=True)
    HD("Follow-Up to the Mini-Round", 3)
    d.body("The mini-round was inconclusive for M1, and its Euclidean conditions at two disk radii were the same problem (the Euclidean plane has no length scale). The follow-up uses five design boundary shares (0.35 to 0.55), three disk radii (0.20, 0.30, and 0.40) for the hyperbolic disk and the cap, one Euclidean condition per boundary share, and fresh seeds 401 to 420 (33 conditions). The number of disks <i>K</i> is computed with polished gradient descent and with L-BFGS-B, and the larger value is used because each is a lower bound. The full text is PREREGISTRATION_FOLLOWUP.md.")
    F(d.table, "Decision Rules of the Follow-Up",
      ["Question", "Decision rule"],
      [["F1: the fixed-radius deficit follows the design boundary share (33 conditions)", "Same thresholds as M1: supported if Spearman <i>r</i><sub>s</sub> is at least .50 and the one-sided permutation <i>p</i> is below .05; not supported if <i>r</i><sub>s</sub> is below .20; otherwise inconclusive."],
       ["F2: curvature adds a deficit at a matched boundary share", "Same rule as M2, fitted with indicators of the two radii other than 0.30 (27 degrees of freedom): detectable if the 95% <i>t</i> interval of the hyperbolic term excludes 0 and the term is at least .05; no detectable effect if the interval lies within \u00b1.05; otherwise inconclusive."],
       ["F3: a boundary-loss model describes the other spaces", "Same rule as M3, with <i>\u03bb</i> fitted on the five Euclidean conditions and used to predict the other 28; at most .05 describes, more than .10 does not. The model is a description, not an explanation."]],
      "The three rules are separate and are not corrected for the nine rules of the earlier rounds. Analyses not listed here, including the second method alone and the mini-round without duplicates, are exploratory. Data: PREREGISTRATION_FOLLOWUP.md.",
      aligns=["l", "l"], widths=[2.4, 4.1], split=True)
    d.body("Tables 6, 7, and 8 give the decision rules of the confirmatory round, the mini-round, and the follow-up. Analyses outside these rules, including the count-based problem and every robustness check, are exploratory and are labeled so in the paper.")

    FL()
    return d


def main():
    d0 = build({"toc": [], "tables": []})
    probe = [(l, t, 0) for l, t in d0.headings]
    tabs = [(n, t, 0) for k, n, t in d0.captions if k == "Table"]
    pg = {"toc": probe, "tables": tabs}
    for it in range(4):
        d = build(pg)
        path = OUT + "/Research_Plan.docx"
        d.save(path)
        pdf = to_pdf(path, OUT)
        pages = pdf_pages(pdf)
        body0 = next(i for i in range(1, len(pages)) if any(l.strip().startswith("Algorithmic Optimization") for l in pages[i].split("\n")[:4]))
```

```
        newtoc = []
        for lvl, text, _ in pg["toc"]:
            if text == "List of Tables":
                pnum = find_exact(pages, [text], start_page=2)[0]
            else:
                pnum = find_exact(pages, [text], start_page=body0)[0] or find_pages(pages, [text], start_page=bo
↳ dy0)[0]
            newtoc.append((lvl, text, pnum))
        newtab = [(n, t, find_exact(pages, [f"Table {n}"], start_page=body0)[0]) for n, t, _ in pg["tables"]]
        new = {"toc": newtoc, "tables": newtab}
        if new == pg:
            break
        pg = new
    import omml
    omml.MODE = "omml"           # final .docx carries native Word equations; the PDF above used typeset images
    build(pg).save(path)
    print("plan pages:", len(pages), "| missing:", [e for k in pg for e in pg[k] if not e[2]])


if __name__ == "__main__":
    main()
```

make_paper.py

Builds the Research Paper; every number and table value is read from the CSV files.

```
"""make_paper.py -- builds Research_Paper.docx / .pdf (version 2).  Every number comes from the CSV files
or the test log; table and figure numbers come from one registry so that text and floats cannot disagree."""
import os
import re
import sys
import json
import confirm_design as _C
import inspect
import platform
import importlib.metadata as md
import numpy as np
import pandas as pd
from docbuilder import *
import algorithms as A
import experiments as E

```

```
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))   # folder that contains code/ and out/
OUT = os.path.join(ROOT, "out")
DATA = OUT + "/data"
FIG = OUT + "/figures"
PAPER_DATE = "December 3, 2026"

runs = pd.read_csv(f"{DATA}/runs_raw.csv")
summ = pd.read_csv(f"{DATA}/summary_by_condition.csv")
pw = pd.read_csv(f"{DATA}/test_pairwise_wilcoxon.csv")
fr = pd.read_csv(f"{DATA}/test_friedman.csv")
hyp = pd.read_csv(f"{DATA}/hypothesis_tests.csv")
over = pd.read_csv(f"{DATA}/overall_by_space.csv")
caps = pd.read_csv(f"{DATA}/capacity_fixed_radius.csv")
area = pd.read_csv(f"{DATA}/area_fractions.csv")
bench = pd.read_csv(f"{DATA}/benchmark_values.csv")
rte = pd.read_csv(f"{DATA}/runtime_scaling_exponents.csv")
asm = pd.read_csv(f"{DATA}/assumption_checks.csv")
cfgall = pd.read_csv(f"{DATA}/configurations.csv")
unit_txt = open(f"{OUT}/unit_test_output.txt").read()
ALG = ["gd", "sa", "ls"]
AB = {"gd": "GD", "sa": "SA", "ls": "CP"}
SPACES = ["S2cap", "R2", "H2"]
SLAB = {"S2": "<i>S</i><sup>2</sup>", "S2cap": "Cap", "R2": "<i>R</i><sup>2</sup>", "H2": "<i>H</i><sup>2</sup>"
}
NS2 = sorted(runs[runs.space == "S2"].n.unique())
BALL = E.BALL_RN
SEEDS = runs.seed.nunique()
BUDGET = E.BUDGET_SWEEPS
NRUN = len(runs)
NCOND = runs.condition_id.nunique()
_kn = sorted(int(x) for x in runs[(runs.space == "S2") & (runs.reference_type == "published best known")].n.uniq
ue())
_bf = sorted(int(x) for x in runs[(runs.space == "S2") & (runs.reference_type == "best found")].n.unique())


def _nlist(v):
    return (", ".join(map(str, v[:-1])) + (" and " if len(v) > 1 else "") + str(v[-1])) if v else ""


KNOWN_TXT = ("<i>N</i> = " + _nlist(_kn) + "; Sloane et al., n.d.; Cohn, n.d.") if _kn else "none was entered"
STARNOTE = ((("a published best-known value (<i>N</i> = " + _nlist(_kn) + ")") if _kn else "") + ("; " if _kn an
d _bf else "")
            + (("the best value found by any run (<i>N</i> = " + _nlist(_bf) + ")") if _bf else ""))

TABLES = ["stages", "area", "params", "vtests", "s2res", "ballres", "capacity", "sweeps", "pairwise", "hyp", "gd
```

```
    ↳ p", "timing", "equal", "cmeth", "cdec", "crel", "cdef", "cbud", "ccount", "cwall", "r3a", "r3b", "cmini", "cmini
    ↳ 2", "cfollow"]
FIGS = ["rank", "framework", "sphere", "curv", "conv", "perf", "runtime", "dist", "cdef", "cbud", "r3def"]
APP = {"budget": "A1", "sens": "A2", "spread": "A3"}        # sensitivity tables, in Appendix A
CONFDIR = OUT + "/conf"
TN = {k: i + 1 for i, k in enumerate(TABLES)}
FN = {k: i + 1 for i, k in enumerate(FIGS)}



def p10(x):
    """Format a power of ten such as 1e-05 as 10<sup>-5</sup>."""
    import math
    e = int(round(math.log10(x)))
    return f"10<sup>\u2212{abs(e)}</sup>" if e < 0 else f"10<sup>{e}</sup>"

def Tb(k):
    return f"Table {APP[k]}" if k in APP else f"Table {TN[k]}"


def Fg(k):
    return f"Figure {FN[k]}"


# ------------------------------------------------------------ formatting helpers
def f(x, d=3, lead=True):
    s = f"{x:.{d}f}"
    if not lead and s.startswith("0."):
        s = s[1:]
    if not lead and s.startswith("-0."):
        s = "-" + s[2:]
    return s.replace("-", "\u2212")


def pstr(p):
    return "<i>p</i> < .001" if p < 0.001 else "<i>p</i> = " + f(p, 3, lead=False)


def S(space, n, alg, R=None):
    g = summ[(summ.space == space) & (summ.n == n) & (summ.algorithm == alg)]
    if R is not None:
        g = g[g.R == R]
    return g.iloc[0]


```

```
def PW(space, n, a, b, R=None):
    g = pw[(pw.space == space) & (pw.n == n) & (pw.alg_a == a) & (pw.alg_b == b)]
    if R is not None:
        g = g[g.R == R]
    return g.iloc[0]


def ov(space, alg, col):
    return float(over[(over.space == space) & (over.algorithm == alg)][col].iloc[0])


def wins(space, a, b, who):
    g = pw[(pw.space == space) & (pw.alg_a == a) & (pw.alg_b == b)]
    return int((g.winner == who).sum()), len(g)


def words(s):
    return len(re.findall(r"[A-Za-z0-9\u2212\-\.]+", re.sub(r"<[^>]+>", "", s)))


def best_count(space, alg):
    k = 0
    for R, n in BALL:
        m = {a: S(space, n, a, R=R).mean_score for a in ALG}
        k += int(max(m, key=m.get) == alg)
    return k


def sci(x):
    m, e = f"{x:.1e}".split("e")
    return f"{m} \u00d7 10<sup>\u2212{abs(int(e))}</sup>" if int(e) < 0 else f"{m} \u00d7 10<sup>{int(e)}</sup>"


def test_err(pattern):
    m = re.search(pattern + r".*?\[(?:max rel err|max err) ([0-9.e+-]+)", unit_txt)
    return float(m.group(1))


dflt = {n: {k: v.default for k, v in inspect.signature(fn).parameters.items() if v.default is not inspect._empty
}
        for n, fn in (("gd", A.run_gd), ("sa", A.run_sa), ("ls", A.run_ls))}

# ---------------------------------------------------------- derived headline numbers
mrel = {sp: {a: ov(sp, a, "mean_relative_score") for a in ALG} for sp in ("S2", "S2cap", "R2", "H2")}
mrank = {sp: {a: ov(sp, a, "mean_rank") for a in ALG} for sp in ("S2", "S2cap", "R2", "H2")}
```

```
best_rank = {sp: min(ALG, key=lambda a: mrank[sp][a]) for sp in mrank}
NAMEL = {"gd": "gradient descent", "sa": "simulated annealing", "ls": "compression"}
assert all(mrank[sp]["ls"] > 2.9 for sp in mrank)          # compression ranks last in every space
fr_sig, fr_n = int(fr.significant.sum()), len(fr)
H1o = hyp[(hyp.hypothesis == "H1") & (hyp.test == "gd-sa")].iloc[0]
H1c = hyp[(hyp.hypothesis == "H1c") & hyp.test.str.contains("across K")]
H1c_gdsa = H1c[H1c.test.str.startswith("gd-sa")].iloc[0]
H1c_h2 = hyp[(hyp.hypothesis == "H1c") & hyp.test.str.contains("H2 minus R2")].iloc[0]
H1c_cap = hyp[(hyp.hypothesis == "H1c") & hyp.test.str.contains("S2cap minus R2")].iloc[0]
h1_a = bool((H1c.p_holm < 0.05).any())                          # criterion (a)
h1_b = len(set(best_rank[s] for s in SPACES)) > 1               # criterion (b)
H1_DECISION = ("Supported" if (h1_a and h1_b) else ("Partly supported" if (h1_a or h1_b) else "Not supported"))
↳ + " (exploratory)"
h2_gd = hyp[(hyp.hypothesis == "H2") & (hyp.test == "slope ls - slope gd")].iloc[0]
h2_sa = hyp[(hyp.hypothesis == "H2") & (hyp.test == "slope ls - slope sa")].iloc[0]
ls_higher = [(h2_gd.mean_diff > 0 and h2_gd.p_holm < .05), (h2_sa.mean_diff > 0 and h2_sa.p_holm < .05)]
H2_DECISION = ("Supported" if all(ls_higher) else ("Partly supported" if any(ls_higher) else "Not supported")) +
↳  " (exploratory)"
h3 = {a: hyp[(hyp.hypothesis == "H3") & (hyp.test.str.startswith(a + ":"))].iloc[0] for a in ALG}
n3 = sum(int(h3[a].supported) for a in ALG)
H3_DECISION = ("Supported" if n3 == 3 else ("Partly supported" if n3 > 0 else "Not supported")) + " (exploratory
↳ )"
s2sm = runs[(runs.space == "S2") & (runs.n <= 14)]
rate1 = {a: s2sm[s2sm.algorithm == a].groupby("n")["success_1pct"].mean() for a in ALG}
below1 = {a: [int(n) for n, v in rate1[a].items() if v < 0.9] for a in ALG}
gd4 = runs[(runs.space == "S2") & (runs.n == 4) & (runs.algorithm == "gd")]
stuck = int((gd4.score < 1.6).sum())
sc_ls_small = {n: S("S2", n, "ls").success_rate_0p1pct for n in (4, 5, 6)}
vers = {p: md.version(p) for p in ("numpy", "scipy", "pandas", "matplotlib", "python-docx", "Pygments", "pdfplum
↳ ber", "Pillow")}
PYV = platform.python_version()


def table_group_counts():
    lines = [l for l in unit_txt.split("\n") if l.startswith("PASS") or l.startswith("FAIL")]

    def cnt(pred):
        sel = [l for l in lines if pred(l)]
        return len(sel), sum(1 for l in sel if l.startswith("PASS"))
    return [
        ("Sphere distance", "300 random pairs against the arctangent form; closed forms", "Error < 10<sup>\u2212
↳ 9</sup>; closed forms < 10<sup>\u221212</sup> (antipode < 10<sup>\u22127</sup>)",
         *cnt(lambda l: "S2 distance matches" in l or "S2 closed-form" in l)),
        ("Hyperbolic distance", "Poincaré formula, radius tanh(<i>s</i>/2), law of cosines, area integral", "Rel
↳ ative error < 10<sup>\u22127</sup>; others < 10<sup>\u22129</sup>, 10<sup>\u22128</sup>, 10<sup>\u22126</sup>",
```

```
          *cnt(lambda l: "H2 hyperboloid" in l or "H2 point" in l or "H2 distance equals" in l or "H2 disk area" in l)),
        ("Euclidean control", "Distance of (0,0) and (3,4); 7-disk hexagon in a ball of radius 3", "Error < 10<sup>\u221212</sup>", *cnt(lambda l: "R2 distance of" in l or "R2 hexagonal" in l)),
        ("Spherical cap", "Pole, equator, antipode, area, sampling, Euclidean limit, clipping", "Area error < 10<sup>\u22126</sup>; sampling within 4 s.e.; limit within 0.2%", *cnt(lambda l: "S2 cap:" in l)),
        ("Metric axioms", "Symmetry and 4,000 random triples per space", "Asymmetry < 10<sup>\u221212</sup>; triangle violation < 10<sup>\u22129</sup>", *cnt(lambda l: "symmetry and triangle" in l)),
        ("Exponential map", "Step length equals vector length; point stays on the manifold", "Error < 10<sup>\u2127</sup>; manifold error < 10<sup>\u22129</sup>", *cnt(lambda l: "exponential map" in l)),
        ("Gradients", "Analytic Riemannian gradient against central finite differences", "Difference < 10<sup>\u22125</sup> max(1, |gradient|)", *cnt(lambda l: "analytic Riemannian gradient" in l)),
        ("Known Tammes optima", "Three polyhedra; optimizers on seeds 9101 to 9105", "Distance error < 10<sup>\u221212</sup>; best gap < 0.002 rad", *cnt(lambda l: "Closed-form Tammes" in l or "Optimizer suite recovers" in l)),
    ]


REFS = [
    "Absil, P.-A., Mahony, R., & Sepulchre, R. (2008). <i>Optimization algorithms on matrix manifolds</i>. Princeton University Press.",
    "Byrd, R. H., Lu, P., Nocedal, J., & Zhu, C. (1995). A limited memory algorithm for bound constrained optimization. <i>SIAM Journal on Scientific Computing, 16</i>(5), 1190–1208.",
    "Cohn, H. (n.d.). <i>Spherical codes</i> [Table of best known spherical codes]. Massachusetts Institute of Technology. https://cohn.mit.edu/spherical-codes/",
    "Danzer, L. (1963). <i>Endliche Punktmengen auf der 2-Sphäre mit möglichst grossem Minimalabstand</i> [Habilitation thesis]. Universität Göttingen.",
    "Fejes Tóth, L. (1943). Über eine Abschätzung des kürzesten Abstandes zweier Punkte eines auf einer Kugelfläche liegenden Punktsystems. <i>Jahresbericht der Deutschen Mathematiker-Vereinigung, 53</i>, 66–68.",
    "Hajek, B. (1988). Cooling schedules for optimal annealing. <i>Mathematics of Operations Research, 13</i>(2), 311–329. https://doi.org/10.1287/moor.13.2.311",
    "Liu, D. C., & Nocedal, J. (1989). On the limited memory BFGS method for large scale optimization. <i>Mathematical Programming, 45</i>(1–3), 503–528.",
    "Musin, O. R., & Tarasov, A. S. (2015). The Tammes problem for N = 14. <i>Experimental Mathematics, 24</i>(4), 460–468. https://doi.org/10.1080/10586458.2015.1022842",
    "Robinson, R. M. (1961). Arrangement of 24 points on a sphere. <i>Mathematische Annalen, 144</i>, 17–48.",
    "Saff, E. B., & Kuijlaars, A. B. J. (1997). Distributing many points on a sphere. <i>The Mathematical Intelligencer, 19</i>(1), 5–11.",
    "Schütte, K., & van der Waerden, B. L. (1951). Auf welcher Kugel haben 5, 6, 7, 8 oder 9 Punkte mit Mindestabstand Eins Platz? <i>Mathematische Annalen, 123</i>, 96–124.",
    "Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., ... SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. <i>Nature Methods, 17</i>(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2",
```

```
    "Wales, D. J., & Doye, J. P. K. (1997). Global optimization by basin-hopping and the lowest energy structures of Lennard-Jones clusters containing up to 110 atoms. <i>The Journal of Physical Chemistry A, 101</i>(28), 5111–5116. https://doi.org/10.1021/jp970984n",
    "Specht, E. (n.d.). <i>Packomania: Circles in a circle</i> [Data set]. http://www.packomania.com/",
    "Sloane, N. J. A., Hardin, R. H., Smith, W. D., & others. (n.d.). <i>Tables of spherical codes</i> [Data set]. http://neilsloane.com/packings/",
    "Tammes, P. M. L. (1930). On the origin of number and arrangement of the places of exit on the surface of pollen-grains. <i>Recueil des Travaux Botaniques Néerlandais, 27</i>, 1–84.",
    "Böröczky, K. (1978). Packing of spheres in spaces of constant curvature. <i>Acta Mathematica Academiae Scientiarum Hungaricae, 32</i>, 243\u2013261. https://doi.org/10.1007/BF01902361",
    "Cannon, J. W., Floyd, W. J., Kenyon, R., & Parry, W. R. (1997). Hyperbolic geometry. In S. Levy (Ed.), <i>Flavors of geometry</i> (pp. 59\u2013115). Cambridge University Press.",
    "Holm, S. (1979). A simple sequentially rejective multiple test procedure. <i>Scandinavian Journal of Statistics, 6</i>(2), 65\u201370.",
    "Kirkpatrick, S., Gelatt, C. D., Jr., & Vecchi, M. P. (1983). Optimization by simulated annealing. <i>Science, 220</i>(4598), 671\u2013680. https://doi.org/10.1126/science.220.4598.671",
    "Lubachevsky, B. D., & Stillinger, F. H. (1990). Geometric properties of random disk packings. <i>Journal of Statistical Physics, 60</i>, 561\u2013583. https://doi.org/10.1007/BF01025983",
    "Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H., & Teller, E. (1953). Equation of state calculations by fast computing machines. <i>The Journal of Chemical Physics, 21</i>(6), 1087\u20131092. https://doi.org/10.1063/1.1699114",
    "Musin, O. R., & Tarasov, A. S. (2012). The strong thirteen spheres problem. <i>Discrete & Computational Geometry, 48</i>(1), 128\u2013141. https://doi.org/10.1007/s00454-011-9392-2",
    "Wilcoxon, F. (1945). Individual comparisons by ranking methods. <i>Biometrics Bulletin, 1</i>(6), 80\u201383. https://doi.org/10.2307/3001968",
]

DEC = json.load(open(CONFDIR + "/decisions.json"))
_FD = json.load(open(CONFDIR + "/followup_decisions.json"))["primary"]
_MD = json.load(open(CONFDIR + "/mini_decisions.json"))
_cr = pd.read_csv(f"{DATA}/conf_runs.csv")
_cr = _cr[(_cr.label == "Tammes N = 13")] if "label" in _cr.columns else _cr[_cr.condition_id == 1001]
_N13H, _N13N = int(_cr["success_0.1pct"].sum()), len(_cr)
_R3 = json.load(open(CONFDIR + "/round3_decisions.json"))
abstract = (
    f"This study compares optimization algorithms for dense packings on the sphere, the Euclidean plane, and the hyperbolic plane under one protocol with an equal budget and paired seeds. "
    f"In a preregistered confirmatory round (six methods, 33 conditions, three budgets; {DEC['n_runs']:,} runs), polished gradient descent had the best mean rank ({f(DEC['C4']['mean_rank_by_method']['gdp'], 2)}), and L-BFGS-B or basin hopping beat both gradient descent and simulated annealing in all {DEC['C4']['n_conditions']} conditions; compression ranked last. "
    f"The shortfall of a packing from the density bound rose with the share of the ball near its wall (Spearman {f(DEC['C1']['spearman'], 2, lead=False)}, partly circular because both come from the radius found; {f(_FD['F1']['spearman'], 2, lead=False)} at a fixed radius). "
    f"A final preregistered round of {_R3['A1']['n_conditions']} equal-area conditions found no curvature effect
```

```
    ↳  larger than .02 of the bound (hyperbolic {f(_R3['A1']['g_H'], 3)}, cap {f(_R3['A1']['g_S'], 3)}), though both i
    ↳ ntervals excluded zero. "
234     f"A rule, the number of disks at the bound times (1 \u2212 0.46 times the boundary share), predicted the num
    ↳ ber of disks that fit within one disk in all {_R3['B1']['n_conditions']} new conditions. "
235     f"No method reached a .90 success share for 13 points ({_N13H} of {_N13N} runs). Results concern the budgets
    ↳  and implementations tested and set no records."
236  )
237
238  KEYWORDS = "sphere packing, Tammes problem, hyperbolic plane, optimization algorithms, curvature"
239
240
241  def build(pg):
242      d = APADoc(title=TITLE)
243      d.title_page(PAPER_DATE)
244      d.new_page()
245      d.front_heading("Abstract")
246      p = d._para(indent_first=0.0, align=WD_ALIGN_PARAGRAPH.LEFT)
247      d.add_marked(p, abstract)
248      p = d._para(indent_first=0.5, align=WD_ALIGN_PARAGRAPH.LEFT)
249      set_run_font(p.add_run("Keywords:"), italic=True)
250      set_run_font(p.add_run(" " + KEYWORDS))
251      d.new_page()
252      d.contents_page("Table of Contents", "Chapter", pg["toc"], "toc", field_instr='TOC \\o "1-3" \\z \\u')
253      d.new_page()
254      d.contents_page("List of Figures", "Figure", pg["figs"], "list", in_toc=True)
255      d.new_page()
256      d.contents_page("List of Tables", "Table", pg["tabs"], "list", in_toc=True)
257      d.new_page()
258
259      floats = []
260
261      appfloats = []
262
263      def FF(kind, key, *a, **k):
264          (appfloats if key in APP else floats).append((kind, key, a, k))
265
266      def FL():
267          while floats:
268              kind, key, a, k = floats.pop(0)
269              if kind == "table":
270                  assert d.ntab + 1 == TN[key], (key, d.ntab, TN[key])
271                  d.table(*a, **k)
272              else:
273                  assert d.nfig + 1 == FN[key], (key, d.nfig, FN[key])
274                  d.figure(*a, **k)
```

```
275
276     def HD(text, level=1):
277         FL()
278         return d.heading(text, level)
279
280     p = d._para(align=WD_ALIGN_PARAGRAPH.CENTER)
281     set_run_font(p.add_run(TITLE), bold=True)
282
283     # ======================================================================== Introduction
284     HD("Introduction", 1)
285     d.body("This paper compares optimization methods for dense packings on the sphere, the Euclidean plane, and
    ↳ the hyperbolic plane: three methods in a first, exploratory study, and six in the preregistered rounds that foll
    ↳ owed. This section reviews what is known, states the gap, the questions, and the hypotheses, and separates estab
    ↳ lished knowledge from the contribution of this study.")
286     HD("Background and Related Work", 2)
287     d.body("On the unit sphere, the Tammes problem asks for <i>N</i> points whose smallest pairwise geodesic dis
    ↳ tance is as large as possible. The problem is named for Tammes (1930). Optimal configurations have been proved f
    ↳ or <i>N</i> = 3, 4, 6, and 12 (Fejes Tóth, 1943), <i>N</i> = 5, 7, 8, and 9 (Schütte & van der Waerden, 1951), <
    ↳ i>N</i> = 24 (Robinson, 1961), <i>N</i> = 10 and 11 (Danzer, 1963), <i>N</i> = 13 (Musin & Tarasov, 2012), and <
    ↳ i>N</i> = 14 (Musin & Tarasov, 2015). For most other <i>N</i>, the best known configurations come from computer
    ↳ search (Sloane et al., n.d.; Cohn, n.d.), and their optimality is unproven. Energy-minimization methods for spre
    ↳ ading many points over a sphere are surveyed by Saff and Kuijlaars (1997).")
288     d.body("Curvature changes the geometry of packing. In the Euclidean plane the area of a disk grows as the sq
    ↳ uare of its radius; in the hyperbolic plane it grows exponentially (Cannon et al., 1997), so most of the area of
    ↳  a large hyperbolic disk lies near its boundary. Böröczky (1978) proved density bounds for packings in spaces of
    ↳  constant curvature, where the definition of density is itself delicate. Computational methods for packing inclu
    ↳ de energy minimization, Monte Carlo and annealing methods (Kirkpatrick et al., 1983; Metropolis et al., 1953), a
    ↳ nd compression of growing disks (Lubachevsky & Stillinger, 1990).")
289     HD("Research Gap and Contribution", 2)
290     d.body("Established knowledge in this area includes the optimal Tammes configurations listed above, the area
    ↳  formulas of the three plane geometries, and the existence of the three families of methods. I did not find a st
    ↳ udy that runs several of these methods on the sphere, the Euclidean plane, and the hyperbolic plane under one sh
    ↳ ared protocol with an equal budget and paired seeds. The contribution of this project is that comparison: three
    ↳ simple optimizers, one bounded disk-packing problem posed identically in geodesic balls of curvature +1, 0, and
    ↳ \u22121, the Tammes problem on the whole sphere, validation against closed-form and published values, and a full
    ↳  report of failures. The study proves no theorem and sets no packing record.")
291     HD("Research Questions and Hypotheses", 2)
292     d.body(f"RQ1: under an equal budget, which algorithm reaches the best packings in each problem? RQ2: how doe
    ↳ s performance change with the number of points or disks? RQ3: for numbers of points with a proven optimum, how o
    ↳ ften does each algorithm recover it within a stated tolerance? RQ4 (exploratory): when the same bounded-disk pro
    ↳ blem is solved in spaces of curvature +1, 0, and −1, does the ranking of the algorithms change? Three hypotheses
    ↳  were stated in the Research Plan, and {Tb('hyp')} gives each one with its measurable decision rule and its resu
    ↳ lt. Because the plan was finalized after some results had been seen (see Deviations From the Research Plan), eve
    ↳ ry hypothesis result is reported as exploratory. RQ4 and H1 are exploratory in a stronger sense: curvature and a
    ↳ rea are confounded, because a ball of the same radius has a different area in each space; only six settings per
```

```
    ↳ space were run; and the ball of radius 3 on the sphere covers nearly the whole sphere. H1 therefore refers to th
    ↳ e space, not to curvature alone.")
293
294         R3 = json.load(open(CONFDIR + "/round3_decisions.json"))
295         import conf_figures as _CF
296         _mrk = pd.read_csv(f"{DATA}/conf_mean_ranks.csv", index_col=0)
297         _conds = {c["cid"]: c for c in _C.conditions()}
298         _rk = DEC["C4"]["mean_rank_by_method"]
299         HD("Main Results at a Glance", 1)
300         d.body(f"Four results carry the paper, and the rest of the text supports them. First, the ranking of the met
    ↳ hods: across the {DEC['C4']['n_conditions']} conditions of the preregistered confirmatory round, polished gradie
    ↳ nt descent had the best mean rank ({f(_rk['gdp'], 2)}), followed by L-BFGS-B ({f(_rk['lbfgsb'], 2)}), basin hopp
    ↳ ing ({f(_rk['bh'], 2)}), simulated annealing ({f(_rk['sa'], 2)}), gradient descent ({f(_rk['gd'], 2)}), and the
    ↳ simplified compression method ({f(_rk['ls'], 2)}); the better of L-BFGS-B and basin hopping beat both gradient d
    ↳ escent and simulated annealing in all {DEC['C4']['n_conditions']} conditions ({Fg('rank')}). "
301                 f"Second, the boundary: the share by which a packing falls short of the upper bound on the density ri
    ↳ ses with the share of the ball&rsquo;s area near its wall, in all three spaces (Spearman {f(DEC['C1']['spearman'
    ↳ ], 2, lead=False)} in the confirmatory round, a test that is partly circular, and {f(json.load(open(CONFDIR + '/
    ↳ followup_decisions.json'))['primary']['F1']['spearman'], 2, lead=False)} at a fixed disk radius in a follow-up).
    ↳  "
302                 f"Third, curvature: in a final preregistered round of {R3['A1']['n_conditions']} equal-area condition
    ↳ s, once the boundary share is in the model the hyperbolic disk differed from the Euclidean disk by {f(R3['A1']['
    ↳ g_H'], 3)} (95% interval {f(R3['A1']['g_H_ci'][0], 3)} to {f(R3['A1']['g_H_ci'][1], 3)}) and the cap by {f(R3['A
    ↳ 1']['g_S'], 3)}, so no curvature effect larger than .02 of the upper bound was found, where three earlier rounds
    ↳  had been inconclusive. "
303                 f"Fourth, a rule: the number of disks of a given radius that fit in a ball is predicted by the number
    ↳  at the upper bound times (1 &minus; 0.46 times the boundary share), within one disk in {f(R3['B1']['share_withi
    ↳ n_1'] * 100, 0)}% of {R3['B1']['n_conditions']} new conditions.".replace("&rsquo;", "’").replace("&minus;", "−")
    ↳ )
304         d.body("Two cautions apply to all four. The results hold for the budgets, implementations, and sizes tested
    ↳ (the rule for the number of disks was tested only for balls of at most 46 disks), and no packing found is claime
    ↳ d to be optimal. The first study, which preceded these rounds and is exploratory, is reported in the Findings; t
    ↳ he mini-round, the follow-up, and a decision that was withdrawn are in Appendix B.")
305         FF("figure", "rank", "Mean Rank of the Six Methods in the 33 Conditions of the Confirmatory Round", _CF.fig_
    ↳ rank(_mrk, _conds), 4.7,
306             "Polished gradient descent has the best mean rank over the 33 conditions and the best or second-best rank
    ↳  in most of them; compression has the worst mean rank in almost every condition.",
307             "Each cell is the mean rank (1 = best, 6 = worst) of the method within seeds at 1,500 sweeps, over 20 see
    ↳ ds; darker blue = better rank. Columns are ordered by mean rank over all conditions (bottom row, in bold). GD =
    ↳ gradient descent; GD+P = GD with a polishing step; SA = simulated annealing; CP = compression.",
308             "Heat map of the mean rank of six methods in 33 conditions, with the mean over conditions in the bottom r
    ↳ ow.")
309         FDEC = json.load(open(CONFDIR + "/followup_decisions.json"))
310         MDEC = json.load(open(CONFDIR + "/mini_decisions.json"))
311         _hs = open(CONFDIR + "/PREREGISTRATION.sha256").read().split()
```

```
    _ad = open(CONFDIR + "/PREREG_ADDENDA.sha256").read().split("\n")
    _fu = open(CONFDIR + "/FOLLOWUP.sha256").read().split("\n")
    _u = lambda t: t.replace("T", " ")[:16] + " UTC"
    _r3h = open(CONFDIR + "/ROUND3.sha256").read().split("\n")[1].split(": ")[1]
    d.body(f"The study has six stages, which {Tb('stages')} lists in order so that the findings in the following pages can be placed. Only the stages with a hash were written before their runs, and all hashes were recorded by the author.")
    FF("table", "stages", "Stages of the Study", ["Stage", "Purpose and data", "Preregistered", "Main result"],
       [["First study", f"Three methods (gradient descent, simulated annealing, compression) in {NCOND} conditions on the sphere, the Euclidean disk, the hyperbolic disk, and the spherical cap; seeds 1 to 30; {NRUN:,} runs.", "No. The plan was finalized after results had been seen, so every result is exploratory.",
         f"Simulated annealing had the best mean rank on the sphere, in the cap, and in the Euclidean disk, and gradient descent in the hyperbolic disk; compression was last; no dependence on the space after correction (Holm-adjusted {pstr(H1c_gdsa.p_holm)})."],
        ["Confirmatory round", f"Six methods, three budgets, 33 conditions including equal-area balls; seeds 101 to 120; {DEC['n_runs']:,} runs.", f"Yes; hash recorded {_u(_hs[-1])}, before the pilot tuning and every evaluation run.",
         f"Polished gradient descent had the best mean rank ({f(DEC['C4']['mean_rank_by_method']['gdp'], 2)}); the added baselines beat gradient descent and annealing; the deficit followed the boundary share (Spearman {f(DEC['C1']['spearman'], 2, lead=False)}); curvature was inconclusive; no method recovered every proven optimum."],
        ["Addendum", "Changes decided after seeing the confirmatory results: Packomania references, wider tuning ranges for two methods and a rerun, saved configurations, wall analysis, wording of the reference density.", f"Written and hashed {_u(_ad[-2].split(": ")[1])} before the runs it describes; it adds no hypothesis.", "No decision changed; compression moved inside its tuning range, and L-BFGS-B stayed at the edge."],
        ["Mini-round (Appendix B)", "Fixed disk radius and matched boundary share, 18 conditions (15 distinct), polished gradient descent; seeds 301 to 320.", f"Yes; hash recorded {_u(_ad[-2].split(": ")[1])}, before its runs.", f"M1 {MDEC['M1']['decision']}; M2 computed as \u201c{MDEC['M2']['decision']}\u201d, a decision withdrawn because 3 of the 18 conditions duplicated others (inconclusive without them, post hoc); M3 {MDEC['M3']['decision']}."],
        ["Follow-up (Appendix B)", "Five boundary shares, three radii, 33 distinct conditions, two methods; seeds 401 to 420.", f"Yes; hash recorded {_u(_fu[1].split(': ')[1])}, before its runs.", f"F1 {FDEC['primary']['F1']['decision']}; F2 {FDEC['primary']['F2']['decision']}; F3 {FDEC['primary']['F3']['decision']}."],
        ["Final round", f"Equal-area balls ({R3['A1']['n_conditions']} conditions, 48 area-by-count cells, seeds 501 to 520) and a count rule at fixed radius (20 conditions, seeds 601 to 620); two methods.", f"Yes; hash recorded {_u(_r3h)}, before its runs (06:42 to 06:52 UTC on October 31).", f"No curvature effect larger than .02 (hyperbolic {f(R3['A1']['g_H'], 3)}, cap {f(R3['A1']['g_S'], 3)}); the rule for the number of disks {R3['B1']['decision']}."]],
       "The confirmatory round and the final round have their own sections; the addendum, mini-round, and follow-up, and a decision that was withdrawn, are in Appendix B. Hashes are SHA-256 values recorded by the author (see the section on the Confirmatory Round for what they do and do not show). UTC = Coordinated Universal Time.",
       aligns=["l", "l", "l", "l"], widths=[1.0, 2.1, 1.6, 1.8], split=True)

    # ====================================================================== Framework
    HD("Framework", 1)
    d.body(f"{Fg('framework')} is a conceptual illustration of the study design; it shows no data. The study mov
```

```
    ↳ es from the research problem to the geometric spaces and the definitions of distance and score, then to the thre
    ↳ e algorithms and their implementation. The implementation is accepted only if the unit tests pass; otherwise it
    ↳ is corrected and retested. Passing code is run in repeated trials with fixed seeds, the trials are summarized by
    ↳  performance measures, and the measures are analyzed statistically to evaluate the hypotheses. This section defi
    ↳ nes the geometry, the problems and their comparability, the experimental design, the algorithms, and the validat
    ↳ ion, statistical, and reproducibility procedures.")
332     FF("figure", "framework", "Framework of the Study", f"{FIG}/framework.png", 4.2,
333        "Conceptual illustration of the study design (no experimental data). Rectangles are processes, rounded re
    ↳ ctangles are inputs and outputs, and the diamond is a decision. If any unit test fails, the implementation is co
    ↳ rrected and retested before trials begin.",
334        "", "Flowchart from the research problem through geometric spaces, distance definitions, algorithms, impl
    ↳ ementation, unit tests, a pass-or-fail decision, repeated trials, performance measures, statistical analysis, an
    ↳ d findings.")
335     HD("Geometry of the Spaces", 2)
336     d.body("The unit sphere <i>S</i><sup>2</sup> has curvature +1. Its points are unit vectors <b><i>x</i></b> i
    ↳ n <i>R</i><sup>3</sup>, and the geodesic distance is given by Equation 1. The hyperbolic plane <i>H</i><sup>2</s
    ↳ up> has curvature \u22121. Computations use the hyperboloid model, where a point is <b><i>x</i></b> = (<i>x</i><
    ↳ sub>0</sub>, <i>x</i><sub>1</sub>, <i>x</i><sub>2</sub>) with <i>x</i><sub>0</sub> > 0 and Lorentz product [<b><
    ↳ i>x</i></b>, <b><i>y</i></b>] = \u2212<i>x</i><sub>0</sub><i>y</i><sub>0</sub> + <i>x</i><sub>1</sub><i>y</i><su
    ↳ b>1</sub> + <i>x</i><sub>2</sub><i>y</i><sub>2</sub> = \u22121 for <b><i>y</i></b> = <b><i>x</i></b>, and figure
    ↳ s use the Poincaré disk. The Euclidean plane <i>R</i><sup>2</sup> has the usual distance.")
337     d.equation("cos <i>d</i> = <b><i>x</i></b> \u00b7 <b><i>y</i></b>", 1)
338     d.equation("cosh <i>d</i> = \u2212[<b><i>x</i></b>, <b><i>y</i></b>]", 2)
339     d.body("The Poincaré disk coordinate of a hyperboloid point is <i>z</i> = (<i>x</i><sub>1</sub> + <i>i</i> <
    ↳ i>x</i><sub>2</sub>)/(1 + <i>x</i><sub>0</sub>), and the distance between disk points <i>z</i> and <i>w</i> is g
    ↳ iven by Equation 3. A hyperbolic ball of radius <i>R</i> centered at the origin is the Euclidean circle of radiu
    ↳ s tanh(<i>R</i>/2).")
340     d.equation("cosh <i>d</i> = 1 + 2|<i>z</i> \u2212 <i>w</i>|<sup>2</sup> / [(1 \u2212 |<i>z</i>|<sup>2</sup>)
    ↳ (1 \u2212 |<i>w</i>|<sup>2</sup>)]", 3)
341     FL()
342     d.body("<i>Proof sketch that Equations 2 and 3 agree.</i> The inverse of the coordinate map is <i>x</i><sub>
    ↳ 0</sub> = (1 + |<i>z</i>|<sup>2</sup>)/(1 \u2212 |<i>z</i>|<sup>2</sup>) and (<i>x</i><sub>1</sub>, <i>x</i><sub
    ↳ >2</sub>) = 2<i>z</i>/(1 \u2212 |<i>z</i>|<sup>2</sup>). Substituting into \u2212[<b><i>x</i></b>, <b><i>y</i></
    ↳ b>], where <i>w</i><sup>*</sup> is the complex conjugate of <i>w</i>, gives [(1 + |<i>z</i>|<sup>2</sup>)(1 + |<
    ↳ i>w</i>|<sup>2</sup>) \u2212 4 Re(<i>z</i> <i>w</i><sup>*</sup>)] / [(1 \u2212 |<i>z</i>|<sup>2</sup>)(1 \u2212
    ↳ |<i>w</i>|<sup>2</sup>)]. The right side of Equation 3 has the same value, because (1 \u2212 |<i>z</i>|<sup>2</s
    ↳ up>)(1 \u2212 |<i>w</i>|<sup>2</sup>) + 2|<i>z</i> \u2212 <i>w</i>|<sup>2</sup> = (1 + |<i>z</i>|<sup>2</sup>)(1
    ↳  + |<i>w</i>|<sup>2</sup>) \u2212 4 Re(<i>z</i> <i>w</i><sup>*</sup>). The disk areas follow from the area eleme
    ↳ nts sin <i>\u03c1</i> d<i>\u03c1</i> d<i>\u03b8</i>, <i>\u03c1</i> d<i>\u03c1</i> d<i>\u03b8</i>, and sinh <i>\u
    ↳ 03c1</i> d<i>\u03c1</i> d<i>\u03b8</i>, and are given by Equation 4.")
343     d.equation("Area(<i>r</i>) = 2\u03c0(1 \u2212 cos <i>r</i>) on <i>S</i><sup>2</sup>;  \u03c0<i>r</i><sup>2</
    ↳ sup> on <i>R</i><sup>2</sup>;  2\u03c0(cosh <i>r</i> \u2212 1) on <i>H</i><sup>2</sup>", 4)
344     d.body(f"{Tb('area')} gives the area of a ball of radius <i>R</i> and the share of it that lies within dista
    ↳ nce 1 of its boundary, computed from Equation 4 and stored in area_fractions.csv. In the Euclidean plane this sh
    ↳ are shrinks toward zero as <i>R</i> grows, but in the hyperbolic plane it tends to 1 \u2212 <i>e</i><sup>\u22121
```

```
    ↳ </sup>, which is .632 to three decimals. On the sphere a ball can have radius at most \u03c0, so the table has n
    ↳ o spherical entries for larger <i>R</i>. Density is therefore always reported relative to the area of the contai
    ↳ ner.")
345     arows = []
346     for r in area[area.R.isin([2.0, 3.0, 6.0])].itertuples():
347         sph = ("\u2014", "\u2014") if np.isnan(r.spherical_area) else (f(r.spherical_area, 2), f(r.spherical_fra
    ↳ ction_within_1_of_boundary, 3, lead=False))
348         arows.append([f"{r.R:g}", sph[0], sph[1], f(r.euclidean_area, 2), f(r.euclidean_fraction_within_1_of_bou
    ↳ ndary, 3, lead=False), f(r.hyperbolic_area, 2), f(r.hyperbolic_fraction_within_1_of_boundary, 3, lead=False)])
349     FF("table", "area", "Area Within Distance 1 of the Boundary of a Ball of Radius R",
350        ["<i>R</i>", "<i>S</i><sup>2</sup> area", "<i>S</i><sup>2</sup> share", "<i>R</i><sup>2</sup> area", "<i>
    ↳ R</i><sup>2</sup> share", "<i>H</i><sup>2</sup> area", "<i>H</i><sup>2</sup> share"], arows,
351        "",
352        aligns=["c"] * 7, widths=[0.5, 1.0, 1.0, 0.95, 0.95, 1.05, 1.05])
353
354     HD("Packing Problems and Comparability", 2)
355     d.body(f"Problem A is the Tammes problem on <i>S</i><sup>2</sup>: place <i>N</i> points to maximize the smal
    ↳ lest pairwise distance. Problem B places <i>n</i> equal disks in a closed geodesic ball of radius <i>R</i> about
    ↳  a fixed center, in each of the three spaces (a spherical cap, a Euclidean disk, and a hyperbolic disk), and max
    ↳ imizes the common radius <i>r</i>. A disk of radius <i>r</i> whose center is at distance <i>\u03c1</i><sub><i>i<
    ↳ /i></sub> from the center of the ball lies inside the ball when <i>\u03c1</i><sub><i>i</i></sub> \u2264 <i>R</i>
    ↳  \u2212 <i>r</i>, and two disks do not overlap when <i>d</i><sub><i>ij</i></sub> \u2265 2<i>r</i>. Both problems
    ↳  maximize <i>\u2113</i><sub>min</sub> in Equation 5, where the second term is absent for Problem A; the score is
    ↳  <i>\u2113</i><sub>min</sub> for Problem A and <i>r</i> = <i>\u2113</i><sub>min</sub>/2 for Problem B. Packing d
    ↳ ensity in Problem B is <i>n</i> Area(<i>r</i>)/Area(<i>R</i>) from Equation 4.")
356     d.equation("<i>\u2113</i><sub>min</sub> = min{ min<sub><i>i</i> < <i>j</i></sub> <i>d</i><sub><i>ij</i></sub
    ↳ >, min<sub><i>i</i></sub> 2(<i>R</i> \u2212 <i>\u03c1</i><sub><i>i</i></sub>) }", 5)
357     d.body(f"Problem B fixes <i>n</i> and <i>R</i> and maximizes <i>r</i>, which gives a continuous score that a
    ↳ ll three algorithms can improve smoothly; the number of disks of a fixed radius that fit is derived from it. The
    ↳  comparability of the problems is limited, and the limits shape every conclusion. Problems A and B differ in obj
    ↳ ective (a minimum distance against a common radius), in constraints (no boundary against a boundary), and in sca
    ↳ le, so no result of Problem A is compared with a result of Problem B. Statements about curvature rest on Problem
    ↳  B alone, which has the same formulation, the same <i>R</i> and <i>n</i>, and the same code in all three spaces.
    ↳  Even there, a ball of radius <i>R</i> has a different area in each space ({Tb('area')}), so a difference betwee
    ↳ n spaces can come from curvature, from area, or from both, and this study does not separate them. A comparison a
    ↳ t equal area is a different design; a supplementary one was run after the main study and is reported separately
    ↳ as exploratory (see Supplementary Exploratory Analyses), and it is not part of the hypothesis tests.")
358
359     HD("Experimental Design", 2)
360     d.body(f"Problem A uses <i>N</i> = {', '.join(str(int(n)) for n in NS2[:-1])}, and {int(NS2[-1])}. Problem B
    ↳  uses the six settings (<i>R</i>, <i>n</i>) = {', '.join(f'({r:g}, {n})' for r, n in BALL)} in each of the three
    ↳  spaces. The evaluation seeds are 1 to {SEEDS}. The initial configuration depends only on the seed and the condi
    ↳ tion, so the three algorithms start from the same configuration, and initial points are uniform on <i>S</i><sup>
    ↳ 2</sup> for Problem A and uniform by area in the ball of radius <i>R</i> \u2212 0.05 for Problem B. Pilot runs f
    ↳ or parameter tuning used seeds 9001 to 9004 only, which never appear in the evaluation. The total is {NRUN:,} ru
```

```
    ↳ ns.")
361     ntam = len(NS2) * SEEDS * 3
362     nb = len(BALL) * SEEDS * 3
363     d.body(f"The Tammes problem accounts for {ntam:,} runs ({len(NS2)} values of <i>N</i> \u00d7 {SEEDS} seeds \
    ↳ u00d7 3 algorithms), and each of the three spaces accounts for {nb:,} runs in Problem B ({len(BALL)} settings \u
    ↳ 00d7 {SEEDS} seeds \u00d7 3 algorithms).")
364     d.body(f"Every run receives a budget of {BUDGET:,} sweeps, where one sweep is the <i>n</i>(<i>n</i> \u2212 1
    ↳ )/2 pair-distance evaluations needed to compute all distances once. A gradient step costs one sweep plus one swe
    ↳ ep per line-search trial (in Problem B each evaluation also includes the <i>n</i> wall clearances), a simulated-
    ↳ annealing move costs <i>n</i> \u2212 1 evaluations, and a compression step costs two sweeps. Counting evaluation
    ↳ s keeps the comparison independent of implementation speed. Gradient descent and compression stop when their nex
    ↳ t step would exceed the budget. Simulated annealing fixes its number of moves from the budget in advance and doe
    ↳ s not charge proposals rejected for lying outside the ball, so in the ball problems it used slightly less than t
    ↳ he budget (at least {runs[runs.algorithm == 'sa'].sweeps_used.min():,.0f} of {BUDGET:,} sweeps in every run); th
    ↳ is can only disadvantage annealing. Every run reports the best configuration it visited, scored exactly from its
    ↳  coordinates.")
365     d.body(f"The reference value of a condition is the proven optimum for Problem A with <i>N</i> = 4 to 14 and
    ↳ 24 (credited to Fejes Tóth, 1943; Schütte & van der Waerden, 1951; Robinson, 1961; Danzer, 1963; and Musin & Tar
    ↳ asov, 2012, 2015; the values for <i>N</i> = 4, 6, and 12 were also derived in closed form); for the other values
    ↳  of <i>N</i> it is the published best-known value where one was entered ({KNOWN_TXT}) and otherwise the highest
    ↳ score found by any run in that condition. A best-known value is not proven optimal, and a best-found value is a
    ↳ computational result, so relative scores near 1 mean agreement with that reference, not optimality. The publishe
    ↳ d best-known values for <i>N</i> = 30, 40, and 60 replaced the best values found after the main results were see
    ↳ n (see Deviations From the Research Plan). The relative score of a run is its score divided by the reference val
    ↳ ue. A run succeeds when its relative score is at least .999, and a 1% tolerance (at least .99) is a sensitivity
    ↳ check. Convergence is the number of sweeps after which the best score first reached 99% of the run\u2019s own fi
    ↳ nal score. Scaling is the per-seed slope of relative score on log<sub>10</sub> <i>N</i> for <i>N</i> \u2265 13 a
    ↳ nd the log\u2013log slope of mean run time on <i>N</i>. Cost is the wall-clock time, reported descriptively beca
    ↳ use the budget is fixed in distance evaluations.")
366
367     HD("Optimization Algorithms", 2)
368     d.body(f"All three algorithms move points along geodesics and keep points on the manifold. {Tb('params')} gi
    ↳ ves every parameter.")
369     FF("table", "params", "Algorithm Parameters", ["Algorithm and parameter", "Value"],
370        [["GD: energy, exponent", f"<i>E<sub>p</sub></i> of Equation 6 over pair distances and, in Problem B, dou
    ↳ bled wall clearances; <i>p</i> from {dflt['gd']['p0']:g} to {dflt['gd']['pmax']:g}, geometric in the share of bu
    ↳ dget used"],
371         ["GD: step, line search", "<i>\u03b7</i> = min(<i>m</i> \u2113<sub>min</sub><sup>2</sup>/<i>p</i>, 0.25
    ↳ \u2113<sub>min</sub>/max|<i>g</i>|); <i>m</i> starts at 0.5, \u00d7 1.3 after success (maximum 2), \u00d7 0.5 af
    ↳ ter failure (minimum 0.001); on failure multiply <i>\u03b7</i> by 0.4, at most 5 trials"],
372         ["SA: temperature", f"Geometric from {dflt['sa']['T0f']:g} to {p10(dflt['sa']['T1f'])} times the length
    ↳ scale \u221a(area/<i>n</i>)"],
373         ["SA: move, adaptation", "Random geodesic step of length \u03b4(0.1 + 0.9<i>u</i>), <i>u</i> uniform on
    ↳ (0, 1); \u03b4 starts at 0.3 times the length scale; every 100 proposals \u03b4 \u00d7 1.15 if acceptance > .40,
    ↳  \u00d7 0.87 if < .25"],
```

```
374          ["SA: point choice", f"Probability {dflt['sa']['focus']:g}: an endpoint of the limiting pair or the poin
   ↳ t nearest the wall; otherwise uniform"],
375          ["CP: time step, growth", f"Time step {dflt['ls']['dt_f']:g} times the length scale at unit root-mean-sq
   ↳ uare speed; common radius grows by \u03b3 d<i>t</i> per step, \u03b3 falling linearly from {dflt['ls']['g0']:g}
   ↳ to {dflt['ls']['g1']:g}, capped by half the smallest distance"],
376          ["CP: collisions", "Velocity components along the line of centers exchanged if approaching; overlaps rem
   ↳ oved; wall reflection in Problem B; kinetic energy renormalized each step"]],
377         "GD = gradient descent; SA = simulated annealing; CP = compression. The length scale is \u221a(area/<i>n<
   ↳ /i>). Defaults are read from algorithms.py.",
378         aligns=["l", "l"], widths=[1.6, 4.9])
379     FL()
380     d.body("The parameter values were chosen on pilot seeds 9001 to 9004 at the same budget in four pilot condit
   ↳ ions (Tammes <i>N</i> = 13 and 30, hyperbolic disk (<i>R</i>, <i>n</i>) = (2, 12), Euclidean disk (3, 19)) from
   ↳ grids of 11 settings for gradient descent, 4 for annealing, and 10 for compression, ranked by mean score. The gr
   ↳ adient-descent and annealing settings used are grid points; the compression setting lies between grid points and
   ↳  was set by inspection. The spherical-cap family was not used for tuning, and the parameters were not changed fo
   ↳ r it. All values were fixed before the evaluation.")
381     HD("Gradient Descent on a Soft-Minimum Energy", 3)
382     d.body(f"Let <i>\u2113</i><sub>1</sub>, \u2026, <i>\u2113</i><sub><i>m</i></sub> be the pair distances and,
   ↳ in Problem B, the doubled wall clearances 2(<i>R</i> \u2212 <i>\u03c1</i><sub><i>i</i></sub>). The energy in Equ
   ↳ ation 6 is a smooth stand-in for \u2212ln <i>\u2113</i><sub>min</sub>. It satisfies the bound in Equation 7, bec
   ↳ ause the sum lies between its largest term and <i>m</i> times that term. The exponent <i>p</i> grows geometrical
   ↳ ly from {dflt['gd']['p0']:g} to {dflt['gd']['pmax']:g} over the budget, and each step follows the negative Riema
   ↳ nnian gradient with backtracking.")
383     d.equation("<i>E<sub>p</sub></i> = (1/<i>p</i>) ln \u03a3<sub><i>k</i></sub> <i>\u2113<sub>k</sub></i><sup>\
   ↳ u2212<i>p</i></sup>", 6)
384     d.equation("\u2212ln <i>\u2113</i><sub>min</sub> \u2264 <i>E<sub>p</sub></i> \u2264 \u2212ln <i>\u2113</i><s
   ↳ ub>min</sub> + (ln <i>m</i>)/<i>p</i>", 7)
385     HD("Simulated Annealing", 3)
386     d.body(f"Simulated annealing moves one point at a time by a random geodesic step and accepts the move by the
   ↳  Metropolis rule (Metropolis et al., 1953) applied to \u2212<i>\u2113</i><sub>min</sub>, with a geometric coolin
   ↳ g schedule (Kirkpatrick et al., 1983). With probability {dflt['sa']['focus']:g} the point chosen is one that cur
   ↳ rently limits <i>\u2113</i><sub>min</sub>. Because of this focused choice and the nonuniform step distribution,
   ↳ the method is an optimizer, not a sampler of the Boltzmann distribution.")
387     HD("Compression Method", 3)
388     d.body("The compression method follows the idea of Lubachevsky and Stillinger (1990) in a simplified, time-s
   ↳ tepped form, not an event-driven one. Points move with random initial velocities; overlapping disks exchange the
   ↳  velocity components along the line of centers and are pushed apart; the common radius grows at a rate that fall
   ↳ s linearly over the budget, limited by the room available. Because contacts are resolved only at the end of each
   ↳  time step, the scheme is approximate, which limits its final precision.")
389     HD("Properties of the Algorithms: Proven and Observed", 2)
390     d.body("This subsection separates what is mathematically established from what this study only observed. Not
   ↳ hing below is a guarantee that an algorithm finds a globally optimal packing, and the empirical statements are g
   ↳ iven in the Findings, not here.")
391     d.body("<i>Gradient descent.</i> Proven here: Equation 7 holds because <i>\u2113</i><sub>min</sub><sup>\u221
```

```
    ↳ 2<i>p</i></sup> \u2264 \u03a3<sub><i>k</i></sub> <i>\u2113<sub>k</sub></i><sup>\u2212<i>p</i></sup> \u2264 <i>m<
    ↳ /i> <i>\u2113</i><sub>min</sub><sup>\u2212<i>p</i></sup>, so the energy approaches \u2212ln <i>\u2113</i><sub>mi
    ↳ n</sub> as <i>p</i> grows, with an error of at most (ln <i>m</i>)/<i>p</i>. Not established: the energy is not c
    ↳ onvex, so a stationary point can be a local optimum or a saddle; the exponent <i>p</i> changes at every step, so
    ↳  the energy does not decrease monotonically over a run; and the standard convergence theory for Riemannian gradi
    ↳ ent methods with a backtracking line search (Absil et al., 2008) concerns a fixed smooth objective, so it does n
    ↳ ot cover this schedule. The true objective <i>\u2113</i><sub>min</sub> is not differentiable where distances tie
    ↳ , and its maxima are rigid configurations with many contacts. No theorem is claimed that gradient descent reache
    ↳ s a stationary point of <i>\u2113</i><sub>min</sub> or a global maximum.")
392     d.body("<i>Simulated annealing.</i> Proven for finite-state Markov chains (Hajek, 1988): the probability of
    ↳ being at a global optimum tends to 1 under a logarithmic cooling schedule <i>T</i><sub><i>k</i></sub> = <i>c</i>
    ↳ /ln(<i>k</i> + 1) if and only if <i>c</i> is at least the depth of the deepest local minimum that is not global.
    ↳  Not applicable here: the state space is continuous, the cooling is geometric over a fixed number of moves, the
    ↳ step size adapts, and 30% of the moves target the limiting pair. The method is therefore a heuristic optimizer w
    ↳ ithout a convergence guarantee in this study, and its final configuration is not a sample from a known distribut
    ↳ ion.")
393     d.body("<i>Compression.</i> The method of Lubachevsky and Stillinger (1990) is an event-driven simulation of
    ↳  disks whose radii grow, and this study uses a simplified time-stepped version that resolves contacts only at th
    ↳ e end of each step. No theorem guaranteeing convergence of either version to a densest packing is known to me, a
    ↳ nd none is claimed. Because contacts are resolved at the end of each step, positions are accurate only to about
    ↳ the distance a disk moves in one step, which is a plausible reason for the lower final precision of this impleme
    ↳ ntation; this explanation was not tested separately.")
394     d.body(f"<i>Stopping and numerical precision.</i> All three methods use the same stopping rule, a fixed budg
    ↳ et of pair-distance evaluations with no early stopping, so differences in the final score are not caused by diff
    ↳ erent stopping criteria; the exception, unused budget in simulated annealing near the wall, is described under E
    ↳ xperimental Design. Because the budget is fixed, the results describe performance at that budget and not converg
    ↳ ence to a limit ({Fg('conv')} shows how the best score rose during the budget). Distances use arcsine and arcsin
    ↳ h forms that stay accurate for small distances; floating-point error is examined in the Findings.")
395     HD("Validation Procedures", 2)
396     d.body(f"Unit tests checked the geometry and the code before the experiments: the 23 tests of the sphere, hy
    ↳ perbolic, Euclidean, and optimizer code passed before the first 27 conditions were run, and the 6 spherical-cap
    ↳ tests passed before the cap conditions were run. The test log in this paper is the complete run of all 29 tests
    ↳ with the final code, and three of the first 27 conditions (Tammes <i>N</i> = 13, hyperbolic disk (2, 12), Euclid
    ↳ ean disk (2, 7)) were rerun with the final code and reproduced every score exactly. {Tb('vtests')} states how ea
    ↳ ch group of tests is performed and what counts as passing; a single failure in any group would have blocked the
    ↳ experiments. The test for recovery of known optima requires the best of the three optimizers on five test seeds
    ↳ (9101 to 9105), because recovery by each optimizer is hypothesis H3 and not a validation criterion.")
397     HD("Statistical Analysis", 2)
398     d.body(f"For each condition the observations are the {SEEDS} final scores of each algorithm. Scores of the s
    ↳ ame seed come from the same starting configuration and form a block. The Friedman test compares the three algori
    ↳ thms across the {SEEDS} blocks, and the Wilcoxon signed-rank test (Wilcoxon, 1945) compares each pair on the {SE
    ↳ EDS} within-seed differences. Nonparametric paired tests are used because scores are bounded, are tied or cluste
    ↳ red at local optima, and are not normally distributed. Effect sizes are Kendall\u2019s <i>W</i> for the Friedman
    ↳  test and the matched-pairs rank-biserial correlation (<i>r</i><sub>rb</sub>) with the standardized mean differe
    ↳ nce (<i>d<sub>z</sub></i>) for pairs. Means carry 95% confidence intervals based on the <i>t</i> distribution, w
```

```
    hich are approximate for non-normal scores and used descriptively.")
399     d.body("The tests assume independent blocks, which holds because each seed gives an independent random start
    . The Wilcoxon test also assumes that the paired differences are symmetric about their median. This was checked
    through the skewness of each set of differences (an absolute value above 1 is flagged) and the share of zero dif
    ferences, which are stored in assumption_checks.csv. Tests were corrected with the Holm procedure (Holm, 1979),
    taking as one family all tests of the same kind within one space; hypothesis tests are corrected across their ow
    n pairs. A result is statistically significant when the Holm-adjusted <i>p</i> is below .05. Practical importanc
    e is judged separately: a difference is practically important when its mean is at least 0.5% of the reference va
    lue. This threshold and the assumption checks were formalized after the first 27 conditions had been run. The Fr
    iedman test assumes independent blocks whose scores can be ranked; ties are corrected by the implementation, and
     with three algorithms and 30 blocks the chi-square approximation is generally considered adequate. The Wilcoxon
     test discards zero differences and uses an exact <i>p</i> value only when there are no ties or zeros (otherwise
     the normal approximation of the implementation). The rank-biserial correlation and <i>d</i><sub><i>z</i></sub>
    describe the size of a paired difference, and the intervals are <i>t</i> intervals of the mean, checked against
    bootstrap intervals for two headline differences (see Supplementary Exploratory Analyses). Holm\u2019s procedure
     controls the familywise error rate without assuming that the tests are independent (Holm, 1979), which matters
    because all tests in one space share the same 30 seeds.")
400     HD("Reproducibility and Data Integrity", 2)
401     files = sorted(f_ for f_ in os.listdir(DATA) if f_.endswith(".csv"))
402     d.body(f"All code is in Code_Listing.docx and, as source files, in the code folder of CSV_Data.zip. The repo
    rted runs used Python 3.12.3 with numpy 2.4.4, scipy 1.17.1, pandas 3.0.2, and matplotlib 3.10.8; an independent
     full rerun (python3 build_all.py --rerun) under Python 3.13.18 with numpy 2.5.3, scipy 1.18.1, pandas 3.0.5, an
    d matplotlib 3.11.2 reproduced every score, test statistic, and p-value in the CSV files exactly; only the wall-
    clock times differ. The command python3 tests.py runs the validation tests, and python3 build_all.py runs the te
    sts, all experiments, the statistics, the figures, and the building of every document; adding --rerun recomputes
     every run, which took {runs.time_s.sum() / 60:.0f} minutes of run time on one core in the run reported here. Se
    eds are fixed (evaluation seeds 1 to {SEEDS}, pilot seeds 9001 to 9004, test seeds 9101 to 9105), so a rerun rep
    roduces every value. CSV_Data.zip holds {len(files)} CSV files (among them runs_raw.csv with all {NRUN:,} runs,
    configurations.csv with every final configuration, and convergence_traces.csv), unit_test_output.txt, README.md,
     which defines every column of every file, and MANIFEST.sha256, whose SHA-256 hashes can be checked with sha256s
    um. Every table and figure note names the file from which its values were computed. No run was excluded: all {NR
    UN:,} runs are in the data, including failures such as the {stuck} of {SEEDS} seeds in which gradient descent st
    alled for <i>N</i> = 4 (see Conclusions). The forms required by the competition are submitted separately and are
     not reproduced here.")
403     HD("Deviations From the Research Plan", 2)
404     d.body("The Research Plan was finalized after the results of all 33 conditions had been seen (see Appendix A
     of the Research Plan), so the hypotheses and decision rules are not strictly pre-registered. Its Appendix A, Hi
    story of Work Done, lists the changes, which are summarized here. First, the compression method was implemented
    as a time-stepped scheme, not an event-driven one. Second, the spherical-cap condition family was added after th
    e results of the other 27 conditions had been seen, to obtain a common formulation across curvatures; its analys
    is was written into the code before the cap runs were executed, but the resulting evaluation of H1 is explorator
    y with respect to the earlier results. Third, the practical-importance threshold and the assumption checks were
    formalized after the first 27 conditions had been run. Fourth, the validation criterion for optimizer recovery w
    as changed to the best of three optimizers after compression failed an earlier per-optimizer version. Pilot tuni
    ng and the validation tests preceded the main runs they support (see Validation Procedures); the data package co
```

```
    ntains no dated log of when each run was executed. After the main results had been seen, five further changes we
    re made, all recorded in the plan: the references for <i>N</i> = 30, 40, and 60 were replaced by published best-
    known values (they had been the best values found, which changed the relative scores for those values of <i>N</i
    > and therefore the scaling slopes); the citations of the proven optima were corrected; the result of every hypo
    thesis is labeled exploratory; the supplementary analyses and a polished gradient-descent variant were added; an
    d a statement on AI assistance was added. None of them changes a run, a seed, a parameter, or a decision rule of
    the main study. The Confirmatory Round was added after all of these, with its own preregistration, hashed befor
    e its pilot tuning and before every evaluation run; it did not change any result of the first study. After the r
    esults of the Confirmatory Round had been seen, the following were added, each listed in a second preregistratio
    n file and in the section on the Confirmatory Round: Packomania reference values for the Euclidean conditions (e
    ntered after the first analysis of the round; the decisions were the same with both references), extended tuning
    ranges and a re-run of compression and L-BFGS-B, the saved final configurations with the wall analysis, a corre
    ction of the wording of the reference density, and a mini-round with a fixed radius and a matched boundary share
    , whose own preregistration was hashed before its runs. A withdrawn mini-round decision, a follow-up, and a fina
    l round followed; the final round was preregistered, its rules were written in the Research Plan before the pape
    r, and its runs followed the adviser email that the author reports sending. The withdrawal is in Appendix B.")
405
406     # ============================================================================ Findings
407     HD("Disclosure of AI Assistance and Researcher\u2019s Contribution", 2)
408     d.body("Claude, an AI assistant developed by Anthropic, was used to assist with the writing of this document
    , including drafting, grammar checking, and language refinement. Other uses of AI assistance in this project, in
    cluding its use in programming and analysis, are described in the separate AI Disclosure Agreement submitted to
    the research adviser and the Science Research Committee. GitHub was used to support code management and access t
    o programming resources. The author\u2019s original contributions include the selection of the research problem,
    the formulation of the research questions, and the decisions regarding the research design. The author personal
    ly ran and checked the code, verified the results, and can explain and defend the programming procedures, method
    s, and findings. The author accepts full responsibility for the accuracy and integrity of every statement, numer
    ical result, citation, and conclusion presented in this research. The code and data are provided to support repr
    oducibility and independent verification of the results.")
409
410     HD("Findings", 1)
411     d.body("This section reports the validation of the code, the results on the sphere and in geodesic balls, co
    nvergence, scaling and cost, the statistical comparison, and the resolution of the hypotheses. Every value comes
    from the files named in the table and figure notes.")
412     HD("Validation Results", 2)
413     gc = table_group_counts()
414     tot_n = sum(g[3] for g in gc)
415     tot_p = sum(g[4] for g in gc)
416     assert tot_n == int(re.search(r"(\d+) of (\d+) tests passed", unit_txt).group(2)) and tot_p == int(re.search
    (r"(\d+) of (\d+) tests passed", unit_txt).group(1))
417     d.body(f"All {tot_p} of {tot_n} unit tests passed ({Tb('vtests')}).")
418     FF("table", "vtests", "Validation Tests, Pass Criteria, and Results", ["Test group", "How it is performed",
    "Pass criterion", "Passed"],
419        [[g[0], g[1], g[2], f"{g[4]} of {g[3]}"] for g in gc] + [["Total", "", "", f"{tot_p} of {tot_n}"]],
420        "Full output is in unit_test_output.txt. Passed = tests passed of tests run in the group.",
```

```
        aligns=["l", "l", "l", "c"], widths=[1.15, 2.35, 2.2, 0.8])

    FL()
    d.body(f"Distances on <i>S</i><sup>2</sup> matched the arctangent form to a maximum error of {sci(test_err('S2 distance matches'))} rad, and distances on <i>H</i><sup>2</sup> matched the Poincaré disk formula to a maximum relative error of {sci(test_err('H2 hyperboloid distance'))}. The analytic gradients matched finite differences, the closed-form tetrahedron, octahedron, and icosahedron gave the Tammes values arccos(\u22121/3), \u03c0/2, and arctan 2 for <i>N</i> = 4, 6, and 12, and the spherical-cap code reproduced the Euclidean limit. These tests show that the geometry code measures distances correctly and that the optimizers move points as the mathematics prescribes; they do not show that any optimizer finds optimal packings.")
    HD("Packing Performance on the Sphere", 2)
    d.body(f"The reference values are stored in benchmark_values.csv with the source of each (in degrees and radians; the radian values are in the Reference column of {Tb('s2res')}); the proofs are credited in the Introduction. {Fg('sphere')} shows the best configuration found for six values of <i>N</i>, and {Tb('s2res')} gives the mean final minimum distance and the success rate for every <i>N</i>.")
    FF("figure", "sphere", "Best Configurations on the Sphere for Six Values of N", f"{FIG}/fig_sphere_configs.png", 5.4,
       "Orthographic view of the best configuration over all algorithms and seeds for <i>N</i> = 12, 13, 14, 24, 30, and 60. Data: configurations.csv.",
       "Points: filled = on the visible hemisphere, open = on the hidden hemisphere. Circles: spherical caps whose angular radius is half the minimum distance of that configuration. Black arcs: great-circle arcs joining points whose distance is within 0.0001 rad of the minimum. Color: algorithm that found the configuration (blue = gradient descent, red = simulated annealing, green = compression).",
       "Six orthographic views of points on a sphere with circles around each point and great-circle arcs joining touching points, for 12, 13, 14, 24, 30, and 60 points.")
    r12 = {a: S("S2", 12, a) for a in ALG}
    r13 = {a: S("S2", 13, a) for a in ALG}
    r60 = {a: S("S2", 60, a) for a in ALG}
    d.body(f"For <i>N</i> = 12, gradient descent and simulated annealing both reached the icosahedral value (<i>M</i> = {f(r12['gd'].mean_score)} and {f(r12['sa'].mean_score)} rad; the optimum is {f(r12['gd'].reference_value)} rad), whereas compression stopped lower (<i>M</i> = {f(r12['ls'].mean_score)}, <i>SD</i> = {f(r12['ls'].sd_score)}). For <i>N</i> = 13, no algorithm reached the proven optimum of {f(r13['gd'].reference_value)} rad on average (gradient descent <i>M</i> = {f(r13['gd'].mean_score)}, simulated annealing {f(r13['sa'].mean_score)}, compression {f(r13['ls'].mean_score)}). For <i>N</i> = 60, simulated annealing led with <i>M</i> = {f(r60['sa'].mean_score)}, ahead of gradient descent ({f(r60['gd'].mean_score)}) and compression ({f(r60['ls'].mean_score)}); there the reference is the best value found, not a proven optimum.")
    from geometry import Space as _Sp
    def contacts(cid):
        g = runs[runs.condition_id == cid].sort_values("score", ascending=False).iloc[0]
        X = cfgall[(cfgall.condition_id == cid) & (cfgall.algorithm == g.algorithm) & (cfgall.seed == g.seed)].sort_values("index")[["x0", "x1", "x2"]].values
        Dm = _Sp("S2").pair_dist(X)
        c = (Dm < g.score + 1e-3)
        np.fill_diagonal(c, False)
        return int(c.sum() // 2), int((c.sum(1) == 0).sum())
```

```
    cinfo = {n: contacts(100 + NS2.index(n)) for n in (12, 13, 14, 24, 30, 60)}
    d.body("Counting pairs whose distance is within 0.001 rad of the minimum shows how rigid each displayed conf
↳ iguration is (contacting pairs, points without contact): " + "; ".join(f"<i>N</i> = {n}, {c[0]} pairs and {c[1]}
↳  free points" for n, c in cinfo.items()) + ". Free points can move without changing the minimum distance. Free p
↳ oints in the larger configurations (<i>N</i> = 30 and 60) suggest that the search has not reached a jammed packi
↳ ng within the budget; for a small <i>N</i> such as 13, a free point can also belong to an optimal packing.")
    GDNEAR = ", ".join(str(int(n)) for n in NS2 if n <= 14 and S("S2", n, "gd").success_rate_0p1pct == 0 and S("
↳ S2", n, "gd").mean_relative >= 0.995)
    SA_ALL = ", ".join(str(int(n)) for n in NS2 if n <= 14 and S("S2", n, "sa").success_rate_0p1pct >= 0.999)
    GD_ALL = ", ".join(str(int(n)) for n in NS2 if n <= 14 and S("S2", n, "gd").success_rate_0p1pct >= 0.999)
    d.body(f"At the 0.1% tolerance, simulated annealing succeeded in all {SEEDS} seeds for <i>N</i> = {SA_ALL},
↳ and gradient descent for <i>N</i> = {GD_ALL}. Gradient descent never met the tolerance for <i>N</i> = {GDNEAR} e
↳ ven though its mean relative scores there were at least .995, which suggests that its final precision, not its s
↳ earch, limits the success rate. Compression met the tolerance in at most {f(max(sc_ls_small.values()) * 100, 0)}
↳ % of seeds for <i>N</i> = 4 to 6 and never beyond. For <i>N</i> \u2265 24 no algorithm succeeded in more than {f
↳ (max(S('S2', n, a).success_rate_0p1pct for n in NS2 if n >= 24 for a in ALG) * 100, 0)}% of seeds, so the large-
↳ <i>N</i> results are comparisons of average quality, not recoveries of optima.")
    rows = []
    for n in NS2:
        row = [str(int(n)), f(S("S2", n, "gd").reference_value) + ("*" if S("S2", n, "gd").reference_type != "ve
↳ rified optimum" else "")]
        for a in ALG:
            s_ = S("S2", n, a)
            row.append(f"{f(s_.mean_score)} ({f(s_.sd_score)})")
        row.append(" / ".join(f"{S('S2', n, a).success_rate_0p1pct * 100:.0f}" for a in ALG))
        rows.append(row)
    FF("table", "s2res", "Final Minimum Distance and Success Rate on the Sphere by Algorithm", ["<i>N</i>", "Ref
↳ erence (rad)", "GD <i>M</i> (<i>SD</i>)", "SA <i>M</i> (<i>SD</i>)", "CP <i>M</i> (<i>SD</i>)", "Success (%) GD
↳ / SA / CP"], rows,
       f"Distances in radians over {SEEDS} seeds. Success = percentage of seeds with relative score \u2265 .999.
↳  GD = gradient descent; SA = simulated annealing; CP = compression. An asterisk marks a reference that is not pr
↳ oven optimal: {STARNOTE}. Data: summary_by_condition.csv.",
       aligns=["c"] * 6, widths=[0.55, 0.85, 1.1, 1.1, 1.1, 1.8])

    HD("Packing Performance in Geodesic Balls", 2)
    d.body(f"{Fg('curv')} shows the best packing of 19 disks in a ball of radius 3 in each of the three spaces.
↳ {Tb('ballres')} reports the mean radius and the packing density for all 18 conditions, and {Tb('capacity')} turn
↳ s the best radii into a statement about capacity: the largest tested number of disks that fit when the radius mu
↳ st reach a target value. These capacity entries are lower bounds from the tested values of <i>n</i> only.")
    FF("figure", "curv", "Best Packings of 19 Disks in a Ball of Radius 3 in Three Spaces", f"{FIG}/fig_curvatur
↳ e_packing.png", 6.4,
       "Best packing found for <i>R</i> = 3 and <i>n</i> = 19 in each space (best run over all algorithms and se
↳ eds), with the resulting radius <i>r</i> in the panel title. The panels use different projections: the spherical
↳  cap is shown in azimuthal equidistant projection, where the distance from the center is preserved and shapes ar
↳ e distorted; the Euclidean disk is shown undistorted; the hyperbolic disk is shown in the Poincaré disk, where h
```

```
   ↳ yperbolic disks are exact Euclidean circles. Data: configurations.csv.",
465         "Shaded regions: disks of radius <i>r</i>. Dots: disk centers. Solid circle: container, a ball of radius
   ↳ 3 (Euclidean radius tanh(3/2) in the Poincaré disk). Dotted circle: boundary at infinity of the Poincaré disk. C
   ↳ olor: algorithm that found the packing (blue = gradient descent, red = simulated annealing).",
466         "Three panels showing 19 disks in a ball of radius 3: stretched disks in a spherical cap, equal circles i
   ↳ n a Euclidean disk, and circles that look smaller toward the edge in a hyperbolic disk.")
467     gdh = {sp: runs[(runs.condition_id == (400 if sp == "S2cap" else 300 if sp == "R2" else 200) + 4)].sort_valu
   ↳ es("score", ascending=False).iloc[0] for sp in SPACES}
468     d.body(f"The best radii were {f(gdh['S2cap'].score, 4)} in the cap, {f(gdh['R2'].score, 4)} in the Euclidean
   ↳  disk, and {f(gdh['H2'].score, 4)} in the hyperbolic disk. These radii are not comparable as measures of packing
   ↳  quality, because the balls have different areas; the densities in {Tb('ballres')} are the better guide. In the
   ↳ hyperbolic plane, gradient descent had the highest mean radius in {best_count('H2', 'gd')} of 6 settings and sim
   ↳ ulated annealing in {best_count('H2', 'sa')}. In the Euclidean disk, simulated annealing had the highest mean ra
   ↳ dius in {best_count('R2', 'sa')} of 6 and gradient descent in {best_count('R2', 'gd')}. In the cap, simulated an
   ↳ nealing had the highest mean radius in {best_count('S2cap', 'sa')} of 6 and gradient descent in {best_count('S2c
   ↳ ap', 'gd')}. Compression never had the highest mean radius ({best_count('H2', 'ls') + best_count('R2', 'ls') + b
   ↳ est_count('S2cap', 'ls')} of 18 conditions).")
469     rows = []
470     for sp in SPACES:
471         for R, n in BALL:
472             c = [S(sp, n, a, R=R) for a in ALG]
473             rows.append([SLAB[sp], f"{R:g}", str(n)] + [f"{f(x.mean_score)} ({f(x.sd_score)})" for x in c] + ["
   ↳ / ".join(f"{x.mean_density * 100:.0f}" for x in c)])
474     FF("table", "ballres", "Mean Disk Radius and Packing Density in a Ball by Algorithm", ["Space", "<i>R</i>",
   ↳ "<i>n</i>", "GD <i>r</i> (<i>SD</i>)", "SA <i>r</i> (<i>SD</i>)", "CP <i>r</i> (<i>SD</i>)", "Density (%) GD / S
   ↳ A / CP"], rows,
475         f"Radius <i>r</i> over {SEEDS} seeds, in the units of the space. Density = percentage of the ball\u2019s
   ↳ area covered by the disks, averaged over seeds. Cap = spherical cap. GD = gradient descent; SA = simulated annea
   ↳ ling; CP = compression. Data: summary_by_condition.csv.",
476         aligns=["l", "c", "c", "c", "c", "c", "c"], widths=[0.7, 0.4, 0.4, 1.1, 1.1, 1.1, 1.7])
477     rows = []
478     for sp in SPACES:
479         for R in (2.0, 3.0):
480             g = caps[(caps.space == sp) & (caps.R == R)].sort_values("target_radius")
481             cells = [f"{int(x.largest_tested_n_that_fits)} ({f(x.best_radius_at_that_n)})" if x.largest_tested_n
   ↳ _that_fits > 0 else "none" for x in g.itertuples()]
482             rows.append([SLAB[sp], f"{R:g}"] + cells)
483     FF("table", "capacity", "Largest Tested Number of Disks That Fit at a Target Radius", ["Space", "<i>R</i>",
   ↳ "<i>r</i> \u2265 0.4", "<i>r</i> \u2265 0.5"], rows,
484         "Largest tested <i>n</i> (best radius). Cap = spherical cap. Data: capacity_fixed_radius.csv.",
485         aligns=["l", "c", "c", "c"], widths=[1.0, 0.8, 2.35, 2.35])
486
487     HD("Convergence, Scaling, and Cost", 2)
488     sw = {a: S("S2", 60, a).mean_sweeps_to_99 for a in ALG}
489     d.body(f"Convergence effort is the number of sweeps a run needed to first reach 99% of its own final score (
```

```
    ↳ {Tb('sweeps')}). For <i>N</i> = 60 the mean was {sw['gd']:,.0f} sweeps for gradient descent, {sw['sa']:,.0f} for
    ↳  simulated annealing, and {sw['ls']:,.0f} for compression, out of a budget of {BUDGET:,}. A value near the budge
    ↳ t means that the run was still improving when the budget ended, so its final score is a lower bound on what a lo
    ↳ nger run could reach. {Fg('conv')} shows how the best score so far rose with the budget, {Fg('perf')} shows the
    ↳ final relative score for every condition, {Fg('runtime')} shows run time, and {Fg('dist')} shows the spread over
    ↳  seeds.")
490     rows = []
491     for lab, sp, n, R in (("<i>S</i><sup>2</sup>, <i>N</i> = 60", "S2", 60, None),
492                           ("Cap, <i>R</i> = 3, <i>n</i> = 19", "S2cap", 19, 3.0), ("<i>R</i><sup>2</sup>, <i>R</
    ↳ i> = 3, <i>n</i> = 19", "R2", 19, 3.0), ("<i>H</i><sup>2</sup>, <i>R</i> = 3, <i>n</i> = 19", "H2", 19, 3.0)):
493         rows.append([lab] + [f"{S(sp, n, a, R=R).mean_sweeps_to_99:,.0f}" for a in ALG])
494     FF("table", "sweeps", "Mean Sweeps Needed to Reach 99% of the Final Score", ["Condition", "GD", "SA", "CP"],
    ↳  rows,
495        f"Budget = {BUDGET:,} sweeps. Cap = spherical cap. Data: summary_by_condition.csv.",
496        aligns=["l", "c", "c", "c"], widths=[2.6, 1.3, 1.3, 1.3])
497     lsm = lambda n: S("S2", n, "ls").mean_relative
498     d.body(f"On the sphere, gradient descent and simulated annealing had relative scores of at least {f(min(S('S
    ↳ 2', n, a).mean_relative for n in NS2 if n <= 12 for a in ('gd', 'sa')), 3, lead=False)} for <i>N</i> \u2264 12,
    ↳ whereas compression first fell below .990 at <i>N</i> = {int(min(n for n in NS2 if lsm(n) < 0.99))} and then dec
    ↳ lined steadily, reaching a mean of {f(r60['ls'].mean_relative, 2)} of the reference at <i>N</i> = 60, against {f
    ↳ (r60['gd'].mean_relative, 2)} for gradient descent and {f(r60['sa'].mean_relative, 2)} for simulated annealing.
    ↳ Simulated annealing took the longest and its time grew almost linearly with <i>N</i> (log\u2013log slope {f(floa
    ↳ t(rte[rte.algorithm == 'sa'].time_exponent.iloc[0]), 2)}), because each of its moves costs one row of distances.
    ↳  Gradient descent and compression took much less time (slopes {f(float(rte[rte.algorithm == 'gd'].time_exponent.
    ↳ iloc[0]), 2)} and {f(float(rte[rte.algorithm == 'ls'].time_exponent.iloc[0]), 2)}), because their vectorized ste
    ↳ ps are dominated by fixed overhead at these sizes; these slopes describe this implementation, not the algorithms
    ↳  in general.")
499     FF("figure", "conv", "Convergence of the Best Score During the Budget", f"{FIG}/fig_convergence.png", 4.7,
500        f"Mean of the best score so far divided by the reference value, over {SEEDS} seeds, for six conditions. T
    ↳ he horizontal axis is logarithmic and the vertical axes do not start at zero. Data: convergence_traces.csv.",
501        "Lines: mean of the best score so far (blue solid with circles = gradient descent, red dashed with square
    ↳ s = simulated annealing, green dash-dot with triangles = compression). Shaded bands: 95% confidence intervals ov
    ↳ er seeds. Dotted line: reference value (1.0). Cap = spherical cap, <i>R</i> = ball radius, <i>n</i> = number of
    ↳ disks, <i>N</i> = number of points.",
502        "Six line charts of best score versus fraction of budget for three algorithms.")
503     FF("figure", "perf", "Final Relative Score by Condition", f"{FIG}/fig_performance_vs_size.png", 4.4,
504        f"Mean relative score over {SEEDS} seeds with 95% confidence intervals. The top panel is the Tammes probl
    ↳ em and the other panels are disks in a ball; vertical axes do not start at zero. Data: summary_by_condition.csv.
    ↳ ",
505        "Markers and lines as in the previous figure (blue circles = gradient descent, red squares = simulated an
    ↳ nealing, green triangles = compression). Error bars: 95% confidence intervals. Dotted line: reference value (1.0
    ↳ ), which is a proven optimum for <i>N</i> ≤ 14 and <i>N</i> = 24, a published best-known value where stated in t
    ↳ he text, and the best value found otherwise. Horizontal labels in the lower panels: ball radius <i>R</i>, number
    ↳  of disks <i>n</i>.",
506        "Four stacked line charts of relative score for three algorithms: the Tammes problem by number of points,
```

```
  ↳  and the spherical cap, Euclidean disk, and hyperbolic disk by setting.")
507      FF("figure", "runtime", "Mean Run Time on the Sphere by Number of Points", f"{FIG}/fig_runtime_scaling.png",
  ↳  4.5,
508         f"Mean wall-clock time per run over {SEEDS} seeds at equal budget. Data: summary_by_condition.csv.",
509         "Blue circles = gradient descent, red squares = simulated annealing, green triangles = compression. Error
  ↳  bars: 95% confidence intervals. Both axes logarithmic; time in seconds.",
510         "Log-log line chart of run time versus number of points for three algorithms.")
511      q = {a: np.percentile(runs[(runs.space == "S2") & (runs.n == 60) & (runs.algorithm == a)].relative_score, [2
  ↳ 5, 75]) for a in ALG}
512      dis = lambda a, b: a[1] < b[0] or b[1] < a[0]
513      nov60 = dis(q["gd"], q["sa"]) and dis(q["gd"], q["ls"]) and dis(q["sa"], q["ls"])
514      d.body(f"For <i>N</i> = 60 the interquartile ranges of the three algorithms {'do not overlap' if nov60 else
  ↳ 'overlap in part'} ({Fg('dist')}).")
515      FF("figure", "dist", "Distribution of Relative Scores Over Seeds", f"{FIG}/fig_distribution.png", 4.5,
516         f"Box plots over {SEEDS} seeds for six conditions; the pair in parentheses is (<i>R</i>, <i>n</i>). Data:
  ↳  runs_raw.csv.",
517         "Box: interquartile range. Black line: median. Whiskers: 1.5 interquartile ranges. Markers: outliers. Dot
  ↳ ted line: reference value. Blue = gradient descent, red = simulated annealing, green = compression. Cap = spheri
  ↳ cal cap.",
518         "Box plots of relative score for three algorithms in six conditions.")
519
520      HD("Statistical Comparison of Algorithms", 2)
521      mx = float(fr.p_holm.max())
522      d.body(f"The Friedman test found a significant difference among the three algorithms in {fr_sig} of {fr_n} c
  ↳ onditions (largest Holm-adjusted <i>p</i> {('< .001' if mx < 0.001 else '= ' + f(mx, 3, lead=False))}), with Ken
  ↳ dall\u2019s <i>W</i> between {f(float(fr.kendall_W.min()), 2, lead=False)} and {f(float(fr.kendall_W.max()), 2,
  ↳ lead=False)}. {Tb('pairwise')} counts, for each space and pair, the conditions in which one algorithm was signif
  ↳ icantly higher after Holm correction and, in the last column, how many of those differences were also practicall
  ↳ y important (mean difference at least 0.5% of the reference). Compression was significantly lower than gradient
  ↳ descent in {sum(wins(sp, 'gd', 'ls', 'gd')[0] for sp in ('S2', 'S2cap', 'R2', 'H2'))} of {fr_n} conditions and l
  ↳ ower than simulated annealing in {sum(wins(sp, 'sa', 'ls', 'sa')[0] for sp in ('S2', 'S2cap', 'R2', 'H2'))} of {
  ↳ fr_n}.")
523      rows = []
524      for sp in ("S2", "S2cap", "R2", "H2"):
525          for a, b in (("gd", "sa"), ("gd", "ls"), ("sa", "ls")):
526              g = pw[(pw.space == sp) & (pw.alg_a == a) & (pw.alg_b == b)]
527              ka, nn = wins(sp, a, b, a)
528              kb, _ = wins(sp, a, b, b)
529              rows.append([SLAB[sp], f"{AB[a]} vs. {AB[b]}", str(ka), str(kb), str(nn - ka - kb), str(int(g.signif
  ↳ icant_and_practical.sum()))])
530      FF("table", "pairwise", "Conditions in Which One Algorithm Was Significantly Higher", ["Space", "Comparison"
  ↳ , "First higher", "Second higher", "No difference", "Also practical"], rows,
531         "Counts of conditions (15 for <i>S</i><sup>2</sup>, 6 for each other space) from Wilcoxon signed-rank tes
  ↳ ts with Holm-adjusted <i>p</i> < .05. First = first-named algorithm. Also practical = significant differences of
  ↳  at least 0.5% of the reference. Cap = spherical cap. GD = gradient descent; SA = simulated annealing; CP = comp
```

```
   ↳ ression. Data: test_pairwise_wilcoxon.csv.",
532         aligns=["l", "l", "c", "c", "c", "c"], widths=[0.9, 1.4, 1.0, 1.1, 1.1, 1.0])
533     d.body(f"Significance does not always mean a large difference. For <i>N</i> = 5, 6, and 12, gradient descent
   ↳  was significantly higher than simulated annealing with <i>r</i><sub>rb</sub> = 1.00, yet the mean differences w
   ↳ ere only {f(PW('S2', 5, 'gd', 'sa').mean_diff * 1000, 1)}, {f(PW('S2', 6, 'gd', 'sa').mean_diff * 1000, 1)}, and
   ↳  {f(PW('S2', 12, 'gd', 'sa').mean_diff * 1000, 1)} milliradians, which is below the 0.5% threshold. Of the {int(
   ↳ pw.significant.sum())} significant pairwise results, {int(pw.significant_and_practical.sum())} were also practic
   ↳ ally important. For <i>N</i> = 60 on the sphere, gradient descent minus simulated annealing was {f(PW('S2', 60,
   ↳ 'gd', 'sa').mean_diff, 3)} rad, 95% CI [{f(PW('S2', 60, 'gd', 'sa').ci_lo, 3)}, {f(PW('S2', 60, 'gd', 'sa').ci_h
   ↳ i, 3)}], <i>r</i><sub>rb</sub> = {f(PW('S2', 60, 'gd', 'sa').rbc, 2, lead=False)}, <i>d<sub>z</sub></i> = {f(PW(
   ↳ 'S2', 60, 'gd', 'sa').dz, 2)}, {pstr(PW('S2', 60, 'gd', 'sa').p_holm)}. For 19 hyperbolic disks in a ball of rad
   ↳ ius 3 the same difference was {f(PW('H2', 19, 'gd', 'sa', R=3.0).mean_diff, 3)}, 95% CI [{f(PW('H2', 19, 'gd', '
   ↳ sa', R=3.0).ci_lo, 3)}, {f(PW('H2', 19, 'gd', 'sa', R=3.0).ci_hi, 3)}], <i>r</i><sub>rb</sub> = {f(PW('H2', 19,
   ↳ 'gd', 'sa', R=3.0).rbc, 2, lead=False)}, <i>d<sub>z</sub></i> = {f(PW('H2', 19, 'gd', 'sa', R=3.0).dz, 2)}, {pst
   ↳ r(PW('H2', 19, 'gd', 'sa', R=3.0).p_holm)}.")
534     flagged = pw[pw.skewness.abs() > 1.0]
535     allzero = int((pw.n_nonzero == 0).sum())
536     clear = int((flagged.rbc.abs() >= 0.90).sum())
537     zero_txt = "in no comparison were all paired differences zero" if allzero == 0 else f"in {allzero} compariso
   ↳ ns all paired differences were zero"
538     d.body(f"The assumption checks show where the Wilcoxon results need care. The absolute skewness of the paire
   ↳ d differences exceeded 1 in {len(flagged)} of {len(pw)} comparisons, so the symmetry assumption is doubtful ther
   ↳ e, and {zero_txt}. Of the flagged comparisons, {clear} had an absolute rank-biserial correlation of at least .90
   ↳ , where the sign of the difference is nearly constant and the conclusion does not rest on symmetry; the remainin
   ↳ g {len(flagged) - clear} should be read with caution (all values are in test_pairwise_wilcoxon.csv). Mean ranks
   ↳ (1 = best of three) were {f(mrank['S2']['sa'], 2)} for simulated annealing, {f(mrank['S2']['gd'], 2)} for gradie
   ↳ nt descent, and {f(mrank['S2']['ls'], 2)} for compression on the sphere; {f(mrank['S2cap']['sa'], 2)}, {f(mrank[
   ↳ 'S2cap']['gd'], 2)}, and {f(mrank['S2cap']['ls'], 2)} in the cap; {f(mrank['R2']['sa'], 2)}, {f(mrank['R2']['gd'
   ↳ ], 2)}, and {f(mrank['R2']['ls'], 2)} in the Euclidean disk; and {f(mrank['H2']['gd'], 2)} for gradient descent,
   ↳  {f(mrank['H2']['sa'], 2)} for simulated annealing, and {f(mrank['H2']['ls'], 2)} for compression in the hyperbo
   ↳ lic disk.")
539
540     HD("Hypothesis Evaluation", 2)
541     d.body(f"{Tb('hyp')} states the decision rules and resolves the three hypotheses; the text after it gives th
   ↳ e reasoning.")
542     h1_txt = f"Kruskal\u2013Wallis for GD\u2013SA across spaces, <i>\u03c7</i><sup>2</sup>(2) = {f(H1c_gdsa.chi2
   ↳ , 2)}, Holm {pstr(H1c_gdsa.p_holm)}; lowest mean rank: cap {AB[best_rank['S2cap']]}, Euclidean {AB[best_rank['R2
   ↳ ']]}, hyperbolic {AB[best_rank['H2']]}"
543     FF("table", "hyp", "Hypotheses, Decision Rules, and Results", ["Hypothesis", "Decision rule", "Result", "Dec
   ↳ ision"],
544         [["H1: the ranking depends on the space",
545           "Disks in a geodesic ball: Kruskal\u2013Wallis test across the three spaces of the per-setting mean pai
   ↳ red difference in relative score, per algorithm pair, Holm-adjusted over three pairs; and the lowest-mean-rank a
   ↳ lgorithm in each space. Supported if some pair has adjusted <i>p</i> < .05 and the lowest-mean-rank algorithm is
   ↳  not the same in all spaces; partly if exactly one holds.",
```

```
          h1_txt, H1_DECISION],
         ["H2: compression scales better",
          "Tammes problem, <i>N</i> \u2265 13: Wilcoxon tests of the compression slope of relative score on log<sub>10</sub> <i>N</i> against each other slope, Holm-adjusted over two tests. Supported if higher than both (adjusted <i>p</i> < .05); partly if higher than one.",
          f"Slope per decade of <i>N</i>: CP {f(h2_gd.mean_slope_a, 3)}, GD {f(h2_gd.mean_slope_b, 3)}, SA {f(h2_sa.mean_slope_b, 3)}; Holm {pstr(h2_gd.p_holm)}", H2_DECISION],
         ["H3: all recover the proven optimum for small <i>N</i>",
          "Tammes problem, <i>N</i> = 4 to 14: share of seeds with relative score \u2265 .999. Supported if the share is \u2265 .90 at every <i>N</i> for all three algorithms; partly if for one or two.",
          f"<i>N</i> with share \u2265 .90: GD {11 - int(h3['gd'].n_N_below_0p9)} of 11, SA {11 - int(h3['sa'].n_N_below_0p9)} of 11, CP {11 - int(h3['ls'].n_N_below_0p9)} of 11", H3_DECISION]],
        "GD = gradient descent; SA = simulated annealing; CP = compression. A 1% tolerance (relative score \u2265 .99) is a sensitivity check only. Data: hypothesis_tests.csv, scaling_slopes_per_seed.csv.",
        aligns=["l", "l", "l", "l"], widths=[1.1, 2.5, 1.9, 1.0])
    d.body(f"<i>H1</i> is {H1_DECISION.lower()}. Criterion (b) holds: simulated annealing has the lowest mean rank in the cap and in the Euclidean disk, and gradient descent in the hyperbolic disk. Criterion (a) fails: the difference between gradient descent and simulated annealing does not differ significantly across the three spaces after Holm correction (raw {pstr(H1c_gdsa.p)}, adjusted {pstr(H1c_gdsa.p_holm)}), and the other two pairs show no significant difference (adjusted <i>p</i> = {f(float(H1c[H1c.test.str.startswith('gd-ls')].p_holm.iloc[0]), 3, lead=False)} and {f(float(H1c[H1c.test.str.startswith('sa-ls')].p_holm.iloc[0]), 3, lead=False)}). The median per-setting difference (gradient descent minus simulated annealing, in relative score) was {f(H1c_gdsa.median_diff_S2cap, 4)} in the cap, {f(H1c_gdsa.median_diff_R2, 4)} in the Euclidean disk, and {f(H1c_gdsa.median_diff_H2, 4)} in the hyperbolic disk, so the cap resembles the Euclidean disk and the hyperbolic disk differs. A paired contrast of that difference against the Euclidean disk over the six settings gave raw {pstr(H1c_h2.p)} (adjusted {pstr(H1c_h2.p_holm)}) for the hyperbolic disk and raw {pstr(H1c_cap.p)} for the cap. With six settings per space the smallest possible two-sided <i>p</i> of this contrast is .031, so the study has little power here. For completeness, the contrast between the sphere (Tammes) and the hyperbolic disk that was specified first gave an adjusted <i>p</i> of {f(H1o.p_holm, 3, lead=False)} for the gradient descent\u2013simulated annealing pair; it compares different problems and is not used to attribute the difference to curvature. In summary, the evidence is consistent with a difference in the hyperbolic disk, but it is not significant after correction, it cannot separate curvature from area, and it was explored after the first results were seen.")
    d.body(f"<i>H2</i> is {H2_DECISION.lower()}: compression scaled worse, not better. Its relative score fell by {f(abs(h2_gd.mean_slope_a), 3)} per decade of <i>N</i> for <i>N</i> \u2265 13, much faster than that of gradient descent ({f(abs(h2_gd.mean_slope_b), 3)}) or simulated annealing ({f(abs(h2_sa.mean_slope_b), 3)}), with {pstr(h2_gd.p_holm)} against both. For <i>N</i> = 30, 40, and 60 the reference is not a proven optimum; a reference that is too low raises all relative scores in a condition by the same factor, so it cannot reverse the order of the three algorithms within a condition, but it can change slopes across <i>N</i>. The compression method is also a simplified version, so this result does not describe the event-driven original.")
    d.body(f"<i>H3</i> is {H3_DECISION.lower()} at the 0.1% tolerance. Success rates reached .90 at {11 - int(h3['sa'].n_N_below_0p9)} of 11 values of <i>N</i> for simulated annealing, {11 - int(h3['gd'].n_N_below_0p9)} for gradient descent, and {11 - int(h3['ls'].n_N_below_0p9)} for compression. As a sensitivity check, at the 1% tolerance gradient descent fell below .90 only at <i>N</i> = {', '.join(str(n) for n in below1['gd'])}, simulated annealing at <i>N</i> = {', '.join(str(n) for n in below1['sa'])}, and compression at {len(below1['ls'])} of 11 values. The bound in Equation 7 shows that the smoothing error of gradient descent can reach (ln <i>m</i>)/<i>p</i>
```

```
    ↳ > in the logarithm of the score, which at the final exponent is not small compared with a 0.1% tolerance; the sh
    ↳ ortfall of gradient descent is consistent with this bias, but the study did not test that explanation.")
558
559     # ============================================================================ Conclusions
560     import supp_section
561     supp_section.build_section(d, FF, HD, Tb, f, pstr, DATA, runs, BUDGET)
562     import conf_section
563     conf_section.build_section(d, FF, HD, Tb, Fg, f, pstr, DATA, CONFDIR, part="main")
564     import r3_section
565     r3_section.build_section(d, FF, HD, Tb, Fg, f, pstr, DATA, CONFDIR)
566
567     HD("Conclusions", 1)
568     d.body("This section summarizes how the results answer the research questions, states the limits of the stud
    ↳ y, and points to implications and next steps.")
569     HD("Summary of Findings", 2)
570     FD = json.load(open(CONFDIR + "/followup_decisions.json"))["primary"]
571     MD = json.load(open(CONFDIR + "/mini_decisions.json"))
572     _tm = pd.read_csv(f"{DATA}/conf_summary_by_condition.csv")
573     _tm = _tm[(_tm.budget == 1500) & _tm.label.str.startswith("Tammes")]
574     _tp = _tm.pivot(index="label", columns="algorithm", values="mean_rel")
575     _bs = _tm.groupby("label").success_0p1.max()
576     _mr = DEC["C4"]["mean_rank_by_method"]
577     d.body(f"The findings rest first on the preregistered confirmatory round (fresh seeds, three budget levels,
    ↳ six methods, 33 conditions, {DEC['n_runs']:,} runs) and then on the first study, which is exploratory. "
578            f"Answering RQ1 on the confirmatory round, polished gradient descent had the best mean rank ({f(_mr['
    ↳ gdp'], 2)}, against {f(_mr['lbfgsb'], 2)} for L-BFGS-B, {f(_mr['bh'], 2)} for basin hopping, {f(_mr['sa'], 2)} f
    ↳ or simulated annealing, {f(_mr['gd'], 2)} for gradient descent, and {f(_mr['ls'], 2)} for compression), and the
    ↳ better of the added L-BFGS-B and basin-hopping baselines beat both gradient descent and simulated annealing in a
    ↳ ll {DEC['C4']['n_conditions']} conditions, so the ranking of the first study, in which simulated annealing and g
    ↳ radient descent led, is a ranking among weaker methods; the best method at 1,500 sweeps was the best at 3,000 sw
    ↳ eeps in only {f(DEC['C5']['share_same_top_method_1500_vs_3000'] * 100, 0)}% of conditions, although the whole or
    ↳ dering was stable across budgets (median Kendall’s <i>W</i> {f(DEC['C5']['median_W'], 2, lead=False)}). "
579            f"Answering RQ2, the relative score of compression fell from {f(_tp.loc['Tammes N = 9', 'ls'], 3, lea
    ↳ d=False)} at <i>N</i> = 9 to {f(_tp.loc['Tammes N = 60', 'ls'], 3, lead=False)} at <i>N</i> = 60 and that of gra
    ↳ dient descent from {f(_tp.loc['Tammes N = 9', 'gd'], 3, lead=False)} to {f(_tp.loc['Tammes N = 60', 'gd'], 3, le
    ↳ ad=False)}, while polished gradient descent, L-BFGS-B, and basin hopping stayed at or above {f(min(_tp.loc[:, 'g
    ↳ dp'].min(), _tp.loc[:, 'lbfgsb'].min(), _tp.loc[:, 'bh'].min()), 3, lead=False)} for <i>N</i> from 13 to 60, so
    ↳ the number of points hurts the weaker methods most. "
580            f"Answering RQ3, no method recovered the proven optimum within 0.1% in at least 90% of seeds for all
    ↳ of <i>N</i> = 9, 13, 14, and 24 (C6 not supported): the best shares were {f(DEC['C6']['by_N']['9']['best_share']
    ↳ , 2, lead=False)} at <i>N</i> = 9, {f(DEC['C6']['by_N']['13']['best_share'], 2, lead=False)} at <i>N</i> = 13, {
    ↳ f(DEC['C6']['by_N']['14']['best_share'], 2, lead=False)} at <i>N</i> = 14, and {f(DEC['C6']['by_N']['24']['best_
    ↳ share'], 2, lead=False)} at <i>N</i> = 24. "
581            f"Answering RQ4, the ranking of gradient descent and simulated annealing depended on the space ({pstr
    ↳ (DEC['C3']['kruskal_p'])}) but by less than .01 in relative score. The confirmatory round, the mini-round, and t
```

```
    he follow-up could not settle whether curvature itself lowers the density; the final round of {R3['A1']['n_condi
    tions']} equal-area conditions did: with the boundary share in the model, the hyperbolic term was {f(R3['A1']['g
    _H'], 3)} (95% interval {f(R3['A1']['g_H_ci'][0], 3)} to {f(R3['A1']['g_H_ci'][1], 3)}) and the cap term {f(R3['
    A1']['g_S'], 3)} ({f(R3['A1']['g_S_ci'][0], 3)} to {f(R3['A1']['g_S_ci'][1], 3)}), so the preregistered decision
     is that no curvature effect as large as .02 of the density bound was detected. The intervals exclude zero, with
     opposite signs, so small effects of about 1% of the bound are not ruled out; the earlier withdrawn decision is
    in Appendix B. "
582             f"The boundary-share prediction is supported in the confirmatory round (Spearman {f(DEC['C1']['spearm
    an'], 2, lead=False)}), but that test is partly circular, because the deficit and the boundary share are both co
    mputed from the radius that the methods find, and its 26 conditions are paired. At a fixed radius, which removes
     the circularity, the mini-round was inconclusive (M1 {f(MD['M1']['spearman'], 2, lead=False)}) and the follow-u
    p supported the correlation ({f(FD['F1']['spearman'], 2, lead=False)}, {pstr(FD['F1']['perm_p_one_sided'])}); th
    e loss model is a description, not an explanation. "
583             f"Beyond benchmarking, the final round tested a rule for the number of disks that fit in a ball: it w
    as within one disk in {f(R3['B1']['share_within_1'] * 100, 0)}% of {R3['B1']['n_conditions']} new conditions (me
    an absolute error {f(R3['B1']['mean_abs_error_disks'], 2)} disks), for balls of at most 46 disks.")
584         d.body(f"The first study (exploratory; {SEEDS} paired seeds, {NCOND} conditions, one budget, three methods)
    found that gradient descent and simulated annealing were close to each other and clearly ahead of the simplified
     compression method in all four settings (on the sphere, mean relative scores of {f(mrel['S2']['gd'], 3, lead=Fa
    lse)} for gradient descent, {f(mrel['S2']['sa'], 3, lead=False)} for simulated annealing, and {f(mrel['S2']['ls'
    ], 3, lead=False)} for compression). Simulated annealing had the best mean rank on the sphere, in the cap, and i
    n the Euclidean disk, and gradient descent in the hyperbolic disk, and the difference between spaces was not sig
    nificant after correction. "
585             "In exploratory follow-up runs (see Supplementary Exploratory Analyses), a max-min polishing step rai
    sed the mean relative score of gradient descent and the number of values of <i>N</i> with a success share of at
    least .90 (its linear programs are not charged in the sweep count and added about 0.3 seconds per run), which th
    e confirmatory round then confirmed for the ranking, and at equal area the ranking did not differ detectably acr
    oss the three spaces, a result that the preregistered comparison C3 did not reproduce, although the difference i
    t found is smaller than .01 in relative score. "
586             f"For <i>N</i> = 13, no method reached a success share of .90, and {_N13H} of {_N13N} runs recovered
    the optimum within 0.1% (see Confirmatory Round).")
587         d.body(f"<b>What we can actually conclude.</b> Five conclusions are supported by the preregistered rounds an
    d do not depend on exploratory labels. First, at an equal budget of distance evaluations, the stronger baselines
     (L-BFGS-B and basin hopping) and polished gradient descent beat plain gradient descent and simulated annealing
    in every one of 33 conditions, and compression, as implemented here, is the weakest. Second, the order of the me
    thods is stable across the three budgets, but the single best method changes with the budget. Third, packing den
    sity in a bounded ball falls short of the whole-space upper bound by an amount that grows with the share of the
    ball near its wall, in all three spaces. Fourth, recovery of the proven or best-known configuration within 0.1%
    depended on <i>N</i>: at 1,500 sweeps some method reached it in at least {f(min(_bs['Tammes N = 9'], _bs['Tammes
     N = 14'], _bs['Tammes N = 24'], _bs['Tammes N = 30']) * 100, 0)}% of seeds for <i>N</i> = 9, 14, 24, and 30, bu
    t none did for <i>N</i> = 13, and the best shares were only {f(_bs['Tammes N = 40'], 2, lead=False)} for <i>N</i
    > = 40 and {f(_bs['Tammes N = 60'], 2, lead=False)} for <i>N</i> = 60. "
588             "Fifth, in the final preregistered round no curvature effect larger than .02 of the density bound was
     found at equal area, and the number of disks that fit in a ball was predicted within one disk by a one-coeffici
    ent rule. Three things are not concluded: that curvature has exactly no effect (the final-round intervals exclud
```

```
    e zero, with opposite signs, within about 1% of the bound), whether the boundary causes the loss (the boundary-l
    oss model describes the data and does not explain them), and whether the configurations found are optimal. The p
    ractical recommendation rests on the first two conclusions: for a fixed budget, use polished gradient descent, L
    -BFGS-B, or basin hopping, not plain gradient descent, annealing, or this compression method; the polishing step
     helps in the confirmatory round and costs wall-clock time that the sweep count does not charge.")
589     cfgs = cfgall
590     srow = gd4[gd4.score < 1.6].iloc[0]
591     X4 = cfgs[(cfgs.condition_id == srow.condition_id) & (cfgs.algorithm == "gd") & (cfgs.seed == srow.seed)].so
    rt_values("index")[["x0", "x1", "x2"]].values
592     coplanar = np.linalg.svd(X4, compute_uv=False)[2] < 1e-6
593     STUCKDESC = "four points on one great circle" if coplanar else "an arrangement of four points that is not th
    e tetrahedron"
594     assert abs(float(srow.score) - np.pi / 2) < 1e-3
595     d.body(f"One failure is worth recording. In {stuck} of {SEEDS} seeds, gradient descent for <i>N</i> = 4 conv
    erged to {STUCKDESC}, with a minimum distance of \u03c0/2 instead of the tetrahedral value, which shows why repe
    ated seeds matter even for four points.")
596     HD("Limitations on the Research Design and Material", 2)
597     d.body(f"The conclusions are limited as follows. (a) The first study used one budget level ({BUDGET:,} sweep
    s); the confirmatory round used three levels (500, 1,500, and 3,000), and a larger budget could still change the
     ordering, especially for compression. (b) The compression method is a simplified, time-stepped version of the m
    ethod of Lubachevsky and Stillinger (1990). (c) In the first study, algorithm parameters were tuned on four pilo
    t seeds in four pilot conditions with grids of different sizes, and unequal tuning effort could favor one method
    ; the confirmatory round used the same effort for all six methods (one parameter, six settings), but two methods
     were selected at the edge of their range, and after their ranges were extended (post hoc, nine settings for tho
    se two methods) L-BFGS-B was again selected at the edge. (d) Where no optimum is proven, the reference is a publ
    ished best-known value ({KNOWN_TXT}) or else the best value found, so relative scores near 1 do not establish op
    timality; the polished variant exceeded the best value found in some ball conditions, which shows that a best-fo
    und reference can lie below what is reachable. (e) Problem B maximizes the radius for fixed <i>n</i> and <i>R</i
    >, and uses {len(BALL)} settings with <i>R</i> = 2 or 3. The first study did not compare equal areas, so curvatu
    re and area are confounded in it; the confirmatory round compares equal areas, but only up to 4\u03c0 for the sp
    here, and its boundary-share and curvature analyses rest on 26 conditions; and the cap with <i>R</i> = 3 covers
    almost the whole sphere. (f) The comparison of curvatures rests on six settings per space and an exploratory ana
    lysis, which is why RQ4 and H1 are exploratory. (g) The Wilcoxon symmetry assumption was doubtful in part of the
     comparisons, and the 0.1% success tolerance interacts with the exponent cap of gradient descent. (h) Run times
    describe this Python implementation on one machine. (i) No claim of global optimality is made for any configurat
    ion found. (j) Equal sweeps are not equal time: simulated annealing took several times longer per run than gradi
    ent descent ({Tb('timing')}), and the polishing step is charged in sweeps but not for the time to solve its line
    ar programs. (k) The polishing step is one refinement (max-min by sequential linear programming), and its time i
    s not charged in sweeps. (l) The confirmatory round used one machine with two processes, 20 seeds, six methods w
    ith one tuned parameter each, a reference that is the best score found in {DEC['reference_types'].get('best foun
    d in this round', 0)} of 33 conditions, and a self-recorded preregistration hash. (m) The boundary-share test C1
     is partly circular, because the density deficit and the boundary share are both computed from the radius that t
    he methods find, and the 26 ball conditions are paired by area or by number of disks, so its permutation test as
    sumes more independence than the design has. The mini-round designed to remove the circularity was inconclusive
    for M1 (Spearman {f(MDEC['M1']['spearman'], 2, lead=False)}), its decision M2 was withdrawn when three duplicate
```

```
    d Euclidean conditions were removed, and the follow-up, with a second method, supported F1 (Appendix B); the bou
    ndary-loss model is a description, not an explanation.  (n) The final round used two methods at one budget (1,50
    0 sweeps), balls of at most 46 disks for the counting rule (a lower bound on the number that fit, from 20 seeds)
    , and a loss coefficient (0.46) fitted in the follow-up; its curvature decision means that no effect above .02 w
    as detected, not that the effect is zero, and its self-recorded hash is independent of the adviser\u2019s email
    only by the author\u2019s report. (o) The wall ring (every disk within one disk radius of the wall) occurred onl
    y in the two largest hyperbolic balls (radii 6 and 8) and is not a general result.")
598     HD("Implications and Future Directions", 2)
599     d.body("Well-spread point sets on spheres are used for sampling and for placing sensors or antennas, and pac
    kings in hyperbolic space relate to embeddings of tree-like networks. Within the limits above, the results sugge
    st a practical rule that rests on the confirmatory round: at a fixed budget, use polished gradient descent, L-BF
    GS-B, or basin hopping, which beat gradient descent and annealing in all 33 conditions, and not the simplified c
    ompression method; the polishing step\u2019s linear programs are not charged in the sweep count and cost extra w
    all-clock time, and the first study\u2019s preference for gradient descent or annealing applies only among those
     weaker methods. Natural extensions are an event-driven compression method, larger budgets, more ball radii, equ
    al-area and equal-density designs at areas beyond what the sphere can hold (in the Euclidean and hyperbolic plan
    es only), a count-based version of Problem B for balls larger than the 46 disks tested here, and proofs or certi
    fied bounds for the best configurations found.")
600
601     FL()
602     d.new_page()
603     HD("References", 1)
604     for r in sorted(REFS, key=lambda s: s.replace("ö", "o").lower()):
605         d.reference(r)
606     d.new_page()
607     HD("Appendix A: Sensitivity of the Main Results to Budget, Parameters, and Starting Configuration", 1)
608     d.body(f"This appendix holds the three sensitivity tables referred to in the Supplementary Exploratory Analy
    ses: {Tb('budget')} gives the mean relative score at three budgets, {Tb('sens')} the effect of changing one para
    meter at a time, and {Tb('spread')} the spread over starting configurations.")
609     while appfloats:
610         kind, key, a, k = appfloats.pop(0)
611         d.table(*a, label=APP[key], **k)
612     d.new_page()
613     HD("Appendix B: Mini-Round, Follow-Up, and the Withdrawn Decision", 1)
614     d.body("This appendix keeps the history of the work between the confirmatory round and the final round, incl
    uding a decision that was withdrawn. It is not needed to follow the main results. The Research Plan holds the da
    ted version history in its Appendix A.")
615     conf_section.build_section(d, FF, HD, Tb, Fg, f, pstr, DATA, CONFDIR, part="appendix")
616     FL()
617     return d
618
619
620 def main():
621     d0 = build({"toc": [], "figs": [], "tabs": []})
622     pg = {"toc": [(l, t, 0) for l, t in d0.headings],
```

```
            "figs": [(n, t, 0) for k, n, t in d0.captions if k == "Figure"],
            "tabs": [(n, t, 0) for k, n, t in d0.captions if k == "Table"]}
    fast = "--fast" in sys.argv and os.path.exists(OUT + "/paper_pages.json")
    if fast:
        old = json.load(open(OUT + "/paper_pages.json"))
        if all(len(old[k]) == len(pg[k]) for k in pg):
            pg = {k: [(int(e[0]) if k == "toc" else int(e[0]), e[1], int(e[2])) for e in v] for k, v in old.items()}
    for it in range(1 if fast else 5):
        d = build(pg)
        path = OUT + "/Research_Paper.docx"
        d.save(path)
        pdf = to_pdf(path, OUT)
        pages = pdf_pages(pdf)
        body0 = next(i for i in range(1, len(pages)) if any(l.strip().startswith("Algorithmic Optimization") for l in pages[i].split("\n")[:4]))
        newtoc = []
        for (l, t, _) in pg["toc"]:
            if t in ("List of Figures", "List of Tables"):
                pnum = find_exact(pages, [t], start_page=3)[0]
            else:
                pnum = find_exact(pages, [t], start_page=body0)[0] or find_pages(pages, [t], start_page=body0)[0]
            newtoc.append((l, t, pnum))
        newfigs = [(n, t, find_pages(pages, [f"Figure {n}. {t}"], start_page=body0)[0]) for n, t, _ in pg["figs"]]
        newtabs = [(n, t, find_exact(pages, [f"Table {n}"], start_page=body0)[0]) for n, t, _ in pg["tabs"]]
        new = {"toc": newtoc, "figs": newfigs, "tabs": newtabs}
        if new == pg:
            break
        pg = new
    import omml
    omml.MODE = "omml"           # final .docx carries native Word equations; the PDF above used typeset images
    build(pg).save(path)
    json.dump({k: [list(map(str, e)) for e in v] for k, v in pg.items()}, open(OUT + "/paper_pages.json", "w"))
    print("pages:", len(pages), "| body starts at page", body0 + 1)
    print("abstract words:", words(abstract))
    print("entries without page:", [e for k in ("toc", "figs", "tabs") for e in pg[k] if not e[2]])


if __name__ == "__main__":
    main()
```

make_code_listing.py

Builds this code listing.

```
"""make_code_listing.py -- builds Code_Listing.docx (all code, syntax-colored, line-numbered)."""
import os
import sys
import json
import platform
import importlib.metadata as md
from pygments import lex
from pygments.lexers import PythonLexer
from pygments.token import Token
from docbuilder import *
from docx.enum.text import WD_LINE_SPACING

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))   # folder that contains code/ and out/
OUT = os.path.join(ROOT, "out")
CODE = os.path.join(ROOT, "code")
LISTING_DATE = "December 3, 2026"
MONO = "Courier New"
SIZE = 9
GAP_ABOVE = 4.0      # extra space (pt) between the code above and the text
GAP_BELOW = 13.9      # extra space (pt) between the text and the code below
WIDTH = 112          # code characters per line in landscape (9 in at 9 pt Courier New, minus line numbers)

FILES = [
    ("build_all.py", "Master script. Runs the unit tests, all experiments, the statistics, the figures, and buil
↳ ds every deliverable."),
    ("geometry.py", "Geometry of the sphere, the hyperbolic plane (hyperboloid model), and the Euclidean plane:
↳ distances, gradients, exponential maps, sampling."),
    ("algorithms.py", "The packing problems and the three optimizers: gradient descent on a soft-minimum energy,
↳  simulated annealing, and compression."),
    ("tests.py", "Validation tests: closed-form distances, metric axioms, finite-difference gradients, and the k
↳ nown Tammes optima for 4, 6, and 12 points."),
    ("experiments.py", "Runs every condition with 30 paired seeds under an equal budget and writes the raw CSV f
↳ iles."),
    ("analyze.py", "Summary statistics, Friedman and Wilcoxon tests with Holm correction, effect sizes, and the
↳ hypothesis tests."),
    ("make_figures.py", "All figures, drawn only from the CSV files, with the mathematical accuracy assertions."
↳ ),
    ("docbuilder.py", "Shared APA 7 document builder used for the plan, paper, and this listing."),
```

```
    ("make_plan.py", "Builds the Research Plan (written before the experiments)."),
    ("make_paper.py", "Builds the Research Paper; every number and table value is read from the CSV files."),
    ("make_code_listing.py", "Builds this code listing."),
    ("make_package.py", "Builds the data package: README, seed list, SHA-256 manifest, and the ZIP file."),
    ("supplementary.py", "Supplementary exploratory analyses of the first study: equal area, sensitivity, polish
↳ ed gradient descent, timing, checks."),
    ("supp_section.py", "Writes the Supplementary Exploratory Analyses section of the paper from the supplementa
↳ ry CSV files."),
    ("omml.py", "Native Word equations and their image versions."),
    ("newalgs.py", "Added baselines of the confirmatory round: SciPy L-BFGS-B and basin hopping on the soft-mini
↳ mum energy."),
    ("confirm_design.py", "Conditions, reference densities, boundary share, and density deficit of the confirmat
↳ ory round."),
    ("record_env.py", "Records hardware and software versions."),
    ("confirm_tune.py", "Equal-effort pilot tuning of all six methods on pilot seeds 9101 to 9104."),
    ("confirm_run.py", "Runs the confirmatory round: fresh seeds 101 to 120, three budgets, six methods."),
    ("confirm_count.py", "Count-based version of the disk problem (exploratory)."),
    ("confirm_analyze.py", "Analysis of the confirmatory round exactly as preregistered."),
    ("conf_figures.py", "Figures of the confirmatory round."),
    ("conf_section.py", "Writes the Confirmatory Round section of the paper from the conf_* files."),
    ("confirm_configs.py", "Reruns the confirmatory round with the same seeds and saves every final configuratio
↳ n (configs/cfg_<cid>.npz) and the wall metrics."),
    ("confirm_retune.py", "Extended candidate grids for compression and L-BFGS-B on the pilot conditions (ADDEND
↳ UM 1)."),
    ("confirm_run_retuned.py", "Reruns compression and L-BFGS-B on the confirmatory conditions with the re-tuned
↳  parameters."),
    ("confirm_extra.py", "Post-hoc analyses of ADDENDUM 1 (re-tuned baselines, wall metrics) and the boundary-lo
↳ ss model."),
    ("confirm_mini.py", "Matched-boundary mini-round: design, runs, and rules M1 to M3."),
    ("confirm_followup.py", "Follow-up to the mini-round: two methods, rules F1 to F3, and L-BFGS-B on the mini-
↳ round conditions."),
    ("confirm_round3.py", "Final preregistered round: equal-area balls (part A) and the predictive rule for the
↳ number of disks (part B)."),
    ("r3_section.py", "Writes the Final Round section of the paper from the round-3 files."),
    ("checks.py", "Automated checks of the final DOCX and PDF files."),
    ("fill.py", "Helper that reports unused space at the bottom of each PDF page."),
    ("pilot.py", "Pilot runs on separate seeds used to check speed and sanity before tuning."),
    ("tune.py", "Pilot tuning of gradient descent and simulated annealing on seeds 9001 to 9004."),
    ("tune2.py", "Further pilot tuning of gradient descent and compression."),
    ("tune3.py", "Final pilot tuning of the compression method."),
]

COL = {Token.Keyword: "0033B3", Token.Literal.String: "067D17", Token.Comment: "7A7A7A", Token.Literal.Number: "
↳ 1750EB",
       Token.Name.Builtin: "871094", Token.Name.Function: "00627A", Token.Name.Class: "00627A", Token.Name.Decor
```

```
  ↳ ator: "9E880D",
        Token.Operator.Word: "0033B3"}


def color_of(tt):
    while tt is not None:
        if tt in COL:
            return COL[tt], tt in (Token.Comment,) or str(tt).startswith("Token.Comment")
        tt = tt.parent
    return None, False


def shade(p):
    ppr = p._p.get_or_add_pPr()
    bd = OxmlElement("w:pBdr")
    for edge in ("top", "left", "bottom", "right"):
        e = OxmlElement("w:" + edge)
        e.set(qn("w:val"), "single")
        e.set(qn("w:sz"), "4")
        e.set(qn("w:space"), "3")
        e.set(qn("w:color"), "BFBFBF")
        bd.append(e)
    sh = OxmlElement("w:shd")
    sh.set(qn("w:val"), "clear")
    sh.set(qn("w:color"), "auto")
    sh.set(qn("w:fill"), "F2F2F2")
    after = ("tabs", "suppressAutoHyphens", "kinsoku", "wordWrap", "overflowPunct", "topLinePunct", "autoSpaceDE
  ↳ ", "autoSpaceDN",
             "bidi", "adjustRightInd", "snapToGrid", "spacing", "ind", "contextualSpacing", "mirrorIndents", "su
  ↳ ppressOverlap", "jc",
             "textDirection", "textAlignment", "textboxTightWrap", "outlineLvl", "divId", "cnfStyle", "rPr", "se
  ↳ ctPr", "pPrChange")
    nxt = next((c for c in ppr if c.tag in [qn("w:" + t) for t in after]), None)
    for el in (bd, sh):
        if nxt is not None:
            nxt.addprevious(el)
        else:
            ppr.append(el)
    # the box edge sits on the page margin: indent the text by the border space plus the line width
    p.paragraph_format.left_indent = Pt(3.5)
    p.paragraph_format.right_indent = Pt(3.5)


def line_runs(src):
    """Return a list (one per source line) of lists of (text, color, italic)."""
```

```
    lines = [[]]
    for tt, val in lex(src, PythonLexer()):
        c, it = color_of(tt)
        parts = val.split("\n")
        for k, part in enumerate(parts):
            if k > 0:
                lines.append([])
            if part:
                lines[-1].append((part.replace("\t", "    "), c, it))
    if lines and not lines[-1]:
        lines.pop()
    return lines


def add_code(d, src):
    lines = line_runs(src)
    for no, runs in enumerate(lines, 1):
        chunks, cur, n = [], [], 0
        for text, c, it in runs:
            while text:
                room = WIDTH - n
                if room <= 0:
                    chunks.append(cur)
                    cur, n = [], 0
                    continue
                piece, text = text[:room], text[room:]
                cur.append((piece, c, it))
                n += len(piece)
        chunks.append(cur)
        for k, ch in enumerate(chunks):
            p = d.doc.add_paragraph()
            pf = p.paragraph_format
            pf.line_spacing = 1.0
            pf.space_after = Pt(0)
            pf.space_before = Pt(0)
            pf.first_line_indent = Inches(0)
            pf.alignment = WD_ALIGN_PARAGRAPH.LEFT
            shade(p)
            lab = f"{no:>4}  " if k == 0 else "    \u21b3 "
            r = p.add_run(lab)
            set_run_font(r, name=MONO, size=SIZE, color="8C8C8C")
            for text, c, it in ch:
                r = p.add_run(text)
                set_run_font(r, name=MONO, size=SIZE, color=c if c else "000000", italic=it or None)
    return len(lines)
```

```
def between(p, before=0.0, after=0.0):
    """Text between code blocks: exact double spacing, with equal room above and below."""
    pf = p.paragraph_format
    pf.line_spacing_rule = WD_LINE_SPACING.EXACTLY
    pf.line_spacing = Pt(27.6)
    pf.space_before = Pt(before)
    pf.space_after = Pt(after)


def landscape_section(d):
    d.doc.add_section(WD_SECTION.NEW_PAGE)
    sec = d.doc.sections[-1]
    sec.orientation = WD_ORIENT.LANDSCAPE
    sec.page_width, sec.page_height = Inches(11), Inches(8.5)
    sec.header.is_linked_to_previous = True


REPORTED = [("Python", "3.12.3"), ("numpy", "2.4.4"), ("scipy", "1.17.1"), ("pandas", "3.0.2"), ("matplotlib", "
↳ 3.10.8"),
            ("python-docx", "1.2.0"), ("Pygments", "2.20.0"), ("pdfplumber", "0.11.9"), ("Pillow", "12.1.1")]


def versions():
    return REPORTED


def _versions_current():
    out = [("Python", platform.python_version())]
    for pk in ("numpy", "scipy", "pandas", "matplotlib", "python-docx", "Pygments", "pdfplumber", "Pillow"):
        try:
            out.append((pk, md.version(pk)))
        except md.PackageNotFoundError:
            pass
    return out


def build(pg):
    d = APADoc(title=TITLE)
    d.title_page(LISTING_DATE)
    d.new_page()
    d.contents_page("Table of Contents", "Chapter", pg["toc"], "toc", field_instr='TOC \\o "1-3" \\z \\u')
    d.new_page()
    p = d._para(align=WD_ALIGN_PARAGRAPH.CENTER)
```

```
    set_run_font(p.add_run(TITLE), bold=True)
    d.heading("How to Run the Code", 1)
    d.body("This document lists all code used in the study. The sections below give the software environment and the exact commands that reproduce every result.")
    d.heading("Software Environment", 2)
    d.body("The reported runs used the versions below; an independent full rerun (python3 build_all.py --rerun) under Python 3.13.18 with numpy 2.5.3, scipy 1.18.1, pandas 3.0.5, and matplotlib 3.11.2 reproduced every score, test statistic, and p-value in the CSV files exactly; only the wall-clock times differ. No other libraries are needed, and the optimizers use only numpy and scipy-compatible routines from numpy.")
    for name, v in versions():
        p = d._para(indent_first=0.0, left=0.5, spacing=1.0)
        d.add_marked(p, f"{name} {v}")
    p.paragraph_format.space_after = Pt(13.8)   # the list ends one double-spaced step above the next heading
    d.heading("Commands", 2)
    d.body("Place all files listed below in one folder and run the commands in that order. The first command runs the validation tests and stops if any test fails. The second command runs the unit tests, all experiments, the statistics, the figures, and the builds of the plan, paper, listing, and data package; add --rerun to recompute every run from scratch (about 17 minutes on one core for the first study). Seeds 1 to 30 are used for evaluation and are fixed, so a rerun reproduces every value. Run them from that folder.")
    for cmd in ("python3 tests.py", "python3 build_all.py", "python3 build_all.py --rerun"):
        p = d._para(indent_first=0.0, left=0.5, spacing=1.0)
        set_run_font(p.add_run(cmd), name=MONO, size=SIZE)
    landscape_section(d)
    between(d.heading("Source Code Listing", 1))
    between(d.body("Each file is listed in full with line numbers and syntax colors. A long line continues on the next row, marked with an arrow, and its indentation is kept."))
    total = 0
    for fn, desc in FILES:
        between(d.heading(fn, 2), before=GAP_ABOVE)
        between(d.body(desc), after=GAP_BELOW)
        src = open(f"{CODE}/{fn}").read()
        total += add_code(d, src)
    between(d.heading("Unit Test Output", 1), before=GAP_ABOVE)
    between(d.body("The output below is the complete result of running python3 tests.py before the experiments."), after=GAP_BELOW)
    add_code_text = open(f"{OUT}/unit_test_output.txt").read()
    for line in add_code_text.rstrip("\n").split("\n"):
        p = d.doc.add_paragraph()
        p.paragraph_format.line_spacing = 1.0
        p.paragraph_format.space_after = Pt(0)
        p.paragraph_format.first_line_indent = Inches(0)
        shade(p)
        txt = line
        while True:
            r = p.add_run(txt[:WIDTH + 6])
```

```
            set_run_font(r, name=MONO, size=SIZE)
            txt = txt[WIDTH + 6:]
            if not txt:
                break
            p = d.doc.add_paragraph()
            p.paragraph_format.line_spacing = 1.0
            p.paragraph_format.space_after = Pt(0)
            p.paragraph_format.first_line_indent = Inches(0)
            shade(p)
    return d, total


def main():
    d0, total = build({"toc": []})
    entries = [(l, t) for l, t in d0.headings]
    pg = {"toc": [(l, t, 0) for l, t in entries]}
    for it in range(4):
        d, total = build(pg)
        path = OUT + "/Code_Listing.docx"
        d.save(path)
        pdf = to_pdf(path, OUT)
        pages = pdf_pages(pdf)
        body0 = next(i for i in range(1, len(pages)) if any(l.strip().startswith("Algorithmic Optimization") for
 l in pages[i].split("\n")[:4]))
        newtoc = []
        for l, t, _ in pg["toc"]:
            pnum = find_exact(pages, [t], start_page=body0)[0]
            newtoc.append((l, t, pnum))
        new = {"toc": newtoc}
        if new == pg:
            break
        pg = new
    print("listing pages:", len(pages), "| code lines:", total, "| missing:", [e for e in pg["toc"] if not e[2]]
)


if __name__ == "__main__":
    main()
```

make_package.py

Builds the data package: README, seed list, SHA-256 manifest, and the ZIP file.

```
"""make_package.py -- builds CSV_Data.zip (CSV files, README.md, seed list, SHA-256 manifest)."""
import csv
import glob
import hashlib
import os
import shutil
import zipfile

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))   # folder that contains code/ and out/
OUT = os.path.join(ROOT, "out")
DATA = OUT + "/data"
PKG = OUT + "/CSV_Data"
TITLE = "Algorithmic Optimization of Dense Sphere Packings in Non-Euclidean Spaces"

COLS = {
    "method": "Method that produced the row (gdp = gradient descent with compression, lbfgsb = L-BFGS-B; or the name of the best method in r3a_conditions.csv).",
    "K_gdp": "Largest disk count reached in at least half of the seeds by gradient descent with compression.",
    "K_lbfgsb": "Largest disk count reached in at least half of the seeds by L-BFGS-B.",
    "K2": "Larger of K and K_lbfgsb (both methods combined).",
    "deficit0_gdp": "Fixed-radius density deficit using K_gdp.",
    "deficit0_lbfgsb": "Fixed-radius density deficit using K_lbfgsb.",
    "deficit0_best": "Fixed-radius density deficit using K2 (best of both methods).",
    "K_pred": "Predicted disk count, floor(n_ref (1 - 0.46 b0) + 0.5) (round 3, part B, rule B1).",
    "b": "Boundary share of the best configuration (share of the covered area that lies within one radius of the wall).",
    "cell": "Cell label of round 3 part A: area and disk count joined by an underscore.",
    "design": "Supplementary equal-area design label.",
    "area": "Container area in the equal-area comparison (same in all three spaces).",
    "mean_gd": "Mean relative score of gradient descent.", "mean_sa": "Mean relative score of simulated annealing.", "mean_ls": "Mean relative score of compression.",
    "gd_minus_sa": "mean_gd minus mean_sa.", "p_wilcoxon_gd_sa": "Unadjusted Wilcoxon signed-rank p, GD vs SA.", "p_friedman": "Friedman p across the three algorithms.", "p_holm_gd_sa": "Holm-adjusted Wilcoxon p, GD vs SA, within the equal-area family.",
    "density": "Packing density of the disks in the container.", "sweeps_used": "Budget used in sweeps (1 sweep = n(n-1)/2 pair-distance evaluations).", "recomputed": "Score recomputed from final coordinates.",
    "reference_value": "Reference value used for the relative score.", "relative_score": "score divided by reference_value.", "reference_type": "verified optimum, published best known, or best found.",
    "status": "Reference status: verified optimum, published best known, or best found.", "source": "Source of the reference value.",
    "check": "Independent recomputation of the score from final coordinates.", "time_s": "Wall-clock seconds of the run.",
    "variant": "Sensitivity variant (budget, soft-minimum exponent, annealing schedule, or starting configuration).", "budget": "Budget in sweeps for the variant.",
    "value": "Value of the named check.", "min": "Minimum over seeds.", "median": "Median over seeds.", "max": "
```

```
  ↳ Maximum over seeds.", "std": "Standard deviation over seeds.", "label": "Condition label.",
36      "space": "Geometry: S2 = unit sphere, H2 = hyperbolic plane (curvature -1), R2 = Euclidean plane (control)."
  ↳ ,
37      "condition_id": "Integer id of the condition: 100-114 sphere (N = 4,5,...,14,24,30,40,60), 200-205 hyperboli
  ↳ c ball, 300-305 Euclidean ball, 400-405 spherical-cap ball; the six ball settings are (R,n) = (2,7),(2,12),(2,19
  ↳ ),(3,12),(3,19),(3,30).",
38      "n": "Sphere: number of points N. Ball problems: number of disks n.",
39      "R": "Container (ball) radius in the units of the space; empty for the sphere.",
40      "algorithm": "gd = gradient descent, sa = simulated annealing, ls = compression (Lubachevsky-Stillinger-styl
  ↳ e), gdp = gradient descent with a final max-min polishing step (supplementary files only).",
41      "seed": "Random seed 1-30 shared by all algorithms (the initial configuration depends only on seed and condi
  ↳ tion).",
42      "score": "Final score recomputed from the final coordinates: minimum geodesic distance in radians (sphere) o
  ↳ r common disk radius (ball problems, units of the space).",
43      "density": "Ball problems only: share of the container area covered by the n disks.",
44      "pair_evals": "Pair-distance evaluations used (the budget unit).",
45      "sweeps_used": "pair_evals divided by n(n-1)/2.",
46      "iterations": "Algorithm-specific count: gradient steps (gd), proposed moves (sa), time steps (ls).",
47      "time_s": "Wall-clock seconds for the run.",
48      "sweeps_to_99pct": "Sweeps used when the best score so far first reached 99% of the run's own final score.",
49      "recomputed_score": "Score recomputed again from the stored final configuration (integrity check; equals sco
  ↳ re).",
50      "reference_type": "verified optimum (published value) or best found (highest score of any run in that condit
  ↳ ion).",
51      "reference_value": "Reference score in the same units as score.",
52      "relative_score": "score divided by reference_value.",
53      "gap_to_reference": "reference_value minus score.",
54      "success_0.1pct": "1 if relative_score >= 0.999, else 0.",
55      "success_1pct": "1 if relative_score >= 0.99, else 0.",
56      "index": "Index of the point or disk within the configuration (0-based).",
57      "x0": "Coordinate 1. Sphere: x. Hyperbolic plane: hyperboloid coordinate x0 (x0 = cosh of distance from orig
  ↳ in). Euclidean plane: x.",
58      "x1": "Coordinate 2. Sphere: y. Hyperbolic plane: hyperboloid coordinate x1. Euclidean plane: y.",
59      "x2": "Coordinate 3. Sphere: z. Hyperbolic plane: hyperboloid coordinate x2. Euclidean plane: empty.",
60      "budget_fraction": "Fraction of the budget used (41 log-spaced values from 0.001 to 1).",
61      "best_score": "Best score so far at that budget fraction (same units as score).",
62      "n_seeds": "Number of seeds.",
63      "mean_score": "Mean of score over seeds.", "sd_score": "Standard deviation of score (n-1).",
64      "ci95_lo": "Lower end of the 95% t confidence interval of the mean score.", "ci95_hi": "Upper end of the sam
  ↳ e interval.",
65      "median_score": "Median score.", "min_score": "Minimum score.", "max_score": "Maximum score.",
66      "mean_relative": "Mean relative_score.", "sd_relative": "Standard deviation of relative_score.",
67      "rel_ci95_lo": "Lower end of the 95% t interval of mean relative score.", "rel_ci95_hi": "Upper end.",
68      "mean_gap": "Mean gap_to_reference.",
69      "success_rate_0p1pct": "Share of seeds with success_0.1pct = 1.", "success_rate_1pct": "Share of seeds with
```

```
  ↳ success_1pct = 1.",
 70     "mean_density": "Mean density (ball problems).", "sd_density": "Standard deviation of density.",
 71     "mean_time_s": "Mean wall-clock seconds.", "sd_time_s": "Standard deviation of seconds.",
 72     "mean_sweeps_to_99": "Mean of sweeps_to_99pct.", "mean_pair_evals": "Mean pair-distance evaluations.",
 73     "chi2": "Friedman chi-square statistic (2 df), or chi-square of the slope Friedman test.", "df": "Degrees of
  ↳  freedom.",
 74     "p": "Raw p value.", "p_holm": "Holm-adjusted p value (family = all tests of the same kind within one space;
  ↳  H1 and H2 families as in analyze.py).",
 75     "kendall_W": "Kendall's W = chi2 / (n_seeds * (k - 1)) with k = 3.",
 76     "significant": "1 if p_holm < .05.",
 77     "alg_a": "First algorithm of a pair.", "alg_b": "Second algorithm of a pair.",
 78     "W": "Smaller of the two signed-rank sums of the Wilcoxon test.",
 79     "rbc": "Matched-pairs rank-biserial correlation (positive = alg_a higher).",
 80     "dz": "Standardized mean difference of the paired differences (mean / SD).",
 81     "mean_diff": "Mean of (alg_a score - alg_b score) over seeds; in hypothesis_tests.csv, difference of mean sl
  ↳ opes.",
 82     "ci_lo": "Lower end of the 95% t interval of mean_diff.", "ci_hi": "Upper end.",
 83     "n_nonzero": "Number of seeds with a nonzero paired difference.",
 84     "winner": "Algorithm significantly higher after Holm correction, or none.",
 85     "best_mean_algorithm": "Algorithm with the highest mean score in the condition.",
 86     "mean_rank": "Mean within-seed rank (1 = highest score of the three).",
 87     "mean_relative_score": "Mean relative score pooled over all conditions of the space.",
 88     "hypothesis": "H1, H2, or H3.", "test": "Description of the test.",
 89     "n_S2": "H1: number of sphere conditions.", "n_H2": "H1: number of hyperbolic conditions.",
 90     "median_diff_S2": "H1: median over sphere conditions of the per-condition mean paired difference in relative
  ↳  score.",
 91     "median_diff_H2": "H1: same for hyperbolic conditions.", "U": "Mann-Whitney U statistic.",
 92     "mean_slope_a": "H2: mean over seeds of the slope of relative score on log10 N for the first algorithm.",
 93     "mean_slope_b": "H2: same for the second algorithm.",
 94     "min_success_rate": "H3: smallest success rate over N = 4..14.", "n_N_below_0p9": "H3: number of N in 4..14
  ↳ with success rate below 0.90.",
 95     "supported": "H3: 1 if the success rate is at least 0.90 at every N in 4..14.",
 96     "time_exponent": "Slope of log10 mean time on log10 N (sphere, N >= 13).", "time_intercept": "Intercept of t
  ↳ hat fit.",
 97     "gd": "Slope per decade of N for gradient descent (scaling_slopes_per_seed.csv).",
 98     "sa": "Same for simulated annealing.", "ls": "Same for compression.",
 99     "hyperbolic_area": "2 pi (cosh R - 1).", "euclidean_area": "pi R^2.",
100     "hyperbolic_fraction_within_1_of_boundary": "Share of the hyperbolic disk's area within distance 1 of its bo
  ↳ undary.",
101     "euclidean_fraction_within_1_of_boundary": "Same for the Euclidean disk.",
102     "theta_deg": "Published optimal minimum angular distance in degrees.", "theta_rad": "Same in radians.", "sou
  ↳ rce": "Literature source.",
103     "target_radius": "Target disk radius.", "largest_tested_n_that_fits": "Largest tested n whose best radius re
  ↳ ached the target (0 if none).",
104     "best_radius_at_that_n": "Best radius over all runs at that n.",
```

```
    "mean_rel_diff": "Mean of (alg_a - alg_b) divided by the reference value.",
    "share_zero_diff": "Share of seeds with a zero paired difference.",
    "skewness": "Sample skewness of the paired differences (Wilcoxon symmetry check).",
    "practical": "1 if |mean_rel_diff| >= 0.005 (practical-importance threshold).",
    "significant_and_practical": "1 if significant and practical.",
    "wilcoxon_symmetry_flag": "1 if |skewness| > 1.",
    "spherical_area": "2 pi (1 - cos R) for R <= pi, else empty.",
    "spherical_fraction_within_1_of_boundary": "Share of the spherical cap's area within distance 1 of its boundary.",
    "median_diff_S2cap": "H1: median over spherical-cap settings of the per-setting mean paired difference in relative score.",
    "median_diff_R2": "H1: same for Euclidean settings.",
    "group": "Confirmatory round: tammes, equal_area, or large.", "best_method": "Method with the highest mean score in the condition (1,500 sweeps).",
    "r_best": "That highest mean score (disk radius).", "deficit": "Density deficit 1 - density / reference density.", "boundary_share": "Share of the ball's area within distance r_best of its boundary.",
    "ref_density": "Reference density: upper bound on the density of any packing of disks of radius r_best in the whole space.",
    "mean_rel": "Mean relative score over 20 seeds.", "sd_rel": "Standard deviation of the relative score.", "success_0p1": "Share of seeds with relative score >= .999.", "success_1": "Share of seeds with relative score >= .99.",
    "mean_sweeps": "Mean budget used, in sweeps.", "mean_sweeps_used": "Mean budget used, in sweeps.", "median_time_s": "Median wall-clock seconds per run.",
    "mean_gd_minus_sa": "Mean over seeds of the relative score of GD minus that of SA.", "kendall_W": "Kendall's W of the six methods across the three budget levels.",
    "mean_rank_gd": "Mean rank of GD within seeds (1 = best).", "mean_rank_gdp": "Same for GD+P.", "mean_rank_sa": "Same for SA.", "mean_rank_ls": "Same for compression.", "mean_rank_lbfgsb": "Same for L-BFGS-B.", "mean_rank_bh": "Same for basin hopping.",
    "touch_share": "Share of disks whose gap to the wall limit is at most 1% of the disk radius.", "ring": "1 if every disk lies within one disk radius of the wall limit.", "max_gap_over_r": "Largest gap of a disk to the wall limit, in units of the disk radius.",
    "mean_touch_share": "Mean touch_share.", "ring_share": "Share of runs with ring = 1.", "median_max_gap_over_r": "Median of max_gap_over_r.", "runs": "Number of runs.",
    "r0": "Mini-round: fixed disk radius.", "b0": "Mini-round: design boundary share.", "label": "Condition label or space name.", "area": "Area of the ball.", "deficit0": "Mini-round: fixed-radius density deficit.",
    "success": "1 if the score reached the fixed radius r0 (mini-round) .", "rel_old": "Mean relative score with the original tuned value.", "rel_new": "Mean relative score with the re-tuned value.", "space": "Space of the condition (R2, H2, S2cap, or S2).", "cid": "Condition id of the mini-round.",
    "n_ref": "Reference count of disks.", "K": "Largest number of disks of radius 0.30 reached in at least half of 20 seeds.", "K_over_n_ref": "K divided by the reference count.", "share_at_K": "Share of seeds that reached the radius at K.",
    "radius": "Disk radius of the count-based problem.", "reached": "1 if the score reached the radius.", "value": "Value of the tuned parameter (tuning_runs.csv) or of the check (supp_checks.csv).", "pilot": "Index of the pilot condition.", "rel": "Score relative to the best pilot score.",
    "assertion": "Name of a figure accuracy assertion.", "passed": "True if it passed.", "detail": "Numerical de
```

```
    tail.",
129 }
130
131
132 def sha(path):
133     h = hashlib.sha256()
134     with open(path, "rb") as f:
135         for chunk in iter(lambda: f.read(1 << 20), b""):
136             h.update(chunk)
137     return h.hexdigest()
138
139
140 def main():
141     if os.path.exists(PKG):
142         shutil.rmtree(PKG)
143     os.makedirs(PKG)
144     files = sorted(glob.glob(DATA + "/*.csv"))
145     for f in files:
146         shutil.copy(f, PKG)
147     shutil.copy(OUT + "/unit_test_output.txt", PKG)
148     os.makedirs(PKG + "/confirmatory")
149     for nm in ("PREREGISTRATION.md", "PREREGISTRATION.sha256", "TUNED_AND_ENV.sha256", "tuned_params.json", "har
    dware.json", "decisions.json",
150                "PREREGISTRATION_ADDENDUM_1.md", "PREREGISTRATION_MINIROUND.md", "PREREG_ADDENDA.sha256", "RETUNE
    D.sha256", "tuned_params_retuned.json",
151                "decisions_retuned.json", "decisions_best_found_references.json", "mini_decisions.json", "extra.j
    son", "packomania_values.csv", "PREREGISTRATION_FOLLOWUP.md", "FOLLOWUP.sha256", "PREREGISTRATION_ROUND3.md", "R
    OUND3.sha256", "ROUND3_START.txt", "ROUND3_END.txt", "PLAN_FILES.sha256", "round3_decisions.json", "r3a_runs.csv
    ", "r3a_conditions.csv", "r3b_design.csv", "r3b_scan.csv", "r3b_summary.csv", "round3.log", "followup_decisions.
    json", "CORRECTION_NOTE_1.md", "CORRECTION.sha256", "FOLLOWUP_RUNTIMES.txt"):
152         shutil.copy(OUT + "/conf/" + nm, PKG + "/confirmatory")
153     for f in sorted(glob.glob(OUT + "/conf/*.csv")) + sorted(glob.glob(OUT + "/conf/*.log")):
154         shutil.copy(f, PKG + "/confirmatory")
155     shutil.copytree(OUT + "/conf/configs", PKG + "/confirmatory/configs")
156     shutil.copytree(OUT + "/conf/retuned", PKG + "/confirmatory/retuned")
157     os.makedirs(PKG + "/code")
158     for src in sorted(glob.glob(os.path.join(ROOT, "code", "*.py"))):
159         if "_backup" in src:
160             continue
161         shutil.copy(src, PKG + "/code")
162     shutil.copytree(os.path.join(ROOT, "code", "frozen"), PKG + "/code/frozen")
163     lines = [f"# CSV Data for \"{TITLE}\"", "",
164              "Researcher: Marian Ela J. Alibadbad (Region VIII, MCS-I). Every file was produced by `python3 buil
    d_all.py`.",
165              "Seeds 1-30 are used for evaluation and are listed in `seed_list.csv`; pilot seeds 9001-9004 are ne
```

```
   ↳ ver used in the evaluation.",
166                "All files are UTF-8, comma-separated, with a header row. Scores are in radians (sphere) or in the
   ↳ units of the space (ball problems).", "",
167                "## Files", ""]
168      undefined = set()
169      for f in files:
170          name = os.path.basename(f)
171          with open(f) as fh:
172              rd = csv.reader(fh)
173              header = next(rd)
174              n = sum(1 for _ in rd)
175          lines.append(f"### {name}  ({n:,} rows)")
176          lines.append("")
177          for c in header:
178              if c not in COLS:
179                  undefined.add(c)
180              lines.append(f"- `{c}`: {COLS.get(c, 'UNDEFINED')}")
181          lines.append("")
182      ncon = len(glob.glob(PKG + "/confirmatory/runs_*.csv"))
183      nconrows = sum(sum(1 for _ in open(f)) - 1 for f in glob.glob(PKG + "/confirmatory/runs_*.csv"))
184      lines += ["### confirmatory/", "",
185                f"All files of the confirmatory round and its follow-ups. Frozen originals are never regenerated:
   ↳ the `PREREGISTRATION*.md` files, the `*.sha256` files, `CORRECTION_NOTE_1.md`, `tuned_params*.json` and `hardwar
   ↳ e.json`.",
186                "",
187                "- Preregistration documents: `PREREGISTRATION.md` (main round), `PREREGISTRATION_ADDENDUM_1.md`,
   ↳ `PREREGISTRATION_MINIROUND.md`, `PREREGISTRATION_FOLLOWUP.md`, `PREREGISTRATION_ROUND3.md`; `CORRECTION_NOTE_1.m
   ↳ d` is the correction note. Their SHA-256 hashes, taken before the data they govern were produced, are in `PREREG
   ↳ ISTRATION.sha256`, `PREREG_ADDENDA.sha256`, `FOLLOWUP.sha256`, `ROUND3.sha256`, `CORRECTION.sha256`, `RETUNED.sh
   ↳ a256`, `TUNED_AND_ENV.sha256` and `PLAN_FILES.sha256`; `ROUND3_START.txt` and `ROUND3_END.txt` record the time s
   ↳ tamps of round 3.",
188                f"- `runs_<cid>.csv` ({ncon} files, condition ids 1000-{1000 + ncon - 1}; {nconrows:,} rows in all
   ↳ ): one row per condition, method, budget and seed of the confirmatory round (columns as in `runs_raw.csv` plus `
   ↳ condition_id`, `recomputed_score`, `sweeps_used`).",
189                "- `retuned/runs_<cid>.csv`: the same conditions rerun with the re-tuned parameters of ADDENDUM 1;
   ↳  `tuned_params_retuned.json`, `decisions_retuned.json` and `decisions_best_found_references.json` hold the param
   ↳ eters and the decisions.",
190                "- `walls_<cid>.csv`: wall metrics of the final configurations (conditions 1007-1032, ball problem
   ↳ s); `configs/cfg_<cid>.npz`: every final configuration, arrays `X[budget, seed, method, n, dim]`, `budgets`, `se
   ↳ eds`, `algs`; `configs.log` is the log of that rerun (maximum difference of recomputed scores).",
191                "- `tuning_runs.csv`, `tuning_runs_extended.csv`, `tuned_params.json`: tuning runs and the tuned v
   ↳ alues; `decisions.json`: the decisions C1-C6; `extra.json`: post-hoc analyses; `hardware.json`: the machine and
   ↳ software versions of the runs; `packomania_values.csv`: reference radii.",
192                "- Count-based problem: `count_runs.csv`, `count_summary.csv`. Mini-round: `mini_design.csv`, `min
   ↳ i_scan.csv`, `mini_summary.csv`, `mini_lbfgsb_scan.csv`, `mini_lbfgsb_summary.csv`, `mini_both_methods_summary.c
```

```
   ↳ sv`, `mini_decisions.json`. Follow-up: `followup_design.csv`, `followup_scan.csv`, `followup_summary.csv`, `foll
   ↳ owup_decisions.json`, `FOLLOWUP_RUNTIMES.txt`. Round 3: `r3a_runs.csv` and `r3a_runs_<cid>.csv`, `r3a_conditions
   ↳ .csv`, `r3b_design.csv`, `r3b_scan.csv`, `r3b_summary.csv`, `round3_decisions.json`. Logs: `run.log`, `count.log
   ↳ `, `mini.log`, `followup.log`, `retuned.log`, `round3.log`.",
193                "- Column definitions of the CSV files in this folder are given below.", ""]
194      cfiles = sorted(glob.glob(PKG + "/confirmatory/*.csv"))
195      seen = set()
196      for f in cfiles:
197          name = os.path.basename(f)
198          if name.startswith(("runs_", "walls_", "r3a_runs_")):
199              continue
200          with open(f) as fh:
201              rd = csv.reader(fh)
202              header = next(rd)
203              n = sum(1 for _ in rd)
204          lines.append(f"#### confirmatory/{name}  ({n:,} rows)")
205          lines.append("")
206          for c in header:
207              if c not in COLS:
208                  undefined.add(c)
209              lines.append(f"- `{c}`: {COLS.get(c, 'UNDEFINED')}")
210          lines.append("")
211      for pat in ("runs_*.csv", "walls_*.csv"):
212          fl = sorted(glob.glob(PKG + "/confirmatory/" + pat))
213          with open(fl[0]) as fh:
214              header = next(csv.reader(fh))
215          lines.append(f"#### confirmatory/{pat}  ({len(fl)} files)")
216          lines.append("")
217          for c in header:
218              if c not in COLS:
219                  undefined.add(c)
220              lines.append(f"- `{c}`: {COLS.get(c, 'UNDEFINED')}")
221          lines.append("")
222      lines += ["### code/frozen/", "", "Unchanged copies of the preregistration documents, their `.sha256` files
   ↳ and `packomania_values.csv`, kept next to the code so that the scripts can check them before every run.", ""]
223      lines += ["### code/", "", f"All {len(glob.glob(PKG + '/code/*.py'))} Python source files (the same files as
   ↳  in Code_Listing.docx), including the confirmatory scripts `confirm_*.py`, `newalgs.py`, `supplementary.py` and
   ↳ `record_env.py`. To regenerate every result, run `python3 tests.py` and then `python3 build_all.py` from this fo
   ↳ lder; the scripts create a sibling `out/` folder with all data, figures, and documents (about 17 minutes for the
   ↳  first study, plus about 30 minutes for the supplementary analyses and about 1 hour for the confirmatory round o
   ↳ n two processes; Python 3 with numpy, scipy, pandas, matplotlib, python-docx, Pygments, pdfplumber, Pillow; the
   ↳ document builders also need LibreOffice and the Liberation fonts).", "",
224                "### unit_test_output.txt", "", "Complete output of `python3 tests.py` (validation tests, run befo
   ↳ re the experiments).", "",
225                "## Verification", "", "`MANIFEST.sha256` lists the SHA-256 hash of every file. Check with `sha256
```

```
    ↳ sum -c MANIFEST.sha256` from this folder.", ""]
226     if undefined:
227         raise SystemExit("undefined columns: " + ", ".join(sorted(undefined)))
228     open(PKG + "/README.md", "w").write("\n".join(lines))
229     allf = sorted(p for p in glob.glob(PKG + "/**", recursive=True) if os.path.isfile(p) and not p.endswith("MAN
    ↳ IFEST.sha256"))
230     with open(PKG + "/MANIFEST.sha256", "w") as m:
231         for p in allf:
232             m.write(f"{sha(p)}  {os.path.relpath(p, PKG)}\n")
233     zp = OUT + "/CSV_Data.zip"
234     if os.path.exists(zp):
235         os.remove(zp)
236     with zipfile.ZipFile(zp, "w", zipfile.ZIP_DEFLATED) as z:
237         for p in sorted(glob.glob(PKG + "/**", recursive=True)):
238             if os.path.isfile(p):
239                 z.write(p, "CSV_Data/" + os.path.relpath(p, PKG))
240     print("package:", len(files), "CSV files +", "README, manifest;", os.path.getsize(zp) // 1024, "KB")
241
242
243 if __name__ == "__main__":
244     main()
```

supplementary.py

Supplementary exploratory analyses of the first study: equal area, sensitivity, polished gradient descent, timing, checks.

```
  1 """supplementary.py -- post hoc, exploratory analyses added after the main study (see the Research Plan, version
    ↳  2.1).
  2
  3 None of these results is used to decide H1-H3.  Same algorithms, parameters (except where a parameter is the thi
    ↳ ng varied),
  4 budget (1,500 sweeps unless varied), seeds 1-30, starting configurations, and random streams as the main study.
  5
  6   1. equal-area balls (cap, Euclidean disk, hyperbolic disk) and two larger balls in R2 and H2;
  7   2. budget sensitivity (500 and 3,000 sweeps) and parameter sensitivity (one parameter of each algorithm, two v
    ↳ alues each);
  8   2b. gradient descent followed by a max-min polishing step (sequential linear programming), same total budget;
  9   3. independent recomputation of every score from the saved coordinates, robustness of H1, bootstrap intervals,
    ↳  spread over starts.
 10
 11 Usage:  python3 supplementary.py           (reuses cached run files in out/data if present)
 12         python3 supplementary.py --rerun    (recomputes the runs; about 40 minutes on two cores)
```

```
"""
import os
import sys
import time
import numpy as np
import pandas as pd
from multiprocessing import Pool
from scipy import stats
from algorithms import Problem, RUNNERS, ALGS, run_algorithm

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA = os.path.join(ROOT, "out", "data")
SEEDS = range(1, 31)
BUDGET = 1500
AREAS = [3.0, 6.0, 12.0]
NS = [12, 19, 30]
LARGE = [(4.0, 30), (5.0, 60)]
# five conditions of the main study (same condition ids, hence the same starting configurations)
SENS_CONDS = [("S2", 13, None, 109), ("S2", 30, None, 112), ("S2cap", 19, 3.0, 404), ("R2", 19, 3.0, 304), ("H2"
, 19, 3.0, 204)]
VARIANTS = [("budget 500", "all", 500, {}), ("budget 3000", "all", 3000, {}),
            ("gd pmax=75", "gd", 1500, dict(pmax=75.0)), ("gd pmax=300", "gd", 1500, dict(pmax=300.0)),
            ("sa T0=0.005", "sa", 1500, dict(T0f=0.005)), ("sa T0=0.08", "sa", 1500, dict(T0f=0.08)),
            ("ls g0=0.1", "ls", 1500, dict(g0=0.1)), ("ls g0=0.5", "ls", 1500, dict(g0=0.5))]


def radius(space, A):
    """Radius of the geodesic ball of area A in each space (Equation 4 inverted)."""
    if space == "S2cap":
        return float(np.arccos(1 - A / (2 * np.pi)))
    if space == "R2":
        return float(np.sqrt(A / np.pi))
    return float(np.arccosh(1 + A / (2 * np.pi)))


def _problem(sp, n, R):
    return Problem("S2" if sp == "S2cap" else sp, n, R)


# ------------------------------------------------------------------ 1. equal area
def equal_area_conditions():
    conds = []
    for A in AREAS:
        for n in NS:
            for sp in ("S2cap", "R2", "H2"):
```

```
                conds.append((sp, n, radius(sp, A), "equal-area", A))
    for R, n in LARGE:
        for sp in ("R2", "H2"):
            area = np.pi * R * R if sp == "R2" else 2 * np.pi * (np.cosh(R) - 1)
            conds.append((sp, n, R, "large-ball", float(area)))
    return conds


def _equal_area_job(args):
    cid, (sp, n, R, kind, A) = args
    P = _problem(sp, n, R)
    rows = []
    for seed in SEEDS:
        X0 = P.initial(np.random.default_rng([seed, cid, 0]))
        for aid, alg in ALGS.items():
            res = run_algorithm(alg, P, X0, BUDGET * P.sweep, [seed, cid, aid])
            rows.append(dict(design=kind, area=A, space=sp, condition_id=cid, n=n, R=R, algorithm=alg, seed=seed
↳ ,
                             score=res["score"], density=P.density(res["score"]), sweeps_used=res["pe"] / P.swee
↳ p,
                             recomputed=res["final_check"]))
    return rows


def run_equal_area():
    conds = list(enumerate(equal_area_conditions(), start=500))
    with Pool(2) as pool:
        out = pool.map(_equal_area_job, conds, chunksize=1)
    d = pd.DataFrame([r for o in out for r in o])
    best = d.groupby("condition_id").score.transform("max")
    d["reference_value"] = best
    d["relative_score"] = d.score / best
    d.to_csv(f"{DATA}/supp_equal_area_runs.csv", index=False)
    return d


def summarize_equal_area(d):
    w = d.pivot_table(index=["design", "area", "space", "condition_id", "n", "R", "seed"], columns="algorithm",
↳ values="relative_score").reset_index()
    rows = []
    for (des, A, sp, cid, n, R), g in w.groupby(["design", "area", "space", "condition_id", "n", "R"]):
        diff = g.gd - g.sa
        pg = stats.wilcoxon(diff).pvalue if diff.abs().sum() > 0 else 1.0
        rows.append(dict(design=des, area=round(A, 2), space=sp, n=n, R=round(R, 3), mean_gd=g.gd.mean(), mean_s
↳ a=g.sa.mean(),
```

```
                           mean_ls=g.ls.mean(), gd_minus_sa=diff.mean(), p_wilcoxon_gd_sa=pg,
                           p_friedman=stats.friedmanchisquare(g.gd, g.sa, g.ls).pvalue))
    s = pd.DataFrame(rows)
    s["p_holm_gd_sa"] = np.nan
    for des, idx in s.groupby("design").groups.items():          # Holm family = the conditions of one design
        p = s.loc[idx, "p_wilcoxon_gd_sa"].values
        order = np.argsort(p)
        adj = np.empty(len(p))
        run = 0.0
        for k, i in enumerate(order):
            run = max(run, (len(p) - k) * p[i])
            adj[i] = min(run, 1.0)
        s.loc[idx, "p_holm_gd_sa"] = adj
    s.to_csv(f"{DATA}/supp_equal_area_summary.csv", index=False, float_format="%.6g")
    return s


# ---------------------------------------------------------------- 2. sensitivity
def _sens_job(c):
    sp, n, R, cid = c
    P = _problem(sp, n, R)
    rows = []
    for seed in SEEDS:
        X0 = P.initial(np.random.default_rng([seed, cid, 0]))
        for aid, alg in ALGS.items():
            for vname, valg, bud, kw in VARIANTS:
                if valg not in ("all", alg):
                    continue
                res = RUNNERS[alg](P, X0, bud * P.sweep, np.random.default_rng([seed, cid, aid]), **kw)
                rows.append(dict(space=sp, condition_id=cid, n=n, R=R, algorithm=alg, seed=seed, variant=vname,
budget=bud,
                                 score=res["score"], check=P.score(res["X"])))
    return rows


def run_sensitivity():
    with Pool(2) as pool:
        out = pool.map(_sens_job, SENS_CONDS, chunksize=1)
    d = pd.DataFrame([r for o in out for r in o])
    d.to_csv(f"{DATA}/supp_sensitivity_runs.csv", index=False)
    return d


# ---------------------------------------------------------------- 2b. gradient descent with polishing
def _gdp_job(cond):
```

```
    import experiments as E
    kind, n, R = cond
    cid = E.COND_ID[cond]
    P = _problem(kind, n, R)
    rows = []
    for seed in E.SEEDS:
        X0 = P.initial(np.random.default_rng([seed, cid, 0]))
        t0 = time.perf_counter()
        res = RUNNERS["gdp"](P, X0, E.BUDGET_SWEEPS * P.sweep, np.random.default_rng([seed, cid, 1]))
        dt = time.perf_counter() - t0
        rows.append(dict(space=kind, condition_id=cid, n=n, R=(R if R is not None else float("nan")), algorithm=
↳ "gdp", seed=seed,
                         score=res["score"], check=P.score(res["X"]), sweeps_used=res["pe"] / P.sweep, time_s=dt
↳ ))
    return rows


def run_gdp_all():
    import experiments as E
    with Pool(2) as pool:
        out = pool.map(_gdp_job, list(E.conditions()), chunksize=1)
    d = pd.DataFrame([r for o in out for r in o])
    ref = pd.read_csv(f"{DATA}/runs_raw.csv").groupby("condition_id")[["reference_value", "reference_type"]].fir
↳ st().reset_index()
    d = d.merge(ref, on="condition_id")
    d["relative_score"] = d.score / d.reference_value
    d.to_csv(f"{DATA}/supp_gd_polish_runs.csv", index=False)
    return d


# ------------------------------------------------------------------ 2c. wall-clock benchmark (one process, nothin
↳ g else running)
def run_timing(seeds=range(1, 6)):
    rows = []
    for sp, n, R, cid in SENS_CONDS:
        P = _problem(sp, n, R)
        for seed in seeds:
            X0 = P.initial(np.random.default_rng([seed, cid, 0]))
            for alg, aid in (("gd", 1), ("gdp", 1), ("sa", 2), ("ls", 3)):
                t0 = time.perf_counter()
                res = RUNNERS[alg](P, X0, BUDGET * P.sweep, np.random.default_rng([seed, cid, aid]))
                rows.append(dict(space=sp, condition_id=cid, n=n, R=R, algorithm=alg, seed=seed, time_s=time.per
↳ f_counter() - t0,
                                 sweeps_used=res["pe"] / P.sweep, score=res["score"]))
    d = pd.DataFrame(rows)
```

```
    d.to_csv(f"{DATA}/supp_timing.csv", index=False)
    return d


# ---------------------------------------------------------------- 3. checks
def _indep_score(space, X, R):
    """Score recomputed with formulas that do not use geometry.py (arctan2, Poincare disk)."""
    n = len(X)
    iu = np.triu_indices(n, 1)
    if space in ("S2", "S2cap"):
        cr = np.cross(X[:, None, :], X[None, :, :])
        d = np.arctan2(np.linalg.norm(cr, axis=-1), X @ X.T)[iu]
        man = abs(np.linalg.norm(X, axis=1) - 1).max()
        rho = np.arctan2(np.hypot(X[:, 0], X[:, 1]), X[:, 2])
    elif space == "H2":
        z = (X[:, 1] + 1j * X[:, 2]) / (1 + X[:, 0])
        den = (1 - np.abs(z)[:, None] ** 2) * (1 - np.abs(z)[None, :] ** 2)
        d = np.arccosh(1 + 2 * np.abs(z[:, None] - z[None, :]) ** 2 / den)[iu]
        man = abs(-X[:, 0] ** 2 + X[:, 1] ** 2 + X[:, 2] ** 2 + 1).max()
        rho = 2 * np.arctanh(np.abs(z))
    else:
        d = np.sqrt(((X[:, None, :] - X[None, :, :]) ** 2).sum(-1))[iu]
        man = 0.0
        rho = np.hypot(X[:, 0], X[:, 1])
    l = d.min()
    if R is not None:
        l = min(l, (2 * (R - rho)).min())
    return (l / 2 if R is not None else l), man, (None if R is None else float((rho + l / 2 - R).max()))


def run_checks():
    runs = pd.read_csv(f"{DATA}/runs_raw.csv")
    cfg = pd.read_csv(f"{DATA}/configurations.csv")
    res = {}
    rows = []
    for (sp, cid, alg, seed), g in cfg.groupby(["space", "condition_id", "algorithm", "seed"], sort=False):
        Rv = g.R.iloc[0]
        Rv = None if np.isnan(Rv) else float(Rv)
        X = g[["x0", "x1", "x2"]].values if sp != "R2" else g[["x0", "x1"]].values
        s, man, ex = _indep_score(sp, X, Rv)
        rows.append((sp, cid, alg, seed, s, man, ex))
    ch = pd.DataFrame(rows, columns=["space", "condition_id", "algorithm", "seed", "indep", "manifold_err", "wal
↳ l_excess"])
    m = runs.merge(ch, on=["space", "condition_id", "algorithm", "seed"])
    res["n_scores_checked"] = len(m)
```

```
    res["max_abs_diff_score_vs_independent"] = float((m.score - m.indep).abs().max())
    res["max_abs_diff_score_vs_recomputed"] = float((m.score - m.recomputed_score).abs().max())
    res["max_manifold_error"] = float(m.manifold_err.max())
    res["max_disk_excess_beyond_wall"] = float(m.wall_excess.dropna().max())
    # H1 robustness
    w = runs.pivot_table(index=["space", "condition_id", "seed"], columns="algorithm", values="relative_score").reset_index()
    w["gdsa"] = w.gd - w.sa
    b = w[w.space.isin(["S2cap", "R2", "H2"])]
    mean_d = b.groupby(["space", "condition_id"]).gdsa.mean().reset_index()
    med_d = b.groupby(["space", "condition_id"]).gdsa.median().reset_index()

    def kw(df):
        return stats.kruskal(*[df[df.space == s].gdsa.values for s in ("S2cap", "R2", "H2")])

    k1, k2 = kw(mean_d), kw(med_d)
    res["H1_kruskal_chi2_mean_difference"], res["H1_kruskal_p_mean_difference"] = float(k1.statistic), float(k1.pvalue)
    res["H1_kruskal_chi2_median_difference"], res["H1_kruskal_p_median_difference"] = float(k2.statistic), float(k2.pvalue)
    rng = np.random.default_rng(20261028)
    code = mean_d.space.map({"S2cap": 0, "R2": 1, "H2": 2}).values
    vals = mean_d.gdsa.values
    H = lambda lab: stats.kruskal(*[vals[lab == i] for i in range(3)]).statistic
    obs = H(code)
    res["H1_permutation_p_20000"] = float(np.mean([H(rng.permutation(code)) >= obs - 1e-12 for _ in range(20000)]))
    for tag, ids in (("R2_settings", [0, 1, 2]), ("R3_settings", [3, 4, 5])):
        sub = mean_d[mean_d.condition_id.mod(100).isin(ids)]
        kk = kw(sub)
        res[f"H1_kruskal_chi2_{tag}"], res[f"H1_kruskal_p_{tag}"] = float(kk.statistic), float(kk.pvalue)

    def ci(x, B=10000):
        x = np.asarray(x, float)
        h = stats.t.ppf(.975, len(x) - 1) * x.std(ddof=1) / np.sqrt(len(x))
        bs = rng.choice(x, (B, len(x))).mean(1)
        return x.mean(), x.mean() - h, x.mean() + h, np.percentile(bs, 2.5), np.percentile(bs, 97.5)

    for tag, cid in (("S2_N60", 114), ("H2_R3_n19", 204)):
        mu, tlo, thi, blo, bhi = ci(w[w.condition_id == cid].gdsa.values)
        res[f"gd_minus_sa_{tag}_mean_relative"] = float(mu)
        res[f"gd_minus_sa_{tag}_t_lo"], res[f"gd_minus_sa_{tag}_t_hi"] = float(tlo), float(thi)
        res[f"gd_minus_sa_{tag}_boot_lo"], res[f"gd_minus_sa_{tag}_boot_hi"] = float(blo), float(bhi)
    pd.DataFrame({"check": list(res), "value": list(res.values())}).to_csv(f"{DATA}/supp_checks.csv", index=False, float_format="%.10g")
```

```
    # spread of relative score over the 30 starting configurations
    sel = {109: "Tammes N = 13", 112: "Tammes N = 30", 404: "Cap, R = 3, n = 19", 304: "Euclidean, R = 3, n = 19", 204: "Hyperbolic, R = 3, n = 19"}
    sp_ = runs[runs.condition_id.isin(sel)].groupby(["condition_id", "algorithm"]).relative_score.agg(["min", "median", "max", "std"]).reset_index()
    sp_["label"] = sp_.condition_id.map(sel)
    sp_.to_csv(f"{DATA}/supp_initialization_spread.csv", index=False, float_format="%.6g")
    return res


def main(rerun=False):
    os.makedirs(DATA, exist_ok=True)
    t0 = time.time()
    f1, f2 = f"{DATA}/supp_equal_area_runs.csv", f"{DATA}/supp_sensitivity_runs.csv"
    d1 = run_equal_area() if (rerun or not os.path.exists(f1)) else pd.read_csv(f1)
    summarize_equal_area(d1)
    if rerun or not os.path.exists(f2):
        run_sensitivity()
    if rerun or not os.path.exists(f"{DATA}/supp_gd_polish_runs.csv"):
        run_gdp_all()
    if rerun or not os.path.exists(f"{DATA}/supp_timing.csv"):
        run_timing()
    run_checks()
    print("supplementary analyses finished in %.0f s" % (time.time() - t0))


if __name__ == "__main__":
    main(rerun="--rerun" in sys.argv)
```

supp_section.py

Writes the Supplementary Exploratory Analyses section of the paper from the supplementary CSV files.

```
"""supp_section.py -- writes the section 'Supplementary Exploratory Analyses' of the paper from the supp_*.csv files.

Every number in the section is computed here from the data files; nothing is typed in by hand.
"""
import numpy as np
import pandas as pd
from scipy import stats

```

```
LABEL = {109: "Tammes <i>N</i> = 13", 112: "Tammes <i>N</i> = 30", 404: "Cap, <i>R</i> = 3, <i>n</i> = 19",
         304: "Euclidean, <i>R</i> = 3, <i>n</i> = 19", 204: "Hyperbolic, <i>R</i> = 3, <i>n</i> = 19"}
ORDER = [109, 112, 404, 304, 204]
AB = {"gd": "GD", "gdp": "GD+P", "sa": "SA", "ls": "CP"}
FULL = {"gd": "gradient descent", "gdp": "gradient descent with polishing", "sa": "simulated annealing", "ls": "compression"}


def build_section(d, FF, HD, Tb, f, pstr, DATA, runs, BUDGET):
    rd = lambda n: pd.read_csv(f"{DATA}/{n}")
    sens, gdp, tim = rd("supp_sensitivity_runs.csv"), rd("supp_gd_polish_runs.csv"), rd("supp_timing.csv")
    eqr, eqs, chk, spr = rd("supp_equal_area_runs.csv"), rd("supp_equal_area_summary.csv"), rd("supp_checks.csv"), rd("supp_initialization_spread.csv")
    C = dict(zip(chk.check, chk.value))
    ref = runs.groupby("condition_id").reference_value.first()
    sens["rel"] = sens.score / sens.condition_id.map(ref)
    main_mean = runs.groupby(["condition_id", "algorithm"]).relative_score.mean()
    gdp["rel"] = gdp.score / gdp.condition_id.map(ref)

    HD("Supplementary Exploratory Analyses", 2)
    d.body("The analyses in this section were added after the main results had been seen, in response to the limitations listed in the Conclusions. "
           "They use the same seeds, the same starting configurations, and the same budget unless stated, and none of them is used to decide H1, H2, or H3. "
           "Their code is supplementary.py, and their data are the files whose names begin with supp_.")
    # ---- headline numbers for the summary paragraph (same definitions as the tables below)
    _m = {a: runs[(runs.space == "S2") & (runs.n >= 24) & (runs.algorithm == a)].groupby("condition_id").relative_score.mean().mean() for a in ("gd", "sa")}
    _mp = gdp[(gdp.space == "S2") & (gdp.n >= 24)].assign(rel=lambda x: x.score / x.condition_id.map(ref)).groupby("condition_id").rel.mean().mean()
    def _succ(df, a):
        sub = df[(df.space == "S2") & (df.n <= 14) & (df.algorithm == a)]
        return int(((sub.score / sub.condition_id.map(ref) >= 0.999).groupby(sub.n).mean() >= 0.90).sum())
    _sc = {"gd": _succ(runs, "gd"), "gdp": _succ(gdp, "gdp"), "sa": _succ(runs, "sa")}
    _t = tim.groupby(["condition_id", "algorithm"]).time_s.mean()
    _rr = [_t[(c, "sa")] / _t[(c, "gd")] for c in ORDER]
    _extra = float((tim[tim.algorithm == "gdp"].groupby("condition_id").time_s.mean() - tim[tim.algorithm == "gd"].groupby("condition_id").time_s.mean()).mean())
    _ea = eqs[eqs.design == "equal-area"]
    _kw = stats.kruskal(*[_ea[_ea.space == sp].gd_minus_sa.values for sp in ("S2cap", "R2", "H2")])
    d.body("<i>Summary of the supplementary results.</i> " +
           f"(1) Numerical accuracy: all {int(C['n_scores_checked']):,} scores agreed with an independent recomputation to within {C['max_abs_diff_score_vs_independent']:.0e}. "
           f"(2) Polishing: gradient descent followed by a max-min polishing step raised the mean relative score on the sphere for <i>N</i> = 24 to 60 from {f(_m['gd'], 3, lead=False)} to {f(_mp, 3, lead=False)} (simulated a
```

```
  ↳ nnealing: {f(_m['sa'], 3, lead=False)}) "
45              f"and the number of <i>N</i> from 4 to 14 with a success share of at least .90 from {_sc['gd']} to {_
  ↳ sc['gdp']} of 11 (simulated annealing: {_sc['sa']}); the polishing step uses a linear program that the sweep cou
  ↳ nt does not charge, which added {f(_extra, 2)} seconds per run on average. "
46              f"(3) Time: at equal sweeps, simulated annealing took {f(min(_rr), 1)} to {f(max(_rr), 1)} times as l
  ↳ ong as gradient descent. "
47              f"(4) Sensitivity: moving one parameter at a time changed gradient descent and annealing very little
  ↳ and compression a great deal, and compression stayed last at every budget and parameter setting tried. "
48              f"(5) Equal area: the gradient-descent minus annealing difference did not differ detectably across th
  ↳ e three spaces (Kruskal–Wallis {pstr(float(_kw.pvalue))}), while larger balls reproduced the main-study sign pat
  ↳ tern, so the apparent effect of the space is tied to ball size and is not shown to be caused by curvature. All o
  ↳ f this is exploratory.")
49
50      # ------------------------------------------------------------------ numerical accuracy
51      HD("Numerical Accuracy and Independent Recomputation", 3)
52      d.body(f"Every one of the {int(C['n_scores_checked']):,} final scores was recomputed from the saved coordina
  ↳ tes with formulas that do not use the geometry code: the arctangent form of the sphere distance, the Poincaré-di
  ↳ sk formula for the hyperbolic plane, and direct Euclidean distances. "
53              f"The largest difference from the stored score was {C['max_abs_diff_score_vs_independent']:.1e}, the
  ↳ largest manifold error of any saved point was {C['max_manifold_error']:.1e}, and no disk extended beyond the wal
  ↳ l of its ball (largest excess {C['max_disk_excess_beyond_wall']:.1f}). "
54              f"These errors are many orders of magnitude below the 0.1% success tolerance, so rounding does not af
  ↳ fect any success rate. "
55              f"The distance code uses arcsine and arcsinh forms, which stay accurate for small distances where an
  ↳ arccosine would lose digits. These checks show that the stored scores are numerically sound; they do not show th
  ↳ at any configuration is optimal.")
56
57      # ------------------------------------------------------------------ gradient descent with polishing
58      HD("Gradient Descent With a Polishing Step, and Wall-Clock Time", 3)
59      grp = {"Sphere, <i>N</i> = 4 to 14": lambda r: (r.space == "S2") & (r.n <= 14), "Sphere, <i>N</i> = 24 to 60
  ↳ ": lambda r: (r.space == "S2") & (r.n >= 24),
60              "Spherical cap": lambda r: r.space == "S2cap", "Euclidean disk": lambda r: r.space == "R2", "Hyperbol
  ↳ ic disk": lambda r: r.space == "H2"}
61      two = runs.copy(); two["rel"] = two.relative_score
62      allr = pd.concat([two[["space", "condition_id", "n", "algorithm", "seed", "rel"]], gdp[["space", "condition_
  ↳ id", "n", "algorithm", "seed", "rel"]]])
63      rows = []
64      gm = {}
65      for lab, fn in grp.items():
66          sub = allr[fn(allr)]
67          cm = sub.groupby(["condition_id", "algorithm"]).rel.mean().groupby("algorithm").mean()
68          gm[lab] = cm
69          rows.append([lab] + [f(cm[a], 3, lead=False) for a in ("gd", "gdp", "sa", "ls")])
70      sc = {}
71      for a in ("gd", "gdp", "sa", "ls"):
```

```
        sub = allr[(allr.space == "S2") & (allr.n <= 14) & (allr.algorithm == a)]
        sh = (sub.assign(ok=(sub.rel >= 0.999)).groupby("n").ok.mean())
        sc[a] = int((sh >= 0.90).sum())
    rows.append(["<i>N</i> with success share ≥ .90 (of 11)"] + [str(sc[a]) for a in ("gd", "gdp", "sa", "ls")])
    FF("table", "gdp", "Mean Relative Score of Gradient Descent With and Without a Polishing Step",
       ["Problem group", "GD", "GD+P", "SA", "CP"], rows,
       "GD = gradient descent; GD+P = gradient descent on the first 85% of the budget followed by a max-min polishing step on the remaining 15%; SA = simulated annealing; CP = compression. "
       "Cells are the mean relative score over the conditions of the group (30 seeds each); the last row counts the values of <i>N</i> from 4 to 14 at which at least 90% of seeds reached a relative score of .999. "
       "GD, SA, and CP are the main-study runs. A value above 1.000 means the result exceeds the reference, which in the ball problems is the best score found in the main study, so values above 1.000 are possible there. Data: runs_raw.csv, supp_gd_polish_runs.csv.",
       aligns=["l", "c", "c", "c", "c"], widths=[2.7, 0.95, 0.95, 0.95, 0.95])
    s2 = gm["Sphere, <i>N</i> = 4 to 14"]; s3 = gm["Sphere, <i>N</i> = 24 to 60"]
    d.body(f"The main study suggested that the final precision of gradient descent, not its search, limits its success rate (the bound in Equation 7 allows a smoothing bias). To test this, gradient descent was followed by a max-min polishing step: the first 85% of the budget was used by the unchanged gradient descent, and the remaining 15% by a sequential linear program that maximizes the smallest distance within a shrinking trust region, with every polishing iteration charged as one sweep. "
           f"The step is a local refinement of the same kind as the L-BFGS refinement of a smoothed energy (Liu & Nocedal, 1989; Byrd et al., 1995), which was used as a baseline in the Confirmatory Round. With the same total budget, polishing changed the mean relative score on the sphere from {f(s2['gd'], 3, lead=False)} to {f(s2['gdp'], 3, lead=False)} for <i>N</i> = 4 to 14 and from {f(s3['gd'], 3, lead=False)} to {f(s3['gdp'], 3, lead=False)} for <i>N</i> = 24 to 60 ({Tb('gdp')}). "
           f"The number of values of <i>N</i> from 4 to 14 at which at least 90% of seeds reached the proven optimum within 0.1% changed from {sc['gd']} to {sc['gdp']} of 11 (simulated annealing: {sc['sa']}; compression: {sc['ls']}). "
           f"This is exploratory evidence about the explanation, not a revision of H3, which was decided on the three algorithms of the main study.")
    # timing
    tm = tim.groupby(["condition_id", "algorithm"]).time_s.mean()
    rows = []
    for cid in ORDER:
        rows.append([LABEL[cid]] + [f(tm[(cid, a)], 2) for a in ("gd", "gdp", "sa", "ls")] + [f(tm[(cid, "sa")] / tm[(cid, "gd")], 1)])
    FF("table", "timing", "Mean Wall-Clock Time per Run at Equal Budget",
       ["Condition", "GD", "GD+P", "SA", "CP", "SA / GD"], rows,
       f"Seconds per run at {BUDGET:,} sweeps, mean of 5 runs (seeds 1 to 5) per cell, measured in one process with nothing else running on the same computer. SA / GD = ratio of simulated-annealing time to gradient-descent time. "
       "The sweep budget is equal, but the time per sweep differs between methods and between implementations; times describe this Python code only. The main-study times are in summary_by_condition.csv. Data: supp_timing.csv.",
       aligns=["l", "c", "c", "c", "c", "c"], widths=[2.3, 0.7, 0.75, 0.7, 0.7, 0.85])
```

```
    rr = [tm[(c, "sa")] / tm[(c, "gd")] for c in ORDER]
    d.body(f"Equal sweeps do not mean equal time ({Tb('timing')}). Simulated annealing took {f(min(rr), 1)} to {f(max(rr), 1)} times as long as gradient descent per run in these five conditions, so the small advantage of annealing in some conditions was obtained with a much longer wall-clock time; "
           f"the polishing step added {f(float((tim[tim.algorithm == 'gdp'].groupby('condition_id').time_s.mean() - tim[tim.algorithm == 'gd'].groupby('condition_id').time_s.mean()).mean()), 2)} seconds per run on average because each polishing iteration solves a linear program that the sweep count does not charge.")

    # ---------------------------------------------------------------- sensitivity
    HD("Sensitivity to the Budget, the Parameters, and the Starting Configuration", 3)
    rows, same = [], 0
    for cid in ORDER:
        means = {}
        for a in ("gd", "sa", "ls"):
            m500 = sens[(sens.condition_id == cid) & (sens.algorithm == a) & (sens.variant == "budget 500")].rel.mean()
            m3000 = sens[(sens.condition_id == cid) & (sens.algorithm == a) & (sens.variant == "budget 3000")].rel.mean()
            rows.append([LABEL[cid] if a == "gd" else "", AB[a], f(m500, 3, lead=False), f(main_mean[(cid, a)], 3, lead=False), f(m3000, 3, lead=False)])
            means[a] = (m500, main_mean[(cid, a)], m3000)
        best = [max(("gd", "sa", "ls"), key=lambda a: means[a][k]) for k in range(3)]
        same += int(len(set(best)) == 1)
    FF("table", "budget", "Mean Relative Score at Three Budgets",
       ["Condition", "Algorithm", "500 sweeps", "1,500 sweeps", "3,000 sweeps"], rows,
       "Mean over 30 seeds, relative to the reference value of the main study (a value above 1.000 would exceed it). The 1,500-sweep column is the main study; the other columns use the same starting configurations and random streams. "
       "GD = gradient descent; SA = simulated annealing; CP = compression. Data: runs_raw.csv, supp_sensitivity_runs.csv.",
       aligns=["l", "l", "c", "c", "c"], widths=[2.4, 1.0, 1.0, 1.1, 1.1])
    gain = {a: np.mean([sens[(sens.condition_id == c) & (sens.algorithm == a) & (sens.variant == "budget 3000")].rel.mean() - main_mean[(c, a)] for c in ORDER]) for a in ("gd", "sa", "ls")}
    d.body(f"Tripling the budget from 500 to 1,500 sweeps and doubling it to 3,000 raised the mean relative score for every algorithm, and doubling from 1,500 to 3,000 sweeps changed it by {f(gain['gd'], 3, lead=False)} for gradient descent, {f(gain['sa'], 3, lead=False)} for simulated annealing, and {f(gain['ls'], 3, lead=False)} for compression on average over the five conditions ({Tb('budget')}). "
           f"The algorithm with the highest mean was the same at all three budgets in {same} of 5 conditions. The budget therefore matters, the conclusion that compression is last holds at every budget tested, and the ordering of gradient descent and simulated annealing depends on the condition and the budget.")
    # parameters
    rows = []
    spec = [("GD", "gd", "<i>p</i><sub>max</sub> = 75", "gd pmax=75"), ("GD", "gd", "<i>p</i><sub>max</sub> = 150 (default)", None), ("GD", "gd", "<i>p</i><sub>max</sub> = 300", "gd pmax=300"),
            ("SA", "sa", "<i>T</i><sub>0</sub> = 0.005 (default 0.02)", "sa T0=0.005"), ("SA", "sa", "<i>T</i><s
```

```
    ↳ ub>0</sub> = 0.02 (default)", None), ("SA", "sa", "<i>T</i><sub>0</sub> = 0.08", "sa T0=0.08"),
125              ("CP", "ls", "<i>γ</i><sub>0</sub> = 0.10", "ls g0=0.1"), ("CP", "ls", "<i>γ</i><sub>0</sub> = 0.25
    ↳ (default)", None), ("CP", "ls", "<i>γ</i><sub>0</sub> = 0.50", "ls g0=0.5")]
126      spread_ = {}
127      for ab, a, lab, var in spec:
128          vals = [(main_mean[(c, a)] if var is None else sens[(sens.condition_id == c) & (sens.algorithm == a) & (
    ↳ sens.variant == var)].rel.mean()) for c in ORDER]
129          spread_.setdefault(a, []).append(vals)
130          rows.append([f"{ab}: {lab}"] + [f(v, 3, lead=False) for v in vals] + [f(float(np.mean(vals)), 3, lead=Fa
    ↳ lse)])
131      FF("table", "sens", "Mean Relative Score With One Parameter Changed",
132         ["Setting"] + ["<i>N</i> = 13", "<i>N</i> = 30", "Cap", "Euclid.", "Hyperb."] + ["Mean"], rows,
133         "Mean relative score over 30 seeds at 1,500 sweeps; <i>N</i> = 13 and <i>N</i> = 30 are Tammes problems;
    ↳ “Cap”, “Euclid.”, and “Hyperb.” are the balls with <i>R</i> = 3 and <i>n</i> = 19. "
134         "<i>p</i><sub>max</sub> = final exponent of gradient descent; <i>T</i><sub>0</sub> = initial temperature
    ↳ of simulated annealing as a multiple of the length scale; <i>γ</i><sub>0</sub> = initial growth rate of compress
    ↳ ion. One parameter was changed at a time. Data: runs_raw.csv, supp_sensitivity_runs.csv.",
135         aligns=["l", "c", "c", "c", "c", "c", "c"], widths=[2.1, 0.7, 0.7, 0.6, 0.75, 0.75, 0.6])
136      rng_ = {a: (max(np.mean(v) for v in spread_[a]) - min(np.mean(v) for v in spread_[a])) for a in spread_}
137      d.body(f"Changing one parameter at a time within the ranges in {Tb('sens')} moved the mean relative score by
    ↳  at most {f(rng_['gd'], 3, lead=False)} for gradient descent, {f(rng_['sa'], 3, lead=False)} for simulated annea
    ↳ ling, and {f(rng_['ls'], 3, lead=False)} for compression (differences between the best and worst setting, averag
    ↳ ed over the five conditions). "
138              f"Only three of the nine values were tuned on the pilot seeds, so this is a local sensitivity check,
    ↳ not a new tuning: it asks whether the main conclusions depend on the exact default values, and it does not searc
    ↳ h for better ones.")
139      # initialization spread
140      rows = []
141      for cid in ORDER:
142          row = [LABEL[cid]]
143          for a in ("gd", "sa", "ls"):
144              r = spr[(spr.condition_id == cid) & (spr.algorithm == a)].iloc[0]
145              row.append(f"{f(r['median'], 3, lead=False)} ({f(r['min'], 3, lead=False)}\u2013{f(r['max'], 3, lead
    ↳ =False)})")
146          rows.append(row)
147      FF("table", "spread", "Spread of the Relative Score Over 30 Starting Configurations",
148         ["Condition", "GD", "SA", "CP"], rows,
149         "Median relative score, with the minimum and maximum in parentheses, of single runs at 1,500 sweeps (seed
    ↳ s 1 to 30); each seed gives one starting configuration shared by the three algorithms. GD = gradient descent, SA
    ↳  = simulated annealing, CP = compression. Standard deviations are in supp_initialization_spread.csv. Data: runs_
    ↳ raw.csv.",
150         aligns=["l", "c", "c", "c"], widths=[2.2, 1.45, 1.45, 1.45])
151      sd13 = spr[(spr.condition_id == 109)].set_index("algorithm")["std"]
152      d.body(f"The spread over starting configurations ({Tb('spread')}) shows local optima directly. For the Tamme
    ↳ s problem with <i>N</i> = 13 the standard deviation of the relative score was {f(sd13['gd'], 3, lead=False)} for
```

```
 gradient descent, {f(sd13['sa'], 3, lead=False)} for simulated annealing, and {f(sd13['ls'], 3, lead=False)} for compression, "
            f"and no run reached the proven optimum, so different starts end in nearby but different local optima, and the best of 30 starts is a better estimate of what the method can reach than a single start. Whether a given local optimum is a strict local maximum was not tested.")

    # ---------------------------------------------------------------- equal area
    HD("Equal-Area and Larger-Ball Comparison", 3)
    ea = eqs[eqs.design == "equal-area"]
    rows = []
    for A in sorted(ea.area.unique()):
        for sp, lab in (("S2cap", "Cap"), ("R2", "Euclidean"), ("H2", "Hyperbolic")):
            g = ea[(ea.area == A) & (ea.space == sp)]
            rows.append([f"{A:g}", lab, f(g.R.min(), 2) + "–" + f(g.R.max(), 2) if g.R.nunique() > 1 else f(g.R.iloc[0], 2),
                         f(g.mean_gd.mean(), 3, lead=False), f(g.mean_sa.mean(), 3, lead=False), f(g.mean_ls.mean(), 3, lead=False), f(g.gd_minus_sa.mean(), 3, lead=False)])
    lg = eqs[eqs.design == "large-ball"]
    for r in lg.itertuples():
        rows.append([f"{r.area:.0f}", {"R2": "Euclidean", "H2": "Hyperbolic"}[r.space] + f", <i>n</i> = {r.n}", f(r.R, 2), f(r.mean_gd, 3, lead=False), f(r.mean_sa, 3, lead=False), f(r.mean_ls, 3, lead=False), f(r.gd_minus_sa, 3, lead=False)])
    FF("table", "equal", "Balls of Equal Area in the Three Spaces, and Larger Balls",
       ["Area", "Space", "<i>R</i>", "GD", "SA", "CP", "GD − SA"], rows,
       "The first nine rows are balls of equal area (3, 6, and 12) in the three spaces, averaged over <i>n</i> = 12, 19, and 30 (the radius <i>R</i> is the same for all <i>n</i>); the last four rows are single larger balls in the Euclidean and hyperbolic planes, which the sphere cannot contain. "
       "Cells are mean relative scores over 30 seeds, relative to the best score found in the same condition; GD − SA is the mean paired difference. Data: supp_equal_area_runs.csv, supp_equal_area_summary.csv.",
       aligns=["c", "l", "c", "c", "c", "c", "c"], widths=[0.6, 1.9, 0.9, 0.7, 0.7, 0.7, 0.9])
    gdsa = {sp: ea[ea.space == sp].gd_minus_sa.values for sp in ("S2cap", "R2", "H2")}
    kw = stats.kruskal(*gdsa.values())
    rk = eqr[eqr.design == "equal-area"].copy()
    rk["rank"] = rk.groupby(["condition_id", "seed"]).score.rank(ascending=False)
    mr = rk.groupby(["space", "algorithm"])["rank"].mean().unstack()
    best_eq = {sp: mr.loc[sp].idxmin() for sp in mr.index}
    h2l = lg[lg.space == "H2"]; r2l = lg[lg.space == "R2"]
    d.body(f"The main study could not separate curvature from area. To control for area, balls of the same area (3, 6, and 12) were packed in the three spaces with <i>n</i> = 12, 19, and 30 disks ({Tb('equal')}). "
           f"At equal area, the difference between gradient descent and simulated annealing did not differ detectably across the three spaces (median {f(np.median(gdsa['S2cap']), 4)} in the cap, {f(np.median(gdsa['R2']), 4)} in the Euclidean disk, and {f(np.median(gdsa['H2']), 4)} in the hyperbolic disk; Kruskal–Wallis <i>χ</i><sup>2</sup>(2) = {f(kw.statistic, 2)}, {pstr(kw.pvalue)}, nine settings per space), "
           f"and the algorithm with the lowest mean rank was {FULL[best_eq['S2cap']]} in the cap, {FULL[best_eq['R2']]} in the Euclidean disk, and {FULL[best_eq['H2']]} in the hyperbolic disk (mean ranks in the hyperbolic di
```

```
    ↳ sk: {f(mr.loc['H2','gd'], 2)} for gradient descent and {f(mr.loc['H2','sa'], 2)} for simulated annealing). "
182             f"In the two larger balls, which can exist only in the Euclidean and hyperbolic planes, the sign patt
    ↳ ern of the main study returned: gradient descent minus simulated annealing was {f(r2l.gd_minus_sa.min(), 3, lead
    ↳ =False)} to {f(r2l.gd_minus_sa.max(), 3, lead=False)} in the Euclidean disk and {f(h2l.gd_minus_sa.min(), 3, lea
    ↳ d=False)} to {f(h2l.gd_minus_sa.max(), 3, lead=False)} in the hyperbolic disk "
183             f"(Holm-adjusted {pstr(float(h2l.p_holm_gd_sa.min()))} at the smallest in the hyperbolic disk, two se
    ↳ ttings only). "
184             f"Together these results suggest that the apparent dependence of the ranking on the space in the main
    ↳  study is tied to the size of the ball and not shown to be caused by curvature, but this design cannot establish
    ↳  that: equal area is possible only up to 4π on the sphere, where hyperbolic curvature has little room to act, an
    ↳ d the larger balls differ in both area and curvature.")
185
186         # ------------------------------------------------------------------ robustness of H1
187         d.heading("Robustness of the H1 Result", 3)
188         d.body(f"The H1 contrast (gradient descent minus simulated annealing across the cap, the Euclidean disk, and
    ↳  the hyperbolic disk) gave a Kruskal–Wallis statistic of {f(C['H1_kruskal_chi2_mean_difference'], 2)} ({pstr(C['
    ↳ H1_kruskal_p_mean_difference'])} before Holm adjustment). "
189             f"Replacing the per-setting mean difference by the median gave {f(C['H1_kruskal_chi2_median_differenc
    ↳ e'], 2)} ({pstr(C['H1_kruskal_p_median_difference'])}); a permutation test of the same statistic over the 18 set
    ↳ tings (20,000 random relabelings of the three spaces) gave {pstr(C['H1_permutation_p_20000'])}; "
190             f"restricting to the three settings with <i>R</i> = 2 gave {pstr(C['H1_kruskal_p_R2_settings'])} and
    ↳ to the three with <i>R</i> = 3 gave {pstr(C['H1_kruskal_p_R3_settings'])}. "
191             f"All of these unadjusted values are of the same size as the original one, and multiplying any of the
    ↳ m by 3 for the three algorithm pairs puts it above .05, so the conclusion is unchanged: the evidence is suggesti
    ↳ ve and not significant after correction. "
192             f"For two headline paired differences, bootstrap percentile intervals agreed with the <i>t</i> interv
    ↳ als reported in the Statistical Comparison: for <i>N</i> = 60 on the sphere (gradient descent minus simulated an
    ↳ nealing, in relative score) the mean was {f(C['gd_minus_sa_S2_N60_mean_relative'], 4)}, 95% <i>t</i> interval [{
    ↳ f(C['gd_minus_sa_S2_N60_t_lo'], 4)}, {f(C['gd_minus_sa_S2_N60_t_hi'], 4)}] and bootstrap interval [{f(C['gd_minu
    ↳ s_sa_S2_N60_boot_lo'], 4)}, {f(C['gd_minus_sa_S2_N60_boot_hi'], 4)}]; "
193             f"for the hyperbolic ball with <i>R</i> = 3 and <i>n</i> = 19 the mean was {f(C['gd_minus_sa_H2_R3_n1
    ↳ 9_mean_relative'], 4)}, [{f(C['gd_minus_sa_H2_R3_n19_t_lo'], 4)}, {f(C['gd_minus_sa_H2_R3_n19_t_hi'], 4)}] and [
    ↳ {f(C['gd_minus_sa_H2_R3_n19_boot_lo'], 4)}, {f(C['gd_minus_sa_H2_R3_n19_boot_hi'], 4)}]. "
194             f"These robustness checks use the same 18 settings and the same seeds as the main analysis; they cann
    ↳ ot add power, and they do not address the confounding of curvature with area.")
```

omml.py

Native Word equations and their image versions.

```
  1 """omml.py -- real Word equation objects (Office Math, OMML) for every displayed equation.
  2
```

```
Each entry of EQUATIONS is keyed by the marked-up text that docbuilder.equation() receives, so call sites
need no change.  Output is native OMML: Word shows it as an editable equation; LibreOffice renders it as a formula.
"""
from xml.sax.saxutils import escape

NS = ('xmlns:m="http://schemas.openxmlformats.org/officeDocument/2006/math" '
      'xmlns:w="http://schemas.openxmlformats.org/wordprocessingml/2006/main"')
FONT = '<w:rPr><w:rFonts w:ascii="Cambria Math" w:hAnsi="Cambria Math" w:cs="Cambria Math"/><w:sz w:val="24"/></w:rPr>'


def R(t, sty=None):
    """Run.  sty: None (italic variable), 'p' (upright), 'b' (bold), 'bi' (bold italic)."""
    pr = f'<m:rPr><m:sty m:val="{sty}"/></m:rPr>' if sty else ""
    return f'<m:r>{pr}{FONT}<m:t xml:space="preserve">{escape(t)}</m:t></m:r>'


def P(t):      # upright text (function names, words)
    return R(t, "p")


def V(t):      # bold italic vector symbol
    return R(t, "bi")


def sub(b, s):
    return f"<m:sSub><m:e>{b}</m:e><m:sub>{s}</m:sub></m:sSub>"


def sup(b, s):
    return f"<m:sSup><m:e>{b}</m:e><m:sup>{s}</m:sup></m:sSup>"


def subsup(b, s, u):
    return f"<m:sSubSup><m:e>{b}</m:e><m:sub>{s}</m:sub><m:sup>{u}</m:sup></m:sSubSup>"


def frac(n, d):
    return f"<m:f><m:num>{n}</m:num><m:den>{d}</m:den></m:f>"


def delim(beg, end, *items, sep=None):
    s = f'<m:sepChr m:val="{sep}"/>' if sep else ""
    es = "".join(f"<m:e>{i}</m:e>" for i in items)
```

```
    return f'<m:d><m:dPr><m:begChr m:val="{beg}"/>{s}<m:endChr m:val="{end}"/></m:dPr>{es}</m:d>'


def nary(ch, lo, body):
    return (f'<m:nary><m:naryPr><m:chr m:val="{ch}"/><m:limLoc m:val="undOvr"/><m:supHide m:val="1"/></m:naryPr>
↳ '
            f"<m:sub>{lo}</m:sub><m:sup/><m:e>{body}</m:e></m:nary>")


def minlim(lo, body):
    return (f"<m:func><m:fName><m:limLow><m:e>{P('min')}</m:e><m:lim>{lo}</m:lim></m:limLow></m:fName>"
            f"<m:e>{body}</m:e></m:func>")


def fn(name, arg):
    return f"<m:func><m:fName>{P(name)}</m:fName><m:e>{arg}</m:e></m:func>"


def wrap(*parts):
    return f"<m:oMath {NS}>" + "".join(parts) + "</m:oMath>"


ell, d_, x_, y_, z_, w_, r_, R_, rho = R("ℓ"), R("d"), V("x"), V("y"), R("z"), R("w"), R("r"), R("R"), R("ρ")
i_, j_, k_, p_, m_ = R("i"), R("j"), R("k"), R("p"), R("m")
bar = lambda t: delim("|", "|", t)

EQUATIONS = {
    # Equation 1: sphere distance
    "cos <i>d</i> = <b><i>x</i></b> · <b><i>y</i></b>": wrap(P("cos"), R(" "), d_, R("="), x_, R("·"), y_),
    # Equation 2: hyperboloid distance
    "cosh <i>d</i> = −[<b><i>x</i></b>, <b><i>y</i></b>]": wrap(P("cosh"), R(" "), d_, R("=−"), delim("[", "]",
↳ x_, y_, sep=",")),
    # Equation 3: Poincare disk distance
    "cosh <i>d</i> = 1 + 2|<i>z</i> − <i>w</i>|<sup>2</sup> / [(1 − |<i>z</i>|<sup>2</sup>)(1 − |<i>w</i>|<sup>2
↳ </sup>)]":
        wrap(P("cosh"), R(" "), d_, R("=1+"),
             frac(R("2") + sup(bar(z_ + R("−") + w_), R("2")),
                  delim("(", ")", R("1−") + sup(bar(z_), R("2"))) + delim("(", ")", R("1−") + sup(bar(w_), R("2"
↳ ))))),
    # Equation 4: areas
    "Area(<i>r</i>) = 2π(1 − cos <i>r</i>) on <i>S</i><sup>2</sup>;  π<i>r</i><sup>2</sup> on <i>R</i><sup>2</su
↳ p>;  2π(cosh <i>r</i> − cosh0) on <i>H</i><sup>2</sup>": None,
    # Equation 5: l_min
    "<i>ℓ</i><sub>min</sub> = min{ min<sub><i>i</i> < <i>j</i></sub> <i>d</i><sub><i>ij</i></sub>, min<sub><i>i<
↳ /i></sub> 2(<i>R</i> − <i>ρ</i><sub><i>i</i></sub>) }":
```

```
        wrap(sub(ell, P("min")), R("="), delim("{", "}",
             minlim(i_ + R("<") + j_, sub(d_, i_ + j_)),
             minlim(i_, R("2") + delim("(", ")", R_ + R("−") + sub(rho, i_))), sep=",")),
    # Equation 6: soft minimum energy
    "<i>E<sub>p</sub></i> = (1/<i>p</i>) ln Σ<sub><i>k</i></sub> <i>ℓ<sub>k</sub></i><sup>−<i>p</i></sup>":
        wrap(sub(R("E"), p_), R("="), frac(R("1"), p_), fn("ln", nary("∑", k_, subsup(ell, k_, R("−") + p_)))),
    # Equation 7: bounds
    "−ln <i>ℓ</i><sub>min</sub> ≤ <i>E<sub>p</sub></i> ≤ −ln <i>ℓ</i><sub>min</sub> + (ln <i>m</i>)/<i>p</i>":
        wrap(R("−"), fn("ln", sub(ell, P("min"))), R("≤"), sub(R("E"), p_), R("≤−"), fn("ln", sub(ell, P("min"))),
             R("+"), frac(fn("ln", m_), p_)),
}
# Equation 4 is written explicitly (the key above is a placeholder removed here).
EQUATIONS.pop([k for k in EQUATIONS if k.startswith("Area(")][0])
EQUATIONS["Area(<i>r</i>) = 2π(1 − cos <i>r</i>) on <i>S</i><sup>2</sup>;  π<i>r</i><sup>2</sup> on <i>R</i><sup>2</sup>;  2π(cosh <i>r</i> − 1) on <i>H</i><sup>2</sup>"] = wrap(
    P("Area"), delim("(", ")", r_), R("="), R("2π"), delim("(", ")", R("1−") + P("cos") + R(" ") + r_), P("  on  "), sup(R("S"), R("2")),
    R(";   "), R("π") + sup(r_, R("2")), P("  on  "), sup(R("R"), R("2")),
    R(";   "), R("2π"), delim("(", ")", P("cosh") + R(" ") + r_ + R("−1")), P("  on  "), sup(R("H"), R("2")))


# ---------------------------------------------------------------------------------------------
# PDF variant: LibreOffice (used to make the PDF) may lack its Math module, so for the PDF every
# displayed equation is typeset by matplotlib's mathtext (STIX fonts) and placed inline as a picture.
# The .docx keeps the native editable equations above.  MODE is "image" while the PDF is built and
# "omml" for the final .docx.
# ---------------------------------------------------------------------------------------------
MODE = "image"
LATEX = {
    "cos <i>d</i> = <b><i>x</i></b> · <b><i>y</i></b>": r"$\cos d=\mathbf{x}\cdot\mathbf{y}$",
    "cosh <i>d</i> = −[<b><i>x</i></b>, <b><i>y</i></b>]": r"$\cosh d=-[\mathbf{x},\mathbf{y}]$",
    "cosh <i>d</i> = 1 + 2|<i>z</i> − <i>w</i>|<sup>2</sup> / [(1 − |<i>z</i>|<sup>2</sup>)(1 − |<i>w</i>|<sup>2</sup>)]":
        r"$\cosh d=1+\dfrac{2\,|z-w|^{2}}{\left(1-|z|^{2}\right)\left(1-|w|^{2}\right)}$",
    "Area(<i>r</i>) = 2π(1 − cos <i>r</i>) on <i>S</i><sup>2</sup>;  π<i>r</i><sup>2</sup> on <i>R</i><sup>2</sup>;  2π(cosh <i>r</i> − 1) on <i>H</i><sup>2</sup>":
        r"$\mathrm{Area}(r)=2\pi(1-\cos r)\ \mathrm{on}\ S^{2};\ \ \pi r^{2}\ \mathrm{on}\ R^{2};\ \ 2\pi(\cosh r-1)\ \mathrm{on}\ H^{2}$",
    "<i>ℓ</i><sub>min</sub> = min{ min<sub><i>i</i> < <i>j</i></sub> <i>d</i><sub><i>ij</i></sub>, min<sub><i>i</i></sub> 2(<i>R</i> − <i>ρ</i><sub><i>i</i></sub>) }":
        r"$\ell_{\min}=\min\left\{\min_{i<j}d_{ij},\ \min_{i}\,2(R-\rho_{i})\right\}$",
    "<i>E<sub>p</sub></i> = (1/<i>p</i>) ln Σ<sub><i>k</i></sub> <i>ℓ<sub>k</sub></i><sup>−<i>p</i></sup>":
        r"$E_{p}=\dfrac{1}{p}\,\ln\sum_{k}\ell_{k}^{-p}$",
    "−ln <i>ℓ</i><sub>min</sub> ≤ <i>E<sub>p</sub></i> ≤ −ln <i>ℓ</i><sub>min</sub> + (ln <i>m</i>)/<i>p</i>":
```

```
        r"$-\ln\ell_{\min}\ \leq\ E_{p}\ \leq\ -\ln\ell_{\min}+\dfrac{\ln m}{p}$",
}


def render_png(key, outdir):
    """Typeset one equation to a transparent PNG; returns (path, width_in, height_in)."""
    import os, hashlib
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    from PIL import Image
    os.makedirs(outdir, exist_ok=True)
    path = os.path.join(outdir, "eq_" + hashlib.md5(key.encode()).hexdigest()[:10] + ".png")
    meta = path + ".txt"
    if not (os.path.exists(path) and os.path.exists(meta)):
        plt.rcParams["mathtext.fontset"] = "stix"
        fig = plt.figure(figsize=(7, 1.2), dpi=450)
        t = fig.text(0.02, 0.5, LATEX[key], fontsize=13, va="baseline", ha="left")
        fig.canvas.draw()
        bb = t.get_window_extent(fig.canvas.get_renderer())
        base_y = 0.5 * fig.get_figheight() * 450
        descent_px = base_y - bb.y0          # extent of the equation below its baseline
        fig.savefig(path, dpi=450, transparent=True, bbox_inches="tight", pad_inches=0.03)
        plt.close(fig)
        open(meta, "w").write("%.6f" % float(descent_px / 450.0 + 0.03))
    descent_in = float(open(meta).read())
    w, h = Image.open(path).size
    return path, w / 450.0, h / 450.0, descent_in
```

newalgs.py

Added baselines of the confirmatory round: SciPy L-BFGS-B and basin hopping on the soft-minimum energy.

```
"""newalgs.py -- stronger baselines for the confirmatory round (added; algorithms.py is unchanged).

  lbfgsb : SciPy L-BFGS-B on the soft-minimum energy with the exponent p raised in stages (continuation)
  bh     : basin hopping (scipy.optimize.basinhopping) with L-BFGS-B local minimization on the soft-minimum ener
↳ gy
Budget is the same pair-distance-evaluation count as the other methods: each energy-and-gradient evaluation
costs one sweep (plus n wall clearances in ball problems).  Points are optimized in an unconstrained chart:
sphere  x = y/|y| (y in R^3);  hyperboloid  x = (sqrt(1+|u|^2), u);  Euclidean plane  x = u.
"""
```

```
import time
import numpy as np
from scipy.optimize import minimize, basinhopping
from algorithms import Problem, _soft_energy, _to_score, RUNNERS, ALG_NAMES


class BudgetExhausted(Exception):
    pass


def to_w(P, X):
    return (X[:, 1:] if P.space.kind == "H2" else X).ravel().copy()


def from_w(P, w):
    k = P.space.kind
    if k == "S2":
        Y = w.reshape(P.n, 3)
        return Y / np.linalg.norm(Y, axis=1, keepdims=True)
    if k == "H2":
        U = w.reshape(P.n, 2)
        return np.c_[np.sqrt(1.0 + (U ** 2).sum(1)), U]
    return w.reshape(P.n, 2)


def grad_w(P, X, G, w):
    """Chain rule from the Riemannian gradient G to the chart coordinates."""
    k = P.space.kind
    if k == "S2":
        Y = w.reshape(P.n, 3)
        return (G / np.linalg.norm(Y, axis=1, keepdims=True)).ravel()
    if k == "H2":
        U = X[:, 1:]
        return (G[:, 1:] - G[:, :1] * U / X[:, :1]).ravel()
    return G.ravel()


class _Obj:
    """Energy-gradient callable in chart coordinates with budget accounting and best-visited tracking."""
    def __init__(self, P, budget):
        self.P, self.budget = P, budget
        self.cost = P.sweep + (P.n if P.ball else 0)
        self.pe = 0.0
        self.best, self.bestX = -np.inf, None
        self.p = 2.0
```

```
        self.nev = 0

    def __call__(self, w):
        if self.pe + self.cost > self.budget:
            raise BudgetExhausted
        P = self.P
        X = from_w(P, w)
        E, G, lmin = _soft_energy(P, X, self.p, True)
        self.pe += self.cost
        self.nev += 1
        if P.ball and float(P.space.rho(X).max()) >= P.R:
            pass
        sc = _to_score(P, lmin)
        if sc > self.best and (not P.ball or float(P.lmin(X)) > 0):
            self.best, self.bestX = sc, X.copy()
        return E, grad_w(P, X, G, w)


def run_lbfgsb(P, X0, budget, rng, pmax=150.0, stages=10, p0=2.0):
    """Continuation in p with L-BFGS-B.  L-BFGS-B stops by its own convergence tests, so when a continuation
    finishes with budget left, the method restarts from a new random configuration (same seeded stream) until
    the budget is used; the best configuration visited is returned."""
    obj = _Obj(P, budget)
    start = X0
    first = True
    try:
        while True:
            used0 = obj.pe
            w = to_w(P, start)
            stage_budget = (budget if first else min(budget - obj.pe, budget)) / stages
            for s in range(stages):
                obj.p = p0 * (pmax / p0) ** (s / (stages - 1))
                mf = max(int(stage_budget // obj.cost), 5)
                r = minimize(obj, w, jac=True, method="L-BFGS-B", options=dict(maxfun=mf, maxiter=100000, ftol=1
  ↳ e-13, gtol=1e-11, maxcor=10))
                w = r.x
                X = from_w(P, w)
                if P.ball:
                    X = P.space.clip_to_ball(X, P.R - 1e-6)
                    w = to_w(P, X)
            first = False
            if budget - obj.pe < 0.5 * (obj.pe - used0) or budget - obj.pe < 20 * obj.cost:
                break
            start = P.initial(rng)
    except BudgetExhausted:
```

```
        pass
    X = obj.bestX if obj.bestX is not None else X0
    sc = P.score(X)
    return dict(X=X, score=sc, trace=[(0.0, sc)], pe=obj.pe, iters=obj.nev)


def run_bh(P, X0, budget, rng, step=0.2, pbh=60.0, temp=0.02, local_maxfun=60):
    obj = _Obj(P, budget)
    obj.p = pbh
    scale = P.scale

    def take_step(w):
        return w + rng.normal(scale=step * scale, size=w.shape)

    mk = dict(method="L-BFGS-B", jac=True, options=dict(maxfun=local_maxfun, ftol=1e-13, gtol=1e-11))
    w0 = to_w(P, X0)
    try:
        basinhopping(obj, w0, niter=10 ** 6, T=temp, take_step=take_step, minimizer_kwargs=mk, seed=int(rng.inte
↳ gers(2 ** 31)))
    except BudgetExhausted:
        pass
    X = obj.bestX if obj.bestX is not None else X0
    if P.ball:
        X = P.space.clip_to_ball(X, P.R - 1e-6)
    sc = P.score(X)
    return dict(X=X, score=sc, trace=[(0.0, sc)], pe=obj.pe, iters=obj.nev)


NEW_RUNNERS = {"lbfgsb": run_lbfgsb, "bh": run_bh}
ALG_NAMES.update({"lbfgsb": "L-BFGS-B (SciPy)", "bh": "Basin hopping (SciPy)"})


def run_any(name, P, X0, budget, seed_seq, **params):
    """Run any of the six methods; extra keyword parameters override tuned defaults."""
    rng = np.random.default_rng(seed_seq)
    fn = NEW_RUNNERS.get(name) or RUNNERS[name]
    t0 = time.perf_counter()
    res = fn(P, X0, budget, rng, **params)
    res["time"] = time.perf_counter() - t0
    res["final_check"] = P.score(res["X"])
    return res
```

confirm_design.py

Conditions, reference densities, boundary share, and density deficit of the confirmatory round.

```
"""confirm_design.py -- conditions, reference densities, and boundary shares of the confirmatory round.

Everything here is fixed by the preregistration (PREREGISTRATION.md) before any evaluation seed is run.
"""
import numpy as np

EVAL_SEEDS = list(range(101, 121))          # fresh seeds, never used before
PILOT_SEEDS = [9101, 9102, 9103, 9104]      # tuning only
BUDGETS = [500, 1500, 3000]                 # sweeps; 1 sweep = n(n-1)/2 pair-distance evaluations
MAIN_BUDGET = 1500
ALGS = {"gd": 1, "sa": 2, "ls": 3, "gdp": 4, "lbfgsb": 5, "bh": 6}
TAMMES_N = [9, 13, 14, 24, 30, 40, 60]
EA_AREAS = [3.0, 6.0, 12.0]
EA_N = [12, 24]
LARGE_RH = [2.0, 3.0, 6.0, 8.0]
LARGE_N = {2.0: 24, 3.0: 36, 6.0: 50, 8.0: 70}


def area(kind, R):
    if kind == "S2cap":
        return 2 * np.pi * (1 - np.cos(R))
    if kind == "H2":
        return 2 * np.pi * (np.cosh(R) - 1)
    return np.pi * R * R


def radius_for_area(kind, A):
    if kind == "S2cap":
        return float(np.arccos(1 - A / (2 * np.pi)))
    if kind == "H2":
        return float(np.arccosh(1 + A / (2 * np.pi)))
    return float(np.sqrt(A / np.pi))


def conditions():
    out, cid = [], 1000
    for N in TAMMES_N:
        out.append(dict(cid=cid, group="tammes", kind="S2", n=N, R=None, area=4 * np.pi, label=f"Tammes N = {N}"
```

```
  ↳ ))
39          cid += 1
40      for A in EA_AREAS:
41          for n in EA_N:
42              for kind, lab in (("S2cap", "cap"), ("R2", "Euclidean"), ("H2", "hyperbolic")):
43                  out.append(dict(cid=cid, group="equal_area", kind=kind, n=n, R=radius_for_area(kind, A), area=A,
  ↳  label=f"A = {A:g}, n = {n}, {lab}"))
44                  cid += 1
45      for Rh in LARGE_RH:
46          A = area("H2", Rh)
47          n = LARGE_N[Rh]
48          for kind, lab in (("R2", "Euclidean"), ("H2", "hyperbolic")):
49              R = radius_for_area(kind, A)
50              out.append(dict(cid=cid, group="large", kind=kind, n=n, R=R, area=A, label=f"R_H = {Rh:g}, n = {n},
  ↳ {lab}"))
51              cid += 1
52      return out
53
54
55  def ref_density(kind, r):
56      """Upper bound on the density of any packing of equal disks of radius r in the whole plane/space (Fejes Toth
  ↳  1943 for the sphere,
57      Thue/Fejes Toth for the plane, Boroczky 1978 for the hyperbolic plane): the density of the disks inside the
  ↳ regular
58      triangle whose vertices are the centers of three mutually tangent disks."""
59      if kind == "R2":
60          return float(np.pi / np.sqrt(12.0))
61      if kind == "H2":
62          c = np.cosh(2 * r)
63          al = np.arccos(c / (c + 1.0))
64          return float(3 * al * (np.cosh(r) - 1) / (np.pi - 3 * al))
65      c = np.cos(2 * r)                       # spherical (cap)
66      al = np.arccos(c / (1.0 + c))
67      return float(3 * al * (1 - np.cos(r)) / (3 * al - np.pi))
68
69
70  def boundary_share(kind, R, r):
71      """Share of the ball's area that lies within distance r of its boundary."""
72      return float(1.0 - area(kind, R - r) / area(kind, R))
73
74
75  def deficit(kind, R, n, r):
76      """Density deficit  1 - (density achieved) / (reference density for disks of radius r)."""
77      dens = n * area(kind, r) / area(kind, R)
78      return float(1.0 - dens / ref_density(kind, r))
```

record_env.py

Records hardware and software versions.

```
"""record_env.py -- records hardware and software used for the confirmatory round (out/conf/hardware.json)."""
import json, os, platform, subprocess, sys
import numpy, scipy, pandas
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))


def sh(c):
    return subprocess.run(c, shell=True, capture_output=True, text=True).stdout.strip()


info = dict(cpu_model=sh("lscpu | grep 'Model name' | cut -d: -f2").strip(), logical_cpus=os.cpu_count(), thread
↳ s_per_core=sh("lscpu | grep 'Thread(s)' | cut -d: -f2").strip(),
            l2_cache=sh("lscpu | grep 'L2 cache' | cut -d: -f2").strip(), memory_gb=round(int(sh("grep MemTotal
↳ /proc/meminfo | awk '{print $2}'")) / 1048576, 1),
            os=platform.platform(), python=platform.python_version(), numpy=numpy.__version__, scipy=scipy.__ver
↳ sion__, pandas=pandas.__version__,
            parallel_workers=2, note="Cloud container; wall-clock times of the confirmatory round were measured
↳ with two worker processes sharing the machine.",
            recorded_utc=sh("date -u +%Y-%m-%dT%H:%M:%SZ"))
os.makedirs(ROOT + "/out/conf", exist_ok=True)
from confirm_run import ensure_frozen
ensure_frozen()
json.dump(info, open(ROOT + "/out/conf/hardware.json", "w"), indent=1)
print(info)
```

confirm_tune.py

Equal-effort pilot tuning of all six methods on pilot seeds 9101 to 9104.

```
"""confirm_tune.py -- equal-effort tuning of all six methods on pilot seeds (never used for evaluation).

Each method has ONE tuned parameter with the same number of candidate settings (six); every setting is run on th
↳ e same
pilot conditions and pilot seeds at the main budget; the setting with the highest mean relative score
(score / best score seen in that pilot condition over all settings and methods) is kept.
```

```
Writes out/conf/tuning_runs.csv and out/conf/tuned_params.json.
"""
import os
import json
import numpy as np
import pandas as pd
from multiprocessing import Pool
from algorithms import Problem
import newalgs
import confirm_design as C
from confirm_run import PARAM_KEY, CONF

GRID = {"gd": [40, 80, 150, 250, 400, 700], "gdp": [0.05, 0.10, 0.15, 0.25, 0.35, 0.50], "sa": [0.005, 0.01, 0.02, 0.04, 0.08, 0.16],
        "ls": [0.10, 0.15, 0.25, 0.35, 0.50, 0.70], "lbfgsb": [40, 80, 150, 250, 400, 700], "bh": [0.05, 0.10, 0.20, 0.40, 0.80, 1.50]}
PILOT = [("S2", 11, None), ("S2", 33, None), ("H2", 16, C.radius_for_area("H2", 8.0)), ("R2", 16, C.radius_for_area("R2", 8.0))]


def job(args):
    alg, val, ci, seed = args
    kind, n, R = PILOT[ci]
    P = Problem(kind, n, R)
    X0 = P.initial(np.random.default_rng([seed, 77, ci]))
    res = newalgs.run_any(alg, P, X0, C.MAIN_BUDGET * P.sweep, [seed, 77, ci, C.ALGS[alg]], **{PARAM_KEY[alg]: val})
    return dict(algorithm=alg, value=val, pilot=ci, seed=seed, score=res["score"], time_s=res["time"])


def main():
    tasks = [(a, v, ci, s) for a, g in GRID.items() for v in g for ci in range(len(PILOT)) for s in C.PILOT_SEEDS]
    with Pool(2) as pool:
        rows = pool.map(job, tasks, chunksize=4)
    d = pd.DataFrame(rows)
    d["rel"] = d.score / d.groupby("pilot").score.transform("max")
    d.to_csv(CONF + "/tuning_runs.csv", index=False)
    m = d.groupby(["algorithm", "value"]).rel.mean().reset_index()
    tuned = {}
    for a, g in m.groupby("algorithm"):
        tuned[a] = float(g.loc[g.rel.idxmax(), "value"])
    json.dump(dict(tuned=tuned, grids=GRID, pilot_seeds=C.PILOT_SEEDS, pilot_conditions=[list(map(str, p)) for p in PILOT],
                   budget_sweeps=C.MAIN_BUDGET, mean_relative_by_setting=m.round(5).to_dict("records")), open(CO
```

```
    ↳ NF + "/tuned_params.json", "w"), indent=1)
 45     print(m.pivot(index="value", columns="algorithm", values="rel").round(4) if False else tuned)
 46
 47
 48 if __name__ == "__main__":
 49     os.makedirs(CONF, exist_ok=True)
 50     main()
```

confirm_run.py

Runs the confirmatory round: fresh seeds 101 to 120, three budgets, six methods.

```
  1 """confirm_run.py -- confirmatory round: fresh seeds, three budget levels, six methods (resumable per condition)
    ↳ .
  2
  3 Usage:  python3 confirm_run.py [--rerun] [--only CID,CID]      writes out/conf/runs_<cid>.csv
  4 Tuned parameters are read from out/conf/tuned_params.json (fixed and hashed before this script is run).
  5 """
  6 import os
  7 import sys
  8 import json
  9 import time
 10 import numpy as np
 11 import pandas as pd
 12 from multiprocessing import Pool
 13 from algorithms import Problem
 14 import newalgs
 15 import confirm_design as C
 16
 17 ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
 18 CONF = ROOT + "/out/conf"
 19 PARAM_KEY = {"gd": "pmax", "gdp": "polish_share", "sa": "T0f", "ls": "g0", "lbfgsb": "pmax", "bh": "step"}
 20
 21
 22 def ensure_frozen():
 23     """The preregistration and its hashes are frozen inputs (code/frozen); copy them into out/conf when absent."
    ↳ ""
 24     import shutil
 25     os.makedirs(CONF, exist_ok=True)
 26     for nm in ("PREREGISTRATION.md", "PREREGISTRATION.sha256", "TUNED_AND_ENV.sha256", "packomania_values.csv",
 27                "PREREGISTRATION_ADDENDUM_1.md", "PREREGISTRATION_MINIROUND.md", "PREREG_ADDENDA.sha256", "RETUNE
    ↳ D.sha256",
```

```
                "PREREGISTRATION_FOLLOWUP.md", "FOLLOWUP.sha256", "CORRECTION_NOTE_1.md", "CORRECTION.sha256", "F
↳ OLLOWUP_RUNTIMES.txt", "PREREGISTRATION_ROUND3.md", "ROUND3.sha256", "ROUND3_START.txt", "ROUND3_END.txt", "PLAN
↳ _FILES.sha256"):
        if not os.path.exists(f"{CONF}/{nm}"):
            shutil.copy(os.path.join(os.path.dirname(os.path.abspath(__file__)), "frozen", nm), CONF)


def load_params():
    return json.load(open(CONF + "/tuned_params.json"))["tuned"]


def problem_of(c):
    return Problem("S2" if c["kind"] in ("S2", "S2cap") else c["kind"], c["n"], c["R"])


def run_condition(c):
    params = load_params()
    P = problem_of(c)
    rows = []
    for bi, B in enumerate(C.BUDGETS):
        for seed in C.EVAL_SEEDS:
            X0 = P.initial(np.random.default_rng([seed, c["cid"], 0]))
            for alg, aid in C.ALGS.items():
                kw = {PARAM_KEY[alg]: params[alg]}
                res = newalgs.run_any(alg, P, X0, B * P.sweep, [seed, c["cid"], aid, bi], **kw)
                sc = res["score"]
                rows.append(dict(condition_id=c["cid"], group=c["group"], space=c["kind"], n=c["n"],
                                 R=(c["R"] if c["R"] is not None else float("nan")), area=c["area"], algorithm=a
↳ lg,
                                 budget=B, seed=seed, score=sc, recomputed_score=res["final_check"],
                                 density=(P.density(sc) if P.ball else float("nan")),
                                 sweeps_used=res["pe"] / P.sweep, time_s=res["time"]))
    return pd.DataFrame(rows)


def _job(c):
    f = f"{CONF}/runs_{c['cid']}.csv"
    t0 = time.time()
    df = run_condition(c)
    df.to_csv(f, index=False)
    return c["cid"], time.time() - t0


def main():
    os.makedirs(CONF, exist_ok=True)
```

```
    conds = C.conditions()
    if "--only" in sys.argv:
        keep = {int(x) for x in sys.argv[sys.argv.index("--only") + 1].split(",")}
        conds = [c for c in conds if c["cid"] in keep]
    if "--rerun" not in sys.argv:
        conds = [c for c in conds if not os.path.exists(f"{CONF}/runs_{c['cid']}.csv")]
    conds.sort(key=lambda c: -c["n"])          # longest first
    limit = None
    if "--budget-seconds" in sys.argv:
        limit = float(sys.argv[sys.argv.index("--budget-seconds") + 1])
    t0 = time.time()
    with Pool(2) as pool:
        for cid, dt in pool.imap_unordered(_job, conds):
            print(f"done {cid} in {dt:.0f}s (elapsed {time.time() - t0:.0f}s)", flush=True)
    print("finished", len(conds), "conditions")


if __name__ == "__main__":
    main()
```

confirm_count.py

Count-based version of the disk problem (exploratory).

```
"""confirm_count.py -- count-based version of the disk problem (original element; exploratory).

For a ball of given area in each space and a fixed disk radius r, K = the largest n for which gradient descent w
↳ ith a polishing step
(GD+P, 1,500 sweeps) reaches a disk radius >= r in at least half of 20 fresh seeds (201-220).  n is scanned down
↳ ward from
n_ref = floor(rho*(r) Area(R) / Area(r)), the number of disks that would fit if the ball were packed at the infi
↳ nite-space reference density
(a heuristic ceiling, not a theorem for a bounded ball).  Writes out/conf/count_runs.csv and out/conf/count_summ
↳ ary.csv.
"""
import os
import numpy as np
import pandas as pd
from multiprocessing import Pool
from algorithms import Problem
import newalgs
import confirm_design as C
```

```
from confirm_run import CONF, load_params

SEEDS = list(range(201, 221))
RADIUS = 0.30
AREAS = [6.0, 12.0]
KINDS = [("S2cap", "cap"), ("R2", "Euclidean"), ("H2", "hyperbolic")]


def job(args):
    kind, A, n, seed = args
    R = C.radius_for_area(kind, A)
    P = Problem("S2" if kind == "S2cap" else kind, n, R)
    X0 = P.initial(np.random.default_rng([seed, 9, n, int(A)]))
    res = newalgs.run_any("gdp", P, X0, C.MAIN_BUDGET * P.sweep, [seed, 9, n, int(A)], polish_share=load_params(
)["gdp"])
    return dict(space=kind, area=A, n=n, seed=seed, score=res["score"], reached=int(res["score"] >= RADIUS))


def main():
    rows, summ = [], []
    with Pool(2) as pool:
        for kind, lab in KINDS:
            for A in AREAS:
                R = C.radius_for_area(kind, A)
                nref = int(np.floor(C.ref_density(kind, RADIUS) * C.area(kind, R) / C.area(kind, RADIUS)))
                K = 0
                for n in range(nref, 0, -1):
                    out = pool.map(job, [(kind, A, n, s) for s in SEEDS])
                    rows += out
                    share = float(np.mean([o["reached"] for o in out]))
                    if share >= 0.5:
                        K = n
                        break
                summ.append(dict(space=kind, area=A, radius=RADIUS, n_ref=nref, K=K, K_over_n_ref=K / nref, shar
e_at_K=share))
                print(summ[-1], flush=True)
    pd.DataFrame(rows).to_csv(CONF + "/count_runs.csv", index=False)
    pd.DataFrame(summ).to_csv(CONF + "/count_summary.csv", index=False)


if __name__ == "__main__":
    main()
```

confirm_analyze.py

Analysis of the confirmatory round exactly as preregistered.

```
"""confirm_analyze.py -- analysis of the confirmatory round, exactly as specified in PREREGISTRATION.md.

Reads out/conf/runs_<cid>.csv (+ optional out/conf/packomania_values.csv), writes the conf_*.csv files to out/data and
out/conf/decisions.json.  Nothing here uses a result to choose a rule.
"""
import glob
import json
import os
import sys
import numpy as np
import pandas as pd
from scipy import stats
import confirm_design as C
from experiments import BENCH_DEG, BEST_KNOWN_DEG

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
CONF, DATA = ROOT + "/out/conf", ROOT + "/out/data"
METHODS = ["gd", "gdp", "sa", "ls", "lbfgsb", "bh"]
MAIN = C.MAIN_BUDGET
rng = np.random.default_rng(20261029)          # fixed seed of the permutation and bootstrap procedures


def load():
    d = pd.concat([pd.read_csv(f) for f in sorted(glob.glob(CONF + "/runs_*.csv"))], ignore_index=True)
    conds = {c["cid"]: c for c in C.conditions()}
    pk = {}
    f = CONF + "/packomania_values.csv"
    if os.path.exists(f):
        p = pd.read_csv(f)
        pk = dict(zip(p.n.astype(int), p.radius.astype(float)))
    refs, rtype = {}, {}
    bf = d.groupby("condition_id").score.max()
    for cid, c in conds.items():
        if c["group"] == "tammes":
            N = c["n"]
            if N in BENCH_DEG:
                refs[cid], rtype[cid] = float(np.radians(BENCH_DEG[N])), "proven optimum"
```

```
            else:
                refs[cid], rtype[cid] = float(np.radians(BEST_KNOWN_DEG[N])), "published best known (Sloane)"
        elif c["kind"] == "R2" and c["n"] in pk:
            refs[cid], rtype[cid] = pk[c["n"]] * c["R"], "published best known (Packomania)"
        else:
            refs[cid], rtype[cid] = float(bf.get(cid, np.nan)), "best found in this round"
    d["reference_value"] = d.condition_id.map(refs)
    d["reference_type"] = d.condition_id.map(rtype)
    d["relative_score"] = d.score / d.reference_value
    d["success_0.1pct"] = (d.relative_score >= 0.999).astype(int)
    d["success_1pct"] = (d.relative_score >= 0.99).astype(int)
    return d, conds


def spearman_perm(x, y, nperm=20000):
    obs = stats.spearmanr(x, y)[0]
    cnt = 0
    for _ in range(nperm):
        if stats.spearmanr(x, rng.permutation(y))[0] >= obs - 1e-12:
            cnt += 1
    return float(obs), (cnt + 1) / (nperm + 1)


def main():
    d, conds = load()
    d.to_csv(DATA + "/conf_runs.csv", index=False)
    m = d.groupby(["condition_id", "budget", "algorithm"]).agg(mean_score=("score", "mean"), mean_rel=("relative_score", "mean"), sd_rel=("relative_score", "std"),
                                                               success_0p1=("success_0.1pct", "mean"), success_1=("success_1pct", "mean"),
                                                               mean_time_s=("time_s", "mean"), mean_sweeps=("sweeps_used", "mean")).reset_index()
    lab = {cid: c["label"] for cid, c in conds.items()}
    m["label"] = m.condition_id.map(lab)
    m["reference_type"] = m.condition_id.map(d.groupby("condition_id").reference_type.first())
    m.to_csv(DATA + "/conf_summary_by_condition.csv", index=False)
    out = {}

    # ---- C1 and C2 : density deficit versus boundary share over the 26 ball conditions
    rows = []
    for cid, c in conds.items():
        if c["group"] == "tammes":
            continue
        g = m[(m.condition_id == cid) & (m.budget == MAIN)]
        best = g.loc[g.mean_score.idxmax()]
```

```
        r = float(best.mean_score)
        rows.append(dict(condition_id=cid, label=c["label"], group=c["group"], space=c["kind"], n=c["n"], R=c["R
"], area=c["area"], best_method=best.algorithm,
                         r_best=r, deficit=C.deficit(c["kind"], c["R"], c["n"], r), boundary_share=C.boundary_sh
are(c["kind"], c["R"], r),
                         ref_density=C.ref_density(c["kind"], r)))
    t = pd.DataFrame(rows)
    t.to_csv(DATA + "/conf_deficit_boundary.csv", index=False)
    rho, p = spearman_perm(t.boundary_share.values, t.deficit.values)
    within = {sp: float(stats.spearmanr(g.boundary_share, g.deficit)[0]) for sp, g in t.groupby("space")}
    out["C1"] = dict(n_conditions=len(t), spearman=rho, perm_p_one_sided=p, within_space=within,
                     decision=("supported" if rho >= 0.70 and p < 0.05 else "not supported" if rho < 0.30 else "
inconclusive"))
    # exploratory robustness of C1 (not part of the decision): without the two hyperbolic balls of radius 6 and
8, which have the largest b
    t2 = t[~((t.space == "H2") & (t.group == "large") & (t.R >= 6))]
    rho2, p2 = spearman_perm(t2.boundary_share.values, t2.deficit.values, nperm=5000)
    out["C1_exploratory_without_two_largest_hyperbolic"] = dict(n_conditions=len(t2), spearman=rho2, perm_p_one_
sided=p2)
    X = np.c_[np.ones(len(t)), t.boundary_share, (t.space == "H2").astype(float), (t.space == "S2cap").astype(fl
oat)]
    beta = np.linalg.lstsq(X, t.deficit.values, rcond=None)[0]
    boots = []
    for _ in range(10000):
        ix = rng.integers(0, len(t), len(t))
        boots.append(np.linalg.lstsq(X[ix], t.deficit.values[ix], rcond=None)[0][2])
    lo, hi = np.percentile(boots, [2.5, 97.5])
    gh = float(beta[2])
    out["C2"] = dict(intercept=float(beta[0]), beta_boundary=float(beta[1]), gamma_hyperbolic=gh, gamma_cap=floa
t(beta[3]), ci95_gamma_hyperbolic=[float(lo), float(hi)],
                     decision=("curvature adds a detectable deficit" if lo > 0 and gh >= 0.02 else "no detectabl
e curvature effect" if lo >= -0.02 and hi <= 0.02 else "inconclusive"))

    # ---- C3 : GD minus SA at equal area across spaces
    w = d[(d.budget == MAIN) & d.condition_id.isin([cid for cid, c in conds.items() if c["group"] == "equal_area
"])].pivot_table(index=["condition_id", "seed"], columns="algorithm", values="relative_score")
    dc = (w.gd - w.sa).groupby("condition_id").mean().rename("mean_gd_minus_sa").reset_index()
    dc["space"] = dc.condition_id.map(lambda c: conds[c]["kind"])
    dc.to_csv(DATA + "/conf_gd_minus_sa_equal_area.csv", index=False)
    grp = {sp: g.mean_gd_minus_sa.values for sp, g in dc.groupby("space")}
    kw = stats.kruskal(*grp.values())
    pairs, inside = {}, []
    for a, b in (("S2cap", "R2"), ("S2cap", "H2"), ("R2", "H2")):
        diffs = [np.median(rng.choice(grp[a], len(grp[a]))) - np.median(rng.choice(grp[b], len(grp[b]))) for _ i
n range(10000)]
```

```
        lo, hi = np.percentile(diffs, [5, 95])
        pairs[f"{a}-{b}"] = [float(lo), float(hi)]
        inside.append(lo >= -0.01 and hi <= 0.01)
    out["C3"] = dict(kruskal_chi2=float(kw.statistic), kruskal_p=float(kw.pvalue), median_d={k: float(np.median(
↳ v)) for k, v in grp.items()}, ci90_median_difference=pairs,
                     decision=("ranking depends on the space" if kw.pvalue < 0.05 else "ranking does not depend
↳ on the space (equivalence)" if all(inside) else "inconclusive"))

    # ---- C4 : baselines and ranks
    g1 = m[m.budget == MAIN].pivot(index="condition_id", columns="algorithm", values="mean_rel")
    new_best = (g1[["lbfgsb", "bh"]].max(axis=1) > g1[["gd", "sa"]].max(axis=1))
    rk = d[d.budget == MAIN].copy()
    rk["rank"] = rk.groupby(["condition_id", "seed"]).score.rank(ascending=False)
    mr = rk.groupby(["condition_id", "algorithm"])["rank"].mean().unstack()
    mr.add_prefix("mean_rank_").to_csv(DATA + "/conf_mean_ranks.csv")
    gdp_top2 = (mr.rank(axis=1)["gdp"] <= 2)
    out["C4"] = dict(share_new_baseline_beats_gd_and_sa=float(new_best.mean()), n_conditions=int(len(g1)), share
↳ _gdp_rank_top2=float(gdp_top2.mean()),
                     mean_rank_by_method=mr.mean().round(3).to_dict(),
                     decision_baselines_matter=bool(new_best.mean() >= 0.5), decision_gdp_competitive=bool(gdp_t
↳ op2.mean() >= 0.5))

    # ---- C5 : Kendall's W across budgets
    Ws, same_top = [], []
    for cid, g in m.groupby("condition_id"):
        P = g.pivot(index="algorithm", columns="budget", values="mean_rel").reindex(METHODS)
        R = P.rank(axis=0)
        S = ((R.sum(axis=1) - R.sum(axis=1).mean()) ** 2).sum()
        Ws.append(12 * S / (3 ** 2 * (6 ** 3 - 6)))
        same_top.append(P[1500].idxmax() == P[3000].idxmax())
    out["C5"] = dict(median_W=float(np.median(Ws)), min_W=float(np.min(Ws)), share_same_top_method_1500_vs_3000=
↳ float(np.mean(same_top)),
                     decision=("ranking stable across budgets" if np.median(Ws) >= 0.80 else "ranking not stable
↳  across budgets"))
    pd.DataFrame(dict(condition_id=sorted(m.condition_id.unique()), kendall_W=Ws)).to_csv(DATA + "/conf_kendall_
↳ w.csv", index=False)

    # ---- C6 : Tammes recovery
    rec = {}
    for N in (9, 13, 14, 24):
        cid = [cid for cid, c in conds.items() if c["group"] == "tammes" and c["n"] == N][0]
        g = m[(m.condition_id == cid) & (m.budget == MAIN)]
        rec[N] = dict(best_share=float(g.success_0p1.max()), best_method=g.loc[g.success_0p1.idxmax(), "algorith
↳ m"])
    out["C6"] = dict(by_N=rec, decision=("supported" if all(v["best_share"] >= 0.9 for v in rec.values()) else "
```

```
    ↳ not supported"))
152
153     # ---- descriptive : time, sweeps used, success
154     tt = d.groupby(["budget", "algorithm"]).agg(median_time_s=("time_s", "median"), mean_sweeps_used=("sweeps_us
    ↳ ed", "mean")).reset_index()
155     tt.to_csv(DATA + "/conf_time_by_method.csv", index=False)
156     out["reference_types"] = d.groupby("condition_id").reference_type.first().value_counts().to_dict()
157     out["n_runs"] = int(len(d))
158     out["max_abs_score_vs_recomputed"] = float((d.score - d.recomputed_score).abs().max())
159     json.dump(out, open(CONF + "/decisions.json", "w"), indent=1, default=float)
160     print(json.dumps(out, indent=1, default=float))
161
162
163 if __name__ == "__main__":
164     main()
```

conf_figures.py

Figures of the confirmatory round.

```
  1 """conf_figures.py -- figures of the confirmatory round, drawn only from out/data/conf_*.csv."""
  2 import os
  3 import numpy as np
  4 import pandas as pd
  5 import matplotlib
  6 matplotlib.use("Agg")
  7 import matplotlib.pyplot as plt
  8 from matplotlib.lines import Line2D
  9 import make_figures as MF          # same fonts, legend style, save()
 10
 11 ROOT = MF.ROOT
 12 DATA = MF.DATA
 13 SP_COL = {"R2": "#D55E00", "H2": "#7B3294", "S2cap": "#0072B2"}
 14 SP_MARK = {"R2": "o", "H2": "s", "S2cap": "^"}
 15 SP_NAME = {"R2": "Euclidean", "H2": "Hyperbolic", "S2cap": "Spherical cap"}
 16 M_COL = {"gd": "#1F4E9E", "sa": "#C62828", "ls": "#2E7D32", "gdp": "#6A1B9A", "lbfgsb": "#EF6C00", "bh": "#6D4C4
    ↳ 1"}
 17 M_MARK = {"gd": "o", "sa": "s", "ls": "^", "gdp": "D", "lbfgsb": "v", "bh": "P"}
 18 M_NAME = {"gd": "GD", "gdp": "GD+P", "sa": "SA", "ls": "CP", "lbfgsb": "L-BFGS-B", "bh": "Basin hopping"}
 19
 20
 21 def fig_deficit(t, dec):
```

```
    fig, ax = plt.subplots(figsize=(6.0, 4.6))
    for sp in ("R2", "H2", "S2cap"):
        g = t[t.space == sp]
        for grp, fill in (("equal_area", True), ("large", False)):
            h = g[g.group == grp]
            if len(h):
                ax.scatter(h.boundary_share, h.deficit, s=48, marker=SP_MARK[sp], edgecolor=SP_COL[sp], linewidth=1.4,
                           facecolor=SP_COL[sp] if fill else "white", zorder=3)
    xs = np.linspace(t.boundary_share.min(), t.boundary_share.max(), 50)
    c2 = dec["C2"]
    for sp in ("R2", "H2", "S2cap"):
        if (t.space == sp).any():
            a = c2["intercept"] + (c2["gamma_hyperbolic"] if sp == "H2" else 0) + (c2["gamma_cap"] if sp == "S2cap" else 0)
            ax.plot(xs, a + c2["beta_boundary"] * xs, color=SP_COL[sp], lw=1.2, ls="--", zorder=2)
    ax.set_xlabel("Boundary share  b  (share of ball area within one disk radius of the wall)")
    ax.set_ylabel("Density deficit  δ")
    ax.grid(True)
    h = [Line2D([0], [0], marker=SP_MARK[s], color="w", markeredgecolor=SP_COL[s], markerfacecolor=SP_COL[s], ms=8, label=SP_NAME[s]) for s in ("R2", "H2", "S2cap")]
    h += [Line2D([0], [0], marker="o", color="w", markeredgecolor="black", markerfacecolor="black", ms=8, label="Equal-area balls"),
          Line2D([0], [0], marker="o", color="w", markeredgecolor="black", markerfacecolor="white", ms=8, label="Large balls"),
          Line2D([0], [0], color="black", ls="--", lw=1.2, label="Fitted line (Equation in text)")]
    fig.subplots_adjust(left=0.13, right=0.97, top=0.97, bottom=0.34)
    MF.legend_box(fig, h, loc="lower center", ncol=2, bbox_to_anchor=(0.55, 0.0))
    return MF.save(fig, "conf_deficit_boundary")


def fig_budget(m, conds):
    groups = [("Tammes problem", lambda c: c["group"] == "tammes"), ("Equal-area balls", lambda c: c["group"] == "equal_area"), ("Large balls", lambda c: c["group"] == "large")]
    fig, axs = plt.subplots(1, 3, figsize=(7.0, 3.6), sharey=True)
    for ax, (title, fn) in zip(axs, groups):
        ids = [cid for cid, c in conds.items() if fn(c)]
        g = m[m.condition_id.isin(ids)].groupby(["algorithm", "budget"]).mean_rel.mean().reset_index()
        for a in M_COL:
            h = g[g.algorithm == a].sort_values("budget")
            ax.plot(range(3), h.mean_rel.values, color=M_COL[a], marker=M_MARK[a], ms=5, lw=1.4)
        ax.set_xticks(range(3))
        ax.set_xticklabels(["500", "1,500", "3,000"])
        ax.set_title(title)
        ax.grid(True)
```

```
    axs[0].set_ylabel("Mean relative score")
    fig.text(0.5, 0.30, "Budget (sweeps)", ha="center")
    fig.subplots_adjust(left=0.10, right=0.99, top=0.91, bottom=0.40, wspace=0.08)
    h = [Line2D([0], [0], color=M_COL[a], marker=M_MARK[a], lw=1.4, ms=5, label=M_NAME[a]) for a in M_COL]
    MF.legend_box(fig, h, loc="lower center", ncol=3, bbox_to_anchor=(0.55, 0.0))
    return MF.save(fig, "conf_budget")


def fig_rank(mr, conds):
    """Heat map of the mean rank (1 = best) of the six methods in each of the 33 conditions at 1,500 sweeps, wit
↳ h the mean over conditions at the bottom."""
    order = [a for a in mr.mean().sort_values().index.str.replace("mean_rank_", "")]
    cols = ["mean_rank_" + a for a in order]
    gname = {"tammes": "Tammes", "equal_area": "Equal area", "large": "Large ball"}
    rows = sorted(mr.index, key=lambda c: (["tammes", "equal_area", "large"].index(conds[c]["group"]), c))
    lab = [f"{gname[conds[c]['group']]}: " + conds[c]["label"].replace("Tammes ", "").replace("R_H", "R_H") for
↳ c in rows]
    Z = np.vstack([mr.loc[rows, cols].values, mr[cols].mean().values[None, :]])
    fig, ax = plt.subplots(figsize=(6.4, 7.4))
    im = ax.imshow(Z, cmap="Blues_r", vmin=1, vmax=6, aspect="auto")
    for i in range(Z.shape[0]):
        for j in range(Z.shape[1]):
            ax.text(j, i, f"{Z[i, j]:.1f}", ha="center", va="center", fontsize=6.5, color="white" if Z[i, j] < 3
↳ .4 else "black", fontweight="bold" if i == Z.shape[0] - 1 else "normal")
    ax.set_xticks(range(len(order)))
    ax.set_xticklabels([M_NAME[a] for a in order], fontsize=7.5)
    ax.xaxis.tick_top()
    ax.set_yticks(range(Z.shape[0]))
    ax.set_yticklabels(lab + ["Mean over the 33 conditions"], fontsize=6.3)
    ax.axhline(Z.shape[0] - 1.5, color="black", lw=1.2)
    for k in (sum(conds[c]["group"] == "tammes" for c in rows) - 0.5, sum(conds[c]["group"] != "large" for c in
↳ rows) - 0.5):
        ax.axhline(k, color="white", lw=2.0)
    cb = fig.colorbar(im, ax=ax, fraction=0.035, pad=0.02)
    cb.set_label("Mean rank within seeds (1 = best)", fontsize=7.5)
    cb.ax.tick_params(labelsize=7)
    fig.subplots_adjust(left=0.40, right=0.93, top=0.94, bottom=0.02)
    return MF.save(fig, "conf_rank")


def fig_r3(best, dec):
    """Density deficit against boundary share for the 144 equal-area conditions of the final round, with the thr
↳ ee fitted lines."""
    fig, ax = plt.subplots(figsize=(6.0, 4.6))
    for sp in ("R2", "H2", "S2cap"):
```

```
        g = best[best.space == sp]
        ax.scatter(g.b, g.deficit, s=22, marker=SP_MARK[sp], edgecolor=SP_COL[sp], facecolor="none", linewidth=1.0, zorder=3, alpha=0.9)
    a1 = dec["A1"]
    xs = np.linspace(best.b.min(), best.b.max(), 50)
    for sp in ("R2", "H2", "S2cap"):
        a = a1["intercept"] + (a1["g_H"] if sp == "H2" else 0) + (a1["g_S"] if sp == "S2cap" else 0)
        ax.plot(xs, a + a1["beta_b"] * xs, color=SP_COL[sp], lw=1.6, zorder=2)
    ax.set_xlabel("Boundary share  b")
    ax.set_ylabel("Density deficit  δ")
    ax.grid(True)
    h = [Line2D([0], [0], marker=SP_MARK[s], color=SP_COL[s], markeredgecolor=SP_COL[s], markerfacecolor="none", ms=7, lw=1.6, label=SP_NAME[s]) for s in ("R2", "H2", "S2cap")]
    MF.legend_box(fig, h, loc="lower center", ncol=3, bbox_to_anchor=(0.55, 0.0))
    fig.subplots_adjust(left=0.12, right=0.98, top=0.97, bottom=0.25)
    return MF.save(fig, "conf_r3")
```

conf_section.py

Writes the Confirmatory Round section of the paper from the conf_* files.

```
"""conf_section.py -- writes the 'Confirmatory Round' section of the paper.  Every number is read from the conf_* files."""
import json
import os
import numpy as np
import pandas as pd
import confirm_design as C
import conf_figures as CF
from make_figures import FIG

M_AB = {"gd": "GD", "gdp": "GD+P", "sa": "SA", "ls": "CP", "lbfgsb": "L-BFGS-B", "bh": "BH"}
METHODS = ["gd", "gdp", "sa", "ls", "lbfgsb", "bh"]


class _Null:
    def __getattr__(self, k):
        return lambda *a, **kw: None


def build_section(d, FF, HD, Tb, Fg, f, pstr, DATA, CONF, part="main"):
    _d0, _HD0, _FF0 = d, HD, FF
```

```
    if part == "appendix":
        d, HD, FF = _Null(), (lambda *a, **k: None), (lambda *a, **k: None)
    dec = json.load(open(CONF + "/decisions.json"))
    tuned = json.load(open(CONF + "/tuned_params.json"))
    m = pd.read_csv(DATA + "/conf_summary_by_condition.csv")
    t = pd.read_csv(DATA + "/conf_deficit_boundary.csv")
    tm = pd.read_csv(DATA + "/conf_time_by_method.csv")
    cnt = pd.read_csv(CONF + "/count_summary.csv")
    hs = open(CONF + "/PREREGISTRATION.sha256").read().split()
    hash_pre, hash_time = hs[0], hs[-1]
    ths = open(CONF + "/TUNED_AND_ENV.sha256").read().split("\n")
    hash_tuned = [l.split()[0] for l in ths if l.endswith("tuned_params.json")][0]
    hw = json.load(open(CONF + "/hardware.json"))
    conds = {c["cid"]: c for c in C.conditions()}
    n_runs = dec["n_runs"]
    ref_types = dec["reference_types"]
    pk = os.path.exists(CONF + "/packomania_values.csv")
    c1, c2, c3, c4, c5, c6 = (dec[k] for k in ("C1", "C2", "C3", "C4", "C5", "C6"))
    ext = json.load(open(CONF + "/extra.json"))
    rdec = json.load(open(CONF + "/decisions_retuned.json"))
    rtun = json.load(open(CONF + "/tuned_params_retuned.json"))
    mini = pd.read_csv(CONF + "/mini_summary.csv")
    mdec = json.load(open(CONF + "/mini_decisions.json"))
    wm = pd.read_csv(DATA + "/conf_walls_by_method.csv")
    wc = pd.read_csv(DATA + "/conf_walls_by_condition.csv")
    ADD = open(CONF + "/PREREG_ADDENDA.sha256").read().split("\n")
    h_add = [l.split()[0] for l in ADD if l.endswith("ADDENDUM_1.md")][0]
    h_mini = [l.split()[0] for l in ADD if l.endswith("MINIROUND.md")][0]
    h_time2 = [l for l in ADD if l.startswith("hashed")][0].split(": ")[1]
    FU = open(CONF + "/FOLLOWUP.sha256").read().split("\n")
    h_fu, h_fu_time = FU[0].split()[0], FU[1].split(": ")[1]
    fdec = json.load(open(CONF + "/followup_decisions.json"))
    CR = open(CONF + "/CORRECTION.sha256").read().split("\n")
    h_cor, h_cor_time = CR[0].split()[0], CR[1].split(": ")[1]
    RT = open(CONF + "/FOLLOWUP_RUNTIMES.txt").read()
    fol = pd.read_csv(CONF + "/followup_summary.csv")
    h_ret = [l for l in open(CONF + "/RETUNED.sha256").read().split("\n") if l.endswith("tuned_params_retuned.js
↳ on")][0].split()[0]
    ex = dec["C1_exploratory_without_two_largest_hyperbolic"]

    HD("Confirmatory Round With Equal-Area Balls and Stronger Baselines", 2)
    d.body("The first study could not separate curvature from area, and its two best methods were not compared w
↳ ith stronger baselines. A second round was therefore designed to test a prediction, and its hypotheses, threshol
↳ ds, and decision rules were written and hashed before any of its evaluation seeds was run. "
           "This round is confirmatory only in that sense: it answers the questions written in its preregistrati
```

```
on, with fresh seeds, and it is separate from the first study, whose results are unchanged. All analyses outside the six preregistered decisions are exploratory and are labeled so.")
    HD("Prediction, Design, and Preregistration", 3)
    d.body(f"The prediction is that the density deficit of a packing follows the share of the ball&rsquo;s area near its boundary, so that most of the lower density of packings in hyperbolic balls is a boundary effect and not an effect of curvature. "
           "For <i>n</i> disks of radius <i>r</i> in a ball of radius <i>R</i> let the reference density <i>&rho;</i>* be an upper bound on the density of any packing of disks of radius <i>r</i> in the whole space (Fej&eacute;s T&oacute;th, 1943; B&ouml;r&ouml;czky, 1978), "
           "the density deficit be <i>&delta;</i> = 1 &minus; [<i>n</i> Area(<i>r</i>) / Area(<i>R</i>)] / <i>&rho;</i>*, and the boundary share be <i>b</i> = 1 &minus; Area(<i>R</i> &minus; <i>r</i>) / Area(<i>R</i>), the share of the ball&rsquo;s area within distance <i>r</i> of its boundary. The reference density is an upper bound for the whole space, not for a bounded ball, so <i>&delta;</i> includes the loss caused by the wall.".replace("&rsquo;", "’").replace("&eacute;", "é").replace("&oacute;", "ó").replace("&ouml;", "ö").replace("&rho;", "ρ").replace("&delta;", "δ").replace("&minus;", "−"))
    d.body(f"The preregistration (PREREGISTRATION.md in the data package) fixes the fresh evaluation seeds 101 to 120, three budget levels of 500, 1,500, and 3,000 sweeps in the main design, six methods, and {len(conds)} conditions: the Tammes problem for <i>N</i> = 9, 13, 14, 24, 30, 40, and 60; "
           "balls of equal area (3, 6, and 12) with 12 and 24 disks in the spherical cap, the Euclidean disk, and the hyperbolic disk; and large balls of hyperbolic radius 2, 3, 6, and 8 (areas 17.4, 57.0, 1,261.1, and 9,358.7) with 24, 36, 50, and 70 disks, paired with Euclidean disks of the same area. "
           "Equal area in all three spaces is possible only up to 4π, because the sphere has finite area, so the large balls compare only the hyperbolic and Euclidean planes. "
           f"Its SHA-256 hash, computed on {hash_time.replace('T', ' at ').replace('Z', ' UTC')} before the pilot tuning and before every evaluation run, is {hash_pre}. The tuned parameters (hash {hash_tuned}) were fixed before the evaluation run as well. "
           "A hash recorded by the author is not an independent time stamp; the record kept by the research adviser is described in Appendix B.")
    rows = []
    NAME = {"gd": ("Gradient descent", "<i>p</i><sub>max</sub>"), "gdp": ("GD with polishing", "polishing share"), "sa": ("Simulated annealing", "initial temperature"),
             "ls": ("Compression", "initial growth rate"), "lbfgsb": ("L-BFGS-B (SciPy)", "<i>p</i><sub>max</sub>"), "bh": ("Basin hopping (SciPy)", "step size")}
    g = tuned["grids"]
    for a in METHODS:
        rows.append([NAME[a][0], NAME[a][1], ", ".join(f"{v:g}" for v in g[a]), f"{tuned['tuned'][a]:g}"])
    FF("table", "cmeth", "Methods, Tuned Parameters, and Candidate Settings of the Confirmatory Round", ["Method", "Tuned parameter", "Candidate settings", "Selected"], rows,
       "Each method had one tuned parameter and six candidate settings, run on the same four pilot conditions (Tammes <i>N</i> = 11 and 33; hyperbolic and Euclidean balls of area 8 with 16 disks) and the same four pilot seeds (9101 to 9104) at 1,500 sweeps; the setting with the highest mean score relative to the best score seen in the pilot condition was kept. "
       "GD = gradient descent; GD+P = GD followed by a max-min polishing step; the polishing share is the fraction of the budget given to the polishing step; the step size and the initial temperature are multiples of the length scale √(area/<i>n</i>). Data: tuning_runs.csv, tuned_params.json.",
```

```
        aligns=["l", "l", "l", "c"], widths=[1.8, 1.5, 2.4, 0.8])
    edge = [NAME[a][0] for a in METHODS if tuned["tuned"][a] in (g[a][0], g[a][-1])]
    d.body(f"{Tb('cmeth')} lists the six methods, their tuned parameters, and the selected values. The two added baselines run the same energy as gradient descent. L-BFGS-B (SciPy; Virtanen et al., 2020; Byrd et al., 1995) minimizes the soft-minimum energy with its exponent raised in ten stages and restarts from a new random configuration, with the same random stream, when it stops by its own convergence test before the budget is used; basin hopping (Wales & Doye, 1997; <i>scipy.optimize.basinhopping</i>) perturbs the points and minimizes locally with L-BFGS-B at <i>p</i> = 60. "
           f"Every energy-and-gradient evaluation is charged one sweep, as in gradient descent. Equal tuning means equal effort (one parameter, six settings, the same pilot runs for every method), not an equal chance of finding the best possible setting: the selected value lies at the edge of its candidate range for {', '.join(edge)}, so those two methods may be tuned less well than the others. "
           f"The reported runs used {hw['cpu_model']} with {hw['logical_cpus']} logical processors, {hw['memory_gb']} GB of memory, {hw['os'].split('-with')[0]}, Python {hw['python']}, numpy {hw['numpy']}, scipy {hw['scipy']}, and pandas {hw['pandas']} (hardware.json), with two worker processes sharing the machine; wall-clock times are comparable only within this round. "
           "The hardware of the computer that produced the first study was not recorded in its data package, so its run times cannot be compared with these.")
    pkn = (f"For the ten Euclidean conditions with 12, 24, 36, 50, or 70 disks the reference is the Packomania value (circles in a circle; Specht, n.d.), entered after the first analysis of this round, which had used the best score found (the decisions were the same with both references; decisions_best_found_references.json), and" if pk else "Packomania values for the Euclidean conditions had not been entered at the time of this analysis, so those conditions too use the best score found in this round, and")
    d.body(f"Reference values: the proven optimum for <i>N</i> = 9, 13, 14, and 24 and the published best-known value (Sloane&rsquo;s tables) for <i>N</i> = 30, 40, and 60, seven conditions in all. {pkn} for the spherical cap and the hyperbolic disk, for which the author knows of no published table of best packings, the reference is the best score found by any method, budget, or seed of this round in the condition ({ref_types.get('best found in this round', 0)} conditions). ".replace("&rsquo;", "’") +
           "A relative score for a best-found reference measures the distance from the best score this study found, not from the optimum. No run was excluded; every run is in conf_runs.csv, and every score agreed with its recomputation from the final coordinates to within " + f"{dec['max_abs_score_vs_recomputed']:.0e}.")

    HD("Results of the Preregistered Decisions", 3)
    cm = dec["C3"]["median_d"]
    _r13 = pd.read_csv(DATA + "/conf_runs.csv")
    _r13 = _r13[_r13.condition_id == [k for k, v in conds.items() if v["group"] == "tammes" and v["n"] == 13][0]]
    n13_hit, n13_all = int(_r13["success_0.1pct"].sum()), len(_r13)
    _h = _r13[_r13["success_0.1pct"] == 1].groupby(["algorithm", "budget"]).size()
    n13_txt = ", ".join(f"{M_AB[a]} {v} at {b:,} sweeps" for (a, b), v in _h.items()) or "none"
    N13 = m[(m.label == "Tammes N = 13") & (m.budget == C.MAIN_BUDGET)].set_index("algorithm").mean_rel
    rows = [["C1: the density deficit follows the boundary share (26 ball conditions)", "Supported if Spearman <i>r</i><sub>s</sub> ≥ .70 and one-sided permutation <i>p</i> < .05; not supported if <i>r</i><sub>s</sub> < .30",
             f"<i>r</i><sub>s</sub> = {f(c1['spearman'], 2, lead=False)}, permutation {pstr(c1['perm_p_one_sided
```

```
    ↳ '])}", c1["decision"].capitalize()],
101             ["C2: curvature adds a deficit beyond the boundary share", "The 95% bootstrap interval of the hyperb
    ↳ olic term excludes 0 and the term is ≥ .02; no effect if the interval lies within ±.02",
102              f"Hyperbolic term {f(c2['gamma_hyperbolic'], 3)}, 95% interval [{f(c2['ci95_gamma_hyperbolic'][0],
    ↳ 3)}, {f(c2['ci95_gamma_hyperbolic'][1], 3)}]", c2["decision"].capitalize()],
103             ["C3: the GD versus SA ranking depends on the space (18 equal-area conditions)", "Kruskal–Wallis <i>
    ↳ p</i> < .05; no dependence if every 90% interval of a difference of medians lies within ±.01",
104              f"<i>χ</i><sup>2</sup>(2) = {f(c3['kruskal_chi2'], 2)}, {pstr(c3['kruskal_p'])}", c3["decision"].ca
    ↳ pitalize()],
105             ["C4: stronger baselines matter; GD+P is competitive (33 conditions)", "L-BFGS-B or BH beats both GD
    ↳  and SA in at least half of the conditions; GD+P ranks first or second in at least half",
106              f"Baselines beat GD and SA in {f(c4['share_new_baseline_beats_gd_and_sa'] * 100, 0)}% of conditions
    ↳ ; GD+P in the top two in {f(c4['share_gdp_rank_top2'] * 100, 0)}%", "Both supported" if c4["decision_baselines_m
    ↳ atter"] and c4["decision_gdp_competitive"] else "Not both supported"],
107             ["C5: the ranking of the methods is stable across budgets", "Median Kendall’s <i>W</i> over the thre
    ↳ e budget levels ≥ .80", f"Median <i>W</i> = {f(c5['median_W'], 2, lead=False)} (minimum {f(c5['min_W'], 2, lead=
    ↳ False)})", c5["decision"].capitalize()],
108             ["C6: some method recovers the proven optimum for <i>N</i> = 9, 13, 14, 24", "Best share of seeds wi
    ↳ th relative score ≥ .999 is ≥ .90 for all four",
109              "Best share: " + ", ".join(f"<i>N</i> = {k}: {f(v['best_share'], 2, lead=False)}" for k, v in c6["b
    ↳ y_N"].items()), c6["decision"].capitalize()]]
110     FF("table", "cdec", "Preregistered Decisions of the Confirmatory Round", ["Question", "Decision rule", "Resu
    ↳ lt", "Decision"], rows,
111        "Relative scores at 1,500 sweeps, 20 fresh seeds per condition. GD = gradient descent; GD+P = GD with a p
    ↳ olishing step; SA = simulated annealing; BH = basin hopping. Data: conf_runs.csv, conf_deficit_boundary.csv, con
    ↳ f_gd_minus_sa_equal_area.csv, conf_mean_ranks.csv, conf_kendall_w.csv, decisions.json.",
112        aligns=["l", "l", "l", "l"], widths=[1.7, 2.0, 1.8, 1.0], split=True)
113     ws = c1["within_space"]
114     d.body(f"{Tb('cdec')} gives the decisions. For C1 the correlation between the boundary share and the density
    ↳  deficit over the 26 ball conditions was {f(c1['spearman'], 2, lead=False)} ({pstr(c1['perm_p_one_sided'])}), so
    ↳  the prediction is supported ({Fg('cdef')}; {Tb('cdef')}). The correlation was positive within each space ({f(ws
    ↳ ['R2'], 2, lead=False)} in the Euclidean disk, {f(ws['H2'], 2, lead=False)} in the hyperbolic disk, and {f(ws['S
    ↳ 2cap'], 2, lead=False)} in the cap, not part of the decision). "
115            f"The two hyperbolic balls of radius 6 and 8 have the largest boundary shares and deficits; without t
    ↳ hem the correlation was {f(ex['spearman'], 2, lead=False)} for {ex['n_conditions']} conditions (exploratory). Th
    ↳ e final configuration of every run was saved in a rerun with identical seeds and settings (every score agreed wi
    ↳ th the original to within 1e-9), and the wall contact of all {ext['walls_overall']['runs']:,} ball runs is repor
    ↳ ted below (exploratory). The 26 conditions are not independent: the equal-area conditions come in triplets of on
    ↳ e area and one number of disks in the three spaces, and each large hyperbolic ball is paired with a Euclidean ba
    ↳ ll of the same area, so the permutation test, which treats the conditions as exchangeable, assumes more independ
    ↳ ence than the design has and its <i>p</i> value is optimistic. The boundary share is a function of the disk radi
    ↳ us that the method finds, so it is not independent of the outcome, and the result shows that the deficit moves w
    ↳ ith the boundary share, not that the boundary share causes it.")
116     d.body(f"For C2, once the boundary share is in the model, the hyperbolic term was {f(c2['gamma_hyperbolic'],
    ↳  3)} (95% bootstrap interval {f(c2['ci95_gamma_hyperbolic'][0], 3)} to {f(c2['ci95_gamma_hyperbolic'][1], 3)}) a
```

```
    nd the cap term was {f(c2['gamma_cap'], 3)}, so the decision is {c2['decision']}: with 26 conditions in which th
    e boundary share and the space are strongly related, a separate effect of curvature could be neither shown nor e
    xcluded at the preregistered thresholds. "
            f"For C3, the median difference between gradient descent and simulated annealing (relative score) was
     {f(cm['R2'], 4)} in the Euclidean disk, {f(cm['S2cap'], 4)} in the cap, and {f(cm['H2'], 4)} in the hyperbolic
    disk, a Kruskal–Wallis difference ({pstr(c3['kruskal_p'])}) of less than .01 in relative score: simulated anneal
    ing ahead of gradient descent in the Euclidean disk and the cap, and a tie in the hyperbolic disk. "
            "This differs from the exploratory equal-area comparison of the first study, which found no detectabl
    e dependence on the space with other areas, other numbers of disks, and other seeds; the difference is small in
    size in both, six conditions per space is a small sample, and the preregistered result is the one that counts as
     confirmatory. Both are reported.")
        mr = c4["mean_rank_by_method"]
        d.body(f"For C4, a stronger baseline mattered in every condition: L-BFGS-B or basin hopping had a higher mea
    n relative score than both gradient descent and simulated annealing in all 33 conditions, and the mean ranks ove
    r seeds and conditions were {f(mr['gdp'], 2)} for GD+P, {f(mr['lbfgsb'], 2)} for L-BFGS-B, {f(mr['bh'], 2)} for
    basin hopping, {f(mr['sa'], 2)} for simulated annealing, {f(mr['gd'], 2)} for gradient descent, and {f(mr['ls'],
     2)} for compression ({Tb('crel')}). "
            "The comparison of gradient descent with simulated annealing in the first study is therefore a compar
    ison of two methods that stronger methods beat at the same budget, and its conclusions about their order should
    be read with that in mind. "
            f"For C5, the ordering of the six methods was stable across the three budget levels (median Kendall’s
     <i>W</i> {f(c5['median_W'], 2, lead=False)}), but the best method at 1,500 sweeps was also the best at 3,000 sw
    eeps in only {f(c5['share_same_top_method_1500_vs_3000'] * 100, 0)}% of conditions ({Tb('cbud')}). "
            f"For C6, no method recovered the proven optimum in at least 90% of seeds for every one of <i>N</i> =
     9, 13, 14, and 24: the best share was {f(c6['by_N']['9']['best_share'], 2, lead=False)} at <i>N</i> = 9, {f(c6[
    'by_N']['13']['best_share'], 2, lead=False)} at <i>N</i> = 13, {f(c6['by_N']['14']['best_share'], 2, lead=False)
    } at <i>N</i> = 14, and {f(c6['by_N']['24']['best_share'], 2, lead=False)} at <i>N</i> = 24, so no method reache
    d the 0.1% tolerance at <i>N</i> = 13. Basin hopping came closest (mean relative score {f(N13['bh'], 3, lead=Fal
    se)}, against {f(N13['gdp'], 3, lead=False)} for GD+P and {f(N13['lbfgsb'], 3, lead=False)} for L-BFGS-B); acros
    s all three budget levels, {n13_hit} of the {n13_all} runs at <i>N</i> = 13 reached it ({n13_txt}).")

    FF("figure", "cdef", "Density Deficit Against Boundary Share in the Confirmatory Round", CF.fig_deficit(t, d
    ec), 4.2,
       "Each point is one of the 26 ball conditions, using the best mean radius over the six methods at 1,500 sw
    eeps. The deficit rises with the share of the ball’s area near its wall in all three spaces.",
       "Symbols: circles = Euclidean disk, squares = hyperbolic disk, triangles = spherical cap; filled = equal-
    area balls, open = large balls. Dashed lines = fitted lines with a common slope and a different intercept for ea
    ch space (hyperbolic and cap terms from the regression described in the text). <i>δ</i> = 1 − density / referenc
    e density; <i>b</i> = share of the ball’s area within one disk radius of the boundary. Both are dimensionless.",
       "Scatter plot of density deficit against boundary share for 26 packing conditions with three fitted lines
    .")
    # relative score table at main budget
    grpname = {("tammes", "S2"): "Tammes problem", ("equal_area", "S2cap"): "Equal area: cap", ("equal_area", "R
    2"): "Equal area: Euclidean", ("equal_area", "H2"): "Equal area: hyperbolic",
               ("large", "R2"): "Large ball: Euclidean", ("large", "H2"): "Large ball: hyperbolic"}
```

```
    mm = m[m.budget == C.MAIN_BUDGET].copy()
    mm["g"] = mm.condition_id.map(lambda c: grpname[(conds[c]["group"], conds[c]["kind"])])
    pv = mm.pivot_table(index="g", columns="algorithm", values="mean_rel").reindex(list(grpname.values()))[METHODS]
    rows, bold = [], set()
    for ri, (gname, r) in enumerate(pv.iterrows()):
        rows.append([gname] + [f(v, 3, lead=False) for v in r.values])
        bold.add((ri, int(np.argmax(r.values)) + 1))
    nconds = {gname: int(mm[mm.g == gname].condition_id.nunique()) for gname in pv.index}
    rows = [[f"{r[0]} ({nconds[r[0]]})"] + r[1:] for r in rows]
    rows.append(["Mean rank over all conditions"] + [f(mr[a], 2) for a in METHODS])
    bold.add((len(rows) - 1, int(np.argmin([mr[a] for a in METHODS])) + 1))
    FF("table", "crel", "Mean Relative Score of the Six Methods at 1,500 Sweeps", ["Condition group (conditions)"] + [M_AB[a] for a in METHODS], rows,
       "Mean over conditions of the mean relative score over 20 fresh seeds; boldface marks the best method in each row. For the Tammes problem the reference is the proven or published best-known value; for the other groups it is the best score found in this round, so values are relative to that score. "
       "The last row is the mean rank of the methods within seeds (1 = best). GD = gradient descent; GD+P = GD with a polishing step; SA = simulated annealing; CP = compression; L-BFGS-B and BH (basin hopping) are SciPy methods. Data: conf_summary_by_condition.csv, conf_mean_ranks.csv.",
       aligns=["l"] + ["c"] * 6, widths=[2.0, 0.7, 0.75, 0.7, 0.7, 0.9, 0.75], bold_cells=bold)

    # figures and deficit table
    ids = [cid for cid in t.condition_id]
    rows = [[r.label.replace("R_H", "<i>R</i><sub>H</sub>").replace("A = ", "<i>A</i> = ").replace(" n = ", " <i>n</i> = "), M_AB[r.best_method], f(r.r_best, 3), f(r.deficit, 3, lead=False), f(r.boundary_share, 3, lead=False)] for r in t.itertuples()]
    FF("table", "cdef", "Density Deficit and Boundary Share of the 26 Ball Conditions", ["Condition", "Best method", "<i>r</i>", "<i>δ</i>", "<i>b</i>"], rows,
       "<i>r</i> = highest mean disk radius over the six methods at 1,500 sweeps (units of the space); <i>δ</i> = density deficit and <i>b</i> = boundary share, defined in the text; <i>A</i> = area of the ball; <i>R</i><sub>H</sub> = hyperbolic radius of the large ball, whose area the Euclidean ball matches. Data: conf_deficit_boundary.csv.",
       aligns=["l", "c", "c", "c", "c"], widths=[2.8, 1.0, 0.9, 0.9, 0.9], split=True)
    HD("Budget Levels, Time, and the Count-Based Problem", 3)
    b = m.pivot_table(index="algorithm", columns="budget", values="mean_rel").reindex(METHODS)
    tmm = tm.pivot(index="algorithm", columns="budget", values="median_time_s").reindex(METHODS)
    sw = tm.pivot(index="algorithm", columns="budget", values="mean_sweeps_used").reindex(METHODS)
    rows = [[M_AB[a]] + [f(b.loc[a, B], 3, lead=False) for B in C.BUDGETS] + [f(tmm.loc[a, C.MAIN_BUDGET], 2), f(sw.loc[a, C.MAIN_BUDGET] / C.MAIN_BUDGET * 100, 0) + "%"] for a in METHODS]
    FF("figure", "cbud", "Mean Relative Score of the Six Methods at Three Budget Levels", CF.fig_budget(m, conds), 6.0,
       "The ordering of the methods is similar at the three budget levels; the gap between the methods is largest at 500 sweeps.",
       "Lines and markers: blue circles = GD, purple diamonds = GD+P, red squares = SA, green triangles = compre
```

```
   ↳ ssion, orange triangles = L-BFGS-B, brown crosses = basin hopping. Each point is the mean relative score over th
   ↳ e conditions of the panel and 20 seeds; relative score = score / reference value (dimensionless).",
162         "Three panels of line plots of mean relative score against budget for six methods.")
163     FF("table", "cbud", "Mean Relative Score at Three Budget Levels, With Time and Budget Used", ["Method", "500
   ↳ ", "1,500", "3,000", "Median time (s)", "Budget used"], rows,
164         "Mean relative score over the 33 conditions and 20 seeds at 500, 1,500, and 3,000 sweeps. Time = median w
   ↳ all-clock seconds per run at 1,500 sweeps with two worker processes on the machine described in the text; Budget
   ↳  used = mean share of the 1,500 sweeps actually used (GD+P and L-BFGS-B stop when their own convergence tests ar
   ↳ e met; the linear programs of GD+P are not charged in sweeps). Data: conf_summary_by_condition.csv, conf_time_by
   ↳ _method.csv.",
165         aligns=["l", "c", "c", "c", "c", "c"], widths=[1.4, 0.8, 0.8, 0.8, 1.5, 1.2])
166     d.body(f"Raising the budget from 500 to 3,000 sweeps raised the mean relative score of every method ({Tb('cb
   ↳ ud')}; {Fg('cbud')}), most for compression ({f(b.loc['ls', 500], 3, lead=False)} to {f(b.loc['ls', 3000], 3, lea
   ↳ d=False)}) and gradient descent ({f(b.loc['gd', 500], 3, lead=False)} to {f(b.loc['gd', 3000], 3, lead=False)})
   ↳ and least for GD+P ({f(b.loc['gdp', 500], 3, lead=False)} to {f(b.loc['gdp', 3000], 3, lead=False)}), so the met
   ↳ hods differ most at the smallest budget. "
167             f"Time per run at 1,500 sweeps was {f(tmm.loc['sa', C.MAIN_BUDGET] / tmm.loc['gd', C.MAIN_BUDGET], 1)
   ↳ } times as long for simulated annealing as for gradient descent, and GD+P ran for {f(tmm.loc['gdp', C.MAIN_BUDGE
   ↳ T], 2)} seconds against {f(tmm.loc['lbfgsb', C.MAIN_BUDGET], 2)} for L-BFGS-B, because its linear programs are n
   ↳ ot charged in sweeps; the sweep counts are equal by construction, the times are not.")
168     rows = [[{"S2cap": "Cap", "R2": "Euclidean", "H2": "Hyperbolic"}[r.space], f"{r.area:g}", str(int(r.n_ref)),
   ↳  str(int(r.K)), f(r.K_over_n_ref, 2, lead=False), f(r.share_at_K, 2, lead=False)] for r in cnt.itertuples()]
169     FF("table", "ccount", "Count-Based Version of the Disk Problem: Largest Number of Disks of Radius 0.30", ["S
   ↳ pace", "Area", "Reference count", "<i>K</i>", "<i>K</i> / reference", "Share at <i>K</i>"], rows,
170         "<i>K</i> = largest number of disks of radius 0.30 for which GD+P at 1,500 sweeps reached a radius of at
   ↳ least 0.30 in at least half of 20 fresh seeds (201 to 220), scanned downward from the reference count; reference
   ↳  count = floor(<i>ρ</i>* Area(<i>R</i>) / Area(0.30)), the number of disks that would fit if the ball were packe
   ↳ d at the reference density of the whole space (a heuristic ceiling, not a theorem for a bounded ball). "
171         "Share at <i>K</i> = share of seeds that reached the radius at <i>K</i>. Exploratory; no decision rule. D
   ↳ ata: count_runs.csv, count_summary.csv.",
172         aligns=["l", "c", "c", "c", "c", "c"], widths=[1.4, 0.8, 1.2, 0.8, 1.2, 1.1])
173     cc = cnt.pivot(index="area", columns="space", values="K")
174     d.body(f"The count-based problem fixes the disk radius at 0.30 and asks how many disks fit ({Tb('ccount')}).
   ↳  This is an original use of the same optimizers: the answer is an integer that can be compared across spaces wit
   ↳ hout choosing a value of <i>n</i> in advance. "
175             f"At area 6 the largest numbers found were {int(cc.loc[6.0, 'S2cap'])} in the cap, {int(cc.loc[6.0, '
   ↳ R2'])} in the Euclidean disk, and {int(cc.loc[6.0, 'H2'])} in the hyperbolic disk, and at area 12 they were {int
   ↳ (cc.loc[12.0, 'S2cap'])}, {int(cc.loc[12.0, 'R2'])}, and {int(cc.loc[12.0, 'H2'])}. "
176             "These are lower bounds on the true maximum, because a failure to find a packing does not prove that
   ↳ none exists, and they depend on the method and the budget.")
177     if part == "appendix":
178         d, HD, FF = _d0, _HD0, _FF0
179         HD("Additions After the First Analysis", 2)
180         d.body(f"Five additions were made after the results above were seen. They were written in a second prere
   ↳ gistration file (PREREGISTRATION_ADDENDUM_1.md, SHA-256 {h_add}), a mini-round file (PREREGISTRATION_MINIROUND.m
```

```
    d, SHA-256 {h_mini}), both hashed on {h_time2.replace('T', ' at ').replace('Z', ' UTC')} before any of the runs they describe, and a follow-up file (PREREGISTRATION_FOLLOWUP.md, SHA-256 {h_fu}), hashed on {h_fu_time.replace('T', ' at ').replace('Z', ' UTC')} before its runs. "
                "The addendum lists what was decided after the first round had been seen: the Packomania references, the extended tuning ranges, the saved configurations and the wall analysis, and a correction of the wording of the reference density (an upper bound on the density, not the density of a densest packing). The first preregistration file was not edited. "
                "A hash recorded by the author is not an independent time stamp; the author reports having emailed the four files to the research adviser on October 30, 2026, after the runs of the confirmatory round and the mini-round and before the runs of the follow-up. The adviser&rsquo;s email, not the author&rsquo;s hash, is the independent record, and it shows only that each file existed by that date; it cannot show that the first three files preceded their runs.".replace("&rsquo;", "’"))
        HD("Mini-Round With a Fixed Radius and a Matched Boundary Share", 2)
        mm1, mm2, mm3 = mdec["M1"], mdec["M2"], mdec["M3"]
        d.body(f"In the main round the boundary share depends on the radius each method finds, and the spaces differ in boundary share, so C1 is partly circular and C2 could not separate curvature from the boundary. The mini-round fixes the disk radius <i>r</i><sub>0</sub> (0.30 or 0.40) and the design boundary share <i>b</i><sub>0</sub> (0.40, 0.45, or 0.50) in advance, and chooses the radius of each ball so that <i>b</i><sub>0</sub> is the same in the Euclidean disk, the hyperbolic disk, and the spherical cap while the curvature differs ({Tb('cmini')}). "
                "The matched boundary share requires much larger hyperbolic balls than Euclidean ones for the same <i>r</i><sub>0</sub>. The Euclidean plane has no length scale, so a packing of disks of radius 0.40 in a disk of radius <i>R</i> is a scaled copy of a packing of disks of radius 0.30 in a disk of radius 0.75 <i>R</i>: with the same <i>b</i><sub>0</sub> the Euclidean conditions at <i>r</i><sub>0</sub> = 0.30 and 0.40 are the same problem and gave the same <i>K</i> ({Tb('cmini')}). The six Euclidean conditions are therefore three distinct ones, and the model fitted on them in M3 rests on three independent points. The hyperbolic disk and the cap have a length scale, so their conditions at different <i>r</i><sub>0</sub> are different problems. For each ball the number of disks <i>K</i> is the largest <i>n</i> for which GD+P at 1,500 sweeps reaches radius <i>r</i><sub>0</sub> in at least 10 of 20 fresh seeds (301 to 320), and the fixed-radius deficit is <i>δ</i><sub>0</sub> = 1 − <i>K</i> Area(<i>r</i><sub>0</sub>) / (Area(<i>R</i>) <i>ρ</i>*). Because <i>b</i><sub>0</sub> and <i>r</i><sub>0</sub> are set by design, <i>δ</i><sub>0</sub> is not circular in the sense of C1; it is still a lower bound on the true deficit, because a failure to find <i>K</i> + 1 disks does not prove that they do not fit.")
        sp_lab = {"R2": "Euclidean", "H2": "Hyperbolic", "S2cap": "Cap"}
        rows = [[f"{r.r0:.2f}", f"{r.b0:.2f}", sp_lab[r.space], f(r.R, 3), str(int(r.n_ref)), str(int(r.K)), f(r.deficit0, 3, lead=False)] for r in mini.sort_values(["r0", "b0", "space"], key=lambda c: c.map({"R2": 0, "H2": 1, "S2cap": 2}) if c.name == "space" else c).itertuples()]
        FF("table", "cmini", "Mini-Round Conditions With a Fixed Disk Radius and a Matched Boundary Share", ["<i>r</i><sub>0</sub>", "<i>b</i><sub>0</sub>", "Space", "<i>R</i>", "Reference count", "<i>K</i>", "<i>δ</i><sub>0</sub>"], rows,
            "<i>r</i><sub>0</sub> = disk radius, <i>b</i><sub>0</sub> = design boundary share, <i>R</i> = radius of the ball (units of the space), reference count = floor(<i>ρ</i>* Area(<i>R</i>) / Area(<i>r</i><sub>0</sub>)), <i>K</i> = largest number of disks found at radius <i>r</i><sub>0</sub> (GD+P, 1,500 sweeps, at least 10 of 20 seeds, seeds 301 to 320), <i>δ</i><sub>0</sub> = fixed-radius density deficit. Data: mini_design.csv, mini_scan.csv, mini_summary.csv, mini_decisions.json.",
            aligns=["c", "c", "l", "c", "c", "c", "c"], widths=[0.6, 0.6, 1.2, 0.9, 1.2, 0.6, 0.8], split=True)
```

```
        d.body(f"For M1 the Spearman correlation between <i>b</i><sub>0</sub> and <i>δ</i><sub>0</sub> over the 18 conditions was {f(mm1['spearman'], 2, lead=False)} (one-sided permutation {pstr(mm1['perm_p_one_sided'])}), which is {mm1['decision']} at the preregistered thresholds (supported at .50 or more with <i>p</i> < .05); the values of <i>K</i> are small integers, so <i>δ</i><sub>0</sub> changes in coarse steps, and 18 conditions give little power. "
                f"For M2, with the boundary share and the radius in the model, the hyperbolic term was {f(mm2['g_H'], 3)} (95% interval {f(mm2['g_H_ci'][0], 3)} to {f(mm2['g_H_ci'][1], 3)}) and the cap term {f(mm2['g_S'], 3)} ({f(mm2['g_S_ci'][0], 3)} to {f(mm2['g_S_ci'][1], 3)}), so the decision computed under the preregistered rule was “{mm2['decision']}”. That decision is withdrawn: it used 13 degrees of freedom, which counted the three duplicated Euclidean conditions as independent, and the sample was 15 conditions, not 18. The withdrawal was decided after the result was seen, so the analysis without the duplicates is post hoc; both analyses are given side by side in {Tb('cmini2')}. A correction note (CORRECTION_NOTE_1.md, SHA-256 {h_cor}, hashed {h_cor_time.replace('T', ' at ').replace('Z', ' UTC')}) records the withdrawal. With the duplicates removed, M2 is {fdec['mini_without_euclid_duplicates']['F2']['decision']} and M1 stays inconclusive, so the mini-round does not show that curvature has no effect at a matched boundary share; the follow-up below was designed to add power. "
                f"For M3 the boundary-loss model <i>δ</i><sub>0</sub> = <i>λ b</i><sub>0</sub> with <i>λ</i> = {f(mm3['lambda_euclid'], 2, lead=False)} fitted on the six Euclidean conditions predicted the twelve other conditions with a mean absolute error of {f(mm3['mean_abs_error'], 3, lead=False)}, so the model {mm3['decision']} at the preregistered thresholds (at most .05). "
                f"Applied after the fact to the 26 ball conditions of the main round (exploratory; <i>λ</i> = {f(ext['boundary_model_round1']['lambda_euclid'], 2, lead=False)} from the {ext['boundary_model_round1']['n_euclid']} Euclidean conditions), the mean absolute error for the other {ext['boundary_model_round1']['n_other']} conditions was {f(ext['boundary_model_round1']['mae_other'], 3, lead=False)} ({f(ext['boundary_model_round1']['mae_by_space']['H2'], 3, lead=False)} in the hyperbolic disk and {f(ext['boundary_model_round1']['mae_by_space']['S2cap'], 3, lead=False)} in the cap). "
                "The mini-round used one method at one budget, so it says what GD+P finds, not what is possible, and the three rules were not corrected for the six rules of the main round.")
        mw = fdec["mini_without_euclid_duplicates"]
        rows = [["M1: Spearman correlation of <i>b</i><sub>0</sub> and <i>δ</i><sub>0</sub> (one-sided permutation <i>p</i>)", f"{f(mm1['spearman'], 2, lead=False)} ({pstr(mm1['perm_p_one_sided'])}): {mm1['decision']}", f"{f(mw['F1']['spearman'], 2, lead=False)} ({pstr(mw['F1']['perm_p_one_sided'])}): {mw['F1']['decision']}"],
                ["M2: hyperbolic term (95% interval)", f"{f(mm2['g_H'], 3)} ({f(mm2['g_H_ci'][0], 3)} to {f(mm2['g_H_ci'][1], 3)}), 13 df: {mm2['decision']} (withdrawn)", f"{f(mw['F2']['terms']['g_H'], 3)} ({f(mw['F2']['g_H_ci'][0], 3)} to {f(mw['F2']['g_H_ci'][1], 3)}), {mw['F2']['df']} df: {mw['F2']['decision']}"],
                ["M2: cap term (95% interval)", f"{f(mm2['g_S'], 3)} ({f(mm2['g_S_ci'][0], 3)} to {f(mm2['g_S_ci'][1], 3)}): {mdec['M2']['decision_cap_secondary']} (withdrawn)", f"{f(mw['F2']['terms']['g_S'], 3)} ({f(mw['F2']['g_S_ci'][0], 3)} to {f(mw['F2']['g_S_ci'][1], 3)}): {mw['F2']['decision_cap_secondary']}"],
                ["M3: mean absolute error of the boundary-loss model", f"{f(mm3['mean_abs_error'], 3, lead=False)} (<i>λ</i> from 6 Euclidean conditions, 3 distinct): {mm3['decision']}", f"{f(mw['F3']['mean_abs_error'], 3, lead=False)} (<i>λ</i> from {mw['F3']['n_euclid']} Euclidean conditions): {mw['F3']['decision']}"]]
        FF("table", "cmini2", "Mini-Round Results as Preregistered and After Removing the Duplicated Euclidean Conditions", ["Rule", "As preregistered (18 conditions)", "Duplicates removed (15 conditions; post hoc)"], rows,
           "The left column applies the preregistered rules to all 18 conditions; the right column drops the three Euclidean conditions at <i>r</i><sub>0</sub> = 0.40, which duplicate those at 0.30, and was decided after the result was seen, so it is post hoc. The preregistered decision of M2 is withdrawn (see text). Data: mini_decisi
```

```
    ons.json, followup_decisions.json.",
204             aligns=["l", "l", "l"], widths=[2.0, 2.3, 2.3], split=True)
205         HD("Follow-Up With More Levels, a Third Radius, and a Second Method", 2)
206         p1, g1, g2, g3, mc = fdec["primary"], fdec["primary"]["F1"], fdec["primary"]["F2"], fdec["primary"]["F3"
    ], fdec["method_comparison"]
207         d.body(f"A follow-up with its own preregistration (SHA-256 {h_fu}, hashed {h_fu_time.replace('T', ' at '
    ).replace('Z', ' UTC')}, before its runs) removed the Euclidean duplicates and added power: five design boundary
     shares (0.35 to 0.55), three disk radii (0.20, 0.30, and 0.40) for the hyperbolic disk and the cap, one Euclide
    an condition for each boundary share because the Euclidean results do not depend on <i>r</i><sub>0</sub>, and fr
    esh seeds 401 to 420, in {len(fol)} conditions ({Tb('cfollow')}). "
208                 "The replacement boundary shares (0.35 and 0.55 were added), the third radius, and the second met
    hod were all written in that file before any of its runs. The runs started at 23:34 UTC on October 29, 2026, abo
    ut one minute after the hash was recorded, and the two-method runs ended at 23:43 UTC (file timestamps; FOLLOWUP
    _RUNTIMES.txt); the author had emailed the four files to the research adviser before the runs were started. F1 t
    o F3 are therefore preregistered by the author’s hash, and the adviser’s email is the only independent record of
     the date of the files; the time stamps of the runs come from the machine that ran them. "
209                 "The number of disks <i>K</i> of a condition was computed with two methods, GD+P and L-BFGS-B (re
    -tuned value 2,500), each scanned downward from the reference count, and the larger of the two is used; a lower
    bound from one method is weak, and a second method can only raise it. "
210                 f"For F1, the Spearman correlation between <i>b</i><sub>0</sub> and <i>δ</i><sub>0</sub> over the
     {g1['n']} conditions was {f(g1['spearman'], 2, lead=False)} ({pstr(g1['perm_p_one_sided'])}), so the prediction
     that the fixed-radius deficit follows the design boundary share is {g1['decision']} at the preregistered thresh
    olds; unlike in the main round, this test does not use a radius found by a method, so it is not circular in the
    way C1 is. "
211                 f"For F2, with <i>b</i><sub>0</sub> and the radius in the model, the hyperbolic term was {f(g2['t
    erms']['g_H'], 3)} (95% interval {f(g2['g_H_ci'][0], 3)} to {f(g2['g_H_ci'][1], 3)}) and the cap term {f(g2['ter
    ms']['g_S'], 3)} ({f(g2['g_S_ci'][0], 3)} to {f(g2['g_S_ci'][1], 3)}), so the decision is {g2['decision']}: both
     terms are positive and small, neither interval excludes 0, and neither interval lies within ±.05, so a curvatur
    e effect could be neither shown nor excluded. "
212                 f"For F3, the boundary-loss model <i>δ</i><sub>0</sub> = <i>λ b</i><sub>0</sub> with <i>λ</i> = {
    f(g3['lambda_euclid'], 2, lead=False)} fitted on the {g3['n_euclid']} Euclidean conditions predicted the other {
    g3['n_other']} with a mean absolute error of {f(g3['mean_abs_error'], 3, lead=False)} ({f(g3['mae_by_space']['H2
    '], 3, lead=False)} for the hyperbolic disk and {f(g3['mae_by_space']['S2cap'], 3, lead=False)} for the cap), so
     the model {g3['decision']} at the thresholds, with the cap error close to the limit of .05. "
213                 f"The second method added nothing to the first: L-BFGS-B found a larger <i>K</i> than GD+P in {mc
    ['lbfgsb_larger']} of {mc['n']} conditions, the same in {mc['equal']}, and a smaller one in {mc['lbfgsb_smaller'
    ]}, and on the distinct conditions of the mini-round (exploratory) it found a larger <i>K</i> in none. The two m
    ethods agree, which supports the values of <i>K</i> as what these optimizers find at this budget; it does not sh
    ow that more disks cannot fit.")
214         rows = [[sp_lab[r.space], f"{r.r0:.2f}", f"{r.b0:.2f}", f(r.R, 3), str(int(r.n_ref)), str(int(r.K_gdp)),
     str(int(r.K_lbfgsb)), str(int(r.K)), f(r.deficit0, 3, lead=False)] for r in fol.assign(o=fol.space.map({"R2": 0
    , "H2": 1, "S2cap": 2})).sort_values(["o", "r0", "b0"]).itertuples()]
215         FF("table", "cfollow", "Follow-Up Conditions and the Number of Disks Found by Two Methods", ["Space", "<
    i>r</i><sub>0</sub>", "<i>b</i><sub>0</sub>", "<i>R</i>", "Reference count", "<i>K</i> GD+P", "<i>K</i> L-BFGS-B
    ", "<i>K</i>", "<i>δ</i><sub>0</sub>"], rows,
```

```
            "<i>K</i> = larger of the two method values. Seeds 401 to 420, 1,500 sweeps, at least 10 of 20 seeds at radius <i>r</i><sub>0</sub>; other symbols as in the mini-round table. The Euclidean disk is run at <i>r</i><sub>0</sub> = 0.30 only because its results do not depend on <i>r</i><sub>0</sub>. Data: followup_design.csv, followup_scan.csv, followup_summary.csv, followup_decisions.json.",
            aligns=["l", "c", "c", "c", "c", "c", "c", "c", "c"], widths=[1.0, 0.5, 0.5, 0.7, 0.9, 0.7, 0.9, 0.5, 0.6], split=True)
        d.body("The boundary-loss model is a description, not an explanation. It has one parameter fitted on the Euclidean disk, it is not derived from a theory of how a wall disturbs a packing, and agreement of its predictions with the other two spaces shows that the deficit is similar at the same boundary share, not that the boundary causes the loss or that curvature does not matter. A different one-parameter family could fit as well. "
                "Also, because the number of disks is an integer that changes in steps, <i>δ</i><sub>0</sub> changes in steps of about .05 to .10 for the smallest balls, which limits what can be seen at a boundary share near 0.55.")
        return
    HD("Baselines Re-Tuned After the First Analysis", 3)
    rc = ext["retuned_mean_rel_change"]
    chg = {(r["algorithm"], r["budget"]): r["d"] for r in rc}
    sel = rtun["tuned"]
    d.body(f"Compression and L-BFGS-B had been selected at the upper edge of their candidate ranges (0.70 and 70 0). Their ranges were extended (compression: 1.0, 1.4, and 2.0 added; L-BFGS-B: 1,000, 1,500, and 2,500 added), the same pilot protocol was applied to the new settings, and the selected values, hashed on {open(CONF + '/RETUNED.sha256').read().split(chr(10))[2].split(': ')[1].replace('T', ' at ').replace('Z', ' UTC')} (SHA-256 {h_ret}) before the evaluation, were {sel['ls']:g} for compression, inside its extended range, and {sel['lbfgsb']:g} for L-BFGS-B, again the largest value tried. "
            f"Both methods were then run again on all 33 conditions, three budgets, and the same 20 seeds. At 1,500 sweeps the mean relative score changed by {f(chg[('ls', 1500)], 3)} for compression and {f(chg[('lbfgsb', 1500)], 3)} for L-BFGS-B (by {f(chg[('ls', 500)], 3)} and {f(chg[('lbfgsb', 500)], 3)} at 500 sweeps). The decisions C1 to C6 recomputed with the re-tuned methods were the same as the preregistered ones; the mean rank of compression stayed last ({f(rdec['C4']['mean_rank_by_method']['ls'], 2)}), that of L-BFGS-B moved from {f(mr['lbfgsb'], 2)} to {f(rdec['C4']['mean_rank_by_method']['lbfgsb'], 2)}, and GD+P kept the best mean rank ({f(rdec['C4']['mean_rank_by_method']['gdp'], 2)}). "
            "The tables above are the preregistered runs; the re-tuned runs are additional (conf_retuned_vs_original.csv, decisions_retuned.json). The two methods received nine candidate settings and the other four methods six, so the effort is unequal, and a gain from still larger values of the L-BFGS-B exponent cannot be excluded; the gain was small.")
    HD("Wall Contact in All Runs", 3)
    w15 = wm[wm.budget == C.MAIN_BUDGET].set_index("algorithm")
    big = [cid for cid, c in conds.items() if c["group"] == "large" and c["kind"] == "H2"]
    big_n = {cid: conds[cid]["label"] for cid in big}
    rows = [[M_AB[a], f(w15.loc[a, "mean_touch_share"], 2, lead=False), f(w15.loc[a, "ring_share"], 3, lead=False), f(w15.loc[a, "median_max_gap_over_r"], 2)] for a in METHODS]
    FF("table", "cwall", "Wall Contact of the Final Configurations of All Ball Runs at 1,500 Sweeps", ["Method", "Disks touching the wall", "Runs with all disks in the outer ring", "Median largest gap / <i>r</i>"], rows,
       "Each row covers the 26 ball conditions and 20 seeds (520 runs). A disk touches the wall if its gap to the wall limit <i>R</i> − <i>r</i> is at most 1% of the disk radius <i>r</i>; a run has all disks in the outer rin
```

```
g if every disk lies within one disk radius of the wall limit; the largest gap / <i>r</i> is the largest such ga
p in a run in units of <i>r</i>. Exploratory; no decision rule. Data: conf_walls_by_method.csv, conf_walls_by_co
ndition.csv.",
       aligns=["l", "c", "c", "c"], widths=[1.4, 1.5, 2.0, 1.6], split=True)
    h6 = wc[(wc.condition_id == big[2]) & (wc.budget == C.MAIN_BUDGET)].set_index("algorithm")
    h8 = wc[(wc.condition_id == big[3]) & (wc.budget == C.MAIN_BUDGET)].set_index("algorithm")
    wcb = wc.assign(nr=wc.ring_share * 20)
    r6, r8 = wcb[wcb.condition_id == big[2]].nr.sum(), wcb[wcb.condition_id == big[3]].nr.sum()
    n_each = len(wcb[wcb.condition_id == big[2]]) * 20
    r_out = wcb[~wcb.condition_id.isin(big[2:])].nr.sum()
    n_out = len(wcb[~wcb.condition_id.isin(big[2:])]) * 20
    d.body(f"In the main round, one run in the hyperbolic ball of radius 6 had shown all 50 centers in a ring al
ong the wall. With the configurations of all runs saved ({Tb('cwall')}), the ring is specific to the two largest
 hyperbolic balls (radii 6 and 8) and is not a general result: all {int(round(r6 + r8))} runs with every disk in
 the outer ring were in these two conditions ({int(round(r6))} of {n_each:,} runs in the ball of radius 6 and {i
nt(round(r8))} of {n_each:,} in the ball of radius 8, over all methods and budgets), and none of the other {n_ou
t:,} ball runs, in 24 conditions, had one ({int(round(r_out))} runs). "
           f"Contact with the wall is more common: over all {ext['walls_overall']['runs']:,} ball runs, {f(ext['
walls_overall']['mean_touch_share'] * 100, 0)}% of the disks touched the wall, and the share differed greatly by
 method (from {f(w15.mean_touch_share.min(), 2, lead=False)} for {M_AB[w15.mean_touch_share.idxmin()]} to {f(w15
.mean_touch_share.max(), 2, lead=False)} for {M_AB[w15.mean_touch_share.idxmax()]} at 1,500 sweeps), because the
 methods that polish or run to convergence push disks against the wall. "
           f"At 1,500 sweeps the share of runs with all disks in the outer ring reached {f(h6.ring_share.max(),
2, lead=False)} in the ball of radius 6 and {f(h8.ring_share.max(), 2, lead=False)} in the ball of radius 8 for
the method with the most such runs ({M_AB[h6.ring_share.idxmax()]} and {M_AB[h8.ring_share.idxmax()]}). "
           "A disk that touches the wall is a disk of the boundary layer, so a high share indicates that the pac
king is dominated by the wall; it does not show that the interior is poorly packed.")
    HD("Limits of the Confirmatory Round", 3)
    d.body(f"Seven limits apply. First, the conditions are few (33, with six per space in the equal-area compari
son) and the boundary share depends on the result, so C1 and C2 describe 26 points, not a large sample. Second,
{ref_types.get('best found in this round', 0)} of 33 references are best-found values from this round, so relati
ve scores for those conditions cannot show how far the methods are from the optimum. "
           "Third, tuning used one parameter per method and six settings; compression and L-BFGS-B were selected
 at the edge of their ranges, and after their ranges were extended (post hoc) compression was selected inside it
s range and L-BFGS-B again at the edge, so the effort is unequal (nine settings for these two methods, six for t
he others) and L-BFGS-B may still be undertuned. Fourth, one round was run, on one machine, with two processes s
haring it. Fifth, the preregistration is a self-recorded hash and is not independently time-stamped. "
           "Sixth, the large hyperbolic balls are dominated by the wall (see the wall contact below), so they te
st the boundary effect and say little about packings in the interior of a hyperbolic ball. "
           "Seventh, the number of disks <i>K</i> in the mini-round and the follow-up is a lower bound from one
or two optimizers at one budget, it is an integer that changes the deficit in steps, the boundary-loss model has
 one fitted parameter and no derivation, and the follow-up balls are small (at most 46 disks), so these analyses
 cannot settle a curvature effect smaller than about .05.")
```

confirm_configs.py

Reruns the confirmatory round with the same seeds and saves every final configuration (configs/cfg_<cid>.npz) and the wall metrics.

```
"""confirm_configs.py -- rerun of the confirmatory round (same seeds, parameters, budgets) that SAVES the final configuration of every run.

Writes out/conf/configs/cfg_<cid>.npz (array X[budget, seed, method, n, dim]) and out/conf/walls_<cid>.csv (wall metrics, ball conditions).
Every score of the rerun must equal the stored score of runs_<cid>.csv to within 1e-9 (assertion; the maximum difference is written to configs/check.csv).
Wall metrics (exploratory, ADDENDUM 1, section 3): for a run with disk radius r, g_i = R - rho_i - r is the gap between disk i and the wall limit;
disk i touches the wall if g_i <= 0.01 r; a wall-ring run has g_i <= r for every disk.
"""
import os
import sys
import time
import numpy as np
import pandas as pd
from multiprocessing import Pool
import newalgs
import confirm_design as C
from confirm_run import CONF, PARAM_KEY, load_params, problem_of

OUTD = CONF + "/configs"


def job(c):
    t0 = time.time()
    params = load_params()
    P = problem_of(c)
    old = pd.read_csv(f"{CONF}/runs_{c['cid']}.csv")
    algs = list(C.ALGS)
    dim = 2 if c["kind"] == "R2" else 3
    X = np.full((len(C.BUDGETS), len(C.EVAL_SEEDS), len(algs), c["n"], dim), np.nan)
    rows, maxdiff = [], 0.0
    for bi, B in enumerate(C.BUDGETS):
        for si, seed in enumerate(C.EVAL_SEEDS):
```

```
            X0 = P.initial(np.random.default_rng([seed, c["cid"], 0]))
            for ai, alg in enumerate(algs):
                res = newalgs.run_any(alg, P, X0, B * P.sweep, [seed, c["cid"], C.ALGS[alg], bi], **{PARAM_KEY[a
lg]: params[alg]})
                X[bi, si, ai] = res["X"]
                sc0 = float(old[(old.budget == B) & (old.seed == seed) & (old.algorithm == alg)].score.iloc[0])
                maxdiff = max(maxdiff, abs(res["score"] - sc0))
                if P.ball:
                    r = res["score"]
                    g = c["R"] - P.space.rho(res["X"]) - r
                    rows.append(dict(condition_id=c["cid"], algorithm=alg, budget=B, seed=seed, score=r,
                                     touch_share=float(np.mean(g <= 0.01 * r)), ring=int(np.all(g <= r)), max_ga
p_over_r=float(g.max() / r)))
    assert maxdiff <= 1e-9, (c["cid"], maxdiff)
    np.savez_compressed(f"{OUTD}/cfg_{c['cid']}.npz", X=X, budgets=C.BUDGETS, seeds=C.EVAL_SEEDS, algs=algs)
    if rows:
        pd.DataFrame(rows).to_csv(f"{CONF}/walls_{c['cid']}.csv", index=False)
    return c["cid"], maxdiff, time.time() - t0


def main():
    os.makedirs(OUTD, exist_ok=True)
    conds = [c for c in C.conditions() if not os.path.exists(f"{OUTD}/cfg_{c['cid']}.npz")]
    conds.sort(key=lambda c: -c["n"])
    res = []
    with Pool(2) as pool:
        for cid, md, dt in pool.imap_unordered(job, conds):
            print(f"done {cid} maxdiff {md:.1e} in {dt:.0f}s", flush=True)
    allc = []
    for c in C.conditions():
        allc.append(c["cid"])
    print("finished")


if __name__ == "__main__":
    main()
```

confirm_retune.py

Extended candidate grids for compression and L-BFGS-B on the pilot conditions (ADDENDUM 1).

```
"""confirm_retune.py -- extended candidate grids for compression (ls) and L-BFGS-B (ADDENDUM 1, section 4).
```

```
 2
 3  Runs only the NEW settings on the same pilot conditions and seeds as confirm_tune.py, combines them with the sto
 ↳ red original pilot runs,
 4  and selects by the same rule (mean score / best pilot score over all settings and methods). Writes out/conf/tuni
 ↳ ng_runs_extended.csv and tuned_params_retuned.json.
 5  """
 6  import json
 7  import numpy as np
 8  import pandas as pd
 9  from multiprocessing import Pool
10  import confirm_design as C
11  import confirm_tune as T
12  from confirm_run import CONF
13
14  NEW = {"ls": [1.0, 1.4, 2.0], "lbfgsb": [1000, 1500, 2500]}
15
16
17  def main():
18      tasks = [(a, v, ci, s) for a, g in NEW.items() for v in g for ci in range(len(T.PILOT)) for s in C.PILOT_SEE
 ↳ DS]
19      with Pool(2) as pool:
20          rows = pool.map(T.job, tasks, chunksize=2)
21      new = pd.DataFrame(rows)
22      old = pd.read_csv(CONF + "/tuning_runs.csv").drop(columns=["rel"])
23      d = pd.concat([old, new], ignore_index=True)
24      d["rel"] = d.score / d.groupby("pilot").score.transform("max")
25      d.to_csv(CONF + "/tuning_runs_extended.csv", index=False)
26      m = d.groupby(["algorithm", "value"]).rel.mean().reset_index()
27      tuned = json.load(open(CONF + "/tuned_params.json"))["tuned"]
28      for a in NEW:
29          g = m[m.algorithm == a]
30          tuned[a] = float(g.loc[g.rel.idxmax(), "value"])
31      grids = {a: sorted(m[m.algorithm == a].value.tolist()) for a in NEW}
32      json.dump(dict(tuned=tuned, extended_grids=grids, mean_relative_by_setting=m[m.algorithm.isin(NEW)].round(5)
 ↳ .to_dict("records"),
33                     at_upper_edge={a: bool(tuned[a] == max(grids[a])) for a in NEW}), open(CONF + "/tuned_params_
 ↳ retuned.json", "w"), indent=1)
34      print(m[m.algorithm.isin(NEW)].round(4).to_string(), tuned)
35
36
37  if __name__ == "__main__":
38      main()
```

confirm_run_retuned.py

Reruns compression and L-BFGS-B on the confirmatory conditions with the re-tuned parameters.

```
"""confirm_run_retuned.py -- compression (ls) and L-BFGS-B rerun with the re-tuned parameters (tuned_params_retuned.json, hashed before this script ran).
Same conditions, seeds, starting configurations, random streams, and budgets as confirm_run.py. Writes out/conf/retuned/runs_<cid>.csv.
"""
import os
import json
import time
import numpy as np
import pandas as pd
from multiprocessing import Pool
import newalgs
import confirm_design as C
from confirm_run import CONF, PARAM_KEY, problem_of

OUTD = CONF + "/retuned"
ALGS = ["ls", "lbfgsb"]


def job(c):
    t0 = time.time()
    par = json.load(open(CONF + "/tuned_params_retuned.json"))["tuned"]
    P = problem_of(c)
    rows = []
    for bi, B in enumerate(C.BUDGETS):
        for seed in C.EVAL_SEEDS:
            X0 = P.initial(np.random.default_rng([seed, c["cid"], 0]))
            for alg in ALGS:
                res = newalgs.run_any(alg, P, X0, B * P.sweep, [seed, c["cid"], C.ALGS[alg], bi], **{PARAM_KEY[alg]: par[alg]})
                rows.append(dict(condition_id=c["cid"], algorithm=alg, budget=B, seed=seed, score=res["score"], recomputed_score=res["final_check"],
                                 sweeps_used=res["pe"] / P.sweep, time_s=res["time"]))
    pd.DataFrame(rows).to_csv(f"{OUTD}/runs_{c['cid']}.csv", index=False)
    return c["cid"], time.time() - t0


if __name__ == "__main__":
```

```
    os.makedirs(OUTD, exist_ok=True)
    cs = sorted([c for c in C.conditions() if not os.path.exists(f"{OUTD}/runs_{c['cid']}.csv")], key=lambda c:
-c["n"])
    with Pool(2) as pool:
        for cid, dt in pool.imap_unordered(job, cs):
            print("done", cid, f"{dt:.0f}s", flush=True)
    print("finished")
```

confirm_extra.py

Post-hoc analyses of ADDENDUM 1 (re-tuned baselines, wall metrics) and the boundary-loss model.

```
"""confirm_extra.py -- post-hoc analyses of ADDENDUM 1 (retuned baselines, wall metrics) and the exploratory bou
ndary-loss model on the round-1 data.
Writes out/conf/decisions_retuned.json, out/data/conf_retuned_vs_original.csv, conf_walls_by_method.csv, conf_wa
lls_by_condition.csv, out/conf/extra.json.
"""
import glob
import io
import json
import os
import shutil
import contextlib
import numpy as np
import pandas as pd
import confirm_analyze as ca
import confirm_design as C

CONF, DATA = ca.CONF, ca.DATA
ALG = ["ls", "lbfgsb"]


def retuned():
    tmp = CONF + "_retuned"
    shutil.rmtree(tmp, ignore_errors=True)
    os.makedirs(tmp + "/_data")
    shutil.copy(CONF + "/packomania_values.csv", tmp)
    key = ["condition_id", "algorithm", "budget", "seed"]
    comp = []
    for f in sorted(glob.glob(CONF + "/runs_*.csv")):
        o = pd.read_csv(f)
        n = pd.read_csv(CONF + "/retuned/" + os.path.basename(f))
```

```
        m = o.merge(n[key + ["score", "recomputed_score", "sweeps_used", "time_s"]], on=key, how="left", suffixes=("", "_new"))
        m["old"] = o.merge(m[key], on=key, how="right").score.values
        for col in ["score", "recomputed_score", "sweeps_used", "time_s"]:
            m[col] = np.where(m.algorithm.isin(ALG), m[col + "_new"], m[col])
        m.drop(columns=[c for c in m.columns if c.endswith("_new")]).to_csv(f"{tmp}/{os.path.basename(f)}", index=False)
        comp.append(m[m.algorithm.isin(ALG)].assign(new=lambda x: x.score_new))
    ca.CONF, ca.DATA = tmp, tmp + "/_data"
    with contextlib.redirect_stdout(io.StringIO()):
        ca.main()
    ca.CONF, ca.DATA = CONF, DATA
    dec = json.load(open(tmp + "/decisions.json"))
    json.dump(dec, open(CONF + "/decisions_retuned.json", "w"), indent=1)
    c = pd.concat(comp)
    ref = pd.read_csv(DATA + "/conf_runs.csv").groupby("condition_id").reference_value.first()
    c["rel_old"], c["rel_new"] = c.old / c.condition_id.map(ref), c.new / c.condition_id.map(ref)
    s = c.groupby(["condition_id", "algorithm", "budget"])[["rel_old", "rel_new"]].mean().reset_index()
    s.to_csv(DATA + "/conf_retuned_vs_original.csv", index=False)
    shutil.rmtree(tmp)
    return dec, s


def walls():
    w = pd.concat([pd.read_csv(f) for f in sorted(glob.glob(CONF + "/walls_*.csv"))], ignore_index=True)
    conds = {c["cid"]: c for c in C.conditions()}
    w["space"] = w.condition_id.map(lambda c: conds[c]["kind"])
    w["group"] = w.condition_id.map(lambda c: conds[c]["group"])
    a = w.groupby(["budget", "algorithm"]).agg(runs=("ring", "size"), mean_touch_share=("touch_share", "mean"), ring_share=("ring", "mean"), median_max_gap_over_r=("max_gap_over_r", "median")).reset_index()
    a.to_csv(DATA + "/conf_walls_by_method.csv", index=False)
    b = w.groupby(["condition_id", "budget", "algorithm"]).agg(mean_touch_share=("touch_share", "mean"), ring_share=("ring", "mean")).reset_index()
    b.to_csv(DATA + "/conf_walls_by_condition.csv", index=False)
    return w, a, b


def boundary_model():
    t = pd.read_csv(DATA + "/conf_deficit_boundary.csv")
    e = t[t.space == "R2"]
    lam = float((e.boundary_share * e.deficit).sum() / (e.boundary_share ** 2).sum())
    o = t[t.space != "R2"]
    err = (o.deficit - lam * o.boundary_share).abs()
    return dict(lambda_euclid=lam, n_euclid=len(e), n_other=len(o), mae_other=float(err.mean()), mae_by_space={k: float(v) for k, v in err.groupby(o.space).mean().items()},
```

```
                mae_euclid_in_sample=float((e.deficit - lam * e.boundary_share).abs().mean()))


if __name__ == "__main__":
    dec, s = retuned()
    w, a, b = walls()
    bm = boundary_model()
    old = json.load(open(CONF + "/decisions.json"))
    ex = dict(boundary_model_round1=bm, retuned_vs_original={k: dec[k] for k in ("C1", "C2", "C3", "C4", "C5", "
↳ C6")},
              original={k: old[k] for k in ("C1", "C2", "C3", "C4", "C5", "C6")},
              retuned_mean_rel_change=s.assign(d=s.rel_new - s.rel_old).groupby(["algorithm", "budget"]).d.mean(
↳ ).round(4).reset_index().to_dict("records"),
              walls_overall=dict(runs=int(len(w)), mean_touch_share=float(w.touch_share.mean()), ring_share=floa
↳ t(w.ring.mean())))
    json.dump(ex, open(CONF + "/extra.json", "w"), indent=1, default=float)
    print(json.dumps(ex, indent=1, default=float)[:6000])
    print(a.round(3).to_string())
```

confirm_mini.py

Matched-boundary mini-round: design, runs, and rules M1 to M3.

```
"""confirm_mini.py -- matched-boundary mini-round (PREREGISTRATION_MINIROUND.md): design, runs, and the rules M1
↳  to M3.

Usage: python3 confirm_mini.py run | analyze
Writes out/conf/mini_design.csv, mini_scan.csv, mini_summary.csv, mini_decisions.json.
"""
import os
import sys
import json
import time
import numpy as np
import pandas as pd
from multiprocessing import Pool
from scipy import stats
from algorithms import Problem
import newalgs
import confirm_design as C
from confirm_run import CONF

```

```
SEEDS = list(range(301, 321))
R0S = [0.30, 0.40]
B0S = [0.40, 0.45, 0.50]
SPACES = [("R2", "Euclidean"), ("H2", "hyperbolic"), ("S2cap", "cap")]
BUDGET = 1500
POLISH = 0.25            # as tuned in the confirmatory round (fixed by the mini-round preregistration)
NEED = 10                # successes needed out of 20 seeds


def radius_for_b(kind, r, b0):
    hi = {"R2": 200.0, "H2": 40.0, "S2cap": np.pi - 1e-9}[kind]
    lo = r * (1 + 1e-9)
    f = lambda R: C.boundary_share(kind, R, r) - b0
    if f(lo) < 0 or f(hi) > 0:
        raise ValueError((kind, r, b0))
    for _ in range(200):
        mid = 0.5 * (lo + hi)
        lo, hi = (mid, hi) if f(mid) > 0 else (lo, mid)
    return 0.5 * (lo + hi)


def design():
    rows, cid = [], 2000
    for r0 in R0S:
        for b0 in B0S:
            for kind, lab in SPACES:
                R = radius_for_b(kind, r0, b0)
                A = C.area(kind, R)
                nref = int(np.floor(C.ref_density(kind, r0) * A / C.area(kind, r0)))
                rows.append(dict(cid=cid, space=kind, label=lab, r0=r0, b0=b0, R=R, area=A, n_ref=nref))
                cid += 1
    return pd.DataFrame(rows)


def one(c, n, seed):
    P = Problem("S2" if c["space"] == "S2cap" else c["space"], n, c["R"])
    X0 = P.initial(np.random.default_rng([seed, c["cid"], n]))
    res = newalgs.run_any("gdp", P, X0, BUDGET * P.sweep, [seed, c["cid"], n, 4], polish_share=POLISH)
    return res["score"]


def scan(c):
    t0 = time.time()
    rows, K = [], 0
    for n in range(int(c["n_ref"]), 0, -1):
```

```
        ok, fail = 0, 0
        for seed in SEEDS:
            sc = one(c, n, seed)
            hit = sc >= c["r0"] - 1e-9
            ok += hit
            fail += (not hit)
            rows.append(dict(cid=c["cid"], n=n, seed=seed, score=sc, success=int(hit)))
            if ok >= NEED or fail > len(SEEDS) - NEED:      # decided; remaining seeds cannot change the verdict
                break
        if ok >= NEED:
            K = n
            break
    return c["cid"], K, rows, time.time() - t0


def run():
    d = design()
    d.to_csv(CONF + "/mini_design.csv", index=False)
    cs = [r for _, r in d.sort_values("n_ref", ascending=False).iterrows()]
    out, scans = [], []
    with Pool(2) as pool:
        for cid, K, rows, dt in pool.imap_unordered(scan, cs):
            print(f"done {cid} K={K} in {dt:.0f}s", flush=True)
            out.append(dict(cid=cid, K=K))
            scans += rows
    pd.DataFrame(scans).sort_values(["cid", "n", "seed"]).to_csv(CONF + "/mini_scan.csv", index=False)
    s = d.merge(pd.DataFrame(out), on="cid")
    s["deficit0"] = [1 - r.K * C.area(r.space, r.r0) / (r.area * C.ref_density(r.space, r.r0)) for r in s.itertu
↳ ples()]
    s.to_csv(CONF + "/mini_summary.csv", index=False)


def perm_p(x, y, nperm=20000, seed=7):
    rng = np.random.default_rng(seed)
    rho = stats.spearmanr(x, y)[0]
    cnt = sum(stats.spearmanr(x, rng.permutation(y))[0] >= rho - 1e-12 for _ in range(nperm))
    return float(rho), float((cnt + 1) / (nperm + 1))


def analyze():
    s = pd.read_csv(CONF + "/mini_summary.csv")
    out = {}
    rho, p = perm_p(s.b0.values, s.deficit0.values)
    out["M1"] = dict(spearman=rho, perm_p_one_sided=p, decision="supported" if (rho >= .5 and p < .05) else ("no
↳ t supported" if rho < .2 else "inconclusive"))
```

```
    X = np.column_stack([np.ones(len(s)), s.b0, (s.space == "H2").astype(float), (s.space == "S2cap").astype(float), (s.r0 == 0.40).astype(float)])
    y = s.deficit0.values
    beta, *_ = np.linalg.lstsq(X, y, rcond=None)
    res = y - X @ beta
    df = len(y) - X.shape[1]
    cov = res @ res / df * np.linalg.inv(X.T @ X)
    tq = stats.t.ppf(0.975, df)
    se = np.sqrt(np.diag(cov))
    ci = lambda i: [float(beta[i] - tq * se[i]), float(beta[i] + tq * se[i])]
    def rule(i):
        lo, hi = ci(i)
        if (lo > 0 or hi < 0) and abs(beta[i]) >= .05:
            return "detectable curvature effect"
        if lo >= -.05 and hi <= .05:
            return "no detectable curvature effect"
        return "inconclusive"
    out["M2"] = dict(df=int(df), beta_b0=float(beta[1]), g_H=float(beta[2]), g_H_ci=ci(2), g_S=float(beta[3]), g_S_ci=ci(3), e_r0=float(beta[4]),
                     decision=rule(2), decision_cap_secondary=rule(3))
    e = s[s.space == "R2"]
    lam = float((e.b0 * e.deficit0).sum() / (e.b0 ** 2).sum())
    o = s[s.space != "R2"]
    err = float(np.mean(np.abs(o.deficit0 - lam * o.b0)))
    out["M3"] = dict(lambda_euclid=lam, mean_abs_error=err, decision="describes the other spaces" if err <= .05 else ("does not" if err > .10 else "inconclusive"))
    out["K_by_space"] = {k: [int(v) for v in g.K] for k, g in s.groupby("space")}
    json.dump(out, open(CONF + "/mini_decisions.json", "w"), indent=1)
    print(json.dumps(out, indent=1))


if __name__ == "__main__":
    {"run": run, "analyze": analyze}[sys.argv[1]]()
```

confirm_followup.py

Follow-up to the mini-round: two methods, rules F1 to F3, and L-BFGS-B on the mini-round conditions.

```
"""confirm_followup.py -- follow-up to the mini-round (PREREGISTRATION_FOLLOWUP.md): design, runs with two methods, and rules F1 to F3.

Usage: python3 confirm_followup.py run | run_mini_lbfgsb | analyze
```

```
Writes out/conf/followup_design.csv, followup_scan.csv, followup_summary.csv, followup_decisions.json,
mini_lbfgsb_scan.csv (exploratory: L-BFGS-B on the 18 mini-round conditions).
"""
import sys
import json
import time
import numpy as np
import pandas as pd
from multiprocessing import Pool
from scipy import stats
from algorithms import Problem
import newalgs
import confirm_design as C
import confirm_mini as M
from confirm_run import CONF

SEEDS = list(range(401, 421))
R0S = [0.20, 0.30, 0.40]
B0S = [0.35, 0.40, 0.45, 0.50, 0.55]
BUDGET = 1500
AID = {"gdp": 4, "lbfgsb": 5}


def design():
    rows, cid = [], 3000
    for kind, lab in (("R2", "Euclidean"), ("H2", "hyperbolic"), ("S2cap", "cap")):
        for r0 in R0S:
            if kind == "R2" and r0 != 0.30:
                continue
            for b0 in B0S:
                if b0 == 0.35 and r0 == 0.40:
                    continue
                R = M.radius_for_b(kind, r0, b0)
                A = C.area(kind, R)
                rows.append(dict(cid=cid, space=kind, label=lab, r0=r0, b0=b0, R=R, area=A, n_ref=int(np.floor(C
.ref_density(kind, r0) * A / C.area(kind, r0)))))
                cid += 1
    return pd.DataFrame(rows)


def one(c, n, seed, method, par):
    P = Problem("S2" if c["space"] == "S2cap" else c["space"], n, c["R"])
    X0 = P.initial(np.random.default_rng([seed, c["cid"], n]))
    kw = {"polish_share": 0.25} if method == "gdp" else {"pmax": par}
    return newalgs.run_any(method, P, X0, BUDGET * P.sweep, [seed, c["cid"], n, AID[method]], **kw)["score"]
```

```


def scan(job):
    c, method, seeds = job
    par = json.load(open(CONF + "/tuned_params_retuned.json"))["tuned"]["lbfgsb"]
    t0, rows, K = time.time(), [], 0
    for n in range(int(c["n_ref"]), 0, -1):
        ok = fail = 0
        for seed in seeds:
            sc = one(c, n, seed, method, par)
            hit = sc >= c["r0"] - 1e-9
            ok += hit
            fail += (not hit)
            rows.append(dict(cid=c["cid"], method=method, n=n, seed=seed, score=sc, success=int(hit)))
            if ok >= M.NEED or fail > len(seeds) - M.NEED:
                break
        if ok >= M.NEED:
            K = n
            break
    return c["cid"], method, K, rows, time.time() - t0


def _run(d, seeds, tag):
    jobs = [(r, m, seeds) for _, r in d.sort_values("n_ref", ascending=False).iterrows() for m in ("gdp", "lbfgs
b")]
    out, scans = [], []
    with Pool(2) as pool:
        for cid, m, K, rows, dt in pool.imap_unordered(scan, jobs):
            print(f"done {cid} {m} K={K} in {dt:.0f}s", flush=True)
            out.append(dict(cid=cid, method=m, K=K))
            scans += rows
    pd.DataFrame(scans).sort_values(["cid", "method", "n", "seed"]).to_csv(f"{CONF}/{tag}_scan.csv", index=False
)
    k = pd.DataFrame(out).pivot(index="cid", columns="method", values="K").reset_index().rename(columns={"gdp":
"K_gdp", "lbfgsb": "K_lbfgsb"})
    s = d.merge(k, on="cid")
    s["K"] = s[["K_gdp", "K_lbfgsb"]].max(axis=1)
    for col, kk in (("deficit0", "K"), ("deficit0_gdp", "K_gdp"), ("deficit0_lbfgsb", "K_lbfgsb")):
        s[col] = [1 - getattr(r, kk) * C.area(r.space, r.r0) / (r.area * C.ref_density(r.space, r.r0)) for r in
s.itertuples()]
    return s


def run():
    d = design()
```

```
    d.to_csv(CONF + "/followup_design.csv", index=False)
    _run(d, SEEDS, "followup").to_csv(CONF + "/followup_summary.csv", index=False)


def run_mini_lbfgsb():
    d = pd.read_csv(CONF + "/mini_design.csv")
    _run(d, M.SEEDS, "mini_lbfgsb").to_csv(CONF + "/mini_lbfgsb_summary.csv", index=False)


def rules(s, kcol="deficit0", r0dummies=True, euclid_baseline=True):
    """F1-F3 on a summary table with columns space, r0, b0 and a deficit column."""
    out = {}
    y = s[kcol].values
    rho, p = M.perm_p(s.b0.values, y)
    out["F1"] = dict(n=len(s), spearman=rho, perm_p_one_sided=p, decision="supported" if (rho >= .5 and p < .05)
 else ("not supported" if rho < .2 else "inconclusive"))
    cols = [np.ones(len(s)), s.b0, (s.space == "H2").astype(float), (s.space == "S2cap").astype(float)]
    names = ["a", "b0", "g_H", "g_S"]
    r0s = sorted(s.r0.unique())
    if len(r0s) > 1:
        base = 0.30 if 0.30 in r0s else r0s[0]
        for v in r0s:
            if v != base:
                cols.append((s.r0 == v).astype(float))
                names.append(f"e_{v:.2f}")
    X = np.column_stack(cols)
    beta = np.linalg.lstsq(X, y, rcond=None)[0]
    res = y - X @ beta
    df = len(y) - np.linalg.matrix_rank(X)
    se = np.sqrt(np.diag(res @ res / df * np.linalg.pinv(X.T @ X)))
    tq = stats.t.ppf(.975, df)
    def rule(i):
        lo, hi = beta[i] - tq * se[i], beta[i] + tq * se[i]
        if (lo > 0 or hi < 0) and abs(beta[i]) >= .05:
            return "detectable curvature effect"
        return "no detectable curvature effect" if (lo >= -.05 and hi <= .05) else "inconclusive"
    out["F2"] = dict(df=int(df), terms={n: float(b) for n, b in zip(names, beta)}, g_H_ci=[float(beta[2] - tq *
se[2]), float(beta[2] + tq * se[2])],
                     g_S_ci=[float(beta[3] - tq * se[3]), float(beta[3] + tq * se[3])], decision=rule(2), decisi
on_cap_secondary=rule(3))
    e = s[s.space == "R2"]
    lam = float((e.b0 * e[kcol]).sum() / (e.b0 ** 2).sum())
    o = s[s.space != "R2"]
    err = float(np.mean(np.abs(o[kcol] - lam * o.b0)))
    out["F3"] = dict(n_euclid=len(e), n_other=len(o), lambda_euclid=lam, mean_abs_error=err,
```

```
                      mae_by_space={k: float(v) for k, v in (o[kcol] - lam * o.b0).abs().groupby(o.space).mean().
↳ items()},
                      decision="describes the other spaces" if err <= .05 else ("does not" if err > .10 else "inc
↳ onclusive"))
    return out


def analyze():
    s = pd.read_csv(CONF + "/followup_summary.csv")
    out = dict(primary=rules(s, "deficit0"), gdp_only=rules(s, "deficit0_gdp"), lbfgsb_only=rules(s, "deficit0_l
↳ bfgsb"))
    out["method_comparison"] = dict(lbfgsb_larger=int((s.K_lbfgsb > s.K_gdp).sum()), equal=int((s.K_lbfgsb == s.
↳ K_gdp).sum()), lbfgsb_smaller=int((s.K_lbfgsb < s.K_gdp).sum()), n=len(s),
                                    mean_K_gdp=float(s.K_gdp.mean()), mean_K_lbfgsb=float(s.K_lbfgsb.mean()))
    m = pd.read_csv(CONF + "/mini_summary.csv")
    md = m[~((m.space == "R2") & (m.r0 == 0.40))]
    out["mini_without_euclid_duplicates"] = rules(md, "deficit0")
    try:
        ml = pd.read_csv(CONF + "/mini_lbfgsb_summary.csv")
        m2 = m.merge(ml[["cid", "K_lbfgsb"]], on="cid")
        m2["K2"] = m2[["K", "K_lbfgsb"]].max(axis=1)
        m2["deficit0_best"] = [1 - r.K2 * C.area(r.space, r.r0) / (r.area * C.ref_density(r.space, r.r0)) for r
↳ in m2.itertuples()]
        m2.to_csv(CONF + "/mini_both_methods_summary.csv", index=False)
        out["mini_both_methods"] = dict(rules=rules(m2[~((m2.space == "R2") & (m2.r0 == 0.40))], "deficit0_best"
↳ ), lbfgsb_larger=int((m2.K_lbfgsb > m2.K).sum()), equal=int((m2.K_lbfgsb == m2.K).sum()), lbfgsb_smaller=int((m2
↳ .K_lbfgsb < m2.K).sum()))
    except FileNotFoundError:
        pass
    # pooled, distinct conditions: mini (without duplicates, GD+P K) + follow-up (primary K); a condition appear
↳ ing in both (same space, r0, b0) is taken from the follow-up
    key = lambda d: set(zip(d.space, d.r0.round(3), d.b0.round(3)))
    extra = md[[k not in key(s) for k in zip(md.space, md.r0.round(3), md.b0.round(3))]]
    pool = pd.concat([s[["space", "r0", "b0", "deficit0"]], extra[["space", "r0", "b0", "deficit0"]]], ignore_in
↳ dex=True)
    out["pooled_distinct"] = rules(pool, "deficit0")
    json.dump(out, open(CONF + "/followup_decisions.json", "w"), indent=1, default=float)
    print(json.dumps({k: (v if k not in ("gdp_only", "lbfgsb_only") else "...") for k, v in out.items()}, indent
↳ =1, default=float))


if __name__ == "__main__":
    {"run": run, "run_mini_lbfgsb": run_mini_lbfgsb, "analyze": analyze}[sys.argv[1]]()
```

confirm_round3.py

Final preregistered round: equal-area balls (part A) and the predictive rule for the number of disks (part B).

```
"""confirm_round3.py -- final preregistered round (PREREGISTRATION_ROUND3.md).

Part A: equal-area balls (3 spaces x 8 areas x 6 numbers of disks = 144 conditions), GD+P and L-BFGS-B, seeds 501-520, 1,500 sweeps -> rule A1 (curvature, C2 settled), A2.
Part B: predictive rule for the number of disks K (count-based problem): K_pred = floor(n_ref (1 - 0.46 b0) + 0.5), 20 new conditions, GD+P, seeds 601-620 -> rule B1.
Usage: python3 confirm_round3.py runA | runB | analyze
Writes out/conf/r3a_runs.csv, r3a_conditions.csv, r3b_design.csv, r3b_scan.csv, r3b_summary.csv, round3_decisions.json.
"""
import sys
import json
import time
import os
import glob
import numpy as np
import pandas as pd
from multiprocessing import Pool
from algorithms import Problem
import newalgs
import confirm_design as C
import confirm_mini as M
import confirm_followup as F
from confirm_run import CONF

SEEDS_A = list(range(501, 521))
SEEDS_B = list(range(601, 621))
AREAS = [2.0, 3.0, 4.0, 5.0, 6.0, 8.0, 10.0, 12.0]
NS = [8, 12, 16, 20, 24, 32]
BUDGET = 1500
LAMBDA = 0.46
R0S_B = [0.25, 0.35]
B0S_B = [0.375, 0.425, 0.475, 0.525]
AID = {"gdp": 4, "lbfgsb": 5}


def conditions_A():
    out, cid = [], 4000
```

```
    for kind in ("R2", "H2", "S2cap"):
        for A in AREAS:
            for n in NS:
                out.append(dict(cid=cid, space=kind, area=A, n=n, R=C.radius_for_area(kind, A)))
                cid += 1
    return out


def job_A(c):
    t0 = time.time()
    par = json.load(open(CONF + "/tuned_params_retuned.json"))["tuned"]["lbfgsb"]
    P = Problem("S2" if c["space"] == "S2cap" else c["space"], c["n"], c["R"])
    rows = []
    for seed in SEEDS_A:
        X0 = P.initial(np.random.default_rng([seed, c["cid"], 0]))
        for m in ("gdp", "lbfgsb"):
            kw = {"polish_share": 0.25} if m == "gdp" else {"pmax": par}
            res = newalgs.run_any(m, P, X0, BUDGET * P.sweep, [seed, c["cid"], AID[m], 0], **kw)
            rows.append(dict(cid=c["cid"], space=c["space"], area=c["area"], n=c["n"], R=c["R"], method=m, seed=
↳ seed, score=res["score"], recomputed=res["final_check"], time_s=res["time"]))
    pd.DataFrame(rows).to_csv(f"{CONF}/r3a_runs_{c['cid']}.csv", index=False)
    return c["cid"], time.time() - t0


def runA():
    cs = [c for c in conditions_A() if not os.path.exists(f"{CONF}/r3a_runs_{c['cid']}.csv")]
    cs.sort(key=lambda c: -c["n"])
    with Pool(2) as pool:
        for cid, dt in pool.imap_unordered(job_A, cs):
            print("A done", cid, f"{dt:.0f}s", flush=True)


def design_B():
    rows, cid = [], 5000
    for kind in ("R2", "H2", "S2cap"):
        for r0 in ([0.30] if kind == "R2" else R0S_B):
            for b0 in B0S_B:
                R = M.radius_for_b(kind, r0, b0)
                A = C.area(kind, R)
                rows.append(dict(cid=cid, space=kind, r0=r0, b0=b0, R=R, area=A, n_ref=int(np.floor(C.ref_densit
↳ y(kind, r0) * A / C.area(kind, r0)))))
                cid += 1
    return pd.DataFrame(rows)


```

```
def runB():
    d = design_B()
    d.to_csv(CONF + "/r3b_design.csv", index=False)
    jobs = [(r, "gdp", SEEDS_B) for _, r in d.sort_values("n_ref", ascending=False).iterrows()]
    scans, out = [], []
    with Pool(2) as pool:
        for cid, m, K, rows, dt in pool.imap_unordered(F.scan, jobs):
            print(f"B done {cid} K={K} in {dt:.0f}s", flush=True)
            out.append(dict(cid=cid, K=K))
            scans += rows
    pd.DataFrame(scans).sort_values(["cid", "n", "seed"]).to_csv(CONF + "/r3b_scan.csv", index=False)
    s = d.merge(pd.DataFrame(out), on="cid")
    s["K_pred"] = np.floor(s.n_ref * (1 - LAMBDA * s.b0) + 0.5).astype(int)
    s["deficit0"] = [1 - r.K * C.area(r.space, r.r0) / (r.area * C.ref_density(r.space, r.r0)) for r in s.itertuples()]
    s.to_csv(CONF + "/r3b_summary.csv", index=False)


def analyze():
    rng = np.random.default_rng(20261030)
    d = pd.concat([pd.read_csv(f) for f in sorted(glob.glob(CONF + "/r3a_runs_*.csv"))], ignore_index=True)
    d.to_csv(CONF + "/r3a_runs.csv", index=False)
    cond = d.groupby(["cid", "space", "area", "n", "R", "method"]).score.mean().reset_index()
    best = cond.loc[cond.groupby("cid").score.idxmax()].rename(columns={"score": "r_best", "method": "best_method"})
    best["deficit"] = [C.deficit(r.space, r.R, r.n, r.r_best) for r in best.itertuples()]
    best["b"] = [C.boundary_share(r.space, r.R, r.r_best) for r in best.itertuples()]
    best["cell"] = best.area.astype(str) + "_" + best.n.astype(str)
    n_excluded = int(best.deficit.isna().sum())      # conditions whose reference density is undefined (none expected)
    best = best[best.deficit.notna()].copy()
    best.to_csv(CONF + "/r3a_conditions.csv", index=False)

    def fit(t):
        X = np.c_[np.ones(len(t)), t.b, (t.space == "H2"), (t.space == "S2cap")].astype(float)
        return np.linalg.lstsq(X, t.deficit.values, rcond=None)[0], X
    beta, X = fit(best)
    res = best.deficit.values - X @ beta
    sig = float(np.sqrt(res @ res / (len(best) - 4)))
    cells = best.cell.unique()
    groups = {c: best[best.cell == c] for c in cells}
    bs = []
    for _ in range(10000):
        pick = rng.choice(cells, len(cells))
        t = pd.concat([groups[c] for c in pick], ignore_index=True)
```

```
        try:
            bs.append(fit(t)[0])
        except np.linalg.LinAlgError:
            pass
    bs = np.array(bs)
    lo_h, hi_h = np.percentile(bs[:, 2], [2.5, 97.5])
    lo_s, hi_s = np.percentile(bs[:, 3], [2.5, 97.5])
    rule = lambda g, lo, hi: "curvature adds a detectable deficit" if (lo > 0 and g >= 0.02) else ("no detectable curvature effect" if (lo >= -0.02 and hi <= 0.02) else "inconclusive")
    out = dict(A1=dict(n_excluded=n_excluded, n_conditions=len(best), n_cells=len(cells), intercept=float(beta[0]), beta_b=float(beta[1]), g_H=float(beta[2]), g_H_ci=[float(lo_h), float(hi_h)], g_S=float(beta[3]), g_S_ci=[float(lo_s), float(hi_s)],
                       residual_sd=sig, decision=rule(beta[2], lo_h, hi_h), decision_cap_secondary=rule(beta[3], lo_s, hi_s)))
    # A2: paired differences at the same area and n
    pv = best.pivot_table(index=["area", "n"], columns="space", values="deficit")
    dh, ds = (pv.H2 - pv.R2).dropna(), (pv.S2cap - pv.R2).dropna()
    def cell_boot(x):
        v = x.values
        m = [np.mean(rng.choice(v, len(v))) for _ in range(10000)]
        return float(v.mean()), [float(np.percentile(m, 2.5)), float(np.percentile(m, 97.5))]
    out["A2"] = dict(hyperbolic_minus_euclid=dict(zip(("mean", "ci95"), cell_boot(dh))), cap_minus_euclid=dict(zip(("mean", "ci95"), cell_boot(ds))), n_cells_H=len(dh), n_cells_S=len(ds))
    out["A_methods"] = dict(best_method_counts=best.best_method.value_counts().to_dict(), max_abs_score_vs_recomputed=float((d.score - d.recomputed).abs().max()), n_runs=int(len(d)))
    # B
    s = pd.read_csv(CONF + "/r3b_summary.csv")
    e = s.K - s.K_pred
    ab = e.abs()
    within1, mae, rmae = float((ab <= 1).mean()), float(ab.mean()), float((ab / s.K.clip(lower=1)).mean())
    bias = {k: float(v) for k, v in e.groupby(s.space).mean().items()}
    ok = within1 >= 0.80 and mae <= 1.0 and rmae <= 0.10 and all(abs(v) <= 1.0 for v in bias.values())
    bad = within1 < 0.60 or mae > 1.5
    out["B1"] = dict(n_conditions=len(s), share_within_1=within1, share_exact=float((e == 0).mean()), mean_abs_error_disks=mae, mean_rel_error=rmae, mean_signed_error_by_space=bias,
                     decision="supported" if ok else ("not supported" if bad else "inconclusive"))
    json.dump(out, open(CONF + "/round3_decisions.json", "w"), indent=1, default=float)
    print(json.dumps(out, indent=1, default=float))


if __name__ == "__main__":
    {"runA": runA, "runB": runB, "analyze": analyze}[sys.argv[1]]()
```

r3_section.py

Writes the Final Round section of the paper from the round-3 files.

```
"""r3_section.py -- the 'Final Round' section of the paper.  Every number is read from the round3_* and r3* files."""
import json
import pandas as pd
import confirm_design as C
import conf_figures as CF


def build_section(d, FF, HD, Tb, Fg, f, pstr, DATA, CONF):
    dec = json.load(open(CONF + "/round3_decisions.json"))
    a1, a2, b1 = dec["A1"], dec["A2"], dec["B1"]
    best = pd.read_csv(CONF + "/r3a_conditions.csv")
    sb = pd.read_csv(CONF + "/r3b_summary.csv")
    r3 = open(CONF + "/ROUND3.sha256").read().split("\n")
    h_r3, t_r3 = r3[0].split()[0], r3[1].split(": ")[1]
    t0 = open(CONF + "/ROUND3_START.txt").read().strip()
    t1 = open(CONF + "/ROUND3_END.txt").read().strip()
    pl = open(CONF + "/PLAN_FILES.sha256").read().split("\n")
    u = lambda t: t.replace("T", " ")[:16] + " UTC"
    sp_lab = {"R2": "Euclidean", "H2": "Hyperbolic", "S2cap": "Cap"}

    HD("Final Round: An Equal-Area Test of Curvature and a Rule for the Number of Disks", 2)
    d.body(f"Three earlier tests of curvature, the confirmatory round, the mini-round, and the follow-up (Appendix B), were inconclusive. The final round was designed to answer that one question with many more equal-area conditions, and to test one original rule for the number of disks. "
           f"Its hypotheses, decision rules, seeds, and analysis code were written in PREREGISTRATION_ROUND3.md (SHA-256 {h_r3}, recorded by the author on {u(t_r3)}), and the file, the Research Plan, and the earlier preregistration files were emailed to the research adviser before any run was started; the author reports this, and the adviser&rsquo;s email is the independent record of the date. "
           f"The runs began at {u(t0)} and ended at {u(t1)} (ROUND3_START.txt and ROUND3_END.txt). The Research Plan, finished before this paper, states the rules, and the analysis below follows them.".replace("&rsquo;", "’"))
    HD("Design and Preregistered Rules", 3)
    d.body(f"Part A used three spaces, eight ball areas (2, 3, 4, 5, 6, 8, 10, and 12), and six numbers of disks (8, 12, 16, 20, 24, and 32), {a1['n_conditions'] + a1['n_excluded']} conditions in {a1['n_cells']} cells of equal area and equal <i>n</i>, with polished gradient descent and L-BFGS-B at 1,500 sweeps and fresh seeds 501 to 520 ({dec['A_methods']['n_runs']:,} runs). "
           "For each condition the radius is the larger of the two methods&rsquo; mean radii, the density defici
```

```
t <i>δ</i> and the boundary share <i>b</i> are computed from it, and rule A1 fits <i>δ</i> = <i>a</i> + <i>β b</i> + <i>g</i><sub>H</sub> [hyperbolic] + <i>g</i><sub>S</sub> [cap]. The interval of each term comes from resampling the cells, not the conditions, because the three balls of a cell are paired. "
            "A curvature effect is detectable if the interval of <i>g</i><sub>H</sub> excludes 0 and <i>g</i><sub>H</sub> is at least .02, and absent if the interval lies within ±.02. "
            "Part B used the rule <i>K</i><sub>pred</sub> = floor(<i>n</i><sub>ref</sub>(1 − 0.46 <i>b</i><sub>0</sub>) + 0.5), where <i>n</i><sub>ref</sub> is the number of disks of radius <i>r</i><sub>0</sub> that would fit in the ball at the upper bound on the density, <i>b</i><sub>0</sub> is the boundary share at <i>r</i><sub>0</sub>, and 0.46 is the loss coefficient fitted in the follow-up and not refitted, for 20 new conditions and seeds 601 to 620.".replace("&rsquo;", "’"))
    HD("Result for Curvature", 3)
    rows = [["Hyperbolic term, <i>g</i><sub>H</sub> (A1)", f(a1["g_H"], 3), f"{f(a1['g_H_ci'][0], 3)} to {f(a1['g_H_ci'][1], 3)}", a1["decision"].capitalize()],
            ["Cap term, <i>g</i><sub>S</sub> (A1, secondary)", f(a1["g_S"], 3), f"{f(a1['g_S_ci'][0], 3)} to {f(a1['g_S_ci'][1], 3)}", a1["decision_cap_secondary"].capitalize()],
            ["Boundary-share slope, <i>β</i>", f(a1["beta_b"], 3), "not part of the rule", "—"],
            ["Hyperbolic − Euclidean at equal area and <i>n</i> (A2)", f(a2["hyperbolic_minus_euclid"]["mean"], 3), f"{f(a2['hyperbolic_minus_euclid']['ci95'][0], 3)} to {f(a2['hyperbolic_minus_euclid']['ci95'][1], 3)}", "Descriptive"],
            ["Cap − Euclidean at equal area and <i>n</i> (A2)", f(a2["cap_minus_euclid"]["mean"], 3), f"{f(a2['cap_minus_euclid']['ci95'][0], 3)} to {f(a2['cap_minus_euclid']['ci95'][1], 3)}", "Descriptive"]]
    FF("table", "r3a", "Curvature Terms in the Final Round at Equal Area", ["Quantity", "Estimate", "95% interval", "Decision"], rows,
       f"{a1['n_conditions']} conditions in {a1['n_cells']} cells; intervals are percentile intervals of 10,000 resamples of the cells (seed 20261030). <i>δ</i> = density deficit; <i>b</i> = boundary share; the decision rule is in the text. Data: r3a_runs.csv, r3a_conditions.csv, round3_decisions.json.",
       aligns=["l", "c", "c", "l"], widths=[2.6, 0.9, 1.6, 1.4])
    FF("figure", "r3def", "Density Deficit Against Boundary Share in the Final Round", CF.fig_r3(best, dec), 4.4,
       f"Each point is one of the {a1['n_conditions']} equal-area conditions, using the larger mean radius of the two methods at 1,500 sweeps. The three fitted lines have a common slope and differ only in their intercepts.",
       "Symbols and lines: circles and orange line = Euclidean disk, squares and purple line = hyperbolic disk, triangles and blue line = spherical cap. <i>δ</i> = 1 − density / upper bound on the density for disks of the radius found; <i>b</i> = share of the ball&rsquo;s area within one disk radius of its wall (both dimensionless).".replace("&rsquo;", "’"),
       "Scatter plot of density deficit against boundary share for 144 packing conditions with three fitted lines.")
    verdict = {"curvature adds a detectable deficit": "curvature adds a detectable deficit: the hyperbolic disk has a larger density deficit than the Euclidean disk at the same boundary share, by the amount of the hyperbolic term",
               "no detectable curvature effect": "there is no detectable curvature effect: the whole interval lies within ±.02, so any difference between the spaces at the same boundary share is smaller than .02 of the upper bound",
               "inconclusive": "the result is again inconclusive"}[a1["decision"]]
    d.body(f"The preregistered answer is that {verdict}. The hyperbolic term was {f(a1['g_H'], 3)} (95% interval
```

```
 {f(a1['g_H_ci'][0], 3)} to {f(a1['g_H_ci'][1], 3)}) and the cap term {f(a1['g_S'], 3)} ({f(a1['g_S_ci'][0], 3)}
 to {f(a1['g_S_ci'][1], 3)}; {a1['decision_cap_secondary']}), with {a1['n_conditions']} conditions and a residua
l standard deviation of {f(a1['residual_sd'], 3, lead=False)}; the plan expected a half-width of about .03 for a
 residual of .058, and the half-width found for the hyperbolic term was {f((a1['g_H_ci'][1] - a1['g_H_ci'][0]) /
 2, 3, lead=False)} ({Tb('r3a')}; {Fg('r3def')}). "
            f"The decision means that no effect as large as .02 was found, not that the effect is zero: both inte
rvals exclude 0, the hyperbolic term is positive and the cap term negative, and both are about 1% of the bound,
so small effects of opposite sign remain possible and are not explained here. The slope on the boundary share wa
s {f(a1['beta_b'], 2)}: in all three spaces the deficit rose with the boundary share. At equal area and equal nu
mber of disks the hyperbolic disk differed from the Euclidean disk by {f(a2['hyperbolic_minus_euclid']['mean'],
3)} and the cap by {f(a2['cap_minus_euclid']['mean'], 3)} in <i>δ</i>, which include the effect of the different
 boundary share and are descriptive. "
            "For each condition the larger mean radius of polished gradient descent and L-BFGS-B was used, so the
 conclusion concerns what these two optimizers find at 1,500 sweeps, not every possible packing.")
    HD("Result for the Rule for the Number of Disks", 3)
    rows = [[sp_lab[r.space], f"{r.r0:.2f}", f"{r.b0:.3f}", str(int(r.n_ref)), str(int(r.K)), str(int(r.K_pred))
, f"{int(r.K - r.K_pred):+d}" if r.K != r.K_pred else "0"] for r in sb.assign(o=sb.space.map({"R2": 0, "H2": 1,
"S2cap": 2})).sort_values(["o", "r0", "b0"]).itertuples()]
    FF("table", "r3b", "Predicted and Found Number of Disks in the Final Round", ["Space", "<i>r</i><sub>0</sub>
", "<i>b</i><sub>0</sub>", "Reference count", "<i>K</i> found", "<i>K</i> predicted", "Error"], rows,
        "<i>K</i> found = largest number of disks of radius <i>r</i><sub>0</sub> reached by polished gradient des
cent in at least 10 of 20 fresh seeds (601 to 620); <i>K</i> predicted = floor(<i>n</i><sub>ref</sub>(1 − 0.46 <
i>b</i><sub>0</sub>) + 0.5); error = found − predicted. Data: r3b_design.csv, r3b_scan.csv, r3b_summary.csv.",
        aligns=["l", "c", "c", "c", "c", "c", "c"], widths=[1.2, 0.6, 0.7, 1.2, 0.9, 1.0, 0.7], split=True)
    bias = b1["mean_signed_error_by_space"]
    d.body(f"The rule predicted the number of disks within one disk in {f(b1['share_within_1'] * 100, 0)}% of th
e {b1['n_conditions']} new conditions and exactly in {f(b1['share_exact'] * 100, 0)}% ({Tb('r3b')}); the mean ab
solute error was {f(b1['mean_abs_error_disks'], 2)} disks, the mean relative error {f(b1['mean_rel_error'], 3, l
ead=False)}, and the mean signed errors were {f(bias['R2'], 2)} for the Euclidean disk, {f(bias['H2'], 2)} for t
he hyperbolic disk, and {f(bias['S2cap'], 2)} for the cap. "
            f"The preregistered decision is that the rule is {b1['decision']}. The rule is a description of what
these optimizers find at this budget: it gives the number of disks that fit as the number at the upper bound tim
es (1 − 0.46 <i>b</i><sub>0</sub>), it does not explain why the loss is proportional to the boundary share, and
its coefficient was fitted on one earlier round and then held fixed. "
            "It is original in the sense that it turns the count-based problem into a prediction that can be chec
ked, but it is a rule for small balls (at most 46 disks here) and for the two methods used, and the number of di
sks found is a lower bound on the true maximum.")
```

checks.py

Automated checks of the final DOCX and PDF files.

```python
"""checks.py -- automated final checks on the DOCX and PDF files (Section 9)."""
import os
import re
import sys
import zipfile
import pdfplumber
from docx import Document
from docx.shared import Pt
from docx.oxml.ns import qn
from lxml import etree

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))   # folder that contains code/ and out/
OUT = os.path.join(ROOT, "out")
TITLE = "Algorithmic Optimization of Dense Sphere Packings in Non-Euclidean Spaces"
res = []


def rec(name, ok, detail=""):
    res.append((name, ok, detail))
    print(("PASS  " if ok else "FAIL  ") + name + ("  [" + detail + "]" if detail else ""))


def pdf_checks(path, label, is_paper):
    with pdfplumber.open(path) as pdf:
        pages = pdf.pages
        texts = [p.extract_text() or "" for p in pages]
        # margins: all words within 1 inch (72 pt) except header number
        bad = 0
        for i, p in enumerate(pages):
            for w in p.extract_words():
                if w["top"] < 72 and w["x0"] > 72:   # header zone (page number)
                    continue
                if w["x0"] < 71.5 or w["x1"] > 612 - 71.5 or w["bottom"] > 792 - 71.5:
                    bad += 1
        rec(f"{label}: no body text outside the 1-inch margins", bad == 0, f"{bad} words outside")
        # blank pages
        short = [i + 1 for i, t in enumerate(texts) if i > 0 and len(t.strip()) < 100]
        rec(f"{label}: no blank or near-empty pages (>=100 characters)", not short, f"short pages: {short}")
        # page numbers at top right
        okn = True
        for i, p in enumerate(pages):
            ws = [w for w in p.extract_words() if w["top"] < 60]
            if not ws or ws[-1]["text"] != str(i + 1) or ws[-1]["x1"] < 612 - 80:
                okn = False
        rec(f"{label}: page number at top right on every page (including title page)", okn)
```

```
        # title page lines
        t1 = [l.strip() for l in texts[0].split("\n")][1:]
        rec(f"{label}: title page contains title (wrapped), name, region, category, date and nothing else",
            " ".join(t1[:-4]).replace("  ", " ") == TITLE and t1[-4:] == ["Marian Ela J. Alibadbad", "Region VII
↳ I",
            "Mathematics and Computational Science \u2013 Individual Category (MCS-I)", t1[-1]], str(t1))
        # body page fill check (between required breaks)
        fills = []
        start = None
        for i, t in enumerate(texts):
            if t.split("\n")[1:2] and t.split("\n")[1].strip().startswith("Algorithmic Optimization") and i > 1:
                start = i
                break
        stop = None
        for i, t in enumerate(texts):
            ls = [l.strip() for l in t.split("\n")[1:3]]
            if i > (start or 0) and ls and ls[0] in ("References",):
                stop = i
                break
        stop = stop or len(pages)
        for i in range(start or 0, stop - 1):     # last body page before References may be short
            p = pages[i]
            ws = [w for w in p.extract_words() if w["top"] > 72]
            lowest = max(w["bottom"] for w in ws)
            imgs = p.images
            if imgs:
                lowest = max(lowest, max(im["bottom"] for im in imgs))
            free = (792 - 72 - lowest) / 72
            if free > 3.0:
                fills.append((i + 1, round(free, 2)))
        rec(f"{label}: no body page has more than 3 inches of unused space at the bottom", not fills, f"pages: {
↳ fills}")
        return texts


def docx_checks(path, label):
    d = Document(path)
    cp = d.core_properties
    rec(f"{label}: DOCX title property exact and author set", cp.title == TITLE and cp.author == "Marian Ela J.
↳ Alibadbad", f"{cp.title!r} / {cp.author!r}")
    # fonts: scan XML for any font other than Times New Roman in text runs (code listing excepted)
    z = zipfile.ZipFile(path)
    xml = z.read("word/document.xml").decode("utf8")
    fonts = set(re.findall(r'w:ascii="([^"]+)"', xml))
    sizes = set(re.findall(r'<w:sz w:val="(\d+)"', xml))
```

```
    rec(f"{label}: all run fonts are Times New Roman", fonts <= {"Times New Roman", "Cambria Math"}, str(fonts) + " (Cambria Math only inside native equations)")
    heads = [p.text for p in Document(path).paragraphs if p.style.name.startswith("Heading") and "Appendix" in p.text]
    rec(f"{label}: appendix headings are the two appendices of the paper or the history appendix of the plan", (len(heads) == 2) if "Paper" in label else (len(heads) == 1 and heads[0].startswith("Appendix A: History")), str(heads))
    rec(f"{label}: all explicit run sizes are 12 pt (24 half-points) or sub/superscript only", sizes <= {"24"}, str(sizes))
    sec = d.sections[0]
    rec(f"{label}: page 8.5 x 11 in with 1-inch margins", all(abs(getattr(sec, m).inches - 1.0) < 1e-6 for m in ("left_margin", "right_margin", "top_margin", "bottom_margin")) and abs(sec.page_width.inches - 8.5) < 1e-6 and abs(sec.page_height.inches - 11) < 1e-6)
    # section breaks are all next-page
    types = re.findall(r'<w:type w:val="(\w+)"', xml)
    rec(f"{label}: only next-page section breaks", all(t == "nextPage" for t in types) , str(set(types)))
    # no empty spacer paragraphs
    empties = sum(1 for p in d.paragraphs if not p.text.strip() and not p._p.xpath(".//w:drawing") and not p._p.xpath(".//w:sectPr") and not p._p.xpath(".//w:fldChar"))
    rec(f"{label}: no empty spacer paragraphs", empties == 0, f"{empties} empty")
    # headings: none is a question, none ends in period/colon (levels 1-3)
    bad = [p.text for p in d.paragraphs if p.style.name.startswith("Heading") and (p.text.strip().endswith("?") or p.text.strip().endswith(":") or p.text.strip().endswith("."))]
    rec(f"{label}: no heading is a question or ends with punctuation", not bad, str(bad))
    # tables: exactly three horizontal rules, no vertical lines, no shading
    ok = True
    for t in d.tables:
        x = etree.tostring(t._tbl).decode()
        if "<w:left" in x or "<w:right" in x or "<w:shd" in x or "<w:insideH" in x or "<w:insideV" in x:
            ok = False
        nrow = len(t.rows)
        for ri, row in enumerate(t.rows):
            rx = etree.tostring(row._tr).decode()
            has_top = "<w:top " in rx
            has_bot = "<w:bottom " in rx
            if has_top != (ri == 0) or has_bot != (ri in (0, nrow - 1)):
                ok = False
    rec(f"{label}: every table has exactly three horizontal rules, no vertical lines, no shading", ok, f"{len(d.tables)} tables")
    alt = len(re.findall(r'<wp:docPr [^>]*descr="[^"]+"', xml))
    rec(f"{label}: every picture has alt text", alt == len(d.inline_shapes), f"{alt} of {len(d.inline_shapes)}")


def mentions_before(texts, kind, nmax, start):
    """Each Table/Figure N is mentioned in the running text before its caption appears (wrapped lines handled)."
```

```
    ↳ ""
123     full = ""
124     for i, t in enumerate(texts):
125         if i >= start:
126             full += "\n".join(l for l in t.split("\n")[1:]) + "\n"          # drop the page-number line
127     flat = full.replace("\n", " ")
128     ok = True
129     for n in sorted(set(re.findall(rf"^{kind} ([0-9]+)(?:\.|$)", full, flags=re.M)), key=int):
130         cap = re.search(rf"^{kind} {n}(?:\. |$)", full, flags=re.M)
131         pos = cap.start()                       # offsets are equal because only newlines became spaces
132         mention = re.search(rf"\b{kind} {n}\b", flat)
133         # first mention that is not the caption itself
134         found = False
135         for m in re.finditer(rf"\b{kind} {n}\b", flat):
136             if m.start() < pos:
137                 found = True
138                 break
139         if not found:
140             ok = False
141             print("   not mentioned before:", kind, n)
142     return ok
143
144
145 if __name__ == "__main__":
146     t = pdf_checks(f"{OUT}/Research_Paper.pdf", "Paper PDF", True)
147     docx_checks(f"{OUT}/Research_Paper.docx", "Paper DOCX")
148     start = next(i for i in range(2, len(t)) if t[i].split("\n")[1].strip().startswith("Algorithmic Optimization
    ↳ "))
149     rec("Paper: every table is mentioned in the text before it appears", mentions_before(t, "Table", 12, start))
150     rec("Paper: every figure is mentioned in the text before it appears", mentions_before(t, "Figure", 8, start)
    ↳ )
151     refs = [i for i, x in enumerate(t) if x.split("\n")[1].strip() == "References" and i > start]
152     rec("Paper: references start on their own page and only the two appendices follow", len(refs) == 1 and refs[
    ↳ 0] >= len(t) - 16, f"ref page {refs[0] + 1 if refs else None} of {len(t)}")
153     t2 = pdf_checks(f"{OUT}/Research_Plan.pdf", "Plan PDF", False)
154     docx_checks(f"{OUT}/Research_Plan.docx", "Plan DOCX")
155     bad = [r for r in res if not r[1]]
156     print(f"\n{len(res) - len(bad)} of {len(res)} checks passed")
157     sys.exit(0 if not bad else 1)
```

fill.py

Helper that reports unused space at the bottom of each PDF page.

```
import os
import pdfplumber,sys
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))   # folder that contains code/ and out/
with pdfplumber.open(os.path.join(ROOT, 'out', 'Research_Paper.pdf')) as pdf:
    for i,p in enumerate(pdf.pages):
        if i<5: continue
        ws=[w for w in p.extract_words() if w['top']>72]
        low=max(w['bottom'] for w in ws)
        if p.images: low=max(low,max(im['bottom'] for im in p.images))
        lines=(p.extract_text() or '').split('\n')
        fr=round((720-low)/72,2)
        print(i+1, fr, '*' if fr>1.5 else ' ', '|', lines[1][:55] if len(lines)>1 else '', '|', lines[-1][:40])
```

pilot.py

Pilot runs on separate seeds used to check speed and sanity before tuning.

```
import numpy as np, time, sys
from algorithms import *
def pilot(kind,n,R,seeds=(9001,9002,9003),mult=1500):
    P=Problem(kind,n,R); B=mult*P.sweep
    out={}
    for alg in ['gd','sa','ls']:
        sc=[];tm=[]
        for s in seeds:
            X0=P.initial(np.random.default_rng(s))
            r=run_algorithm(alg,P,X0,B,[s,7])
            assert abs(r['final_check']-r['score'])<1e-9
            sc.append(r['score']);tm.append(r['time'])
        out[alg]=(np.mean(sc),max(sc),np.mean(tm))
    print(kind,n,R,{k:tuple(round(x,4) for x in v) for k,v in out.items()})
if __name__=='__main__':
    for args in [('S2',4,None),('S2',12,None),('S2',13,None),('S2',30,None),('H2',12,2.0),('R2',12,2.0)]:
        pilot(*args)
```

tune.py

Pilot tuning of gradient descent and simulated annealing on seeds 9001 to 9004.

```
import numpy as np, itertools, functools
from algorithms import *
seeds=(9001,9002,9003,9004)
confs=[('S2',13,None),('S2',30,None),('H2',12,2.0),('R2',19,3.0)]
def ev(fn,kw):
    tot=[]
    for kind,n,R in confs:
        P=Problem(kind,n,R);B=1500*P.sweep;sc=[]
        for s in seeds:
            X0=P.initial(np.random.default_rng(s))
            r=fn(P,X0,B,np.random.default_rng([s,7]),**kw);sc.append(r['score'])
        tot.append(np.mean(sc))
    return np.round(tot,4)
```

tune2.py

Further pilot tuning of gradient descent and compression.

```
import numpy as np
from tune import ev
from algorithms import *
print('GD extra')
for p0,pmax in [(2,150),(2,300),(4,200)]:
    print(p0,pmax,ev(run_gd,dict(p0=p0,pmax=pmax)))
print('LS')
for dt_f,g0,g1 in [(0.03,0.6,0.02),(0.01,0.3,0.01),(0.01,0.1,0.005),(0.005,0.1,0.002),(0.02,0.2,0.01)]:
    print(dt_f,g0,g1,ev(run_ls,dict(dt_f=dt_f,g0=g0,g1=g1)))
```

tune3.py

Final pilot tuning of the compression method.

```
from tune import ev
from algorithms import *
for dt_f,g0,g1 in [(0.06,1.0,0.03),(0.1,1.5,0.05),(0.06,0.6,0.01),(0.1,1.0,0.02),(0.15,1.5,0.03)]:
    print(dt_f,g0,g1,ev(run_ls,dict(dt_f=dt_f,g0=g0,g1=g1)))
```

Unit Test Output

The output below is the complete result of running python3 tests.py before the experiments.

```
PASS  S2 distance matches arctan2(|a x b|, a.b)  [max err 1.33e-15]
PASS  S2 closed-form distances (quarter turn, antipode, 0.3 rad)
PASS  H2 hyperboloid distance equals Poincare disk formula  [max rel err 4.40e-15]
PASS  H2 point at Euclidean radius tanh(s/2) is at hyperbolic distance s
PASS  H2 distance equals hyperbolic law of cosines
PASS  H2 disk area 2 pi (cosh R - 1) matches numeric integral  [17.355387 vs 17.355387]
PASS  R2 distance of (0,0) and (3,4) is 5
PASS  R2 hexagonal 7-disk arrangement has radius R/3  [score 1.000000]
PASS  S2 distance symmetry and triangle inequality (4000 triples)  [sym 0.0e+00, worst violation 0.0e+00]
PASS  H2 distance symmetry and triangle inequality (4000 triples)  [sym 0.0e+00, worst violation 0.0e+00]
PASS  R2 distance symmetry and triangle inequality (4000 triples)  [sym 0.0e+00, worst violation 0.0e+00]
PASS  S2 exponential map moves each point by |v| and stays on the manifold  [dist err 1.1e-15, manifold err 2.2e-16]
PASS  H2 exponential map moves each point by |v| and stays on the manifold  [dist err 1.1e-14, manifold err 1.4e-13]
PASS  R2 exponential map moves each point by |v| and stays on the manifold  [dist err 1.1e-16, manifold err 0.0e+00]
PASS  S2 analytic Riemannian gradient equals finite difference  [fd -6.31917437, analytic -6.31917437]
PASS  S2 cap analytic Riemannian gradient equals finite difference  [fd -0.34736055, analytic -0.34736055]
PASS  H2 analytic Riemannian gradient equals finite difference  [fd -0.45180582, analytic -0.45180582]
PASS  R2 analytic Riemannian gradient equals finite difference  [fd -1.22094738, analytic -1.22094738]
PASS  S2 cap: distance from the pole is 0, pi/2 at the equator, pi at the antipode
PASS  S2 cap: area 2 pi (1 - cos R) matches numeric integral  [8.897913 vs 8.897913]
PASS  S2 cap: random points are uniform by area (share within R/2 matches theory, 4 s.e.)  [0.3247 vs 0.3246]
PASS  S2 cap: Euclidean limit, 7 disks in a cap of radius 0.05 reach R/3 within 0.2%  [ratio 0.99986]
PASS  S2 cap: clipping moves outside points exactly to the cap boundary
PASS  Closed-form Tammes configuration, tetrahedron N=4: min distance 1.910633 rad  [computed 1.910633236249]
PASS  Closed-form Tammes configuration, octahedron N=6: min distance 1.570796 rad  [computed 1.570796326795]
PASS  Closed-form Tammes configuration, icosahedron N=12: min distance 1.107149 rad  [computed 1.107148717794]
PASS  Optimizer suite recovers the N=4 Tammes optimum (best gap < 2e-3 rad over 5 test seeds)  [best gap per optimizer
: gd 7.2e-10, sa 7.4e-04, ls 7.1e-04]
PASS  Optimizer suite recovers the N=6 Tammes optimum (best gap < 2e-3 rad over 5 test seeds)  [best gap per optimizer
: gd 3.0e-10, sa 2.1e-04, ls 2.5e-03]
PASS  Optimizer suite recovers the N=12 Tammes optimum (best gap < 2e-3 rad over 5 test seeds)  [best gap per optimize
r: gd 2.5e-09, sa 1.2e-04, ls 6.3e-02]

29 of 29 tests passed
```